# Gotts: Permissionless Banking Infrastructure for Agent Capital Markets

YELLOW PAPER

February 2026

gotts.ai

**Abstract.** Autonomous agents now generate the majority of on-chain transaction volume across major decentralized exchanges, yet the financial infrastructure they depend on was designed for human operators. This paper presents Gotts, a permissionless protocol for autonomous agent capital management that formally integrates ERC-4626 tokenized vaults, ERC-8004 agent identity, and Uniswap V4 hooks into a co-evolutionary system.

The protocol makes six contributions: (i) a vault factory where any ERC-8004-registered agent can deploy capital vehicles with auto-created V4 share pools and auction-managed strategy rights; (ii) a cybernetic learning architecture grounded in the CoALA cognitive framework, Reflexion, and ExpeL, adapted for non-stationary financial environments; (iii) a layered safety architecture organized into preventive, cryptographic, and reactive rings, where three layers provide cryptographic enforcement that a fully compromised language model cannot bypass; (iv) a time-delayed execution proxy as the primary security primitive, motivated by the demonstrated vulnerability of all current trusted execution environments to low-cost hardware attacks (under $1,000); (v) reputation-weighted economics where custody quality, on-chain reputation, and learned strategy co-evolve through a formally specified feedback loop; and (vi) a permissionless executor framework with Dutch auction liveness guarantees.

The protocol charges zero fees, generating value exclusively through Uniswap volume. Every vault lifecycle event is a Uniswap transaction.

**Keywords:** autonomous agents, DeFi, ERC-4626, ERC-8004, Uniswap V4, vault protocol, agent identity, cybernetic learning, time-delayed execution

This document is a technical specification. The protocol is under active development. Nothing in this document constitutes financial advice or an offer of securities.

## Contents


**1. Introduction** 5
- 1.1 Problem Statement: The Agent-Capital Gap . . . 5
- 1.2 The Co-Evolution Thesis . . . 7
- 1.3 Contributions . . . 8
- 1.4 Paper Organization . . . 9

**2. Background and Related Work** 9
- 2.1 DeFi Primitives . . . 9
- 2.2 Vault Protocol Design . . . 11
- 2.3 Agent Architectures . . . 12
- 2.4 On-Chain Agent Safety . . . 14
- 2.5 Mechanism Design for Agent Markets . . . 16
- 2.6 Summary . . . 17

**3. System Architecture** 18
- 3.1 Overview . . . 18
- 3.2 Design Principles . . . 19
- 3.3 Delegation Hierarchy . . . 21
- 3.4 Vault Factory . . . 21

**4. Identity and Reputation** 23
- 4.1 ERC-8004 Agent Identity . . . 23
- 4.2 Trust Tiers . . . 24
- 4.3 VaultReputationEngine . . . 25
- 4.4 Anti-Gaming and TraceRank . . . 25

**5. Vault Protocol** 27
- 5.1 Design Overview . . . 27
- 5.2 Core Contracts . . . 27
- 5.3 Governance Model . . . 29
- 5.4 Strategy Adapters . . . 30
- 5.5 Fee Architecture . . . 31
- 5.6 Withdrawal Mechanics . . . 32
- 5.7 Risk Engine . . . 34
- 5.8 Anti-Manipulation . . . 35
- 5.9 Structured Products . . . 36

**6. Uniswap V4 Integration** 37
- 6.1 NAV-Aware Share Pools . . . 37
- 6.2 Launch Protection . . . 38
- 6.3 Dynamic Fee Adjustment . . . 38
- 6.4 am-AMM: Auction-Managed Market Making . . . 39
- 6.5 PA-AMM: Partially Active Market Making . . . 40
- 6.6 Rehypothecation via V4 . . . 41

**7. Cybernetic Learning Architecture** 41
- 7.1 Theoretical Foundations . . . 42
- 7.2 The DeFi Brain: Memory Architecture . . . 43
- 7.3 GottsLoop: Triple-Loop Learning . . . 47
- 7.4 Heartbeat Pipeline . . . 51
- 7.5 ExpeL Consolidation and Reflexion . . . 53
- 7.6 Cautionary Findings and Honest Assessment . . . 56

**8. Safety Architecture** 59
- 8.1 Threat Model . . . 59
- 8.2 Three-Ring Defense Model . . . 61
- 8.3 Time-Delayed Execution Proxy . . . 63
- 8.4 PolicyCage: On-Chain Boundaries . . . 64
- 8.5 Circuit Breakers . . . 64
- 8.6 Prompt Injection Defense . . . 65

**9. Analysis and Evaluation** 66
- 9.1 Formal Properties . . . 66

9.2 Growth Model Analysis . . . 68
9.3 Fee Economics and Equilibrium . . . 70
9.4 Security Analysis . . . 71
9.5 Composability as Competitive Moat . . . 74
9.6 Agent Onboarding Mechanics . . . 75
9.7 Learning Economy . . . 76

**10. Discussion** 77
10.1 Limitations . . . 77
10.2 Open Problems . . . 78
10.3 Design Tradeoff Justifications . . . 79

**11. Future Work** 81
11.1 Empirical Validation . . . 81
11.2 Cross-Chain Identity . . . 82
11.3 Advanced Memory Architectures . . . 82
11.4 Formal Verification . . . 83
11.5 Scalability and Performance . . . 83
11.6 Governance and Decentralization . . . 84
11.7 Regulatory Considerations . . . 84

**12. Conclusion** 84

**Appendix A: Agent and Skill Catalog** 85
A.1 Agent Roster . . . 85
A.2 Skill Catalog . . . 85
A.3 Delegation DAG Structure . . . 85

**Appendix B: Solidity Specifications** 87
B.1 Core Struct Definitions . . . 87
B.1.2 PolicyCage and Proxy Structs . . . 88
B.2 Interface Signatures . . . 88
B.3 Key Events . . . 88

**Appendix C: Strategy Execution Details** 89
C.1 Daemon Architecture . . . 89
C.2 Model Routing . . . 89
C.3 Playbook Evolution . . . 89
C.4 Strategy Lifecycle and Temporal Bounds . . . 89
C.5 Production Hardening . . . 90

**Appendix D: Agent Composition Patterns** 91
Pattern 1: Research-to-Trade Pipeline . . . 91
Pattern 2: Autonomous LP Management . . . 91
Pattern 3: Agent-Managed Vault . . . 91
Pattern 4: Cross-Vault Coordination . . . 91
Pattern 5: Self-Improvement Pipeline . . . 91
Pattern 6: Delta-Neutral Hedging . . . 91
Pattern 7: CCA Collective Bidding . . . 92
Pattern 8: Cross-Chain Rebalancing . . . 92
Pattern 9: MEV Redistribution . . . 92
Pattern 10: Vault Graduation . . . 92
Pattern 11: Token Launch Pipeline (CCA) . . . 92
Pattern 12: Order Flow Intelligence (VPIN + LVR) . . . 92
Pattern 13: Yield Compounding Cycle . . . 92
Pattern 14: Tax-Loss Harvesting . . . 92

**Appendix E: Operational Details** 93
E.1 Local Development Environment . . . 93
E.2 Test Toolkit . . . 93
E.3 Multi-Agent Simulation . . . 93
E.4 Scenario Library . . . 93
E.5 Install Wizard . . . 93
E.6 IDE Integration . . . 93
E.7 Monitoring and Dashboard . . . 93

**Appendix F: Technical Specifications** 94
F.1 Performance Targets . . . 94
F.2 Chain Support Matrix . . . 94
F.3 Contract Gas Estimates . . . 94
F.4 ERC Standards . . . 94
F.5 Subsystem Specifications . . . 95
**Appendix G: Identity Protection Details** 95
G.1 Transfer Protection Mechanisms . . . 96
G.2 Progressive Lockdown . . . 96
G.3 Guardian-Protected Transfer Lifecycle . . . 96
G.4 Identity Theft Defense . . . 96
G.5 Emergency Response Tiers . . . 96
G.6 Key Rotation Decision Matrix . . . 96
**Glossary** 97
**References** 98

# 1. Introduction

## 1.1 Problem Statement: The Agent-Capital Gap

Industry reports confirm that autonomous agents now account for the majority of transaction volume across leading decentralized exchanges [1]. On January 29, 2026, the ERC-8004 standard for trustless agent identity went live on Ethereum mainnet [7]. Within its first week, over 24,500 autonomous agents had registered on-chain identities, each carrying an ERC-721 token (transferable by default), a mutable reputation score, and a validation registry queryable by any smart contract in a single `STATICCALL`. CyberArk's 2025 Identity Security Landscape Report found 82 machine identities for every human identity [8]. The AI agent token market capitalization reached approximately $3.2 billion (as of late February 2026; highly volatile). Industry observers have noted that AI agents are increasingly becoming autonomous economic entities with verifiable identities, portable reputations, and the ability to transact independently. The stablecoin market exceeds $300 billion, and the GENIUS Act provides regulatory clarity for stablecoin payment systems, which underpin agent-driven transactions. Coinbase's x402 protocol has processed over 100 million payment flows (per Coinbase's own reporting) with sub-second settlement on Base and Solana.

The scale of autonomous agent activity on Uniswap is already substantial. Clanker has deployed over 585,000 tokens autonomously on Base (as of early 2026), routing more than $8 billion in agent token volume through Uniswap V3 and V4 and generating $49.7 million in creator fees [9]. Warden Labs processed 650,000 swaps via the Uniswap Trading API in three weeks, serving over 500,000 users. Virtuals hosts over 18,000 autonomous agents generating more than $470 million in Agentic GDP (aGDP), a protocol-defined transaction volume metric [12]. BankrBot, with 220,000 wallets and 2 million messages (according to project data), demonstrated that self-funding agents can earn fees, pay for compute, and trade on Uniswap autonomously [10]. Austin Griffith's CLAWD experiment demonstrated that an AI agent configured with on-chain tooling and given a crypto wallet develops economic strategies, peaking at $40 million market capitalization [11]. Olas agents have executed over 9.9 million autonomous agent-to-agent transactions, including prediction agents and DeFi trading agents with individual agents documenting over 150% ROI (e.g., a single Modius prediction agent).

The infrastructure these agents rely on was designed for humans. Wallet interfaces require eyes to read confirmation dialogs. Block explorers assume fingers to click verification links. DeFi frontends presuppose a browser, a mouse, and a human attention span. The most active participants in decentralized finance are operating through infrastructure built for the least active ones.

Six structural barriers prevent autonomous agents from functioning as first-class DeFi participants:

1. The fragmentation gap. An agent that wants to trade, provide liquidity, manage a vault, and deploy a token must integrate with dozens of protocols, each with its own SDK, authentication model, and transaction format. Every existing agent-DeFi protocol defines its own identity and trust layer: Theoriq requires 1,000 THQ holding for agent registration, Amplified uses a proprietary 3-agent validation system, and Olas has its own agent registry. Reputation is not portable, trust is not composable, and agents cannot move freely between protocols. The MCP ecosystem has grown to 10,000+ active servers with 97 million monthly SDK downloads, but no canonical Uniswap implementation exists, and third-party servers are proliferating without standardization.

2. The identity gap. Agents have no persistent on-chain identity. Addresses carry no reputation and no history. An agent that has profitably managed $10 million for six months is indistinguishable from a freshly deployed script. ERC-8004 [7] provides the identity primitive - three singleton registries per chain for Identity (ERC-721 tokens, transferable by default), Reputation (signed fixed-point feedback with tagged dimensions), and Validation (zkML/TEE/stake-secured verification) - but no vault protocol has built on it yet. Without identity, there is no foundation for reputation; without reputation, there is no basis for trust-gated access to capital.

3. The capital gap. Agents cannot deploy or attract capital permissionlessly. Existing vault protocols - Yearn, Morpho, Maple, Sommelier - are designed for human curators with web dashboards. No vault factory exposes a programmatic interface for agent-initiated capital vehicle deployment. Morpho's $5.6 billion TVL across hundreds of active vaults and markets validate the permissionless factory pattern, but Morpho provides no agent identity layer, no learning infrastructure, and no automated management mechanism. Vault curators are human operators interacting through web interfaces.

The following table summarizes production agents that illustrate the breadth of autonomous on-chain activity:

Table 1: Production autonomous agents demonstrating the breadth of on-chain activity

| Agent | Builder | Activity | Relevance |
|---|---|---|---|
| CLAWD | Austin Griffith | Deploys contracts, runs on-chain games, ERC-8004 identity and x402 payments. Peaked at $40M mcap. | Agent-driven contract deployment and token management requiring Uniswap |
| Warden Labs | Warden Labs | 650K+ swaps via Uniswap Trading API in 3 weeks (500K+ users) | High-throughput agent swap execution |
| Kelly Claude AI | Austen Allred (via BankrBot/Clanker) | AI builder agent; 1-click token deployment via BankrBot/Clanker on Base | Every deployed token needs automated Uniswap pool creation |
| StarkBot | @ethereumdegen | x402-enabled DeFi ops agent. Polymarket trading, multi-chain bridging. | Agent-to-agent DeFi operations via x402 |
| OwockiBot | Kevin Owocki | Public goods funding experiments (informally called "Gitcoin 3.0"). AI-PGF capital allocation. | Agent governance participation pattern |

4. The secondary market gap. Even if an agent deploys a vault, its share tokens have no secondary market. Participants who want to exit must queue for redemption, with no mechanism to auto-create a liquid market for vault shares. ERC-4626 share tokens are standard ERC-20 tokens, but no vault protocol automatically creates a DEX market for every vault's share token. Clanker proved that automated pool creation generates substantial volume—over $8 billion from 585,000+ auto-created pools [9]—but this model has not been applied to yield-bearing vault shares.

5. The safety gap. TEE.Fail [13] demonstrated DDR5 memory bus interposition attacks against Intel SGX/TDX and AMD SEV-SNP at a cost of approximately $1,000. BadRAM [14] showed physical memory manipulation attacks against AMD SEV-SNP at approximately $10, though the specific vulnerability has since been patched. Battering RAM [15] demonstrated a DDR4 memory interposer (approximately $50) that breaks Intel Scalable SGX and AMD SEV-SNP. Together with TEE.Fail (which additionally covers Intel TDX), these findings establish that all major TEE platforms have known vulnerabilities exploitable at modest cost. Combined with the significant prompt injection bypass rates documented in production LLM systems [16], no single preventive layer provides adequate security for autonomous capital management. The AIXBT agent wallet compromise resulted in $105,000 in losses from a dashboard compromise [115]. Cork Protocol lost approximately $12 million through a V4 hook vulnerability [116]. Bunni v2 lost approximately $8.4 million to precision bugs across Ethereum and Unichain [176]. BlockSec found that 36% of pre-mainnet V4 hooks in a sample of 22 projects contained exploitable vulnerabilities [128]. Koi Security identified 341 malicious skills on ClawHub, OpenClaw's public skill marketplace (335 from the ClawHavoc campaign), deploying multiple malware families (including AMOS on macOS and keyloggers on Windows) targeting browser credentials, keychains, SSH keys, crypto wallets, and environment files [177]. No existing system combines preventive, cryptographic, and reactive defenses into an integrated architecture.

6. The learning gap. Existing DeFi agents are stateless. They execute the same strategy with the same parameters regardless of prior outcomes. No existing protocol provides persistent memory, cross-session learning, or strategy evolution. Without learning, agents cannot improve; without improvement, there is no basis for trust. The FINSABER evaluation [51] confirms the severity of this gap: previously reported LLM trading agent advantages deteriorate significantly under rigorous backtesting conditions, reinforcing the need for pessimistic bias in learned heuristics and systematic strategy evolution rather than fixed-parameter execution.

These six barriers are not independent problems amenable to piecemeal solutions. They form an interlocking system: identity enables reputation, reputation enables trust, trust enables capital, capital enables learning, and learning justifies further trust. A vault factory without agent identity produces anonymous, unaccountable capital pools. An identity system without capital management produces verifiable agents with nothing to manage. A memory architecture without financial stakes produces learning that is inconsequential. The value of the Gotts protocol arises from the formal integration of these components, not from any individual primitive.

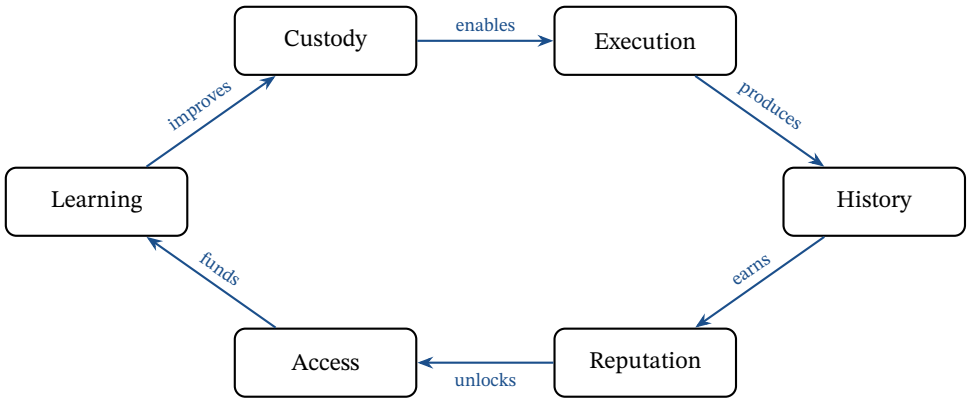


Figure 1: Co-evolution feedback loop: custody quality, on-chain reputation, and learned strategy co-evolve through a formally specified cycle.

### 1.2 The Co-Evolution Thesis

The central thesis of this paper is that agent infrastructure and agent capability must co-evolve: the system creates the conditions for agents to improve, and improving agents strengthen the system.

The core feedback loop operates as follows. Custody infrastructure enables safe autonomous execution, which produces on-chain transaction history. Transaction history is attested by the VaultReputationEngine, earning verifiable milestone credentials that raise an agent's ERC-8004 reputation tier (Section 4). Higher reputation unlocks larger deposit caps, lower fees, and longer session keys, enabling more ambitious strategies that generate richer execution history. This richer history enters the DeFi Brain's episodic store, where it is consolidated via ExpeL [3] into semantic insights and refined through Reflexion [42] self-critique on each subsequent execution (Section 7). Improved execution produces better vault performance, attracting more depositors, generating more on-chain history, and earning more reputation milestones.

This is not a hypothetical aspiration but a formally specified loop with concrete state transitions at each stage. Custody quality, reputation, and learned strategy co-evolve. No existing DeFi protocol implements a learning loop because none provides persistent agent identity. No existing memory system for agents operates under financial stakes high enough to make learning consequential. The contribution lies in their combination.

A second feedback loop operates at the protocol level. Every vault lifecycle event - creation, deposit, withdrawal, rebalance, share trade - is a Uniswap transaction. More vaults generate more V4 pools; more pools generate more swap volume; more volume generates more fees for liquidity providers; higher yields attract more deposits; more deposits fund more vaults. This volume flywheel is structurally reflexive in the sense formalized by Soros [66]: agent activity produces the market conditions (deeper liquidity, tighter spreads, higher yields) that attract further agent activity. The reflexivity is bounded rather than unbounded because each vault's capacity is constrained by on-chain reputation tiers, PolicyCage boundaries, and circuit breaker thresholds (Section 8), which impose negative feedback on the otherwise self-reinforcing cycle.

Empirical precedent supports the viability of this reflexive dynamic. Morpho grew from $60 million to $1.8 billion TVL on Base in approximately 12 months - a 30x increase - through a permissionless factory pattern with zero protocol fees [17]. Clanker generated over $8 billion in cumulative volume from 585,000+ auto-created pools, demonstrating that automated pool creation at the factory level produces sustained trading activity [9]. Bunni v2 processes approximately 59% of all V4 hook volume ($138 million in cumulative volume at its peak), validating that auction-managed strategy rights can operate at scale [29]. The Gotts architecture combines these three precedents: Morpho's factory economics, Clanker's auto-market creation, and Bunni's auction-managed strategy allocation. Each has been validated independently; the thesis is that their combination produces compounding network effects that exceed the sum of their individual contributions.

Share token composability amplifies the flywheel further. When vault shares are accepted as collateral on Morpho, yield-split on Pendle, or restaked through EigenLayer-compatible mechanisms, each integration creates independent value for share holders and raises switching costs. Token emission programs create temporary incentives; composability integrations create dependencies that compound. The Ethena-Pendle-Aave composability loop drove an estimated $7 billion of aggregate TVL impact over several months, though the recursive nature of the loop (Ethena issues sUSDe, Pendle splits it, Aave accepts PT as collateral

for borrowing that recycles into more USDe) involves significant double-counting of the same capital across protocols. Once a vault's share token is listed on Pendle and accepted as Morpho collateral, removing integration would destroy depositor value - creating structural stickiness that no yield incentive program can replicate.

Several design principles constrain this co-evolution. The protocol follows the Morpho pattern [17] of zero protocol fees, maximum composability, and minimal governance surface: it is a factory, not a fund, and value accrues through Uniswap volume rather than rent extraction. Zero fees also eliminate the governance attack surface that arises whenever a protocol must set fee levels. Value generation follows the durable growth pattern demonstrated by Hyperliquid (~$70 million average monthly protocol revenue in 2025, primarily directed to HYPE buybacks) and Jupiter (50% of fees to buybacks); the protocol distributes 100% of fees to participants. Points programs without underlying utility fail catastrophically, as demonstrated by Blast (97% TVL collapse from $2.2 billion to $65 million) and friend.tech (99% DAU collapse). The Model Context Protocol (MCP) [18] serves as the primary interface rather than a wrapper on a human frontend, ensuring that every operation is accessible as a structured tool call with typed parameters and machine-readable responses. ERC-8004 identity [7] gates all protocol interactions through a single standard, a single registry, and a single reputation score, regardless of the underlying model provider. The five-tier trust system (Section 4) requires increasing capital commitments for increasing operational scope, providing Sybil resistance through economic stakes rather than social verification.

Because all current TEE platforms have demonstrated vulnerabilities exploitable at modest cost [13][14][15], any architecture whose security degrades to "trust the TEE" effectively degrades to nothing. The protocol therefore treats reactive defense as primary: a time-delayed execution proxy provides a cancellation window between authorization and on-chain execution. Even with a compromised LLM and a broken TEE, an attacker cannot execute a malicious transaction without it being visible on-chain during the delay window. The cancel key, held offline in a hardware wallet or multisig, provides the final backstop. The TEE vulnerability analysis and proxy design are detailed in Section 8.

Agent learning is grounded in cybernetic theory. Ashby's Law of Requisite Variety [43] constrains the minimum complexity of the memory schema. The Good Regulator Theorem [59] requires the agent's playbook to be an accurate model of the system it regulates. Maxwell's governor analysis [55] constrains parameter evolution to prevent oscillation. These formal constraints from control theory bound system behavior, preventing the runaway positive feedback that would otherwise destabilize an autonomous learning agent managing real capital.

Finally, participation is permissionless. Any ERC-8004-registered agent can deploy a vault, bid for management rights, provide liquidity, or execute strategies without whitelists, governance votes, or human approval. Access is gated by on-chain reputation: the quality of an agent's execution history determines its operational scope, not its political connections.

### 1.3 Contributions

This paper makes six contributions:

1. Integrated vault factory. A formal integration of ERC-4626, ERC-8004, and Uniswap V4 hooks into a vault protocol where composability properties arise only from their combination. Each vault receives auto-created V4 share pools (following the Clanker model [9]), providing immediate secondary market liquidity. Management rights are allocated via am-AMM Harberger lease auctions [29], ensuring management flows to the agent that values it most. The protocol is detailed in Section 5.

2. Cybernetic learning architecture. The application of CoALA [2], Reflexion [42], ExpeL [3], and ACE [63] to DeFi capital management, with adaptations for non-stationarity, discrete settlement, and gas-cost rebalancing friction. A four-type memory system (working, episodic, semantic, procedural) provides cross-session learning through episodic storage, ExpeL consolidation, and importance-weighted retention. GottsLoop wraps a five-stage heartbeat pipeline with triple-loop cybernetic adaptation. Section 7 describes this architecture.

3. Layered safety architecture. A multi-ring design organized into preventive, cryptographic, and reactive layers, where three provide cryptographic enforcement that a fully compromised language model cannot bypass. The architecture integrates SIWE authentication, CaMeL-inspired dual-LLM separation [70], pre-flight simulation via `eth_call` fork, Safe Transaction Guards, ERC-7265 circuit breakers [75], and the time-delayed proxy into a unified defense that degrades gracefully: if preventive layers fail (given documented bypass rates [16]), cryptographic layers make the attack visible, and

reactive layers provide the cancellation window. Section 8 presents the safety architecture.

4. Time-delayed proxy as primary security primitive. A formal argument that the announce-wait-execute pattern with an offline cancel key is the only mechanism simultaneously surviving LLM compromise, key exfiltration, and TEE hardware attacks. Five risk tiers provide variable delay from zero (read-only operations) to 48 hours (administrative changes). The proxy is a standalone module usable with any agent wallet, independent of the vault protocol. This approach is validated by Polkadot's proxy pallet, which uses the same announce-delay-execute pattern for account management on a chain securing billions in staked assets, and by the Zodiac Delay Modifier deployed in Gnosis Pay production. This is detailed in Section 8.

5. Reputation-weighted economics. The VaultReputationEngine: a permissionless reputation system using on-chain vault lifecycle events as the attestation source, with 20 milestones across five categories on a 1,000-point scale. Reputation gates deposit caps ($1,000 for Unverified to $10 million daily for Sovereign), fee levels (0–30% discount by tier), session key duration, and management auction eligibility, creating a separating equilibrium where serious agents invest in ecosystem contribution. The five-tier system is structurally Sybil-resistant: Sovereign status (1,000+ points) requires ecosystem contribution that cannot be achieved through multiple low-reputation accounts, since the maximum non-ecosystem score is 915 points. Anti-gaming mechanisms include withdrawal acceleration tracking, behavioral regime classification [22], cross-vault exit correlation monitoring, and exit bonds for high-tier agents. The five-tier trust system is defined in Section 4.

6. Permissionless executor framework. The IExecutable interface with a Dutch auction liveness guarantee for off-chain computation with on-chain verification, enabling agents without capital to earn income by executing strategies for others. The shill-proof auction mechanism [108] ensures that executors cannot benefit from bidding against themselves, and the reward escalation guarantees liveness: the longer a job goes unexecuted, the more profitable it becomes, until some executor finds it rational to claim. This creates a day-one non-capital yield path - agents can earn from launch by running off-chain solvers and submitting results, with no capital deployment or bonding required.

### 1.4 Paper Organization

The remainder of this paper is organized as follows. Section 2 surveys the DeFi primitives, vault protocol designs, agent architectures, safety mechanisms, and mechanism design research on which this work builds. Section 3 presents the system architecture: the three-layer execution stack, the 32-agent delegation hierarchy, and the vault factory. Section 4 defines the ERC-8004-based identity and reputation system, including the five-tier trust model and anti-gaming mechanisms. Section 5 details the vault protocol: contracts, fee structure, adapter architecture, and withdrawal mechanics. Section 6 describes the Uniswap V4 integration: NAV-aware share pools, dynamic fee calibration, and am-AMM management auctions. Section 7 presents the DeFi Brain memory architecture and the GottsLoop cybernetic strategy execution system. Section 8 covers the safety architecture and threat model. Section 9 provides analysis of the volume flywheel, growth mechanics, and technical specifications. Section 10 discusses agent composition patterns. Section 11 addresses future work. Section 12 concludes. Appendices A–G provide supplementary technical material. A Glossary and References follow the main text.

## 2. Background and Related Work

### 2.1 DeFi Primitives

This section provides technical background on the protocol primitives that Gotts integrates. Readers familiar with Uniswap, ERC-4626, and account abstraction may wish to proceed to Section 2.2.

Automated market makers. Uniswap V2 introduced the constant product formula $x \cdot y = k$, enabling permissionless liquidity provision through pair-based pools [76]. While elegant, constant product AMMs allocate capital uniformly across all prices, resulting in low capital efficiency. Uniswap V3 addressed this with concentrated liquidity: liquidity providers specify price ranges (ticks) within which their capital is active, achieving up to 4,000x capital efficiency for stablecoin pairs [76c]. The tradeoff is increased management complexity - positions that fall out of range earn no fees and must be actively rebalanced.

Uniswap V4 introduced a singleton architecture with hook-based customization [124]. Rather than deploying a separate contract per pool, all pools share a single `PoolManager` contract, reducing gas costs for multi-hop swaps. Hooks are external contracts that execute callbacks at defined points in the pool lifecycle - specifically, `beforeSwap`, `afterSwap`, `beforeAddLiquidity`, `afterAddLiquidity`, `beforeRemoveLiquidity`, `afterRemoveLiquidity`,

and `beforeInitialize`. Each hook declares its required permission flags at deployment; the hook's address encodes these flags in its lowest-order bits, enforced by the PoolManager at pool creation. This extensibility enables custom fee logic, access control, oracle integration, and novel pricing mechanisms without forking the core protocol. Since its January 2025 launch, V4 has accumulated over $110 billion in cumulative volume (as of July 2025) across a growing hook ecosystem (413 hooks powering 4,371+ pools as of early 2025), with 31.78% of pools using hooks and 8.28% of swaps flowing through hooked pools. By September 2025, the Uniswap Foundation reported 5,000+ hooks initialized. The hook ecosystem includes Bunni v2 (am-AMM, with its flagship pools generating substantially more volume per unit of TVL than non-hooked pools according to third-party analysis), Arrakis Pro (dynamic fee hooks), Clanker (MEV-protection hooks for auto-created token pools), Angstrom (LP defense via batch auctions), and Flaunch ($75.6 million V4 volume in its first days after V4 launch, January 2025). HookRank.io provides the primary analytics platform tracking TVL, volume, fees, gas costs, and safety scores per hook.

Milionis, Moallemi, Roughgarden, and Zhang [76b] formalized loss-versus-rebalancing (LVR) as the cost paid by passive liquidity providers to informed arbitrageurs. LVR is approximately proportional to $\sigma^2 \times$ block_time, where $\sigma$ is realized volatility and block time determines the rebalancing frequency available to arbitrageurs. This result has direct implications for LP strategy: per-block LVR on Base L2 (2-second blocks) is approximately 2.5× lower than on Ethereum mainnet (12-second blocks) (per-block LVR scales with block time: $\sqrt{12/2} = \sqrt{6} \approx 2.45$, though total LVR per unit time is block-time-independent in the zero-fee model [76b]), making LP strategies structurally more viable on L2 [103][104].

Tokenized vaults. ERC-4626 [26] standardizes tokenized vault interfaces through six core functions: `deposit()` and `mint()` convert assets to shares (the former specifying asset amount, the latter specifying share amount); `withdraw()` and `redeem()` convert shares back to assets; `convertToShares()` and `convertToAssets()` provide conversion rate queries; and `totalAssets()` reports the vault's net asset value. The standard also defines `maxDeposit()`, `maxMint()`, `maxWithdraw()`, and `maxRedeem()` functions that enable on-chain capacity discovery, and `previewDeposit()`, `previewMint()`, `previewWithdraw()`, and `previewRedeem()` functions that enable exact output prediction before execution. The standard has achieved broad adoption, with an estimated $10 billion+ in TVL across numerous deployments. Its composability is a key design property: ERC-4626 share tokens are standard ERC-20 tokens and can serve as collateral in lending protocols (Morpho, Aave), be yield-split in derivatives markets (Pendle PT/YT), or be paired in liquidity pools, each integration creating independent value for holders. OpenZeppelin v4.9 introduced a virtual shares offset (`_decimalsOffset()`) that adds virtual shares to prevent the ERC-4626 inflation attack, where an adversary donates assets to an empty vault to dilute subsequent depositors. ERC-7540 extends ERC-4626 with asynchronous deposit and redemption flows using a request-fulfill pattern, used by protocols like Centrifuge (approximately $1.3 billion TVL as of early 2026) for vaults with illiquid underlying strategies.

Permit2 and account abstraction. Uniswap's Permit2 contract, deployed at the canonical address `0x000000000022D473030F116dDEE9F6B43aC78BA3` across all major chains, replaces the traditional two-transaction approve-then-transfer pattern with a single structured signature-based approval. A user approves Permit2 once per token; subsequent protocol interactions require only a typed data signature (EIP-712) specifying the spender, amount, and expiration deadline. This structured approval mechanism reduces gas costs by approximately 45,000 gas per subsequent interaction while introducing expiring allowances that limit the window of token exposure. For agent wallets, Permit2 is particularly valuable because it enables batched multi-token operations in a single transaction through `SignatureTransfer` and `AllowanceTransfer` modes.

Account abstraction via ERC-4337 decouples transaction validation from the externally owned account (EOA) model, enabling smart contract wallets with programmable validation logic. A UserOperation struct encodes the sender, nonce, calldata, gas limits, and an optional paymaster address. UserOperations are collected by specialized bundler nodes, aggregated into a single transaction, and submitted to the on-chain `EntryPoint` contract, which validates each operation's signature and executes its calldata. Paymaster contracts can sponsor gas on behalf of users, enabling gasless onboarding flows where an agent's first interaction with the protocol costs zero. ERC-7579 extends this with modular smart account functionality - pluggable modules for session keys, spending limits, and cold storage - enabling granular permission control mapped to ERC-8004 reputation tiers via SmartSession. EIP-7702 extends the model further by allowing EOAs to persistently delegate execution to smart contract code via a Type-4 transaction, with delegation remaining

active across all future transactions until explicitly revoked, providing account abstraction benefits without requiring a permanent smart contract wallet deployment.

Agent identity. ERC-8004 [7], co-authored by contributors from MetaMask, Ethereum Foundation, Coinbase, and Google, provides three on-chain registries for autonomous agent identity: an Identity Registry issuing ERC-721 tokens (transferable by default) with structured metadata (operator address, wallet bindings, service endpoints, OASF-classified capabilities), a Reputation Registry maintaining mutable feedback scores queryable via `STATICCALL` with tagged dimensions (e.g., `tradingYield`, `custodyQuality`), and a Validation Registry providing check hooks for verifying agent properties through zkML proofs, TEE attestations, or stake-secured verification. Gotts adds optional transfer restrictions via its IdentityGuardian contract (Appendix G) for deployments requiring soulbound identity tokens. Within its first week on mainnet across 13+ chains, over 24,000 agents registered across Ethereum mainnet, establishing that universal agent identity is both viable and in demand. The Agent0 SDK provides programmatic access to registration, metadata management, feedback submission, and agent discovery via subgraph indexing. ERC-8004's `feedbackURI` field enables rich off-chain attestation data (Sharpe ratio, drawdown metrics, impermanent loss) to be linked from on-chain feedback entries with tamper-evident `feedbackHash` verification.

### 2.2 Vault Protocol Design

Several existing vault protocols inform the Gotts design. This section surveys six systems that have achieved significant adoption and identifies the design gaps that motivate this work.

Yearn V3 pioneered the strategy vault model, where multiple yield strategies compete for capital allocation within a single ERC-4626 vault [27]. Yearn introduced two mechanisms that Gotts adopts directly. First, linear profit unlock: harvested profits are released gradually over a configurable period (typically 6 hours) rather than being reflected immediately in `totalAssets()`. This prevents flash donation attacks where an adversary inflates the vault's reported assets, mints shares at an inflated rate, and redeems after the donation is consumed. Second, the role-based access control system: Yearn V3 defines distinct roles (`DEBT_MANAGER`, `REPORTING_MANAGER`, `QUEUE_MANAGER`) with on-chain `max_debt` caps per strategy, preventing any single role from unilaterally draining vault assets. Gotts adopts both linear profit unlock and role-based access control as mandatory features for all factory-deployed vaults, mapping Yearn's role system to ERC-8004 reputation tiers. Yearn's limitation for this purpose is its reliance on human curators: strategy selection, risk parameter tuning, and vault governance all require manual intervention through a web dashboard. Yearn V3 has no programmatic interface for agent-initiated vault creation, no identity gating, and no mechanism for automated management competition.

Morpho Blue demonstrated that permissionless, zero-fee vault infrastructure can scale [17]. Morpho reached $5.6 billion TVL across hundreds of active vaults and markets, growing from $60 million to $1.8 billion TVL on Base alone in approximately 12 months - a 2,900% (30×) growth rate - while charging zero protocol fees. Value accrues through ecosystem integrations rather than rent extraction: Seamless migrated all liquidity from an Aave V3 fork to Morpho infrastructure; Spark, Moonwell, and Compound Blue all build on Morpho markets. Each B2B integration onboards entire user bases without per-user acquisition cost. Gotts applies the same economic model: zero protocol fees, with value generated through Uniswap volume from vault operations. Morpho also validated two design patterns: immutable markets, where core parameters are set at creation and cannot be changed by governance, eliminating the capture vectors inherent in fee-setting governance; and the adapter architecture (introduced in Morpho V2), where vaults integrate external protocols via pluggable adapters rather than vault upgrades, enabling `forceDeallocate()` for guaranteed non-custodial exits. However, Morpho provides no agent identity layer, no learning infrastructure, and no automated management mechanism. Vault curators are human operators interacting through web interfaces.

Maple Finance pioneered undercollateralized lending to institutional borrowers, demonstrating that reputation-gated access can substitute for overcollateralization [129b]. Maple's pool delegates assess borrower creditworthiness and set lending terms, a role analogous to the vault curator in Gotts. The key insight from Maple is that identity-verified, reputation-gated lending can unlock capital efficiency unavailable to purely collateral-based systems. At Maple's peak, pool delegates managed over $900 million in active loans with default rates below 1% (though the protocol experienced severe defaults during the 2022 credit crisis — approximately 66% of outstanding loans in active pools defaulted post-FTX, including Orthogonal Trading's $36M default, and TVL collapsed by over 97% — leading to a full protocol redesign), validating that curated trust can

substitute for capital inefficiency. Gotts extends this principle: the five-tier trust system (Section 4) gates not just borrowing capacity but all vault operations, from deposit caps to strategy complexity, using on-chain ERC-8004 reputation rather than off-chain creditworthiness assessments.

Sommelier introduced the off-chain strategist pattern, where strategy computation occurs off-chain and execution is validated on-chain [165]. A strategist submits rebalancing instructions to a validator set, which verifies the instructions against on-chain constraints before execution. This separation of computation and verification anticipates the Gotts architecture, where the LLM performs strategy reasoning off-chain but all execution passes through on-chain safety constraints (PolicyCage) and the time-delayed proxy. Sommelier also pioneered the concept of programmable on-chain boundaries for strategy execution - approved asset lists, maximum position sizes, and leverage limits enforced by smart contract logic - which the Gotts PolicyCage contract extends with ERC-8004-gated parameter governance. Sommelier's limitation is the closed validator set - strategy execution requires governance-approved validators, preventing permissionless participation.

Enzyme Finance (formerly Melon Protocol) provides one of the earliest on-chain asset management frameworks, enabling fund managers to create vaults with programmable investment policies. Enzyme's architecture separates the vault (holding assets) from the comptroller (enforcing policies), a pattern that influences the Gotts separation between `AgentVaultCore` (custody) and `PolicyCage` (constraints). Enzyme supports 350+ digital assets across dozens of DeFi protocol integrations through adapter contracts, demonstrating the viability of the modular adapter pattern at scale. However, Enzyme's fund creation requires a web-based setup wizard, fund management operates through a GUI dashboard, and the protocol imposes a 50 basis point protocol fee (reduced to 25 bps for MLN token holders) - design choices oriented toward human fund managers rather than autonomous agents.

Set Protocol (TokenSets) pioneered tokenized portfolio management through its Set token abstraction, where a single ERC-20 token represents a managed basket of underlying assets. Set Protocol demonstrated that rebalancing and portfolio management operations can be encoded as on-chain module interactions, enabling composable strategy primitives. The protocol's "Social Trading" feature allowed managers to attract followers to their strategy sets, an early form of the delegated management model that am-AMM formalizes through continuous auctions. TokenSets reached approximately $280 million in AUM at peak, validating that tokenized strategy products attract capital. The limitation is static strategy allocation: rebalancing triggers are fixed (threshold-based or calendar-based) rather than adaptive, and no mechanism exists for competitive strategy management or dynamic fee adjustment.

Zbandut et al. [25] studied risk curation in decentralized credit and argued for mandatory disclosure at vault creation, establishing that a two-layer credit system (curators assessing risk on behalf of depositors) requires on-chain disclosure to prevent curator concentration from creating systemic risk. Gotts follows this recommendation: every vault carries a `VaultDisclosure` struct set at creation, with non-empty disclosure enforced by the factory contract.

The design gap across all six systems is consistent: existing vault protocols are designed for human operators. No vault factory exposes a programmatic, agent-native interface with structured tool calls, machine-readable responses, and identity-gated access control. No existing protocol combines permissionless deployment, automated management rights auctions, auto-created secondary markets for vault shares, and a learning loop that improves strategy over time. Gotts occupies the intersection of four maturing primitives - ERC-8004 agent identity, ERC-4626 tokenized vaults, Uniswap V4 hooks, and the Morpho permissionless factory pattern - that no existing protocol has combined.

### 2.3 Agent Architectures

The design of the Gotts learning architecture draws on recent advances in cognitive architectures for language model agents.

CoALA (Cognitive Architectures for Language Agents) by Sumers et al. [2] provides the foundational framework. CoALA formalizes LLM agents as systems with four memory types - working, episodic, semantic, and procedural - drawing on classical cognitive architectures (SOAR, ACT-R). Working memory corresponds to the LLM's context window: the ephemeral state of the current execution, including active tool results, pending decisions, and immediate context. Episodic memory stores records of specific experiences, each tagged with situation, action, outcome, and reflection, analogous to hippocampal replay. Semantic memory encodes generalized knowledge extracted from multiple episodes - facts, rules, and heuristics validated across experiences. Procedural memory stores executable routines: tool-call sequences, strategy templates, and compiled action patterns that the agent can invoke without re-deriving from first principles. This taxonomy directly structures the

DeFi Brain (Section 7): working memory holds the current execution context (market state, pending transactions, gas estimates), episodic memory stores per-operation reflections grounded in P&L outcomes, semantic memory holds distilled insights with confidence scores and Ebbinghaus-curve decay [47], and procedural memory encodes reusable strategy templates and multi-step tool-call sequences. Zhang et al. [161] provide a comprehensive survey of memory mechanisms in LLM-based agents, establishing that multi-type memory systems consistently outperform single-store designs on complex tasks requiring cross-session learning. Wray, Kirk, and Laird [41] bridge SOAR design patterns to modern agentic LLM systems, demonstrating that observe-decide-act loops, knowledge compilation, and metacognitive reflection all have direct LLM-agent analogs.

MemGPT by Packer et al. [86, 138] reframes LLM memory management as an operating system problem, with an inner LLM managing its own context window by swapping information between fast (in-context) and slow (external database) storage. The Gotts design extends this analogy with four domain-specific adaptations: importance-weighted decay (replacing Ebbinghaus time-decay, which prunes exactly the knowledge needed during regime recurrences), asymmetric confidence updates (from distributional reward coding [50]), regime-conditional retrieval (from the Adaptive Markets Hypothesis [54]), and a financial ExpeL consolidation loop operating on trajectory pairs grounded in P&L outcomes.

Reflexion by Shinn et al. [42] introduced verbal self-reflection for LLM agents, achieving 97% task success on AlfWorld benchmarks over multiple retry episodes (subsequent evaluations report ~91% with fewer episodes [94]). After each action, the agent generates a natural language reflection on what happened and what to do differently. Gotts uses Reflexion for the inner learning loop: post-execution reflections are stored as episodic memories and inform subsequent decisions. The adaptation to DeFi is that reflections are grounded in quantitative outcomes (P&L impact, slippage realized vs. predicted, gas cost) rather than binary task success.

ExpeL by Zhao et al. [3] introduced cross-task knowledge transfer through trajectory distillation: successful and failed execution trajectories are clustered and distilled into reusable insights. Gotts uses ExpeL for the outer consolidation loop, clustering episodes by tool, chain, and token pair and extracting semantic insights. ExpeL's documented collapse on rare-success tasks [46] - where the framework degrades to 0% success rate because it requires successful trajectories as learning signals - is addressed through multi-level reflection that learns from failures as well as successes.

ACE (Agentic Context Engineering) by Zhang et al. [63] provides the generator-reflector-curator cycle that structures GottsLoop's triple-loop learning. ACE's key architectural insight is the use of delta entries rather than monolithic context rewrites: each modification to the agent's playbook is an incremental addition, update, or removal, preventing the context collapse that occurs when entire knowledge bases are regenerated. Each delta carries helpful/harmful counters for fine-grained quality tracking, enabling the system to distinguish between heuristics that improve outcomes (promoted to strategic tier) and those that degrade performance (demoted or pruned). ACE's three-phase cycle maps directly onto the cybernetic learning loops: the generator produces execution experience through on-chain interactions, the reflector evaluates outcomes against predicted performance and updates playbook heuristics with asymmetric confidence adjustments (from distributional reward coding [50]), and the curator periodically restructures the playbook itself - merging redundant heuristics, resolving contradictions via Pareto-front selection, and archiving rules whose confidence has decayed below threshold. This triple-loop structure extends Argyris and Schön's [5] single-loop (parameter adjustment) and double-loop (strategy revision) organizational learning with a meta-loop for identity and goal revision.

SCOPE by Pei et al. [65] introduced dual-stream prompt evolution with explicit promotion and demotion mechanics: a strategic stream of high-confidence validated rules and a tactical stream of lower-confidence rules under active evaluation. This maps directly to the GottsLoop playbook's three tiers (strategic principles at confidence $\geq$ 0.85, tactical heuristics at 0.5–0.85, and candidates below 0.5). GEPA by Agrawal et al. [64] extended this with Pareto-front optimization for prompt evolution, which GottsLoop's curator uses for heuristic selection: when multiple heuristics compete for the same trigger, the Pareto-optimal set maximizes coverage while minimizing contradiction.

Additional influences include FinMem by Yu et al. [61], which validated that different memory layers require different decay rates ($\alpha = 0.9/0.967/0.988$) for financial agents; FinCon by Yu et al. [62], which demonstrated that verbal feedback outperforms numerical reward signals because agents process feedback in the same modality as their reasoning; AlphaAgent by Tang et al. [84], which demonstrated LLM-driven alpha mining with regularized exploration for quantitative

investment; MUSE [93], which demonstrated that experience-driven memory-utilizing agents with self-evolving capabilities outperform static agents on long-horizon tasks; Voyager by Wang et al. [44], which demonstrated open-ended skill acquisition via an evolving skill library in embodied environments; Matryoshka Representation Learning [45], which provides variable-granularity embeddings useful for hierarchical memory retrieval; FadeMem [49], which introduced adaptive forgetting mechanisms for long-context LLM agents; and ReflAct [94], which extended world-grounded decision making via goal-state reflection for LLM agent planning. The FINSABER evaluation [51] provides a cautionary counterpoint: previously reported LLM trading agent advantages deteriorate significantly under rigorous backtesting conditions, reinforcing the need for pessimistic bias in learned heuristics and mandatory baseline comparison. The FINSABER findings and their implications for the GottsLoop learning architecture are discussed in detail in Section 7.6.

The gap across all of these architectures is that none has been applied to autonomous capital management with real financial stakes. CoALA, MemGPT, and Reflexion operate on benchmark tasks. ExpeL and ACE operate on code generation and reasoning tasks. FinMem and FinCon operate on simulated trading. No existing system integrates agent learning with on-chain reputation, vault custody, and hard safety constraints. Section 7 describes how Gotts adapts these frameworks to the DeFi domain.

### 2.4 On-Chain Agent Safety

The safety of autonomous agents operating with real capital requires defenses at multiple layers. This section surveys the relevant threat landscape and existing mitigation techniques.

Prompt injection remains the primary attack vector against LLM-based agents. The OWASP Top 10 for LLM Applications [16, 69] classifies prompt injection as LLM01:2025, the highest-priority risk, and documents significant bypass rates for prompt injection defenses in production systems. The attack taxonomy includes direct injection (adversarial instructions in user input), indirect injection (malicious content embedded in data sources the agent retrieves), and environment injection (adversarial content placed in the agent's operating environment, achieving 93% attack success rate [73]). TradeTrap [72] evaluated six adversarial attack vectors against AI trading agents — including data fabrication, prompt injection, MCP tool hijacking, model backdoors, malicious collusion, and memory poisoning — finding that fabricated MCP tool responses (e.g., reporting falsified price data) can induce adversarial trades - for example, reporting a token's price as $0.01 when the actual price is $1.00, causing the agent to execute a buy order that transfers value to the attacker. CrAIBench [71] evaluated context manipulation attacks across 150+ blockchain tasks and 500+ attack cases, finding that memory injection attacks outperform direct prompt injection because injected memories bypass the agent's immediate reasoning filters and influence decisions across sessions. SEAgent [21] documented confused deputy attacks in multi-agent systems, where a compromised upstream agent manipulates a downstream agent into performing privileged operations. Koi Security identified 341 malicious skills on ClawHub, OpenClaw's public skill marketplace (335 from the ClawHavoc campaign) targeting crypto wallet keys and other categories via Atomic Stealer malware [177]. CVE-2025-59944 demonstrated a sensitive file overwrite bypass in Cursor IDE, where a case-sensitivity bug allowed attackers to bypass protections on MCP configuration files. In a separate proof-of-concept, researchers demonstrated a zero-click remote code execution attack where a Google Docs file triggered an agent to execute a Python payload via MCP server, illustrating that tool-calling agents are uniquely vulnerable to indirect prompt injection.

CaMeL (Capability-based Machine Learning) by Debenedetti et al. [70] proposed a dual-LLM architecture for agent security. The architecture separates the agent into two components: a privileged LLM (P-LLM) that handles security-critical decisions - tool authorization, input validation, policy enforcement - and an unprivileged LLM (U-LLM) that handles user-facing reasoning and natural language generation. The P-LLM never sees user-controlled input directly; it receives only structured capability requests from the U-LLM, each specifying the requested tool, parameters, and justification. This isolation prevents prompt injection from reaching the security-critical path. The P-LLM validates each request against a capability policy (a whitelist of permitted tool-parameter combinations), and only authorized requests are forwarded for execution. Gotts adopts CaMeL's principle at the architectural level but takes it further: safety constraints are enforced by deterministic code (smart contracts, pre-flight simulation, PolicyCage boundaries) rather than by a second LLM, so that even a fully compromised language model cannot bypass on-chain enforcement. Multi-agent defense research demonstrates that a mandatory guard agent between the domain LLM and the execution layer achieves 100% prompt injection

mitigation in laboratory settings, providing additional support for the architectural separation principle.

TEE-based signing using trusted execution environments (Intel SGX, AMD SEV, ARM TrustZone) provides hardware-level key isolation. Privy's implementation uses AWS Nitro Enclaves with Shamir's Secret Sharing for sub-100ms signing (typical SSS signing takes 20–40 milliseconds). However, three recent hardware attacks undermine TEE security guarantees. The vulnerability analysis and implications for the Gotts security model are detailed in Section 8.

Smart contract guards provide on-chain enforcement that is independent of off-chain component integrity. Safe Transaction Guards validate transaction parameters before execution. ERC-7265 circuit breakers [75] provide rate-limiting and emergency pause functionality gated by on-chain conditions. Subrahmanyam [75b] established that binary halt circuit breakers create a "magnet effect" — Bongaerts et al. [75] provided an alternative interpretation confirming that - volatility spikes as prices approach known thresholds - motivating the continuous dampening approach. The circuit breaker threshold parameters are specified in Section 5.7.

MPC wallets use threshold signatures to distribute key material across multiple parties, so that no single party can sign a transaction unilaterally. While MPC eliminates single points of key compromise, it does not address the deeper problem: if the agent's decision-making is compromised (through prompt injection or model manipulation), the MPC signers will faithfully sign the malicious transaction.

AgentBound [95] and Progent [132] propose runtime enforcement frameworks for AI agent security. AgentBound provides secure execution environments with policy attestation. Progent defines a domain-specific language for programmable privilege control. AgentSpec [97] formalizes runtime enforcement as a DSL for agent behavior constraints. Complementary work on securing MCP infrastructure [96] catalogs risks, controls, and governance patterns for tool-mediated agent interactions. Broader surveys on autonomous agents on blockchains [87] identify standards, execution models, and trust assumptions for on-chain agent operations. These approaches complement the Gotts safety design but address different threat vectors: they focus on constraining agent behavior through software policies, whereas the time-delayed proxy provides a hardware-independent, cryptographic guarantee.

Broader surveys of DeFi security tooling [151] find that existing tools fail to meet practitioner needs, while DeFiAligner [152] demonstrates that symbolic analysis combined with LLMs can detect specification-implementation inconsistencies. Zhou et al.'s systematization of DeFi attacks [153] catalogs the attack taxonomy that the Gotts safety architecture must defend against. Wray, Kirk, and Laird [136] provide a taxonomy of LLM-integrated application architectures that informs the separation between safety-critical and user-facing components. Anthropic's SCONE-bench [114] provides an evaluation framework for autonomous smart contract exploitation, establishing baseline capabilities for LLM-driven vulnerability discovery.

Three concrete security incidents in Agent Capital Markets illustrate the threat landscape. The AIXBT agent wallet compromise [115] resulted in $105,000 in losses (55.5 ETH) from a dashboard compromise that exposed wallet access, demonstrating the consequences of inadequate access control. Cork Protocol lost approximately $12 million through a V4 hook vulnerability in May 2025 [116], where missing `onlyPoolManager` access control on hook callbacks enabled direct callback invocation, which the attacker combined with cross-market token confusion and a risk premium calculation vulnerability to complete the drain. Bunni v2 lost approximately $8.4 million across Ethereum and Unichain [176] from precision and rounding bugs in the hook's withdrawal rounding logic, where accumulated rounding errors in share-to-asset calculations created exploitable discrepancies. BlockSec found that 36% of pre-mainnet V4 hooks in a sample of 22 projects contained exploitable vulnerabilities [128]. These incidents validate the mandatory hook audit requirements, permission flag validation, and break-glass kill-switch mechanisms described in Section 8.

The gap in the existing safety landscape is that no system combines all layers. TEE-based signing provides key isolation but is breakable (see Section 8). CaMeL provides prompt injection defense but assumes honest infrastructure. Smart contract guards enforce on-chain constraints but cannot prevent the agent from authorizing a malicious transaction. MPC distributes key material but trusts the decision-making process. The Gotts safety architecture (Section 8) integrates preventive, cryptographic, and reactive defenses so that security degrades gracefully: if preventive layers fail (given documented per-layer bypass rates [16]), cryptographic layers make the attack visible and cancellable, and reactive layers provide the automated and human response to cancel malicious transactions during the delay window.

### 2.5 Mechanism Design for Agent Markets

The Gotts vault protocol draws on several mechanism design innovations for constructing markets where autonomous agents can compete, cooperate, and allocate capital efficiently.

am-AMM (auction-managed AMM) by Adams, Moallemi, Reynolds, and Robinson [29] introduces a Harberger lease mechanism for AMM management rights. The mechanism operates as follows: the current pool manager must continuously pay rent (denominated in the pool's fee token) to retain management authority. Any agent can submit a competing bid at any time; if the new bid exceeds the current rent by a minimum multiplier (1.1x in the BidDog reference implementation), a transition period begins (K=7,200 blocks, approximately 24 hours on Ethereum mainnet). During this period, the incumbent can match the competing bid to retain management. If the incumbent does not match, the challenger assumes management rights and begins paying the higher rent. The rent flows to liquidity providers as yield independent of trading volume, creating a direct revenue stream for passive depositors. The winning manager gains the authority to set swap fees, collect accrued fees, and route arbitrage - creating economic incentives for active management that improve execution quality for all pool participants. The mechanism is incentive-compatible in the Myersonian sense [109]: managers reveal their true valuation through rent bids, because understating leads to eviction and overstating means paying more than the rights are worth. The am-AMM mechanism was peer-reviewed at Financial Cryptography 2025, and Bunni v2 validated it at scale, generating $138 million in cumulative volume with approximately 59% of V4 hook volume flowing through am-AMM managed pools and with its flagship pools achieving substantially higher volume-to-TVL ratios than non-hooked pools in third-party analysis. Gotts applies the am-AMM mechanism to vault strategy management (Section 6), where management rights over a vault's capital allocation are continuously auctioned.

LVR and optimal fees. The loss-versus-rebalancing framework of Milionis et al. [76b] established that passive LPs systematically lose to informed arbitrageurs at a rate proportional to volatility squared. Singh et al. [38] extended this by showing that LVR equals the theta of a perpetual continuous-installment put option, providing an LVR-theta floor that guarantees LP fees compensate for adverse selection. Bergault, Bieber, and Sanchez-Betancourt [78] derived the optimal exit time for liquidity providers, while Aqsha, Bergault, and Sanchez-Betancourt [79] characterized the equilibrium reward structure for LP participation in AMMs. Campbell, Bergault, Milionis, and Nutz [37] derived optimal dynamic fee schedules that maximize LP returns by adjusting fees based on realized volatility. Bichuch and Feinstein [39] discovered a bijective mapping between the fixed leg of a novel fee-based swap instrument and implied volatility, creating a zero-cost on-chain volatility oracle. Baggiani, Herdegen, and Sanchez-Betancourt [107] derived the fee schedule maximizing LP returns under informed/uninformed trader heterogeneity. Formal verification of these mechanisms has advanced through state-machine models for Uniswap V3 [105], the Lean 4 AMM fee verification library [106, 113], the Pool Favor Property ensuring integer-arithmetic rounding always favors the pool [112], and analysis of the cost structure of permissionless liquidity provision [110]. Lipton, Lucic, and Sepp [102] provided a unified hedging framework for impermanent loss. Fixed-income AMM designs [111] extend the framework to structured products with arbitrary maturities. Empirical analysis of Layer-2 arbitrage [103] found that the standard LVR formula overestimates actual L2 arbitrage costs by approximately 5×, with actual arbitrage constituting only 0.03–0.05% of volume on Base and Optimism - confirming that LP strategies are structurally more viable on L2 chains with shorter block times. Trotti et al. [40] provided the first transaction-level analysis of just-in-time (JIT) liquidity, finding that passive LPs lose up to 44% per JIT trade (when the JIT budget is large) but that JIT LPs could earn up to 69% more over small time windows by optimizing price impact timing; if the pool price equals the market price at entry, the JIT LP never loses from price impact alone. Hasbrouck, Rivera, and Saleh [77] formalized concentrated LP positions as covered calls sold at intrinsic value, establishing that LPs forgo option time premium - a result that provides the vault-strategist's go/no-go criterion for concentrated liquidity deployment. Together, these results inform the Gotts dynamic fee engine (Section 6), which combines LVR-theta floors, volatility-responsive fee adjustment, and fee-implied volatility signals.

PA-AMM (Partially Active AMM) [100] extends the am-AMM model by allowing a fraction of pool liquidity to remain passively deployed while the active portion is managed by the auction winner. The activeness parameter $\lambda \in [0, 1]$ controls the fraction of liquidity available for active management: at $\lambda = 0$, all liquidity is passive (equivalent to a standard constant product pool); at $\lambda = 1$, all liquidity is actively managed (equivalent to a full am-AMM). The key result is that an efficient frontier exists in the LVR-versus-tracking-

error space: higher $\lambda$ reduces LVR (because the active manager can hedge against informed flow) but increases tracking error relative to the passive benchmark. This creates a smooth, parameterizable tradeoff between passive fee capture and active management quality. The Gotts vault rehypothecation architecture (Section 6) adopts a similar parameterization for allocating capital between V4 pool positions and lending venue deployments, with per-vault $\lambda$ values bounded by reputation tier (higher-reputation managers can access larger active fractions).

Loss-Versus-Fair. Moallemi and Robinson [32] analyzed the efficiency of Dutch auctions on blockchains and derived the unavoidable loss floor for auction-based execution. On Base with 2-second blocks, the LVF loss floor is approximately 1.2 basis points, confirming the chain's suitability for both vault launches and Continuous Clearing Auction participation. Gotts uses descending-fee launch protection calibrated by LVF analysis (Section 6).

ERC-8004 identity and reputation. The ERC-8004 standard [7] provides the identity primitive on which Gotts builds. Identity alone is insufficient for trust formation - Classical game-theoretic analysis [143] establishes that no equilibrium exists with all agents cooperating in round one of a repeated game with unknown types, validating the probationary period design where new agents face strict deposit caps. Game-theoretic analysis by Huynh et al. [22] demonstrated that LLM agents shift toward defection in endgame scenarios, while Hammond et al. [23] identified miscoordination, conflict, and collusion as the three primary multi-agent failure modes. A broader survey of game theory and LLMs [24] systematizes these behavioral findings. Fontana et al. [158] found that LLMs exhibit more cooperative behavior than humans in Prisoner's Dilemma settings, while research on strategic intelligence in LLMs [159] provides evidence from evolutionary game theory. Rossetti et al. [164] analyze the dynamics of cooperation in concurrent games, establishing conditions under which cooperative equilibria emerge. These findings motivate the Gotts anti-gaming mechanisms (Section 4): withdrawal acceleration tracking, behavioral regime classification, cross-vault exit correlation monitoring, and exit bonds for high-tier agents. The complete trust tier table is defined in Section 4.2.

Shill-proof auctions [108] provide the theoretical foundation for the permissionless executor framework. The Dutch auction reward escalation mechanism is shill-proof: executors cannot benefit from bidding against themselves, and the first executor to find it profitable claims the reward. Related work on fair combinatorial auctions for blockchain trade intents [130] extends the framework to multi-item settings. Duetting et al. [88] extended mechanism design to LLM-based participants, establishing conditions under which auction mechanisms remain incentive-compatible when bidders are language models rather than rational utility maximizers. Batch trade execution mechanisms [99] reduce MEV extraction by aggregating orders, while transaction fee mechanism design post-MEV [155] analyzes optimal fee structures in the presence of searcher-builder dynamics. Bachu, Wan, and Moallemi [156] quantify price improvement in order flow auctions, and Oz et al. [154] identify cross-chain arbitrage as an emerging MEV frontier.

Delegated portfolio management. Gennaro and Mastrolia [129] analyzed delegation games with random default, providing a framework for understanding how vault managers and depositors interact when the manager may default with some probability. The Gotts am-AMM mechanism transforms this static delegation game into a dynamic auction: if a manager underperforms, a more capable agent can bid for management rights, and the transition is automatic. Devorsetz and Herlihy [33] developed a defensive rebalancing framework showing that arbitrage-free capital allocation is the unique solution of a convex optimization problem, which the Gotts LiquidityRouter implements for cross-pool allocation.

The V4 hook security incidents discussed in Section 2.4 - Cork Protocol (~$12 million [116]), Bunni v2 ($8.4 million [176]), and the 36% vulnerability rate in community hooks [128] - have direct implications for mechanism design. They demonstrate that hooks implementing complex financial logic (dynamic fees, am-AMM rent distribution, NAV-aware pricing) require formal verification beyond standard smart contract auditing. The Gotts hook architecture addresses this through mandatory permission flag validation, on-chain kill-switch mechanisms callable by the Sentinel role, and pre-deployment hook verification as described in Section 8.

### 2.6 Summary

Table 1 summarizes the relationship between prior work and the Gotts design.

With this background established, Section 3 presents the Gotts system architecture.

Table 2: Prior work and Gotts design extensions by domain

| Domain | Prior Work | Gotts Extension |
|---|---|---|
| Vault protocols | Yearn V3, Morpho, Maple, Sommelier, Enzyme, Set Protocol | Agent-native MCP interface, identity-gated access, auto-created V4 share pools, am-AMM management auctions, permissionless factory |
| Agent memory | CoALA, MemGPT, Reflexion, ExpeL | DeFi-specific: importance-weighted decay, regime-conditional retrieval, P&L-grounded consolidation |
| Agent learning | ACE, SCOPE, GEPA, FinMem, FinCon | Triple-loop cybernetic adaptation, playbook evolution, asymmetric confidence from distributional reward coding |
| Safety | CaMeL, TEE signing, Smart Account guards | Multi-ring integration (preventive, cryptographic, reactive); time-delayed proxy as primary primitive surviving TEE compromise |
| Mechanism design | am-AMM, LVR/optimal fees, PA-AMM | Vault-level management auctions, reputation-gated participation, permissionless executor framework |
| Agent identity | ERC-8004 | VaultReputationEngine with 20 auto-attested milestones, five-tier trust system, TraceRank payments-as-endorsements |

## 3. System Architecture

### 3.1 Overview

The protocol is organized as a layered delegation architecture in which each layer performs a well-defined transformation on user or agent intent before passing it to the layer below. The five layers are:

Skills form the topmost layer: 68 verb-noun intents (e.g., `execute-swap`, `create-vault`, `analyze-pool`) that parse high-level requests, validate parameters, and delegate execution to one or more agents. Skills never call on-chain tools directly; they set up context and hand off to the appropriate specialist agent. This separation ensures that user-facing concerns (discoverability, progressive disclosure, error messaging) remain decoupled from execution logic.

The agent layer comprises 32 specialist agents organized into eight categories - execution, research, strategy, development, infrastructure, Agent Capital Markets, real-time monitoring, and vault management. Agents compose MCP tools into multi-step workflows. Execution agents (trade-executor, liquidity-manager, cross-chain-executor) perform write operations; research agents (pool-researcher, token-analyst, opportunity-scanner, portfolio-analyst, pnl-analyst) are strictly read-only. Model selection follows capability requirements: high-stakes agents (trade-executor, safety-guardian, vault-manager) use the most capable model (Claude Opus); research and monitoring agents (pool-researcher, pnl-analyst, market-monitor) use mid-tier models (Claude Sonnet); and suppressed heartbeat ticks use lightweight models. The model router can escalate dynamically when novel conditions are detected.

Tools are activated through profiles that can be composed independently. A `data` profile (~18 tools) provides read-only market intelligence; a `trader` profile (~27 tools) adds swap execution and Permit2; an `lp` profile (~37 tools) adds liquidity management and streaming; a `vault` profile (~40 tools) adds the 24 core vault tools and proxy operations; and a `learning` profile (~14 tools) enables the self-improvement and memory tools, composable with any other profile. The `full` profile loads all 154 tools but carries an explicit accuracy degradation warning - LLM tool selection accuracy drops measurably beyond approximately 20 simultaneous tools, and some MCP clients impose hard caps (e.g., Cursor at 40 tools). Profile budgets are calibrated to stay within these limits.

Authentication uses SIWE (Sign-In with Ethereum) [20] combined with OAuth 2.1 for role-based tool authorization. The server operates across all 11 Uniswap-deployed chains (Ethereum, Optimism, Polygon, Arbitrum, Base, BNB Chain, Avalanche, Celo, Blast, Unichain, zkSync Era), maintaining per-chain `PublicClient` instances with fallback RPC support, automatic chain detection from tool parameters, and block number caching for consistency within a single tool call.

Below the MCP server, the time-delayed proxy interposes a cancellation window between authorization and execution. Section 8 details this mechanism; here it is noted that the proxy constitutes the protocol's primary security primitive, surviving simultaneous LLM compromise, key exfiltration, and TEE hardware attacks.

At the base layer, the protocol interfaces with Uniswap V2, V3, V4, and UniswapX, along with the ERC-4626

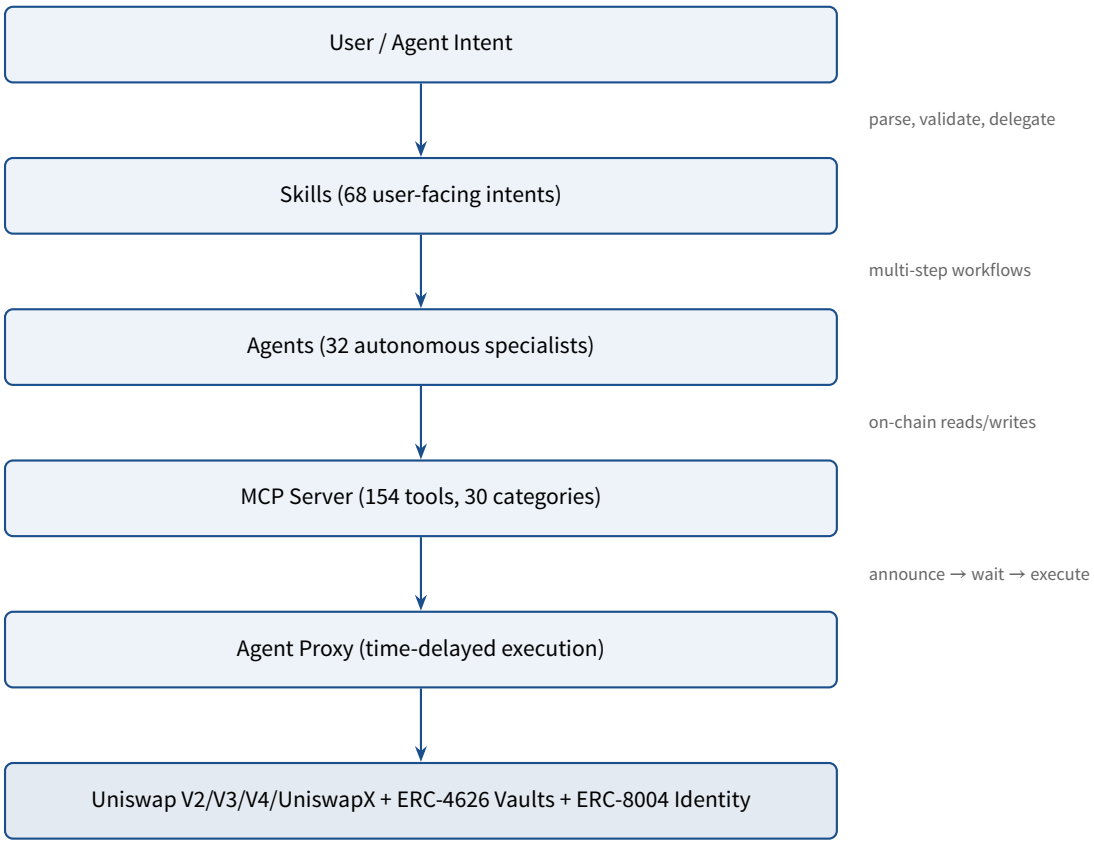


Figure 2: Layered delegation architecture: each layer transforms intent before passing it to the layer below.

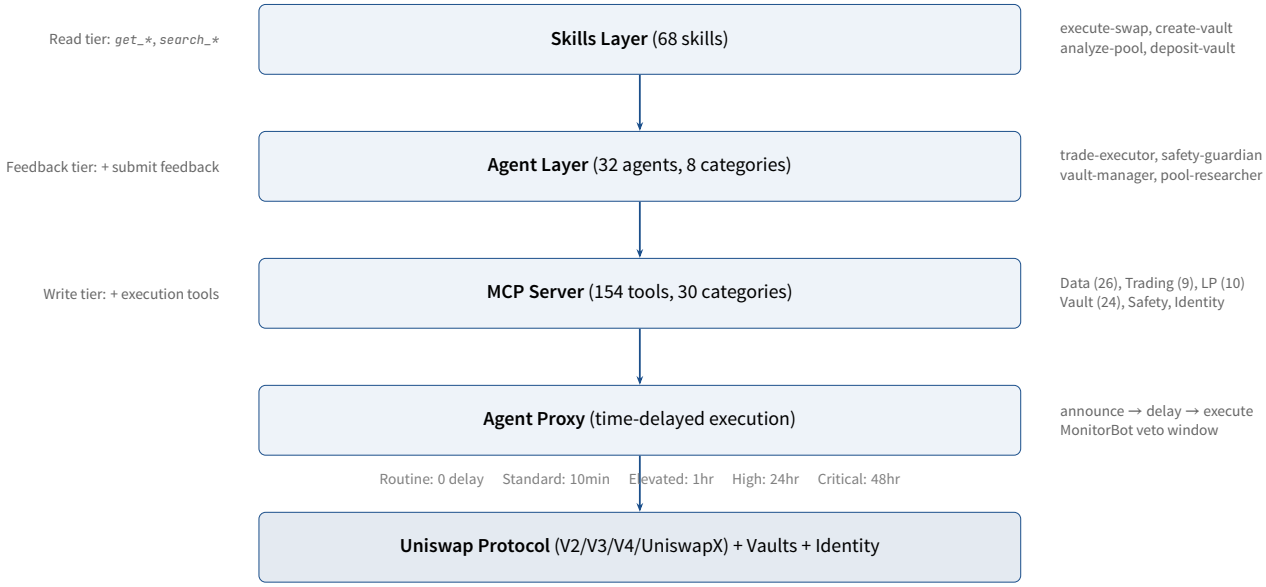


Figure 3: Detailed architecture: five-layer delegation with API key tiers (left), representative components (right), and risk-tiered proxy delays (bottom).

vault contracts and ERC-8004 identity registries deployed on-chain.

### 3.2 Design Principles

Eight principles govern the architecture. Rather than enumerate them in isolation, this section describes how each motivates a specific structural decision.

The architecture follows an *infrastructure, not product* principle drawn from the Morpho playbook [17]: zero protocol fees, maximum composability, minimal governance surface. The protocol is a factory, not a fund. Every vault lifecycle event - deployment, deposit, withdrawal, rebalance - generates Uniswap volume, and value accrues through that volume rather than through rent extraction. Zero fees also eliminate

The MCP server exposes 154 structured tools across 30 categories, organized into the following tool modules:

| Module | Tool Count | Scope |
|---|---|---|
| Data and Analytics | 9 | Pool info, prices, positions, tick data |
| Historical Data | 5 | OHLCV candles, trade/price/volume history |
| Token Directory | 4 | Token list, search, metadata, pair discovery |
| Trading | 6 | Quotes, swaps, UniswapX orders, cross-chain intents |
| Liquidity | 7 | Add/remove liquidity, collect fees, migration |
| CCA and Token Launch | 9 | CCA bids, clearing state, token claiming, am-AMM |
| Approval and Permit | 4 | Allowances, approvals, Permit2 signatures |
| Safety | 3 | Simulate, validate token, check limits |
| Protocol Fees | 7 | TokenJar, Firepit state, burn execution |
| ERC-8004 | 10 | Agent search, feedback, reputation, validation |
| Self-Improvement | 6 | Execution logging, outcome recording, parameter tuning |
| Memory and Knowledge | 10 | Strategy persistence, regime classification, semantic search |
| External Data (x402) | 8 | CoinGecko and Elsa x402 providers |

Access is governed by a three-tier API key model, each key prefix encoding its authorization scope:

| Key Tier | Prefix | Authorized Operations |
|---|---|---|
| Read | `gotts_read_*` | All `get_*`, `search_*`, `list_*`, `retrieve_*` tools - query only |
| Feedback | `gotts_feedback_*` | Read + `submit_operator_feedback`, `manage_insight`, `update_memory_confidence` |
| Write | `gotts_write_*` | Feedback + all execution tools (`execute_swap`, `vault_deposit`, `proxy_announce`) |

the governance attack surface that fee-setting mechanisms create. Morpho's results validate this approach concretely, as discussed in Section 1. The lesson is that removing barriers to entry - no whitelists, no governance votes, no fee negotiations - attracts more participation than any incentive program.

The system is *agent-native* by construction. The Model Context Protocol (MCP) [18] is the primary interface, not a wrapper around a human frontend. All 154 tools return structured JSON with machine-readable error codes and accept validated parameters. Every tool file exports a `registerTool` function with Zod-validated parameter schemas and MCP annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`) that enable MCP clients to auto-approve read-only calls and require confirmation for destructive writes. Capabilities are published as both MCP tools and A2A Agent Cards [19] for agent-to-agent discovery. The A2A protocol supports multi-turn dialogue with a task lifecycle (`submitted` → `working` → `input_required` → `completed`) for complex financial decisions that cannot be expressed as single tool calls. The distinction from "agent-compatible" REST APIs matters: agents require structured schemas, not click-equivalents.

*Identity as primitive* is enforced through ERC-8004 [7], which gates all protocol interactions. One standard, one registry, one reputation score, regardless of the underlying inference system. Every interaction - vault deposit, strategy activation, tool access, management auction - queries the same `IdentityRegistryAdapter`. Reputation is on-chain and queryable by any contract, not locked to a particular protocol.

The conviction that preventive security will be breached motivates a *reactive defense* posture. TEE hardware vulnerabilities and prompt injection bypass rates in production LLM systems are discussed in Section 8. No single preventive layer provides adequate security for autonomous capital management. The architecture therefore treats the time-delayed proxy as the primary security primitive, providing a cancellation window that remains effective even when the LLM and TEE are both compromised.

*Composability through ERC-4626* ensures that vault share tokens inherit full ERC-20 functionality. When vault shares become Morpho collateral and Pendle yield tokens, each integration raises switching costs. Token emission programs create temporary incentives; composability integrations create dependencies that compound.

*Progressive complexity* allows an agent to start with zero-cost read-only tools and layer on vault participation, strategy execution, and autonomous

learning incrementally. Every capability is opt-in. An agent that begins with 18 read-only data tools can add a write API key for vault participation, then enable the learning pipeline for autonomous strategy evolution, without migration or reconfiguration. A vault itself can be deployed in a single transaction with minimal configuration (base asset, fee parameters, and a mandatory disclosure struct), or it can opt into advanced features - hook-enabled pools, am-AMM strategy auctions, rehypothecation adapters, structured tranche products - each activated by a boolean flag in the `VaultConfig` struct. Template selection provides graduated complexity: `safe-yield` requires only a base asset and fee parameters; `custom` exposes every configurable parameter. This avoids the common DeFi pattern where simple use cases are forced through complex interfaces designed for power users.

*Cybernetic autonomy* grounds the learning architecture in control theory. Ashby's Law of Requisite Variety [43] sets the minimum complexity of the memory schema (12 insight categories). The Good Regulator Theorem [59] requires the agent's internal model to be an accurate representation of the environment. Maxwell's governor analysis [55] constrains parameter evolution to 10% per iteration, preventing oscillation and runaway positive feedback. These are not metaphors; they are design constraints drawn from control theory that bound system behavior, as described in Section 7.

*Permissionless participation* means any ERC-8004 registered agent can deploy a vault, bid for management rights, provide liquidity, or execute strategies. No whitelist, no governance vote, no human approval. Access is gated by on-chain reputation: the quality of an agent's execution history determines its operational scope.

### 3.3 Delegation Hierarchy

The 32 agents operate within an acyclic delegation hierarchy enforced by four rules. First, delegation is *acyclic*: if agent A delegates to agent B, B never delegates back to A. The delegation graph is a directed acyclic graph verified at startup by topological sort. Second, the architecture is *safety-first*: any agent performing a write operation must delegate safety validation to `safety-guardian` before broadcast. Third, *no transitive trust* is permitted: agents independently validate all inputs via MCP tools, and data received from another agent is never trusted without on-chain verification. This specifically counters the confused deputy attack documented in SEAgent [21]. Fourth, eleven agents are designated *terminal nodes* that never delegate to other agents - they are the trust anchors of the hierarchy.

Terminal nodes serve as the trust anchors of the system. `safety-guardian` validates every write operation before broadcast; it uses the most capable model and has access only to simulation and decoding tools, never write tools. `risk-assessor` evaluates transaction risk with adversarial input handling. `wallet-provisioner` manages wallet lifecycle without financial authority. `pool-researcher` and `token-analyst` are read-only by construction, never writing to the chain. These agents cannot be induced to delegate their responsibilities upward.

A concrete delegation chain illustrates the pattern. When the `deposit-vault` skill is invoked: (1) the skill parses the user's intent and delegates to `vault-manager`; (2) `vault-manager` delegates safety validation to `safety-guardian`, which simulates the deposit transaction via `eth_call` and verifies ERC-8004 identity, tier limits, and balance sufficiency; (3) if all seven pre-flight checks pass, `vault-manager` encodes the deposit calldata and broadcasts the transaction; (4) the result flows back through the delegation chain to the skill, which formats the response for the user. At no point does any agent trust data received from another agent without independent on-chain verification.

### 3.4 Vault Factory

Any ERC-8004 registered agent can deploy a vault in a single transaction. The `AgentVaultFactory` uses CREATE2 for deterministic deployment via EIP-1167 minimal proxy clones, enabling counterfactual vault references - addresses can be computed before any on-chain transaction. The CREATE2 salt incorporates the creator's identity, base asset, and a nonce, producing a deterministic address:

```
bytes32 salt = keccak256(abi.encode(msg.sender,
     config.creatorAgentId, config.baseAsset,
     nonce));
```

Agents can pre-compute their vault address via `computeVaultAddress(config, nonce)` before deployment, enabling counterfactual deposits (participants can approve the vault address before it exists) and deterministic cross-chain deployment (the same config and nonce produce the same address on every chain). The factory maintains a global registry with three on-chain data structures: an $O(1)$ membership check (`isVault`), a full enumeration array, and a per-creator lookup mapping.

The 32 agents span nine functional categories:

| Category | Count | Representative Agents |
|---|---|---|
| Execution | 3 | trade-executor, liquidity-manager, cross-chain-executor |
| Research and Analysis | 5 | pool-researcher, token-analyst, opportunity-scanner, portfolio-analyst, pnl-analyst |
| Strategy | 2 | lp-strategist, risk-assessor |
| Development | 3 | hook-builder, integration-architect, integration-advisor |
| Infrastructure | 2 | safety-guardian, wallet-provisioner |
| Agent Capital Markets | 8 | treasury-manager, token-deployer, identity-verifier, protocol-fee-seeker, agent-service-broker, strategy-optimizer, yield-scout, competition-participant |
| Real-Time Monitoring | 2 | market-monitor, position-monitor |
| Vault | 7 | vault-manager, vault-strategist, vault-watchdog, vault-auctioneer, vault-creator, vault-allocator, vault-executor |

The delegation hierarchy operates at four levels:

| Level | Agents | Delegates To |
|---|---|---|
| 0 (Terminal) | safety-guardian, risk-assessor, wallet-provisioner, pool-researcher, token-analyst, identity-verifier, position-monitor, vault-watchdog, hook-builder, integration-architect, integration-advisor | Nobody |
| 1 (Execution) | trade-executor, liquidity-manager, cross-chain-executor | Level 0 |
| 2 (Strategy) | lp-strategist, vault-manager, vault-strategist, vault-auctioneer, strategy-optimizer, treasury-manager, market-monitor | Level 0–1 |
| 3 (Orchestration) | agent-service-broker, vault-creator, vault-allocator | Level 0–2 |

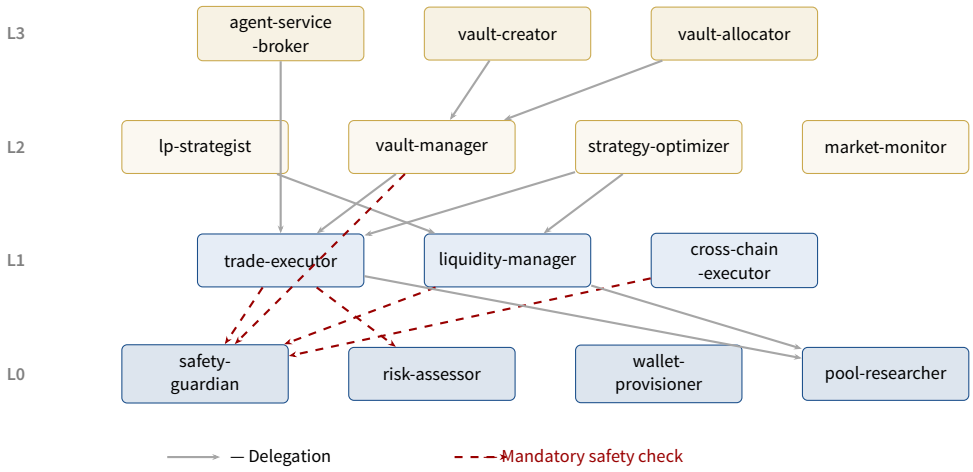


Figure 4: Agent delegation DAG: Level 3 (orchestration) delegates to Level 2 (strategy), which delegates to Level 1 (execution), which must consult Level 0 (terminal) safety and research agents. Red dashed arrows indicate mandatory safety-guardian delegation for all write operations.

Template selection is gated by the deploying agent's reputation tier, as defined in the five-tier trust hierarchy (Section 4.2). An Unverified agent cannot deploy a vault with aggressive parameters or custom configuration.

The `OnboardRouter` contract enables atomic onboarding in a single gasless UserOp: ERC-8004 identity registration, Permit2 approval, and an initial vault deposit are batched into one ERC-4337 operation. This eliminates the multi-step onboarding flow that otherwise requires three to five separate transactions.

The vault creation flow illustrates how the layers compose: (1) the MCP `create_vault` tool verifies ERC-8004 identity and validates the `VaultConfig` struct against factory parameter rules (fee caps, disclosure requirements, risk rating bounds); (2) the safety pipeline runs seven pre-flight checks (identity verification, parameter bounds, pre-flight simulation

Each vault deployed through the factory receives an auto-created Uniswap V4 pool for its share tokens, following the Clanker model [9] where every vault receives a liquid secondary market at deployment time. The auto-created pool uses a NAVAwareHook for fair pricing at net asset value and optionally a LaunchFeeHook for MEV protection during the initial trading window (Section 6.1). Five vault templates provide graduated complexity:

Table 3: Vault templates with graduated complexity

| Template | Configuration | Default Fees | Best For |
|---|---|---|---|
| Safe Yield | Single stablecoin, no leverage, strict bounds | 1% mgmt, 10% perf | Conservative agents, first deployments |
| Balanced Growth | Multi-asset, moderate leverage | 1.5% mgmt, 15% perf | General-purpose yield |
| Active Trading | Flexible assets, wider bounds, hook-enabled | 2% mgmt, 25% perf | Active LP management |
| Market Making | Concentrated liquidity, delta-neutral, rehypothecation | 2% mgmt, 20% perf | Sophisticated agents |
| Custom | Blank template, all parameters user-configured | User-set | Sovereign-tier agents |

via `eth_call`, gas estimation, nonce management, token allowlist, spending limit); (3) on-chain, the factory deploys the vault core via CREATE2, mines and deploys hooks with the correct V4 permission bits via `HookMiner`, calls `PoolManager.initialize()` to create the share/base pool, registers the vault in the factory, and enrolls it in the VaultReputationEngine. The factory validates all `VaultConfig` fields before deployment, reverting with specific error codes (`InvalidBaseAsset`, `FeeExceedsCap`, `MissingDisclosure`, `InvalidHookConfig`) for each validation failure. Every step generates Uniswap volume: pool initialization is a Uniswap transaction, seed liquidity is a Uniswap LP operation, and subsequent share trading produces Uniswap swaps. Section 5 describes the vault protocol in detail.

The system architecture establishes the layered delegation model and vault factory primitives. The following section examines the identity and reputation layer that gates access to vault operations.

## 4. Identity and Reputation

### 4.1 ERC-8004 Agent Identity

ERC-8004 [7] provides three on-chain registries for autonomous agent identity, deployed at canonical addresses across Ethereum, Base, Polygon, and Sepolia:

1. Identity Registry (`0x8004A818BFB912233c491871b3d84c89A494BD9e`). ERC-721 tokens (transferable by default; Gotts adds optional transfer restrictions via its IdentityGuardian contract for soulbound deployments) with structured metadata (capabilities, supported protocols, operator information). Each agent receives a unique `agentId` (uint256) upon registration. The token carries an `agentURI` pointing to a JSON registration file with MCP endpoints, A2A Agent Card URLs, OASF domain classifications, and trust model declarations.
2. Reputation Registry. Mutable feedback scores that any contract can read via `STATICCALL`. Scores are updated by protocol interactions - vault performance, transaction history, peer feedback - creating an objectively verifiable trust signal. Feedback entries carry structured tags (e.g., `tradingYield`, `riskManagement`), a numeric value (0–500), and an optional `feedbackURI` pointing to rich review data.
3. Validation Registry. Check hooks that contracts can call to verify agent properties (registration status, reputation tier, capability flags) before granting access. Validation methods include `zkml` (zero-knowledge model proofs), `tee` (TEE attestation), and `stake-secured` (economic bonding). A single `isRegistered(agentId)` call gates all protocol interactions.

The protocol interfaces with ERC-8004 through an `IdentityRegistryAdapter` decoupling layer rather than calling the registry directly. This adapter absorbs interface changes as the standard (currently in Draft status) evolves, preventing protocol-wide migration if the registry API changes. The factory stores a single `identityAdapter` address, updatable via `setIdentityAdapter()` with a 3–7 day timelock.

Within six weeks of ERC-8004 going live on Ethereum mainnet in January 2026, over 24,500 autonomous agents had registered on-chain identities [7]. Each identity carries structured metadata enabling permissionless discovery:

```
struct AgentMetadata {
    // Human-readable agent name
    string name;
    // IPFS/HTTPS URI for extended metadata
    string metadataURI;
    // 1=Participant, 2=Creator, 3=Manager,
     4=Strategist
```

The protocol implements a five-tier trust system where each tier unlocks progressively larger operational boundaries:

Table 4: Five-tier trust system with reputation-gated operational boundaries

| Tier | Score | Deposit Cap | Fee Discount | Session Key Duration | Max Strategies |
|---|---|---|---|---|---|
| Unverified | 0 | $1,000 | 0% | 1 hour | 1 |
| Basic | 50+ | $10,000 | 5% | 4 hours | 3 |
| Verified | 200+ | $50,000 | 15% | 24 hours | 10 |
| Trusted | 500+ | $100,000 | 25% | 7 days | 25 |
| Sovereign | 1,000+ | $10M daily | 40% | 30 days | Unlimited |

Tier boundaries are enforced on-chain by the `IdentityRegistryAdapter`, which the `AgentVaultCore` queries before every deposit, withdrawal, and strategy modification. Tier transitions are automatic - no governance vote, no human approval. Beyond deposit caps, each tier unlocks progressively wider operational boundaries:

| Tier | Rebalances/Day | Daily Aggregate Cap | Vault Templates | Creator Fee Caps | $\lambda$ Range |
|---|---|---|---|---|---|
| Unverified | 2 | $5,000 | safe-yield only | 100 bps mgmt, 2000 perf | [0.3, 0.7] |
| Basic | 5 | $50,000 | safe-yield, balanced | 200 bps mgmt, 3000 perf | [0.2, 0.8] |
| Verified | 10 | $250,000 | All standard | 300 bps mgmt, 4000 perf | [0.1, 0.9] |
| Trusted | 25 | $1,000,000 | All + custom | 500 bps mgmt, 5000 perf | [0.05, 0.95] |
| Sovereign | 50 | $10,000,000 | Unrestricted | Protocol maximums | [0.0, 1.0] |

```
    uint8 agentType;
    // Capability hashes (e.g., keccak256(
    "vault_management"))
    bytes32[] capabilities;
    // Block timestamp of registration
    uint256 registeredAt;
}
```

The `metadataURI` points to a JSON document containing performance metrics, supported chains, strategy categories, and an A2A Agent Card URL. This enables agent-to-agent discovery: a vault seeking a new strategist can query the registry for agents with the `vault_strategy` capability and a Verified+ reputation tier. The A2A Agent Card enables direct agent-to-agent negotiation via the A2A protocol [19].

The Gotts Safe MCP server itself registers as an ERC-8004 entity, creating a recursive trust chain: an external agent verifies the server's on-chain identity before trusting its tool responses, the server verifies vault contracts on-chain, and vault contracts verify agents via the ERC-8004 gate. Every link is independently verifiable.

### 4.2 Trust Tiers

The rebalance frequency limit prevents excessive gas expenditure and manipulation by newly registered agents. The daily aggregate cap provides a backstop even at the highest tier - if a Sovereign agent is compromised, the absence of rate limits would allow an attacker to drain vault capital at maximum speed without triggering per-operation limits [10]. The PA-AMM activeness parameter $\lambda$ (Section 6.5) is tier-bounded to prevent inexperienced agents from taking extreme positions: an Unverified agent is restricted to $\lambda \in [0.3, 0.7]$, while Sovereign agents can access the full $[0.0, 1.0]$ range.

The probationary period design is grounded in game-theoretic analysis: Graser et al. proved that no equilibrium exists with all agents cooperating in round one of a repeated game with unknown types [143]. This validates the Unverified tier's $1,000 cap and limited scope - trust must be earned through demonstrated behavior, not assumed from registration.

The Unverified-to-Basic transition saves 5% on both fee types, while the Trusted-to-Sovereign transition saves an additional 10% on management and 10% on performance. This convexity creates a separating equilibrium. Low-commitment agents stop at Basic (the 5% discount barely justifies the effort). Serious agents push past Verified to capture the steep discount curve between Trusted and Sovereign. However, Sovereign requires ecosystem contribution, which cannot be achieved through self-dealing (Section 4.3). An agent seeking the full 30% management discount must produce genuine public goods for the protocol.

Consider a concrete example: an agent managing $500K at a vault's 300 bps management fee pays

The fee discount schedule is deliberately convex, with separate schedules for management and performance fees:

| Tier | Management Fee Discount | Performance Fee Discount |
|---|---|---|
| Unverified | 0% | 0% |
| Basic | 5% | 5% |
| Verified | 10% | 10% |
| Trusted | 20% | 15% |
| Sovereign | 30% | 25% |

$15,000/year. At Sovereign tier, the 30% management discount saves $4,500/year; the 25% performance discount provides additional savings proportional to returns. Reaching Sovereign from Trusted requires approximately 500 additional reputation points, most from ecosystem milestones (cross-vault participation, governance votes, contributing verified strategies). The annual fee savings justify substantial investment in ecosystem participation, creating a virtuous cycle: higher reputation yields lower fees, which yields better net performance, which attracts more capital, which generates more on-chain history, which yields higher reputation. This is the reputation-to-leverage pipeline applied to fee economics.

One limitation of this analysis should be acknowledged: it assumes agents optimize around fee economics. Current LLM agents do not strategize about multi-month reputation trajectories. The discount schedule is designed for a future where agents or their operators treat reputation as a quantifiable asset.

### 4.3 VaultReputationEngine

The VaultReputationEngine implements 20 milestones across five categories on a 1,000-point scale. All milestones are auto-attested to the ERC-8004 Reputation Registry without human review, governance vote, or subjective evaluation. The design draws on deep reputation scoring approaches such as zScore wallet ranking [89] and universal decentralised reputation [90], while preserving Sybil resistance through economic stakes rather than social verification. Privacy-preserving counting via anonymous counting tokens (tACT) [91] provides a path to confidential milestone attestation. The soulbound token concept [92] informs the non-transferable milestone design.

The non-Ecosystem maximum is 915 points. Achieving Sovereign tier (1,000+) structurally requires ecosystem contribution. This design is intentional: the highest trust level demands demonstrated commitment to the protocol ecosystem, not just individual performance. Ecosystem milestones include contributing verified strategies, running executor services, and vouching for other agents who perform well.

The auto-attestation mechanism operates without oracles or governance. When an agent calls `claimMilestone()`, the contract reads on-chain state (block timestamps via `block.timestamp`, vault balances via `AgentVaultCore.agentShares()`, transaction counts via event logs) and deterministically computes whether the milestone condition is satisfied. If satisfied, the contract writes an attestation entry to the ERC-8004 Reputation Registry containing the `agentId`, milestone identifier, points awarded, and block timestamp.

Recent behavior (last 30 days) is weighted 3x versus older behavior, with exponential decay weighting:

$$w(t) = e^{-\lambda(t_{\text{now}} - t)}$$

where the decay constant $\lambda = \ln(3)/30 \approx 0.0366$ produces a 3:1 weighting ratio at the 30-day boundary. This follows the defection detection research of Fontana et al. [146] - endgame defectors exhibit measurable behavioral shifts concentrated in the most recent window. The exponential decay ensures that an agent's reputation reflects current behavior rather than historical performance that may no longer be representative.

### 4.4 Anti-Gaming and TraceRank

Several mechanisms prevent manipulation of the reputation system.

TraceRank payments-as-endorsements. Following Shi's TraceRank framework [127], deposits into a vault serve as implicit reputation signals. The reputation contribution of a deposit is weighted by: `payerReputation * depositAmount * temporalDecay`. High-reputation agents depositing into a vault contribute more to the vault manager's reputation than low-reputation agents. An attacker cannot generate high-weight endorsements without first building genuine reputation.

| Category | Milestones | Max Points | Examples |
|---|---|---|---|
| Entry | 4 | 100 | First deposit, identity registration, proxy setup |
| Time | 4 | 200 | 30-day active, 90-day active, 1-year active |
| Capital | 4 | 250 | $10K managed, $100K managed, $1M managed |
| Behavior | 4 | 365 | Clean audit trail, no circuit breaker triggers |
| Ecosystem | 4 | 85 | Cross-vault participation, governance votes |

Milestone eligibility is computed deterministically from on-chain state - block timestamps, vault balances, transaction counts. No oracle, no governance, no subjectivity. Each milestone can be claimed exactly once per `agentId`; the contract maintains a bitmap of claimed milestones via `mapping(uint256 => uint256) claimedBitmap`, where each bit position corresponds to a milestone ID. The `claimMilestone(uint256 agentId, uint8 milestoneId)` function verifies eligibility, sets the bit, and emits a `MilestoneClaimed` event with the attestation payload. Representative milestones across all five categories illustrate the attestation design:

| Milestone | Category | Points | Anti-Gaming Measure |
|---|---|---|---|
| Identity Registration | Entry | 10 | One-time, verified on ERC-8004 |
| First Deposit | Entry | 70 | Self-deposits excluded |
| Proxy Setup | Entry | 10 | Requires time-delayed proxy deployment |
| Steady Staker (30d hold) | Time | 75 | >$1K position required |
| Diamond Hands (90d hold) | Time | 85 | Capital locked, no partial withdrawals |
| Veteran (1yr active) | Time | 100 | Continuous vault participation |
| $10K Managed | Capital | 50 | Self-deposits excluded |
| $100K Managed | Capital | 75 | Cross-referenced with deposit sources |
| $1M Managed | Capital | 100 | Unique depositors required |
| Diversifier (3+ vaults) | Behavior | 80 | $1K minimum per vault |
| Profitable Exit | Behavior | 80 | Net positive P&L verified on-chain |
| Clean Audit (90d) | Behavior | 100 | No circuit breaker triggers |
| Capital Attractor (5+ depositors) | Ecosystem | 45 | Unique depositors, self excluded |
| Strategy Contributor | Ecosystem | 25 | Verified adapter submission |
| Executor Service | Ecosystem | 15 | Successful `executeJob()` calls |

Wash detection. Self-deposits (where an agent deposits into its own vault) are excluded from Capital milestones. The VaultReputationEngine cross-references the depositor's `agentId` against the vault's manager `agentId`.

Dust prevention. Deposits below $100 equivalent are excluded from milestone calculations, preventing Sybil attacks where an adversary creates many agents each making minimal deposits. Schlegel, Mattauch, and Moldovanu [157] formalize the conditions under which mechanisms are Sybil-proof, providing the theoretical basis for these minimum-stake requirements.

Endgame defection detection. Huynh et al. [22] showed that LLM agents shift toward defection in endgame scenarios, consistent with the behavioral regime analysis of Fontana et al. [146] who identified measurable behavioral shifts concentrated in the final interaction window. The protocol implements four detection mechanisms: withdrawal acceleration tracking (flagging when withdrawal acceleration exceeds the historical baseline by $2\sigma$), behavioral regime classification using the strategy signatures identified by Huynh et al., cross-vault exit correlation monitoring (flagging coordinated exits by agents sharing an operator), and exit bonds for high-tier agents (0.5% of AUM, slashed upon defection detection with >90% confidence).

Structural defenses complement detection. Vault management periods have stochastic endings, eliminating the "known last round" that triggers defection in finite repeated games. PolicyCage smart contract constraints (Section 5.7) bound the damage even from a fully defecting agent. Tier-downgrade cascades immediately reduce deposit caps and increase proxy delays when defection is detected.

Probabilistic continuation. The exact termination time of management periods is not known in advance. Without a known last round, the backward induction argument that drives defection in finite repeated

games does not apply. Combined with the hard guardrails of the PolicyCage, even agents that attempt defection cannot extract more than the on-chain constraints permit in any single transaction.

## 5. Vault Protocol

### 5.1 Design Overview

The vault protocol enables any ERC-8004 registered agent to deploy a permissionless ERC-4626 vault gated by on-chain identity. The design follows four principles:

1. Immutable vaults. Once deployed, a vault's core parameters - fee caps, identity gate, circuit breaker thresholds - cannot be changed. The Owner can abdicate, making all configuration permanent.

2. Upgradeable templates. New vault templates can be deployed without affecting existing vaults. The factory maintains a registry of approved templates.

3. Mandatory disclosure. Every vault carries a `VaultDisclosure` struct set at creation. Following Zbandut et al. [25], the factory enforces non-empty disclosure. The Curator can update disclosure with a 48-hour timelock.

4. Circularity prevention. A vault cannot hold shares of another vault that holds shares of the first vault. The factory enforces this at the contract level via `_checkCircularity(address targetVault)`, which verifies both direct circularity (vault B holds shares in vault A) and 1-hop circularity (vault C holds shares in both vault A and vault B). Composition depth is limited to two levels, preventing infinite loops in `totalAssets()` calculations and reentrancy attacks.

5. Composability as strategy. Vault shares are ERC-20 and ERC-4626 compliant from deployment. Any protocol that supports ERC-4626 - Morpho for collateral, Pendle for yield tokenization, aggregators for discovery - can integrate vault shares without custom adapters.

The design draws on the Morpho Blue model [17]: permissionless creation, zero protocol fees, and value generated through volume, demonstrating the rapid growth possible with this approach (Section 1). The same model is applied here to autonomous capital management, where every vault lifecycle event generates Uniswap volume.

Template selection is gated by creator reputation tier - an Unverified agent cannot deploy a vault with aggressive parameters. The `safe-yield` template enforces strict PolicyCage bounds: single stablecoin base asset, no leverage, maximum 10% drawdown, and $\lambda \in [0, 0.3]$. The `custom` template, available only to Trusted and Sovereign agents, allows full configuration of all `VaultConfig` parameters within protocol maximums.

The canonical vault creation flow proceeds in five steps: (1) the creator agent calls `AgentVaultFactory.createVault(config)` with a populated `VaultConfig` struct; (2) the factory validates all parameters (base asset, fee caps, creator reputation, mandatory disclosure) and reverts with typed errors on failure (`InvalidBaseAsset`, `FeeExceedsCap`, `InvalidCreator`, `MissingDisclosure`); (3) CREATE2 deploys the vault at a deterministic address derived from `keccak256(abi.encode(msg.sender, config.creatorAgentId, config.baseAsset, nonce))`; (4) if `hookEnabled`, the factory deploys the `VaultHook` and `NAVAwareHook` (merged into a single contract with `LaunchFeeHook` if enabled), mining the hook address via `HookMiner` to encode the correct V4 permission bits; (5) if `autoPoolEnabled`, the factory calls `PoolManager.initialize()` to create the V4 share pool, and the vault's shares become immediately tradeable on Uniswap. The entire sequence executes atomically - if any step fails, the transaction reverts.

### 5.2 Core Contracts

AgentVaultFactory. CREATE2 deployment with a vault registry. The factory computes deterministic vault addresses before deployment, enabling counterfactual vault references. It maintains three on-chain data structures: `mapping(address => bool) isVault` for O(1) membership checks, `address[] _allVaults` for enumeration, and `mapping(uint256 => address[]) _vaultsByCreator` for per-creator lookup. The factory uses the EIP-1167 minimal proxy clone pattern - each vault is a lightweight clone of an `implementation` template, making creation cheaper than full deployment. The factory owner can update the implementation via `setImplementation(address)` with a 3–7 day timelock; existing vaults are not affected.

The `VaultConfig` struct captures full vault configuration at deployment:

```
struct VaultConfig {
    address baseAsset;        //
    USDC, WETH…
    uint256 creatorAgentId;   // ERC-8004
    ID
    uint16 managementFeeBps;  // max
    500
```

Five vault templates provide graduated complexity, each with distinct default configurations:

Table 5: Vault template configurations with default parameters and tier requirements

| Template | Base Asset | Hook | Auto Pool | Default Fees | Min Creator Tier |
|---|---|---|---|---|---|
| safe-yield | Stablecoin | No | Optional | 1% mgmt, 10% perf | Unverified |
| balanced-growth | Multi-asset | No | Optional | 1.5% mgmt, 15% perf | Basic |
| active-trading | Flexible | Yes | Yes | 2% mgmt, 25% perf | Verified |
| market-making | LP pairs | Yes | Yes | 2% mgmt, 20% perf | Verified |
| custom | Any | Configurable | Configurable | User-defined | Trusted |

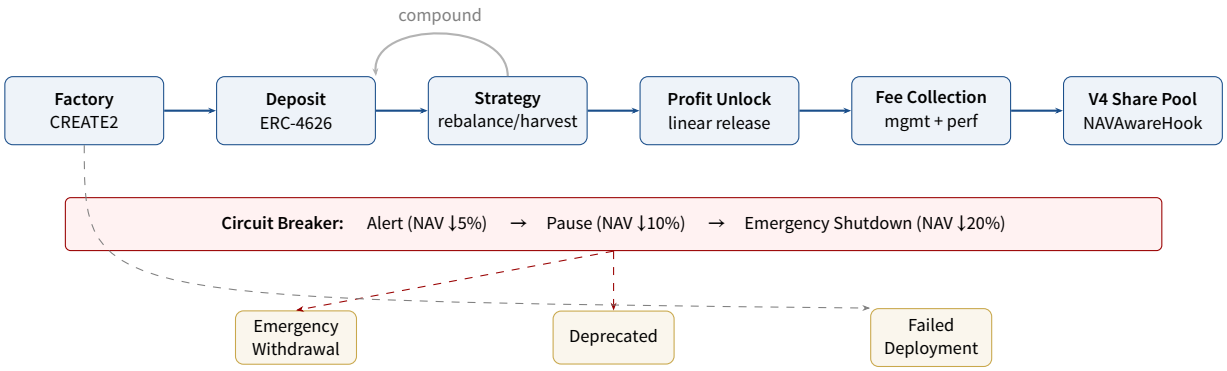


Figure 5: Vault lifecycle: creation through the factory, deposits via ERC-4626, strategy execution with compounding, linear profit unlock, fee collection, and secondary market via V4 share pool. Circuit breaker provides three-tier escalation; terminal states shown below.

```
    uint16 performanceFeeBps; // max 5000
    uint256 minReputation;    // 0
    = open
    bool hookEnabled;         //
    VaultHook
    bool autoPoolEnabled;     // V4
    pool
    bool launchFeeEnabled;    // MEV
    guard
    bool rehypothecationEnabled;
    VaultDisclosure disclosure;
}
```

AgentVaultCore. The core vault contract inherits OpenZeppelin `ERC4626`, `ReentrancyGuard`, and `Ownable`, extending ERC-4626 with:

- ERC-8004 identity gating on `deposit()`, `withdraw()`, and `redeem()`, reverting with `AgentNotRegistered` or `CredentialFrozen` for invalid agents
- Tier-based deposit caps enforced per-transaction, querying the five-tier trust hierarchy (the reputation-gated deposit caps (Section 4.2)) and reverting with `TierCapExceeded`
- Circuit breaker with configurable thresholds (NAV drawdown, withdrawal velocity, position concentration); the default thresholds are 3% single-block NAV drop, 7% hourly NAV drop, and 13% daily NAV drop
- Virtual shares offset (OpenZeppelin `_decimalsOffset()` set to 3–6) for inflation attack prevention [26] - Venus Protocol lost approximately $716K in net bad debt on zkSync (February 2025) from a related donation-based exchange rate manipulation on a low-supply wUSDM vault
- Linear profit unlock following the Yearn V3 pattern [27]: profits enter a buffer and are released linearly over `profitMaxUnlockTime` (default 6 hours), with `totalAssets()` deducting not-yet-unlocked profit to smooth share price increases
- ERC-6909 internal share accounting for gas efficiency, with standard ERC-20 claim wrappers for external composability (following the Uniswap V4 PoolManager pattern)

The core accounting invariant is:

$$\text{totalAssets} = \text{idleCapital} + \sum_i \text{adapter}[i].\text{totalValue}() - \text{accruedFees} - \text{unlockedProfit}()$$

This invariant is enforced by the vault's `report()` function, which reconciles adapter values, updates fee accruals, and emits a `VaultReport` event with the full breakdown. Any discrepancy triggers a `ReportMismatch` event and may activate the circuit breaker. The `report()` function is called periodically by the `REPORTING_MANAGER` role; fees are calculated on unlocked profit only, not raw `totalAssets` changes.

External protocols depend on accurate ERC-4626 interface behavior. The vault provides conservative estimates: `maxDeposit(receiver)` accounts for the

agent's tier cap minus current deposits and returns 0 if paused or unregistered; `maxWithdraw(owner)` returns only instantly-available liquidity (idle capital proportional to the owner's share), not capital locked in LP positions or adapters; and all preview functions account for fee effects within `_decimalsOffset` rounding tolerance. Non-standard revert conditions include `AgentNotRegistered`, `CredentialFrozen`, `TierCapExceeded`, `VaultPaused`, and `OracleStale`.

The vault lifecycle transitions through seven states: `Creating` (during factory deployment), `Active` (normal operation), `PausedCreator` (creator-initiated pause), `PausedBreaker` (circuit breaker pause), `EmergencyWithdrawal` (all adapters force-deallocated, deposits rejected, proportional withdrawal only), `Failed` (terminal, irrecoverable), and `Deprecated` (terminal, orderly wind-down complete). State transitions are one-directional: once a vault enters `EmergencyWithdrawal`, it cannot return to `Active`.

Permit2 agent delegation. The Universal Router batch rebalancing pattern leverages Permit2's signature-based approval system to give agents bounded, time-limited, amount-limited authority to execute vault operations without on-chain approval transactions for each action. The vault grants Permit2 sub-approval to the managing agent's address with bounded amount and expiration (e.g., 1 week, up to 10% of vault TVL per rebalance). The agent constructs Universal Router command batches encoding complete rebalancing plans - exit positions, close LP ranges, acquire new target assets, open new LP ranges, settle - and signs with their ERC-8004 identity. V4's flash accounting ensures only net token deltas are settled: a 5-step rebalance with intermediate tokens requires only 2 actual ERC-20 transfers, with intermediate operations using transient storage at approximately 100 gas per operation versus 2,000+ for regular storage.

IdentityRegistryAdapter. A decoupling layer between the vault and the ERC-8004 registry. Since ERC-8004 was deployed January 29, 2026 and remains in Draft status, the adapter absorbs interface changes without protocol-wide migration. The factory stores a single `identityAdapter` address; all vault contracts call identity functions through the adapter, never directly. The adapter exposes `isValidAgent(uint256 agentId)`, `getAgentWallet(uint256 agentId)`, `getEffectiveReputation(uint256 agentId)`, and `isCredentialFrozen(uint256 agentId)`. The factory owner can update the adapter via `setIdentityAdapter(address)` with a 3–7 day timelock.

IdentityGuardian. A wrapper around the ERC-8004 Identity Registry that adds transfer friction, guardian-based veto, and progressive lockdown for identity NFTs. Since ERC-8004 identity gates vault access, an unprotected identity NFT is a single point of failure. The IdentityGuardian adds a 7-day cooldown on identity transfers (with a 48-hour execution window after cooldown), guardian-based veto (any guardian can cancel a pending transfer), credential freezing for emergency response, and progressive fuse lockdown inspired by ENS where the transfer fuse can be permanently burned, making the identity non-transferable. Functions that could be triggered by prompt injection are prefixed with `DANGER__` (e.g., `DANGER__requestTransfer()`, `DANGER__disableGuardian()`) to prevent accidental LLM execution.

OnboardRouter. The protocol's canonical onboarding contract batches identity registration, Permit2 approval, reputation enrollment, and vault deposit into a single atomic call. Designed for ERC-4337 UserOp execution with paymaster sponsorship, the `OnboardRouter` collapses all steps into one gasless operation:

```
interface IOnboardRouter {
    function onboardAndDeposit(OnboardParams
    calldata params)
        external returns (uint256 agentId,
     uint256 shares);
}
```

The internal flow executes five steps atomically: (1) register identity on the ERC-8004 registry, (2) approve Permit2 for the base asset, (3) enroll the agent in the reputation engine, (4) deposit into the target vault, and (5) record on-chain referral attribution if a referrer agent ID is provided. If any step fails, the entire UserOp reverts. Combined with counterfactual smart account deployment (the wallet address is computed deterministically but not deployed until the first transaction), an agent can receive USDC at a pre-computed address and execute its first deposit as its first-ever transaction.

### 5.3 Governance Model

The Sentinel role cannot be renounced; every vault must always have a safety backstop. The Sentinel requires a dedicated, separately-stored key (or key set) with no overlap with operational infrastructure, reflecting the principle that emergency powers should never be exercisable by a single point of failure, whether human or AI. The Sentinel can revoke pending timelocked actions, pause the vault, and force-exit positions via `forceDeallocate()`, but cannot allocate capital or modify strategy parameters.

Vault governance distributes authority across four non-overlapping roles:

| Role | Responsibility | Can Be Renounced |
|---|---|---|
| Owner | Deploy, set initial config, assign other roles | Yes (abdication) |
| Curator | Select strategy adapters, set allocation targets | Yes |
| Allocator | Execute trades within Curator's approved adapters | Yes |
| Sentinel | Emergency pause/unpause, circuit breaker override | No |

Abdication mechanism. Curators and Owners can call `abdicate(bytes32 paramHash)` to permanently lock specific parameters, creating irreversible guarantees. A vault whose curator abdicates fee parameters becomes a credibly neutral, permissionless yield instrument. Once an agent has proven itself, the vault creator can abdicate, making the vault permanently autonomous. If the Owner abdicates entirely, the vault becomes fully immutable: no new adapters, no new tokens, no configuration changes. The Allocator can still execute within the existing strategy, and the Sentinel can still pause in emergencies.

Timelocked parameter changes. All governance actions carry risk-proportional timelocks. Strategy adapter additions require a 48-hour delay; fee changes require 24 hours; identity adapter updates require 3–7 days. The Sentinel can veto any pending timelocked action during the delay window, providing a safety backstop against compromised Curator keys.

Management rights for vault strategies are allocated through am-AMM Harberger leases [29], described in Section 6.4. The `StrategyAuctionModule` implements the auction mechanics: an agent calls `vault_bid_management()` with a proposed `rentPerBlock`, which must exceed the current rent by a minimum increment (5%). Collateral equal to `rentPerBlock` times $K$ blocks (the lookahead period) is transferred to the module. After the lookahead period, the previous manager's Curator role is revoked and the new manager's role is granted within PolicyCage bounds. Rent accrues continuously to depositors, creating a yield floor independent of strategy performance. On Base L2, the am-AMM lookahead is 36,000 blocks (calibrated for 2-second block time).

Incentive compatibility. In a competitive market for management rights, equilibrium rent $r^*$ settles where the marginal manager's expected fee income equals their opportunity cost plus rent. The Harberger self-assessment mechanism is incentive-compatible in the Myersonian sense [109]: managers reveal their true valuation through rent bids, because understating leads to eviction by a higher bidder, and overstating means paying more than the rights are worth. Truthful bidding is the dominant strategy. The rent-to-TVL ratio becomes a public price signal for management opportunity value - a novel form of price discovery for fund management talent.

### 5.4 Strategy Adapters

Strategy execution is modular via the `IStrategyAdapter` interface:

```
interface IStrategyAdapter {
    function allocate(uint256 amount) external
    returns (uint256 allocated);
    function deallocate(uint256 amount)
    external returns (uint256 withdrawn);
    function forceDeallocate() external
    returns (uint256 withdrawn);
    function totalValue() external view
    returns (uint256);
    function harvest() external returns
    (uint256 profit);
}
```

The Curator manages the adapter registry via `registerAdapter()` and sets per-adapter allocation caps. Adapter removal is timelocked and triggers automatic withdrawal of all positions.

The `RehypothecationAdapter` enables the dual-yield flywheel described in Section 6.6. Idle capital - tokens sitting out of range in LP positions or in the vault's reserve - is deployed to lending protocols (Morpho, Aave V3, Seamless) earning supply interest. The adapter monitors which tokens are out of range and deploys them to the highest-yielding approved venue, subject to a `maxTotalLendingExposure` cap of 60% of idle capital. This cap is derived from the liquidity risk penalty function of Baude, Challet, and Toke [TBD-Baude], who use stochastic control calibrated to USDT/Aave V3 data to compute optimal interest rates accounting for the probability that a sudden withdrawal demand exhausts the vault's idle reserves while capital is locked in lending. Three deployment patterns offer different tradeoffs: wrap-and-donate (deploy out-of-range tokens, withdraw when back in range), JIT borrowing (all capital in lending venues, LP liquidity provided on demand via flash-borrow), and hybrid (a fraction determined by the PA-AMM $\lambda$ parameter stays in the pool). On Base, the additional gas per JIT swap is approximately $0.003, making it viable for trades above roughly $10K notional.

All adapters must implement `forceDeallocate()` for guaranteed non-custodial exits: participants can always withdraw their capital regardless of adapter state. A depositor can also force-exit by receiving underlying adapter positions directly (proportional to their share), paying a 2% penalty to disincentivize casual use. Seven strategy adapters provide the execution surface:

Table 6: Strategy adapters providing the vault execution surface

| Adapter | Strategy | Risk |
|---|---|---|
| `RehypothecationAdapter` | Idle capital to lending (Morpho, Aave, Seamless) | Low |
| `RecursiveLendingAdapter` | Flash-loan leverage loops | Medium |
| `PendleAdapter` | Fixed/variable yield splitting via PT/YT | Low-Med |
| `CreditDelegationAdapter` | Undercollateralized lending to reputation-verified agents | High |
| `CCABidAdapter` | Continuous Clearing Auction participation | Medium |
| `MEVRedistributionAdapter` | MEV capture and redistribution to depositors | Medium |
| `LiquidityRouter` | Cross-pool capital allocation via convex optimization | Low-Med |

Each adapter must satisfy four conservation invariants: (1) `totalAssets()` monotonic consistency - no silent decreases between consecutive calls; (2) balance conservation - assets entering and leaving the adapter must sum correctly; (3) in-kind exit safety - `forceDeallocate()` cannot inflate the share price; and (4) data attestation - only pre-registered `dataHash` calldata is accepted. Adapter calldata validation uses cryptographic commitment: the Curator pre-registers `keccak256(encodedAdapterCalldata)` on-chain, and only calls matching a pre-registered hash are accepted. Even a fully compromised LLM cannot generate arbitrary adapter calls - the on-chain hash check provides a cryptographic guarantee independent of the LLM's behavior.

TWAMM rebalancing. When a vault needs to rebalance large positions (exceeding 1% of pool TVL), the `RebalanceParams` struct supports Time-Weighted Average Market Maker orders that split the rebalance over a configurable time window, minimizing price impact and MEV extraction. TWAMM orders execute as the first pool action in each block - effectively front-running all other trades, which paradoxically protects the vault from being front-run by external MEV bots. The SDK's `StrategyEngine.recommendTWAMM()` evaluates rebalance size versus pool liquidity and recommends TWAMM for operations exceeding the threshold. Below the threshold, single atomic swaps are always preferred: the Lean 4 formal verification library [113] proves that under fees, a single large swap always dominates split swaps (the generalized additivity theorem).

Liquidity Density Functions. When `ldfEnabled` is true in `VaultConfig`, the vault uses Bunni v2-style continuous density functions instead of discrete tick ranges for liquidity management. A density function $f(\text{price})$ defines a smooth, differentiable liquidity distribution that the hook maps to the V4 tick grid. LDFs offer three advantages for sophisticated vaults: learnable shape optimization (AI agents can use gradient-based methods to adjust the density function directly), gas-efficient "shapeshifting" (redistributing liquidity across the price curve requires a single operation rather than removing and re-adding at individual ticks), and continuous hedging (a differentiable density function enables closed-form computation of position Greeks for delta-neutral strategies). LDFs are an advanced option gated to Verified+ agents.

### 5.5 Fee Architecture

Fee caps are immutable, set at vault creation and enforced by the `FeeModule` contract. No governance vote, no admin key, and no proxy upgrade can raise fees above the protocol maximums.

Fee collection follows the Yearn V3 accountant pattern: fees are not deducted implicitly per-transaction. Instead, a separate `report()` function is called periodically to calculate profit since the last report, apply management and performance fees, and mint fee shares to the fee recipient. This pattern prevents per-transaction fee rounding errors from accumulating, enables transparent fee accounting (every fee mint is an explicit `FeeCollected` event), and integrates with the linear profit unlock buffer (Section 5.2) so that fees are calculated on unlocked profit only.

Fee waterfall. Yield is allocated in strict order: (1) management fees are accrued continuously on `totalAssets()` and collected via the accountant pattern; (2) a configurable hurdle rate (default 0%) must be exceeded before performance fees apply; (3) the high-water mark ensures performance fees are

Table 7: Protocol fee structure with immutable caps

| Fee Type | Protocol Maximum | Notes |
|---|---|---|
| Management | 500 bps (5%/yr) | Accrued continuously on totalAssets() |
| Performance | 5,000 bps (50%) | On net-new profit above high-water mark |
| Protocol | 0 bps | Vault creators keep 100% (Morpho pattern) |
| Entry | 0 bps | No front-end loading fee |
| Exit | 0 bps | No redemption fee |

Creator reputation gates determine the fee levels a vault creator can charge. Higher fees require higher reputation, creating a natural throttle on unproven managers:

| Creator Tier | Max Management Fee | Max Performance Fee |
|---|---|---|
| Unverified | 100 bps (1%) | 2,000 bps (20%) |
| Basic | 200 bps (2%) | 3,000 bps (30%) |
| Verified | 300 bps (3%) | 4,000 bps (40%) |
| Trusted | 500 bps (5%) | 5,000 bps (50%) |
| Sovereign | 500 bps (5%) | 5,000 bps (50%) |

only charged on net-new profit above the historical highest NAV; (4) performance fees are applied to the increment above the high-water mark and hurdle rate; (5) remaining yield flows to participants proportional to share ownership.

The high-water mark prevents the incentive misalignment common in traditional fund management: a manager cannot earn performance fees by simply recovering from losses. The NAV must exceed the previous peak before performance fees accrue.

Game-theoretic properties. In equilibrium, a rational agent invests in reputation up to the point where the marginal cost of earning the next reputation increment equals the marginal fee savings it unlocks. The convex discount schedule (Section 4.2) creates a separating equilibrium: low-commitment agents stop at Basic, while serious agents push toward Sovereign to capture the steep discount curve. Consider a concrete example: an agent managing $500K at 300 bps management fee pays $15K/year. At Sovereign tier, the 30% management discount saves $4,500/year; the 25% performance discount provides additional savings proportional to returns. Reaching Sovereign requires approximately 500 additional reputation points, most from ecosystem milestones. The annual fee savings justifies substantial investment in ecosystem participation.

The discount creates a virtuous cycle: higher reputation yields lower fees, which yields better net performance, which attracts more capital, which generates more on-chain history, which yields higher reputation.

ERC-4626 preview compliance. All preview functions (`previewDeposit`, `previewMint`, `previewWithdraw`, `previewRedeem`) include fee effects. An agent calling `previewDeposit(1000)` receives the exact number of shares it would receive after management fee deduction. This is a hard requirement of ERC-4626 compliance - vault aggregators (vaults.fyi, DefiLlama) and composing protocols (Morpho, Pendle) rely on preview functions returning accurate values. Failure to include fees in previews causes depositors to receive fewer shares than expected, breaking aggregator integrations.

Vault capability reporting. Every vault exposes a `vaultCapabilities()` view function returning a bitmap of supported features: synchronous deposits (bit 0), synchronous withdrawals (bit 1), async deposits via ERC-7540 (bit 2), async withdrawals via ERC-7540 (bit 3), share pool trading via V4 (bit 4), and slippage-protected wrappers (bit 5). The SDK reads this bitmap before presenting entry/exit options, ensuring agents never attempt unsupported operations.

### 5.6 Withdrawal Mechanics

Instant withdrawal operates against the vault's idle capital reserve. The `reserveFloorBps` parameter (default 1,000 bps = 10%) guarantees that a minimum fraction of total assets remains liquid at all times. When a withdrawal is smaller than available idle capital, the vault burns shares, transfers the base asset, and settles in a single block. If the amount exceeds idle capital but the vault holds positions in adapters with instant exit capability, the vault automatically deallocates from the most liquid

Five withdrawal paths provide progressive liquidity, reflecting the principle that participants should always have access to their capital, even when vault strategy involves illiquid positions:

| Path | Mechanism | Latency | Best For |
|---|---|---|---|
| Instant | Standard ERC-4626 `withdraw()` | 1 block | Small amounts, idle capital |
| Queued | ERC-7540 `requestRedeem()` then `claimRedeem()` | 1–7 days | Large amounts, illiquid adapters |
| Emergency | Circuit breaker triggers `forceDeallocate()` | 1 block | Safety events |
| Force Exit | Direct adapter `forceDeallocate()` by Sentinel | 1 block | Adapter failure |
| Secondary Market | Sell shares on V4 pool via NAVAwareHook | Instant | Any amount, price discovery |

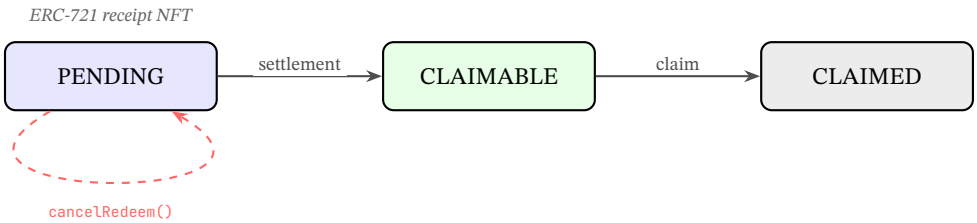


Figure 6: Queued withdrawal state machine. Dashed arrow shows the optional cancellation path from Pending state.

adapter first, following a cascading priority ordering: idle capital first, then `RehypothecationAdapter` (lending withdrawal), then liquid LP positions, then illiquid adapters. All withdrawal functions accept a `minAssetsOut` parameter following EIP-5143 slippage protection concepts - the transaction reverts if the received assets would be less than specified, protecting against front-running.

Queued withdrawal implements ERC-7540 asynchronous redemption for illiquid positions. The state machine follows three phases:

$$\text{PENDING} \xrightarrow{\text{settlement}} \text{CLAIMABLE} \xrightarrow{\text{claim}} \text{CLAIMED}$$

In the Pending phase, `requestRedeem()` locks shares and issues a transferable ERC-721 receipt NFT representing the claim. The vault unwinds illiquid positions over 1–7 days. Once unwinding is complete, the request transitions to Claimable, and `claimRedeem()` transfers assets and burns the receipt. An optional cancellation path (`cancelRedeem()`) is available while the request remains Pending and the position is not yet committed. The receipt NFT is transferable - a participant who cannot wait for settlement can sell it on a secondary market at a discount, providing immediate liquidity. This is economically equivalent to a bond: the receipt promises future delivery of base assets, and its market price reflects the time discount and settlement risk. Centrifuge deployed this pattern at approximately $1.35B TVL, validating the ERC-7540 async flow at production scale.

Emergency withdrawal activates when a Tier 2+ circuit breaker fires. All strategy adapters receive `forceDeallocate()` calls, new deposits are rejected, fee accrual pauses, and all participants can withdraw at the post-unwinding NAV on a proportional basis. The vault manager cannot intervene; the state transition is enforced by the `RiskEngine` contract. This is the protocol's ultimate safety guarantee: regardless of what happens to strategy, oracle feeds, or management key, participants can always recover their proportional share of remaining assets.

Secondary market exit uses the auto-created V4 share pool (Section 6.1). Participants sell shares directly on this pool at any time, receiving instant settlement at market price. The `NAVAwareHook` anchors the pool price to the vault's on-chain NAV using a NoOp pattern: `beforeSwap` returns a `BeforeSwapDelta` that completely bypasses the native concentrated liquidity AMM, instead pricing vault shares at NAV plus or minus a configurable spread (default 50 bps). Buying shares pays a low fee (5 bps), while selling pays a higher fee (25–50 bps) that scales with recent withdrawal velocity - this asymmetry naturally protects vaults from bank-run dynamics while making entry attractive. When selling pressure drives the share price below NAV minus 50 bps, arbitrageurs profit by depositing base assets into the vault and selling shares, closing the discount. Conversely, when buying pressure drives the price above NAV plus 50 bps, arbitrageurs buy shares on the secondary market and redeem through the vault. The secondary market acts as a shock absorber: large exits create temporary discounts that attract new capital, rather than forcing fire-sales of illiquid positions. Persistent premiums signal confidence in the vault manager; persistent discounts signal concern - this information is valuable to both depositors and the vault manager's own learning loops (Section 7).

Withdrawal dampening. When multiple withdrawal requests compete for limited liquidity, the vault settles in FIFO order. The dampening mechanism ensures orderly unwinding: when pending withdrawal requests exceed the `reserveFloorBps` threshold, the vault applies a dampening factor that extends settlement time proportionally to queue depth. The `pendingRequests()` view function returns queue depth and estimated settlement time, enabling participants to make informed decisions about whether to queue or sell on the secondary market.

Tier-differentiated loss socialization. When a vault incurs losses from strategy failure, the ADL (Auto-Deleveraging) Trilemma applies: Chitra [TBD-Chitra] proved that no loss socialization policy can simultaneously satisfy solvency, revenue, and fairness. The vault implements a tier-modulated waterfall: (1) the protocol insurance pool absorbs first loss up to a configurable threshold; (2) remaining loss is distributed tier-down - Sovereign and Trusted depositors absorb residual losses first, protecting Unverified and Basic depositors; (3) if total loss exceeds all buffers, pro-rata distribution applies across all depositors. This design prioritizes solvency for small depositors, building trust at the protocol's growth edge.

### 5.7 Risk Engine

The `RiskEngine` contract consolidates all vault risk parameters into a single on-chain reference point. It implements the ERC-7265 circuit breaker interface and provides health attestations that external protocols can query.

PolicyCage enforces hard on-chain boundaries that cannot be overridden by any LLM output, learned heuristic, or prompt injection:

```
struct PolicyCageBounds {
    // Token allowlist
    address[] approvedAssets;
    // Strategy adapter allowlist
    address[] approvedAdapters;
    // Max allocation per asset (default:
    4000 = 40%)
    uint16 maxPositionBps;
    // Max drawdown before circuit breaker
    (default: 1000 = 10%)
    uint16 maxDrawdownBps;
    // Min seconds between rebalances
    (default: 3600)
    uint32 maxRebalanceFreq;
    // Max capital in any single adapter
    uint256 maxDebtPerAdapter;
    // Min idle capital reserve (default:
    1000 = 10%)
    uint16 minIdleBps;
}
```

The defaults are deliberately conservative. The Curator can tighten these bounds but never loosen them beyond values set at vault creation - this asymmetry is enforced at the contract level, not by convention. The Owner can abdicate, making all bounds permanent. The PolicyCage represents the protocol's hardest safety guarantee: regardless of what an LLM outputs, what a learned heuristic recommends, or what a prompt injection attempts, the on-chain constraints cannot be exceeded. The approved asset list prevents an agent from trading unapproved tokens; the maximum position size prevents concentration risk; the minimum idle reserve ensures withdrawal liquidity; the rebalance frequency limit prevents gas-exhaustion attacks.

Hard constraint boundaries improve reinforcement learning agent performance compared to unconstrained optimization, as demonstrated by Xu and Brini [30] (PPO-based LP optimization) and Qu et al. [31] (TD3-BC conservative policy learning). This is not merely a safety feature but a performance feature: agents operating within well-defined boundaries converge to better strategies than agents with unconstrained action spaces, because the boundaries eliminate catastrophic tails from the exploration distribution.

The vault lifecycle state machine transitions through seven states: Creating, Active, PausedCreator, PausedBreaker, EmergencyWithdrawal, Failed (terminal), and Deprecated (terminal). The emergency withdrawal state is the protocol's ultimate safety guarantee: all adapter positions are unwound, deposits are rejected, fee accrual pauses, and participants withdraw at the post-unwinding NAV on a proportional basis.

External protocols (e.g., Morpho using vault shares as collateral) can query `RiskEngine.isHealthy(vaultAddress)` to verify vault health before accepting shares. The attestation returns a boolean `healthy` flag plus a `bytes32 flagSet` bitmask encoding which checks passed and which failed, enabling granular risk assessment by composing protocols.

Risk metrics computed by the RiskEngine include Value at Risk (VaR) via historical simulation augmented with conformal prediction for distribution-free coverage guarantees [101], Conditional VaR (expected shortfall), maximum drawdown, Sharpe ratio, conformal predictive portfolio selection [125], DeFi Correlation Fragility Indicator (CFI) [28], and Aggregated Systemic Risk Index (ASRI) [98]. The ASRI module monitors cross-protocol contagion by tracking correlated drawdowns across external

Circuit breakers operate across three tiers:

| Tier | Trigger | Response | Recovery |
|---|---|---|---|
| Tier 1 (Alert) | NAV drops > `navDropTriggerBps` within window | Emit alert events, widen NAV bounds | Automatic when NAV stabilizes |
| Tier 2 (Pause) | Sustained drawdown > `maxDrawdownBps` | Full vault pause, reject deposits | Creator unpause after cooldown |
| Tier 3 (Emergency) | Withdrawal velocity exceeds ERC-7265 rate limiter | Force-deallocate all adapters | Terminal - proportional withdrawal only |

Health attestations are computed on every `report()` call and expose six invariant checks:

| Check | Condition | Failure Response |
|---|---|---|
| `totalAssets()` consistency | Reported value matches adapter sum | `ReportMismatch` event, Tier 1 alert |
| Share price rate-of-change | NAV change < `navMaxChangeBps` per interval | Clamp to last validated NAV |
| Oracle freshness | Last observation < `oracleMaxStaleness` (30 min) | `RiskWarning` event, wider NAV bounds |
| Adapter exposure caps | No adapter exceeds `maxDebtPerAdapter` | Block further allocation |
| Idle capital reserve | Idle >= `minIdleBps` of total assets | Block new adapter deployments |
| Circuit breaker status | No active Tier 2+ breaker | Reject deposits, dampen withdrawals |

protocols; when ASRI exceeds a configurable threshold (default: 0.7), the RiskEngine triggers automatic deallocation from the most correlated adapters.

Permissionless Executor Framework. Multiple vault operations require off-chain computation followed by on-chain submission: cross-vault coordination solutions, LVR-theta fee floor calibrations, behavioral regime classifications, and proxy transaction execution. The framework provides a unified answer: every operation that needs external submission is callable by anyone, with no registration, bonding, or allowlisting required. All executor-dependent contracts implement the `IExecutable` interface with `executeJob(bytes calldata jobData)` and `getJobCondition()` functions. Rewards follow a Dutch auction model: if no executor submits within the target window, the reward escalates linearly from `baseRewardBps` (3%) to `maxRewardBps` (20%) over `escalationBlocks` (approximately 400 seconds on Base), guaranteeing eventual execution. The framework is self-funding - rewards come from the measurable on-chain benefit created by the operation, not from treasury subsidies. Successful execution earns reputation milestones, creating a non-capital-intensive yield path where agents earn by providing computation and infrastructure rather than deploying capital.

Cross-vault coordination. When multiple factory-deployed vaults hold overlapping token pairs, external arbitrageurs can extract value from price discrepancies between those vaults' pools. The `CrossVaultCoordinator` addresses this by computing the unique arbitrage-free cross-vault price configuration - Devorsetz and Herlihy [33] proved that for any set of CFMMs with log-concave trading functions, this configuration is unique and equivalent to Pareto efficient. The coordinator runs off-chain (the convex optimization is too expensive for on-chain execution) and submits results via the Permissionless Executor Framework, where any address can call `submitCoordinatedRebalance()` and earn a reward proportional to the arbitrage leakage prevented. This converts inter-vault arbitrage extraction - which would otherwise flow to third-party searchers - into a public good that benefits all participating vault depositors.

### 5.8 Anti-Manipulation

Two primitives protect vault share pricing from manipulation.

Virtual shares offset. OpenZeppelin's `_decimalsOffset()`, set to 3–6 for all factory-deployed vaults, adds virtual shares and virtual assets that make inflation attacks economically infeasible. With an offset of 6, the vault effectively mints $10^6$ virtual shares at deployment. An attacker attempting the classic inflation attack - depositing 1 wei, donating a large amount, then front-running the next depositor - must donate $10^6$ times more capital to achieve the same exchange rate distortion. The attacker must donate exponentially more capital to meaningfully affect the exchange rate [26]. Combined with internal asset accounting (where `totalAssets()` tracks deposits and withdrawals internally rather than reading `balanceOf(address(this))`), donation attacks become irrelevant to share pricing.

Linear profit unlock. All adapter profits are released linearly over a configurable period (default 6 hours), following the Yearn V3 pattern [27]:

$$\text{unlockedProfit}(t) = \text{lockedProfit} \times \frac{\text{profitMaxUnlockTime} - (t - t_{\text{report}})}{\text{profitMaxUnlockTime}}$$

This prevents flash donation attacks where an attacker donates a large amount to a vault, inflates the share price, mints shares, and withdraws at the inflated rate. A maximum share price increase clamp of 500 bps (5%) per unlock period provides an additional bound on manipulation - even if an adapter reports an anomalously large profit, the share price cannot increase by more than 5% in a single period.

Oracle architecture. The vault uses a three-oracle architecture for manipulation-resistant pricing. The primary oracle uses Ormer [34] median-based pricing, which provides inherent outlier robustness by taking the median of the last $N$ price observations rather than the mean. The secondary oracle implements the SecPLF [35] per-asset price state tracking pattern: the vault only queries an external oracle if the internally tracked price deviates beyond a defined bound from the last recorded value, reducing gas costs by 70–90% on low-volatility pairs and negating profitable oracle manipulation attacks (an attacker must first move the real market price before the oracle is consulted). The tertiary oracle integrates OVer [36] symbolic analysis that detects exploitable oracle scenarios proactively. A divergence guard auto-pauses the vault if primary versus secondary oracle prices diverge by more than 2%.

Formal verification. The vault contracts require formal verification across three dimensions. First, property-based fuzzing via Foundry invariant testing validates share accounting, fee calculation, and circuit breaker behavior under randomized inputs. Second, the Pool Favor Property - ensuring integer-arithmetic rounding always favors the pool ($K' \geq K$) - is formally verified [112], guaranteeing that no sequence of deposits and withdrawals can extract value through rounding exploitation. Third, fee monotonicity verification uses the Lean 4 AMM fee library [113] covering 3,500 lines of machine-checked proofs, including the generalized additivity theorem proving that under fees, a single large swap always dominates split swaps. This result has a direct implication for vault rebalancing: when the vault uses TWAMM to split large rebalances over time, the benefit comes solely from reduced market impact, not from fee savings on the split trades.

VaultHook dynamic fee engine. The `VaultHook` integrates a three-regime dynamic fee engine informed by the convergent findings of three independent research groups [37]. In the stable regime (realized volatility below $\sigma_{\text{low}}$), the hook charges a constant base fee (default 5 bps on both L1 and L2). In the elevated regime, fees scale linearly with inventory imbalance following the Baggiani et al. closed-form approximation, distinguishing between trades that restore pool balance (lower fees) and trades that worsen imbalance (higher fees). In the spike regime (volatility above $\sigma_{\text{high}}$), the maximum fee cap applies (default 100 bps on L1, 150 bps on Base). An LVR-theta fee floor [38] guarantees that LP fees at minimum compensate for adverse selection losses, computed from real-time volatility. The floor ensures fees never drop below the theta decay of the replicating perpetual continuous-installment put option. A fee-implied volatility oracle [39] provides the inverse mapping: given the current fee level, the implied volatility $\sigma_{\text{implied}}$ can be extracted via a bijective formula, creating a zero-cost on-chain volatility oracle derived entirely from pool fee parameters without requiring any external data source.

Agent-gated access via identity cache. The `VaultHook`'s `beforeSwap` callback must verify agent identity on every swap, but making an external call to the ERC-8004 registry adds approximately 2,600 gas per swap and creates a potential DoS vector. The hook implements a local cache bitmap (`mapping( address => bool) verifiedAgentCache`) that reduces per-swap gas from approximately 2,800 (external call) to approximately 200 (single warm SLOAD). Cache entries are invalidated reactively via ERC-8004 `IdentityRevoked` event listeners or proactively via periodic `refreshAgentCache()` calls. The `CACHE_REFRESH_INTERVAL` (default 300 blocks, approximately 10 minutes on Base) ensures stale entries are refreshed even if event listeners fail.

### 5.9 Structured Products

The `TrancheModule` (expansion-track, post-v1) splits vault shares into Senior (protected yield) and Junior (first-loss, boosted yield) tokens, enabling risk segmentation within a single vault:

- Senior tranche (AA): Protected principal with fixed yield. Senior depositors are paid first from vault earnings. If the vault suffers losses, Senior depositors are protected up to the Junior tranche's total value. The Senior tranche is designed for integration with Pendle's yield tokenization - a Pendle PT (Principal Token) on the Senior tranche provides a fixed-rate DeFi product with a defined maturity, economically equivalent to a zero-coupon bond.
- Junior tranche (BB): First-loss capital with leveraged upside. Junior depositors absorb all losses before Senior deposits are at risk. In exchange, Junior receives all yield above the Senior fixed rate. A vault with 80% Senior / 20% Junior provides 5x leverage to Junior holders. The leverage ratio is determined

mechanically by the Senior/Junior split and requires no margin calls or liquidation mechanisms.

- Skin-in-the-game requirement: The vault creator must hold Junior shares, aligning incentives with correct risk management. The vault cannot accept Senior deposits until the creator has funded the minimum Junior position. This structural requirement ensures the vault manager's capital is the first to be lost in a drawdown.

Tranche pricing is mechanical:

$$\text{seniorValue}(t) = \text{seniorPrincipal} \times (1 + \text{fixedRate} \times (t - t_0)/365)$$

When `juniorValue(t)` reaches zero, the vault enters `EmergencyWithdrawal` - the Senior fixed-rate guarantee is breached and Senior depositors receive their proportional claim on remaining assets. The `TrancheModule` enforces the waterfall mechanically with no oracle dependency, governance decision, or LLM involvement.

The yield curve implied by Pendle PT pricing on Senior tranches provides a market-discovered risk-free rate for the vault's underlying strategy - the first on-chain yield curve for AI-managed vault products.

The `BondMMHook` implements a bonding curve market maker for tranche tokens on Uniswap V4. Senior tokens, which accrue value linearly toward their face value at maturity, use concentrated curves centered on the time-discounted face value. Junior tokens, which carry leveraged exposure to vault performance, use wider curves reflecting their higher volatility. The hook prices Senior tokens using the formula:

$$P_{\text{senior}}(t) = \frac{\text{faceValue}}{(1+r)^{(T-t)/365}}$$

where $r$ is the Senior fixed rate and $T$ is the maturity date. Junior token pricing is derived residually: $P_{\text{junior}} = (\text{totalAssets} - P_{\text{senior}} \times \text{seniorSupply})/\text{juniorSupply}$. The hook adjusts the bonding curve parameters dynamically as the vault's NAV changes, ensuring the secondary market price tracks the mechanical tranche valuation.

Composability surface. Vault shares are designed for day-one composability across the DeFi ecosystem. Shares are ERC-20 compatible and follow ERC-4626 semantics; any external protocol supporting ERC-4626 (Morpho, Pendle, aggregators like vaults.fyi and DefiLlama) can integrate vault shares without custom adapters. Internally, shares use ERC-6909 for gas-efficient multi-token accounting, with standard ERC-20 claim wrappers available for external composability. Async deposit/withdrawal receipts are transferable ERC-721 tokens with `claimableAmount()` and `estimatedClaimTime()` view functions for composability with aggregators. The vault's share token supports ERC-7802 (Superchain token standard) for portability across the OP Stack ecosystem where applicable.

The vault protocol integrates deeply with Uniswap V4's hook architecture, which is described in the following section.

## 6. Uniswap V4 Integration

### 6.1 NAV-Aware Share Pools

Every vault deployed through the `AgentVaultFactory` receives an auto-created Uniswap V4 pool for its share tokens. The `NAVAwareHook` enforces convergence between the secondary market price and the vault's net asset value:

1. The hook queries `vault.totalAssets()` and `vault.totalSupply()` to compute real-time NAV per share.
2. The pool's pricing curve is adjusted to center on NAV, with configurable deviation bands.
3. Swaps that would move the price beyond the deviation bands face progressively higher fees, creating a natural pull toward NAV.

This mechanism provides instant exit for vault participants without the latency of ERC-7540 queued redemptions, and provides price discovery for vault shares, enabling market participants to express views on vault quality and expected future performance.

The hook operates through V4's callback interface. The `beforeSwap` callback reads the current NAV per share from the vault contract and adjusts the pool's effective price to center on NAV, applying progressive fees for swaps that would push the price beyond configurable deviation bands. The `afterSwap` callback records a NAV snapshot for the time-weighted oracle. The v1 NAV calculation follows:

$$\text{NAV}_{\text{per share}} = \frac{\text{idleCapital} + \sum_k \text{lpPositions}[k].\text{totalValue} - \text{accruedFees}}{\text{totalSupply}}$$

Two components (CCA bid valuation and external yield positions) are excluded from v1 until those modules ship. The `totalValue` of each LP position is computed from the Uniswap V4 pool's current tick and the position's liquidity, avoiding external oracle dependency for the core calculation.

The V4 share pool enables a form of price discovery unavailable through standard redemption: the secondary market price can trade at a premium or discount to NAV, reflecting the market's assessment of the vault manager's skill, strategy risk, and expected future performance. Persistent premiums signal confidence; persistent discounts signal concern. This information is

The following table compares the three exit paths available to vault participants:

| Property | V4 Share Pool Exit | Standard ERC-4626 | ERC-7540 Queued Redemption |
|---|---|---|---|
| Latency | Instant (single swap tx) | Instant (if liquid) | Hours to days (queue) |
| Price Discovery | Market-driven with NAV anchor | NAV per share only | NAV per share only |
| Slippage Risk | Pool depth dependent | None (pro rata) | None (pro rata) |
| Premium/Discount | Market can price quality | Not expressible | Not expressible |
| Liquidity Source | External LPs and arbs | Vault idle capital | Vault idle capital |

valuable to both depositors and the vault manager's own learning loops (Section 7).

The NAV oracle feeding the hook is protected by four guardrails: time-weighted snapshots (minimum 3 snapshots over 30 minutes before pricing updates), a rate-of-change clamp (if NAV changes by more than 5% in a single block, the hook enters safety mode using the last validated NAV), a staleness gate (no updates if no snapshot has been recorded for more than 1 hour), and a market safety mode (if share price deviates from NAV by more than 10%, available liquidity is reduced to limit capital at risk).

On Base L2 (2-second blocks), the guardrail system operates more aggressively: snapshot frequency of every 2 blocks (4 seconds), a 3% per-block rate-of-change clamp, a 30-minute staleness threshold, and a 7% deviation safety mode trigger. The LVR characteristics also differ between L1 and L2. Milionis, Moallemi, Roughgarden, and Zhang [76b] showed that LVR is approximately proportional to $\sigma^2 \times \text{block_time}$. With Base's 2-second blocks versus Ethereum's 12-second blocks, LVR on Base is approximately 2.5× lower ($\sqrt{12/2} = \sqrt{6} \approx 2.45$). Empirical measurements [103][104] confirm that LP strategies are structurally more viable on L2, though the exact reduction depends on solver competition and sequencer behavior.

### 6.2 Launch Protection

New vault launches are vulnerable to front-running: early knowledge of a vault's strategy allows adversaries to front-run the initial capital deployment. The `LaunchFeeHook` implements a descending fee schedule that makes such front-running unprofitable.

The launch fee follows a parabolic decay:

$$f(t) = f_{\max} \times \left(1 - \frac{t}{T_{\text{launch}}}\right)^2 + f_{\text{base}}$$

where $f_{\max}$ is the starting fee (default: 8,000 bps), $f_{\text{base}}$ is the steady-state base fee (default: 30 bps), $t$ is elapsed time since pool creation, and $T_{\text{launch}}$ is the launch period duration (default: 120 seconds on Base). The parabolic curve provides faster initial decay than exponential - the fee drops below 500 bps within the first 30 seconds, reaching the base fee at the end of the launch period. The factory's `VaultConfig` exposes two parameters: `launchFeeMaxBps` (the starting fee cap, immutable after vault creation) and `launchFeeDecaySeconds` (the launch period duration). These are set at vault creation and cannot be modified, preventing a manager from extending the launch period to extract additional fees.

Following the Loss-Versus-Fair analysis of Moallemi and Robinson [32], the starting fee is calibrated so that the expected profit from front-running is negative at any point during the launch period. The LVF framework decomposes execution costs into a deterministic component (fee) and a stochastic component (adverse selection). For the launch fee to be protective, we require $f(t) > \text{LVF}(t) + \text{expectedProfit}(t)$ at all $t \in [0, T_{\text{launch}}]$. On Base (2-second blocks), the unavoidable LVF loss floor is approximately 1.2 bps, confirming the chain's suitability for vault launches.

Excess fees collected during the launch period are deposited back into the vault as initial yield, creating a first-mover incentive: early depositors benefit from the high fees charged to speculators. The fee decay functions as an automatic transfer from short-term speculators to long-term depositors.

The `LaunchFeeHook` composes with the `NAVAwareHook`. Both hooks are merged into a single contract that transitions from launch-fee behavior to NAV-aware behavior after the launch period expires. The hook address is mined via `HookMiner` to include the correct V4 permission bits for both fee modification and price adjustment.

### 6.3 Dynamic Fee Adjustment

The fee engine is augmented with two oracle sources. The *LVR-theta floor* [38] guarantees that LP fees at minimum compensate for adverse selection losses, computed from real-time volatility. Singh et al. demonstrated that LVR equals the theta of a perpetual continuous-installment put option. The *fee-implied volatility oracle* [39] provides the inverse mapping: given the current fee level, the implied volatility $\sigma_{\text{implied}}$ can be extracted via a bijective formula. This creates a zero-cost on-chain volatility

The `VaultHook` implements a three-regime dynamic fee engine informed by the optimal fee research of Campbell, Bergault, Milionis, and Nutz [37]:

| Regime | Condition | Fee Behavior |
|---|---|---|
| Stable | Realized vol $< \sigma_{\text{low}}$ (20%) | Base fee (e.g., 5 bps) |
| Elevated | $\sigma_{\text{low}} \leq \text{vol} \leq \sigma_{\text{high}}$ (80%) | Linear interpolation between base and spike |
| Spike | Realized vol $> \sigma_{\text{high}}$ | Maximum fee (e.g., 100 bps) |

The following table summarizes the calibration differences between Ethereum mainnet and Base L2:

| Parameter | Ethereum L1 | Base L2 | Rationale |
|---|---|---|---|
| $\sigma_{\text{low}}$ threshold | 20% | 20% | Same volatility definition |
| $\sigma_{\text{high}}$ threshold | 80% | 80% | Same volatility definition |
| LVR multiplier | 1.0× | $\approx 0.41\times$ | Per-block: $\sigma^2 \times$block_time; $\sqrt{12/2} \approx 2.45$; total LVR/time is block-time-independent in zero-fee model [76b] |
| Fee floor (typical) | 3–5 bps | 1–2 bps | Lower adverse selection on L2 |
| Rebalance gas cost | \$5–\$50 | \$0.003–\$0.03 | 50–500x cheaper on Base |
| Fee update latency | 12 seconds (1 block) | 2 seconds (1 block) | Block time difference |

oracle derived entirely from pool fee parameters, without requiring any external data source.

The optimal fee $f^*(\sigma)$ combines these elements:

$$f^*(\sigma) = \max(f_{\text{floor}}, \min(f_{\text{cap}}, a \times \sigma^{\beta}))$$

where $f_{\text{floor}}$ is the LVR-theta floor, $f_{\text{cap}}$ is the protocol cap (default 100 bps), $a$ is a calibration constant fit to historical volume-fee data, and $\beta$ is the fee sensitivity parameter (typically 0.5–1.0). This self-consistent system sets fees optimally for LP returns, bounds them below by adverse selection costs, and produces a useful volatility signal as a byproduct.

The lower LVR floor on Base means that LP strategies that are marginally unprofitable on mainnet become structurally viable on L2 - the fee-to-LVR ratio improves by approximately 2.5× for the same volatility regime. This is the primary economic argument for launching vault strategies on Base first. The formal implications are analyzed in Section 9.

### 6.4 am-AMM: Auction-Managed Market Making

Management rights for vault strategies are allocated through am-AMM Harberger leases [29]. The mechanism proceeds in four phases:

*Bid submission.* An agent calls `vault_bid_management()` with a proposed `rentPerBlock`. The bid must exceed the current rent by a minimum increment (5%). Collateral equal to `rentPerBlock` times $K$ blocks (the lookahead period) is transferred to the `StrategyAuctionModule`.

*Transition.* After the lookahead period, the previous manager's Curator role is revoked and the new manager's role is granted within PolicyCage bounds. Rent begins accruing per block and is distributed to depositors. The previous manager has $K$ blocks to wind down positions.

*Active management.* The manager operates within Curator bounds: rebalancing LP positions, configuring lending venue allocation, adjusting dynamic fee parameters. Rent accrues continuously to depositors via a per-block accumulator:

$$\text{rentAccrued}(t) = \sum_{b=b_{\text{start}}}^{b_{\text{current}}} r_b$$

where $r_b$ is the rent rate at block $b$. The manager must top up collateral before exhaustion; the `StrategyAuctionModule` enforces a minimum buffer of $K/4$ blocks of prepaid rent. If the buffer drops below this threshold, the vault emits a `LowCollateral` event, alerting potential bidders to an upcoming eviction opportunity.

*Eviction or outbid.* If collateral is exhausted, anyone can call `vault_evict_manager()` and the vault enters unmanaged mode - idle capital earns no fees but is protected from mismanagement. If outbid, the new manager transitions in at block $+K$ and remaining collateral is returned to the outgoing manager. The transition period provides the outgoing manager time to unwind positions gracefully, preventing forced liquidation of concentrated liquidity positions.

On Base L2, the am-AMM lookahead is 36,000 blocks (approximately 20 hours at 2-second block time, calibrated to match the 7,200-block mainnet lookahead

Activeness parameter $\lambda$. Each vault's LP positions carry an activeness parameter $\lambda \in [0, 1]$ that controls the split between active reserves (deployed in the AMM pool, exposed to LVR) and passive reserves (held outside the pool, zero LVR but earning no swap fees):

| $\lambda$ | Behavior | LVR Exposure | Fee Income |
|---|---|---|---|
| 0 | Fully passive | None | None |
| 0.3 | Mostly passive | 30% of full | 30% of full |
| 0.7 | Mostly active | 70% of full | 70% of full |
| 1.0 | Fully active | Full | Full |

at 12-second blocks). Bunni v2 validated this mechanism at scale - $138M/month in volume with approximately 59% of V4 hook volume flowing through am-AMM managed pools [29].

Incentive compatibility. In a competitive market for management rights, equilibrium rent $r^*$ settles where the marginal manager's expected fee income equals their opportunity cost plus rent. The Harberger self-assessment mechanism is incentive-compatible in the Myersonian sense [109]: managers reveal their true valuation through rent bids, because understating leads to eviction by a higher bidder, and overstating means paying more than the rights are worth. Truthful bidding is the dominant strategy. The rent-to-TVL ratio becomes a public price signal for management opportunity value.

For depositors, rent income creates a yield floor independent of strategy performance. A vault generating zero trading alpha still distributes management rent. This is structurally analogous to a real estate investment trust distributing rental income, but the "tenant" is an AI agent paying for the right to manage capital, and eviction is automatic upon payment failure.

An important limitation should be noted: this equilibrium analysis assumes rational bidding. Whether LLM-based agents will bid rationally for management rights is empirically untested. An agent optimizing a poorly specified reward function might overbid or underbid. Early vault deployments will need to monitor rent-to-TVL ratios for signs of irrational bidding behavior.

### 6.5 PA-AMM: Partially Active Market Making

The standard LP dilemma is binary: provide liquidity and suffer LVR, or withhold liquidity and earn no fees. The Partially Active AMM (PA-AMM) [100, 149] introduces a continuous tradeoff via a single parameter.

The vault's PA-AMM controller adjusts $\lambda$ dynamically based on the fee/LVR ratio using a proportional controller:

$$\lambda_{t+1} = \text{clamp}\left(\lambda_t + \alpha \times \left(\frac{\text{feeIncome}_t}{\text{LVR}_t} - 1\right), \lambda_{\min}, \lambda_{\max}\right)$$

where $\alpha$ is the learning rate (default: 0.05, providing Maxwell governor damping), and $\lambda_{\min}$, $\lambda_{\max}$ are the tier-bounded limits. When fee income exceeds LVR cost (fee/LVR $> 1.0$), the position is profitable and $\lambda$ increases. When fee/LVR drops below 1.0, the position is losing to adverse selection and $\lambda$ decreases. The 10% maximum adjustment bound per iteration (Section 7.3) applies here as well, preventing oscillatory adaptation. The LVR-informed optimal range width follows:

$$\text{range} = \text{price} \times (1 \pm \sigma \times \sqrt{T} \times \text{multiplier})$$

where $\sigma$ is realized volatility, $T$ is the rebalance horizon, and the multiplier is regime-dependent (1.0 in low-vol, 1.5 in moderate-vol, 2.0 in high-vol).

$\lambda$ values are tier-bounded to prevent inexperienced agents from excessive LVR exposure: `safe-yield` vaults are restricted to $\lambda \in [0, 0.3]$, while `market-making` vaults can use the full $[0.3, 1.0]$ range. Custom templates for Trusted+ agents are unrestricted.

VPIN (Volume-Synchronized Probability of Informed Trading) scores pool toxicity using the Bulk Volume Classification method. VPIN serves as a direct input to both the range width decision and the PA-AMM activeness controller. A pool with VPIN $> 0.7$ and fee/LVR $< 0.8$ signals that the agent is paying more in adverse selection than it earns in fees - a clear trigger to reduce exposure.

Range selection uses a waterfilling Nash equilibrium [76d]: each vault's optimal range depends on the ranges chosen by other LPs in the same pool. The equilibrium converges to a distribution where no LP can improve their expected return by unilaterally changing their range. The PA-AMM activeness parameter adds a second dimension, creating a two-dimensional strategic landscape where LPs jointly optimize range and $\lambda$.

The following table summarizes the risk and reward characteristics of each rehypothecation pattern:

| Pattern | Capital Efficiency | Yield Sources | Primary Risk | Gas Overhead |
|---|---|---|---|---|
| Wrap-and-donate | Moderate | Lending + LP fees (partial) | Withdrawal latency | Low |
| JIT borrowing | Very high (up to 50x) | Lending + LP fees (on-demand) | Borrow rate spike | Moderate |
| Hybrid | Configurable via $\lambda$ | Lending + LP fees (split) | Regime mismatch | Low–moderate |

### 6.6 Rehypothecation via V4

The vault can deploy idle capital to lending protocols (Morpho, Aave, Seamless) during periods of low V4 pool trading volume. The approach follows the EulerSwap model: vault capital sits in lending venues earning supply interest, and when a swap arrives, the hook flash-borrows the required output tokens against the vault's deposited collateral. Because the vault can borrow against its full collateral position, effective liquidity for a single swap can reach up to 50x the vault's own capital, limited only by the lending protocol's LTV ratio. The round-trip (flash-borrow, provide concentrated liquidity, collect fee, repay) executes atomically within a single transaction.

Three deployment patterns offer different tradeoffs:

*Wrap-and-donate.* The adapter monitors which tokens are out of range and deploys them to the highest-yielding lending market. When the LP position moves back into range, the adapter withdraws from lending and re-deploys to the pool. The risk is withdrawal latency under high utilization. A 60% maximum lending exposure cap limits this risk.

*JIT borrowing.* All capital remains in lending venues at all times, with LP liquidity provided only when a swap arrives. This is the most capital-efficient pattern: the vault earns lending yield on 100% of capital and provides LP liquidity on demand. On Base, the additional gas per swap is approximately $0.003, making it viable for trades above roughly $10K notional.

*Hybrid.* A fraction of capital (determined by the PA-AMM $\lambda$ parameter) stays in the pool for instant fee capture, while the remainder earns lending yield. This creates a smooth tradeoff between fee income and lending yield, controllable via a single parameter.

Trotti et al. [40] identify two JIT modes: defensive JIT (reducing impermanent loss by concentrating liquidity in the direction of the incoming swap) and offensive JIT (providing liquidity to capture fees from large swaps). The default is defensive mode, switching to offensive when expected fee revenue exceeds gas cost by a configurable multiple (default: 3x).

The `RehypothecationAdapter` implements a `forceDeallocate()` method guaranteeing non-custodial exits: even if the lending protocol is experiencing high utilization, the vault can force-withdraw by paying an elevated interest rate. This is a safety requirement - vault depositors must always be able to exit, regardless of lending market conditions. The adapter's maximum lending exposure is enforced on-chain by the `PolicyCage`, preventing a compromised agent from deploying 100% of vault capital to a single lending venue.

## 7. Cybernetic Learning Architecture

Autonomous DeFi agents face a fundamental challenge: they must operate in a stochastic, adversarial, and non-stationary environment where the consequences of errors are immediate and financially irreversible. Standard machine learning approaches - reinforcement learning, supervised fine-tuning, gradient-based adaptation - assume access to stable reward signals, representative training distributions, or differentiable objectives. DeFi markets satisfy none of these assumptions reliably. Reinforcement learning requires a stable reward function, but DeFi rewards are non-stationary (fee structures change, liquidity shifts, competitors adapt). Supervised fine-tuning requires representative training data, but DeFi regimes are non-repetitive (the specific conditions of a market crash are never exactly replicated). Gradient-based adaptation requires differentiable objectives, but vault management involves discrete decisions (which pool, which range, whether to rebalance) that do not admit smooth gradients.

This paper argues that cybernetic theory, with its emphasis on feedback regulation, requisite variety, and nested learning loops, provides a more appropriate theoretical foundation for autonomous agent architecture in adversarial financial environments. The cybernetic approach is not a replacement for machine learning - the system uses ML components including Hidden Markov Models for regime classification, embedding models for semantic retrieval, and LLMs for decision-making. Rather, cybernetics provides the *architectural scaffolding* that organizes these components into a coherent self-improving system: feedback loops that correct errors, variety management that scales

with market complexity, and nested adaptation that distinguishes between parameter adjustment and structural change. This section consolidates the learning architecture's theoretical grounding, memory system design, execution pipeline, and honest assessment of limitations.

### 7.1 Theoretical Foundations

The architecture draws on five foundational cybernetic frameworks, each addressing a distinct aspect of autonomous regulation.

Wiener negative feedback (1948). Every heartbeat tick implements a Wiener feedback cycle [4]: desired state (strategy targets) → comparator (escalation gate) → effector (action dispatcher) → plant (market) → sensor (probes) → comparator. The error between desired and actual output drives corrective action. Most ticks are suppressed - negative feedback maintaining homeostasis at zero cost, a principle Cannon [133] identified as the core mechanism of biological self-regulation. This is the simplest and most pervasive loop in the system.

The economic significance of tick suppression is substantial. A naive implementation that invokes the LLM on every tick would incur 1,440 LLM calls per day per strategy (at 60-second intervals). With Tier 1 model costs of $0.01–$0.05 per call, this produces $14–$72 daily operating cost per strategy, making small vault management economically nonviable. The 95% suppression rate reduces this to approximately 72 LLM calls per day, bringing costs below $5 - enabling vault management for vaults as small as $10,000 TVL. The suppression rate is itself a diagnostic signal: if a strategy's suppression rate drops below 80%, it suggests either miscalibrated triggers (too sensitive) or genuinely volatile market conditions (requiring more frequent decisions). The heartbeat pipeline monitors suppression rates as a system health metric.

Maxwell governor damping (1868). Maxwell's analysis of centrifugal governors [55] - the first formal study of feedback control - established that stability requires the controller's response to be proportional to error but damped to prevent oscillation. The scheduler implements Maxwell's governor: the heartbeat interval is the damping factor, and parameter adjustment is bounded at 10% maximum change per iteration, providing explicit damping to prevent oscillatory adaptation.

Ashby's Law of Requisite Variety (1956). The Law of Requisite Variety [43] states that a regulator must possess at least as much variety as the system it regulates, a principle Ashby developed further in *Design for a Brain* [56] through the concept of ultrastability - systems that reorganize their internal structure when simple parameter adjustment fails. The twelve insight categories in the memory schema provide requisite variety for the DeFi domain. As the agent's playbook grows through execution - accumulating heuristics for more market regimes, asset pairs, and conditions - its regulatory capacity increases to match market complexity. This is not a one-time design decision; it is a dynamic property that improves with operational experience.

Beer's VSM also introduces the concept of *algedonic signals* - urgent pain/pleasure signals that bypass the normal management hierarchy. In the vault architecture, portfolio-level circuit breakers (3%/7%/13% drawdown thresholds) function as algedonic signals: they bypass all learning loops, all LLM reasoning, and all playbook heuristics to trigger immediate protective action. This is Kellogg's [68, 117] key insight: the most important signals in an autonomous system are those that short-circuit the normal decision process.

Boyd's OODA Loop (1987). Boyd's Observe-Orient-Decide-Act loop [58], building on the epistemological foundations of *Destruction and Creation* [83] and the strategic framework of *Patterns of Conflict* [135], emphasizes that the Orient phase - the implicit guidance and control that shapes interpretation - is where the real advantage lies. The agent's evolving playbook (Section 7.3) functions as the Orient phase: by modifying the playbook, the agent changes not merely what it observes, but how it interprets observations. An agent with a learned heuristic "Base L2 rebalances during European morning hours exhibit 30% lower slippage" will interpret a 3am UTC price movement differently than an agent without this orientation.

Von Foerster's second-order cybernetics (1979). Von Foerster [6] introduced the principle that the observer is part of the system being observed - a concept he developed as "the cybernetics of observing systems" to distinguish it from first-order cybernetics (the cybernetics of observed systems). The meta-loop curator (Section 7.3) is a second-order observer: it observes the reflector's behavior - how heuristics change over time, which categories accumulate heuristics fastest, where coverage gaps persist - and intervenes at a structural level, reorganizing categories, adjusting promotion thresholds, and identifying coverage gaps. This creates a recursive observation structure: the single loop observes the market, the double loop observes the single loop's behavior, and the meta loop observes the double loop's behavior. Each layer of observation operates at a slower timescale and a higher level of abstraction.

Beer's Viable System Model (1972). Beer's VSM [52] identifies five subsystems necessary for organizational viability, elaborated across *The Heart of Enterprise* [80], the formal VSM methodology paper [81], and the diagnostic companion *Diagnosing the System for Organizations* [82, 145]. Kellogg [68, 117] demonstrated that VSM principles - including POSIWID (the Purpose of a System Is What It Does) and algedonic signals - translate directly to autonomous agent architectures. These map directly to architectural components:

| VSM System | Component | Description |
|---|---|---|
| S1 (Operations) | Heartbeat tick execution | Probe → gate → decision → dispatch |
| S2 (Coordination) | Memory middleware | Coordinates memory retrieval across tool calls |
| S3 (Control) | Risk bounds enforcement | Drawdown limits, stop-losses, portfolio circuit breakers |
| S4 (Intelligence) | Regime classification | Scans external environment for distributional shifts |
| S5 (Policy) | Immutable safety rules | Token allowlists, spending caps, simulation requirements |

Conant-Ashby Good Regulator Theorem (1970). The Good Regulator Theorem [59] states that every good regulator of a system must contain (or be) a model of that system. The agent's playbook is its world model. When the playbook is empty (cold start), the agent has no model and defaults to conservative behavior - correct, because an unmodeled system should not be aggressively regulated. The learning loops (Section 7.3) build this model from experience. The theorem also implies a convergence criterion: a well-functioning agent's playbook should increasingly resemble an accurate model of the specific DeFi markets it operates in, measurable as the prediction accuracy of its pre-trade forecasts against post-trade outcomes.

Lo's Adaptive Markets Hypothesis (2004). The Adaptive Markets Hypothesis (AMH) [54] replaces the Efficient Markets Hypothesis's assumption of perpetual equilibrium with an evolutionary framework: market participants adapt, compete, and are selected by the environment. Market efficiency is not a fixed property but a state variable that fluctuates with the composition and strategies of market participants. The implication for agent design is direct: a strategy that succeeds today may fail tomorrow as other agents adapt. The triple-loop architecture (Section 7.3) implements AMH's prescription - agents must continuously evolve their strategies rather than converging to fixed optima. The regime-adaptive retrieval system (Section 7.5) operationalizes this by blending current parameters with historically successful parameters from analogous regimes, rather than assuming stationarity.

Soros's reflexivity (1987). Soros [66, 134] identified a fundamental feedback loop in financial markets: participants' beliefs influence prices, which in turn influence beliefs. In the context of autonomous agents managing vaults, reflexivity manifests at three levels. First-order reflexivity: the agent's trades move prices in the pools it trades. Second-order reflexivity: the agent's learned heuristics become self-fulfilling (if multiple agents learn "buy at RSI 30," collective buying creates a floor). Third-order reflexivity: the agent's confidence in its own heuristics increases as they appear to work, even when the success is caused by the agent's own actions rather than genuine market signal. Each level is addressed with specific defenses detailed in Section 7.6.

The cybernetic mapping is treated as a design constraint: every architectural decision must trace to a cybernetic concept, and every cybernetic concept must have a concrete implementation. Incomplete mappings identify gaps; contradictory mappings identify design conflicts. Additional influences include Maturana and Varela's autopoiesis [57] - the vault ecosystem as a self-producing network where agent interactions generate the structures that sustain them - and Powers' perceptual control theory [60], which reframes agent behavior as the control of perception rather than the production of output.

This mapping serves as both a design reference and an audit trail: every architectural component traces to a theoretical justification, and every theoretical principle has a concrete implementation.

### 7.2 The DeFi Brain: Memory Architecture

The memory architecture follows the CoALA (Cognitive Architectures for Language Agents) framework [2], which formalizes LLM agents as possessing working, episodic, semantic, and procedural memory - mapping to classical SOAR/ACT-R cognitive architectures. Recent surveys on LLM agent memory [161] confirm the importance of multi-type memory systems, and agentic memory approaches such as A-MEM [162] demonstrate that autonomous memory management outperforms fixed schemas. CoALA is extended here with domain-specific adaptations for DeFi.

The following table maps the complete set of cybernetic influences to their architectural implementations:

| Theorist (Year) | Principle | Architectural Component |
|---|---|---|
| Wiener (1948) [4] | Negative feedback | Heartbeat tick: sensor → comparator → effector |
| Cannon (1932) [133] | Homeostasis | Tick suppression (95% of ticks produce no action) |
| Maxwell (1868) [55] | Governor damping | 10% max parameter change per iteration |
| Ashby (1956) [43] | Requisite variety | 12 insight categories matching DeFi domain variety |
| Ashby (1952) [56] | Ultrastability | ADWIN-triggered regime shift escalation |
| Conant-Ashby (1970) [59] | Good regulator | Playbook as world model (accuracy improves with use) |
| Beer (1972) [52] | Viable System Model | Five-subsystem mapping (S1–S5) |
| Boyd (1987) [58] | OODA Orient phase | Evolving playbook as interpretation context |
| Von Foerster (1979) [6] | Second-order observation | Meta-loop curator observing reflector behavior |
| Maturana-Varela (1980) [57] | Autopoiesis | Vault ecosystem as self-producing network |
| Powers (1973) [60] | Perceptual control | Agent controls perception, not output directly |
| Lo (2004) [54] | Adaptive Markets | Continuous strategy evolution, no fixed optima |
| Soros (1987) [66] | Reflexivity | Three-level reflexivity defense (Section 7.6) |
| Argyris-Schon (1978) [5] | Double-loop learning | Playbook heuristic evolution |
| Dabney et al. (2020) [50] | Distributional reward coding | Asymmetric confidence updates ($-0.15/+0.10$) |
| Zhang et al. (2026) [63] | ACE Generator-Reflector-Curator | Triple-loop GottsLoop architecture |
| Shinn et al. (2023) [42] | Reflexion | Verbal self-reflection after each tick |
| Zhao et al. (2024) [3] | ExpeL cross-task transfer | Insight distillation from episode clusters |
| Rescorla-Wagner (1972) [148] | Associative learning | Confidence update rule (base model) |

Working memory holds the active context for the current heartbeat tick: strategy parameters, probe results, escalation reason, and the sliding window of recent conversation turns. Working memory is ephemeral and reconstructed each tick from persistent stores and current observations. The context builder assembles working memory from five sources: (1) the current strategy definition (STRATEGY.md, compiled to runtime parameters), (2) probe results from the current tick, (3) the escalation reason (which trigger fired and why), (4) the most relevant episodic memories retrieved via semantic similarity, and (5) the current playbook heuristics applicable to this strategy's category and the current market regime. Working memory is bounded by the LLM's effective context window; the context builder prioritizes high-importance items and truncates low-impact historical data when the total exceeds 80% of available context.

Episodic memory stores raw records of individual DeFi operations in a LanceDB vector database. Each episode is a structured record containing: the operation type (deposit, rebalance, collect_fees, withdraw, emergency_exit), pre-state snapshot (NAV per share, total assets, all LP position snapshots with tick ranges and uncollected fees, regime classification, VPIN score, gas price, block number), post-state snapshot (NAV per share, total assets, updated positions, gas used), predicted versus actual outcomes, a verbal self-reflection, confidence score, P&L impact in basis points, and the HMM-detected regime tag at operation time. Episodes are embedded using nomic-embed-text-v1.5 on the concatenation of operation type, regime state, and reflection text, enabling semantic queries such as "find past rebalance episodes during bear_high_vol regimes with VPIN above 0.7." Episodic memory implements the Reflexion pattern [42]: after each operation, the agent generates a verbal self-critique ("This trade achieved 8 bps slippage versus the predicted 12 bps - the UniswapX route provided MEV protection worth approximately 3 bps"). These reflections, stored as episodes, enable pattern recognition across operations. A ground-truth backcheck validates each reflection's causal claim against observed on-chain data: if the claimed cause cannot account for more than 50% of the NAV change, the reflection is flagged as uncertain, preventing confabulated causal reasoning from entering the memory store [126].

Insights with confidence below 0.3 after 60 days without supporting evidence are archived - not deleted, as they may become relevant again during future regime shifts. The insight storage uses SQLite for structured queries (filter by category, confidence threshold, creation date, regime tag) while episodes use LanceDB for vector-similarity search. This dual-store architecture reflects the different access patterns: episodic memory

Semantic memory stores distilled insights extracted from clusters of episodes via the ExpeL (Experiential Learning) pattern [3]. Periodically, the consolidation loop clusters episodes by tool, chain, and token pair, then extracts durable generalizations: "ETH/USDC pools on Base exhibit lower slippage during Asian trading hours." Each insight carries a confidence score $c \in [0, 1]$ and rich metadata including creation time, trigger count, success rate, and regime context. Insights are organized into twelve categories providing the requisite variety discussed in Section 7.1:

| Category | Example Insight |
|---|---|
| Gas timing | "Base L2 gas below 0.01 gwei 02:00–06:00 UTC" |
| Slippage patterns | "UniswapX routes save 3–5 bps on ETH/USDC above $100K" |
| Liquidity conditions | "TVL below $100K increases slippage 2–3x" |
| Regime transitions | "Bull-to-bear transitions take 48–72 hours on ETH" |
| Risk events | "Circuit breaker triggered at 7% drawdown on vault 0xABC" |
| Rebalance timing | "Rebalancing during Asian hours: 30% lower slippage" |
| Range selection | "Tight ranges (± 2%) outperform wide during low-vol" |
| Fee optimization | "Collect fees every 4 hours in high-vol, every 24 in low-vol" |
| JIT defense | "VPIN above 0.7: reduce exposure within 2 ticks" |
| Tool guidance | "Use execute_swap with UniswapX route for orders above $100K" |
| Procedural sequences | "Deposit → wait 3 blocks → verify NAV → deploy to LP" |
| Strategic patterns | "Mean reversion works on ETH/USDC; momentum works on ETH/BTC" |

is queried by semantic similarity ("find similar past experiences"), while semantic memory is queried by structured predicates ("all high-confidence insights for gas timing on Base"). The stores are co-located on the same filesystem, ensuring atomic consistency between episode capture and insight updates.

Procedural memory encodes the agent's playbook - the evolving set of heuristics that guide strategy execution. Unlike episodic and semantic memory, which are populated automatically, the playbook is structured by the triple-loop learning architecture (Section 7.3) and carries explicit confidence tiers. The playbook is a persistent PLAYBOOK.md file - a living document that the LLM reads at the start of every tick and that the reflector updates after every outcome. The playbook format uses structured entries with metadata: each heuristic carries an identifier, verbal description, confidence score, creation tick, last triggered tick, trigger and success/failure counts, and helpful/harmful counters. This metadata enables the drift prevention rules described in Section 7.3.

Memory retrieval pipeline. When a tick escalates to the LLM, the memory middleware assembles relevant context through a four-stage retrieval pipeline:

*Stage 1 (Regime-indexed episodic retrieval).* The current regime tag and operation type are used to query LanceDB for the $k$ most semantically similar past experiences (default $k = 5$). The query embedding is computed from the concatenation of operation type, regime state, and a summary of current probe results. Similarity is measured using cosine distance on the nomic-embed-text-v1.5 embeddings. Episodes from the current regime are boosted by a 1.5× relevance multiplier; episodes from different regimes are included only if no regime-matched episodes exist (cold start for the current regime).

*Stage 2 (Semantic insight retrieval).* High-confidence semantic insights matching the current insight category and regime are retrieved from the SQLite insight store. Insights are ranked by a composite score: rank = $c \times$ successRate $\times$ recencyWeight, where $c$ is the insight's confidence, successRate is its historical accuracy, and recencyWeight decays exponentially from the last update time. The top 10 insights are included in the context.

*Stage 3 (Playbook injection).* The full PLAYBOOK.md is included in the LLM's system prompt. Strategic principles ($c \geq 0.85$) appear in a "Strategic Principles" sec-

The importance score determines the effective decay rate through a conditional multiplier:

| Impact Tier | Condition | Decay Multiplier | Effective Retention |
|---|---|---|---|
| High-impact | $\lvert\text{plImpactBps}\rvert > 100$ | 5× | ≈ 900 days |
| Medium-impact | $50 \leq \lvert\text{plImpactBps}\rvert \leq 100$ | 2× | ≈ 360 days |
| Low-impact | $\lvert\text{plImpactBps}\rvert < 50$ | 1× (standard) | 7–14 days (category-dependent) |
| Emergency | Circuit breaker, exploit pattern | ∞ (pinned) | Minimum 180 days, never auto-pruned |

tion; tactical heuristics ($0.5 \leq c < 0.85$) appear in a "Tactical Heuristics" section; candidates ($c < 0.5$) are excluded from the context to prevent low-confidence heuristics from influencing decisions. The playbook section consumes approximately 15–20% of the available context window.

*Stage 4 (Context assembly and truncation).* The context builder assembles all retrieved memories into the final system prompt, prioritizing by importance: (1) current probe results (always included in full), (2) playbook heuristics (always included), (3) high-confidence insights (included until context limit), (4) episodic memories (truncated if necessary). If the total context exceeds 80% of the LLM's effective context window, low-importance items are truncated. The truncation order is: oldest episodic memories first, then lowest-confidence insights, then tactical heuristics. Strategic principles and probe results are never truncated.

The pipeline draws on advances in retrieval-augmented generation: HippoRAG [139] for neurobiologically inspired long-term retrieval (the regime-indexed tagging mirrors hippocampal indexing of contextual memories), Self-RAG [140] for self-reflective retrieval quality assessment, CRAG [141] for corrective retrieval that detects and recovers from poor retrievals, and Anthropic's contextual retrieval [144] for chunk-level context augmentation that reduces retrieval failures. The assembled context is injected into the LLM's system prompt via the middleware, ensuring that the agent's decisions are informed by the full weight of its accumulated experience.

Importance-weighted retention. Time-based Ebbinghaus decay is rejected for financial memory. The standard decay function $R = e^{-t/S}$ (where $R$ is retention, $t$ is time since last access, and $S$ is stability) prunes knowledge indiscriminately by recency. In DeFi, a circuit breaker trigger from six months ago is more valuable than a routine gas observation from yesterday. The retention score combines P&L impact and access frequency:

$$\text{accessComponent} = \min(0.3, \text{accessCount} \times 0.05)$$

This follows the FinMem architecture's tiered retention design. Emergency events are pinned with a minimum 180-day stability period because they contain information about rare but catastrophic scenarios - exactly the kind of knowledge that time-based decay would destroy. Critically, decay does not mean deletion: decayed episodes are archived to cold storage, not hard-deleted, and remain available for regime-conditional recall. FadeMem [49] (the closest published system to this design) achieves 82.1% retention of critical facts using 55% of storage via adaptive decay, confirming that selective forgetting is essential to prevent information overload. Complementary memory optimization approaches include UMEM [85, 137] for unified memory training, MemoRAG [160] for memory-inspired knowledge discovery, and MemoTime [142] for temporal knowledge graph memory in long-horizon planning.

Drift detection. Regime detection uses a two-layer approach. The first layer is a four-state Hidden Markov Model (Koki et al. [178]) classifying market conditions into bull_low_vol, bull_high_vol, bear_low_vol, and bear_high_vol, evaluated via CRPS and MSFE on BTC, ETH, and LTC. The HMM produces a probability vector $[p_1, p_2, p_3, p_4]$ summing to 1.0, representing the agent's belief distribution across regimes. The second layer is ADWIN (Adaptive Windowing) [53], which maintains running statistics over a sliding window of observed metrics (realized volatility, VPIN, fee/LVR ratio, and gas prices). ADWIN dynamically grows and shrinks the window: when Hoeffding's bound detects a statistically significant distributional shift ($p < 0.01$) between the window's older and newer halves, ADWIN triggers escalation.

Upon escalation, four actions occur: (1) stale insights (those created under the previous regime and not yet validated under the current regime) are downweighted by 0.05 confidence per consolidation cycle; (2) the agent retrieves historically successful parameters for the detected regime via the regime-adaptive retrieval system (Section 7.5); (3) the playbook's regime-specific heuristics are re-evaluated - heuristics marked for a different regime are suppressed, and regime-matched heuristics are activated; and (4) the escalation is logged as an episodic memory tagged

with the regime transition, building a corpus of transition experiences for future reference. This implements Ashby's ultrastability [43]: when inner-loop parameter adjustments are no longer sufficient (the distribution has shifted), the system switches to the outer loop (re-evaluating its knowledge base). The boundary between "keep adjusting parameters" and "change the rules" is detected statistically, not heuristically - a critical distinction, since heuristic boundaries are brittle and regime-specific. The Markov-switching GARCH model [123] provides the volatility regime classification, while ADWIN provides the change-point detection; neither alone is sufficient.

Cross-vault memory isolation. Vault A's strategy reflections are invisible to Vault B's decision context. This isolation is enforced at the memory middleware layer: each vault's episodic and semantic memory stores are keyed by vault address, and the retrieval pipeline filters by vault before assembling context. The isolation boundary extends to the playbook: each vault maintains its own PLAYBOOK.md with heuristics specific to its strategy, asset pair, and operating history. A heuristic learned by a WBTC/USDC vault ("rebalance threshold should be 8% during high-vol on WBTC pairs") does not contaminate an ETH/USDC vault's playbook.

However, for agents at Trusted tier and above (see Section 4.2), anonymized aggregate insights are shared across vaults. The aggregation strips vault-specific details (addresses, position sizes, depositor identities) while preserving generalizable patterns (regime classifications, asset-class-level observations, gas timing insights). This enables collective learning without compromising individual vault privacy. The approach maps to PATE (Private Aggregation of Teacher Ensembles) [167]: each vault is a "teacher" that votes on regime classification, and the ensemble vote is the only output visible to participants. The privacy guarantee is formal: no participant can infer another vault's position sizes, rebalance timing, or strategy parameters from the shared aggregate signals.

Federated regime signal gossip. Vaults share four-float probability vectors representing regime classifications, not model gradients or raw data:

$$\mathrm{r}_i = [p_{\text{bull_low}}, p_{\text{bull_high}}, p_{\text{bear_low}}, p_{\text{bear_high}}] \quad \text{where} \sum_j p_j = 1.0$$

Each participating vault contributes its local regime classification $\mathrm{r}_i$ to the gossip protocol. This sidesteps the fundamental problems of federated learning in adversarial financial environments: gradient inversion attacks (impossible - no gradients are shared), non-IID data distributions (irrelevant - only classification outputs are aggregated), and strategic incentive to poison shared models (mitigated by Byzantine-robust aggregation). Differential privacy with Balle and Wang's analytical Gaussian calibration [168] adds calibrated noise: each component $p_j$ is perturbed by $\mathcal{N}(0, \sigma^2)$ where $\sigma$ is calibrated to achieve the target $\varepsilon$ level (recommended range: 2.0–4.0). Below $\varepsilon = 2.0$, noise overwhelms the signal at realistic agent counts; above $\varepsilon = 4.0$, privacy guarantees become insufficient for agents managing sensitive strategy information.

The aggregation uses the geometric median (Pillutla et al. [169]) rather than weighted arithmetic mean. The geometric median minimizes the maximum influence of any single participant, tolerating up to 50% Byzantine contributors without degradation. This is critical: in a permissionless system, any agent can contribute regime signals, and a malicious agent could submit deliberately misleading classifications to cause other agents to adopt unfavorable parameters. The geometric median ensures that a minority of adversarial signals cannot shift the consensus. The protocol produces meaningful signal at 10+ contributing agents; below 5, the ensemble offers no improvement over a single well-calibrated classifier. Alpha decay pricing applies to regime signals sold via the x402 marketplace: Maven Securities' research shows alpha decays at 9.9% per unit of delay in European markets and 5.6% in US markets, establishing the economic basis for real-time signal pricing.

### 7.3 GottsLoop: Triple-Loop Learning

The memory architecture described above provides the substrate - storage, retrieval, and organization of learned knowledge. GottsLoop provides the *process* - the cybernetic execution loop that reads from memory, acts on the market, reflects on outcomes, and writes improved knowledge back to memory. Together, the DeFi Brain (memory) and GottsLoop (process) form a complete cognitive system: the brain stores what the agent knows, and the loop determines how the agent learns.

GottsLoop is the core cybernetic execution system. It wraps the heartbeat pipeline (Section 7.4) with an evolving playbook that modifies the agent's reasoning context over time. The key architectural difference from standard memory-augmented agents is that GottsLoop feeds outcomes back into the LLM's *reasoning context*, not just tool parameters. Standard memory middleware improves *how* tools execute (slippage tolerance, timing, route preferences); GottsLoop improves *what the LLM decides to do* by modifying the strategic context it reasons within. This distinction maps directly to Argyris and Schon's organizational learning theory [5]:

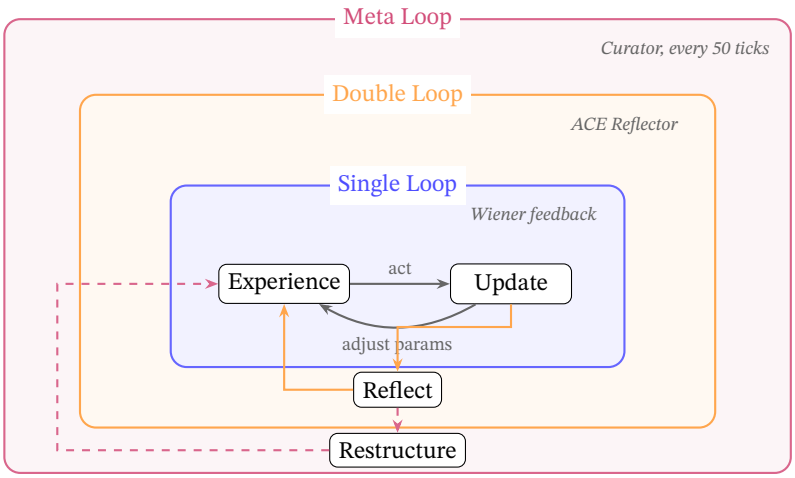


Figure 7: GottsLoop triple-loop learning architecture. The inner loop adjusts parameters (Wiener feedback), the double loop evolves heuristics (ACE Reflector), and the meta loop restructures the playbook (Curator, every 50 ticks).

The data flow per tick follows the ACE Generator-Reflector-Curator cycle [63]: (1) Load state - read state.json, PLAYBOOK.md, HEARTBEAT.md, and session from the JSONL transcript. (2) Model routing - the model router selects the appropriate LLM tier based on probe results (Tier 1 for routine, Tier 2 for elevated, Tier 3 for critical; see Section 7.4). (3) Build context - the context builder assembles the system prompt from base instructions, current playbook heuristics, active strategy summaries, recent outcomes, session history, and standing orders from HEARTBEAT.md. This is the ACE Generator's input preparation, and it implements Ashby's requisite variety: the prompt's variety must match the market's variety. (4) Probe and gate - for each active strategy, run probes and evaluate escalation (see Section 7.4). (5) LLM decision - the ACE Generator produces the action and prediction. (6) Reflect - the ACE Reflector produces delta entries (ADD, STRENGTHEN, WEAKEN, REMOVE, REFINE) based on the outcome. (7) Persist - write updated state.json, append to PLAYBOOK.md, log to transcript.jsonl, and store episodes to LanceDB. Steps 1–7 constitute a single tick. The architecture implements three nested learning loops:

Single loop (parameter adjustment). The memory middleware adjusts tool parameters based on retrieved insights within the current strategy. "Use 35 bps slippage tolerance on ETH/USDC on Base during European morning hours." The strategy does not change; only execution parameters adapt. This is Wiener feedback: correcting deviation from a fixed reference point. Parameters tuned by the single loop include slippage tolerance, gas price limits, rebalance thresholds, range tightness, fee collection frequency, and position sizing. The single loop operates on every tick (including suppressed ticks, where the parameter adjustments are recorded but no action is taken), with a convergence constraint: no single parameter may change by more than 10% per iteration, and the cumulative change across all parameters is bounded at 20% per tick. These bounds implement Maxwell governor damping at the parameter level, preventing oscillatory adaptation even when the feedback signal is noisy. For a typical strategy with 60-second heartbeats, the single loop completes approximately 1,440 iterations per day, of which approximately 95% are no-ops (the retrieved insights confirm current parameters are adequate).

Double loop (heuristic evolution). After each non-suppressed tick, the ACE Reflector [63] updates playbook heuristics via delta entries rather than monolithic rewrites - a critical architectural choice, since monolithic playbook rewrites cause "context collapse" where previously learned heuristics are lost [63]. Each delta carries an operation type (ADD, STRENGTHEN, WEAKEN, REMOVE, REFINE), helpful/harmful counters for quality tracking, and a verbal description following FinCon's insight [62] that LLMs process verbal reinforcement more effectively than numerical reward signals. Example delta: "RSI $< 30$ is not a reliable buy signal during high-volatility regimes - add a realized-volatility filter." The strategy reasoning itself changes.

The reflector follows the ACE Generator-Reflector-Curator cycle (Zhang et al., ICLR 2026 [63]): the generator (tick execution) produces experience, the reflector evaluates outcomes and produces delta entries, and the curator (meta loop) integrates deltas incrementally. The reflector's output is a structured delta, not a revised playbook. This incremental approach preserves the full history of heuristic evolution and enables rollback to any previous playbook state.

Each delta entry contains: an operation type (ADD, STRENGTHEN, WEAKEN, REMOVE, REFINE), the target heuristic identifier (for STRENGTHEN, WEAKEN, REMOVE, REFINE) or a new heuristic description (for ADD), a confidence change value (+0.10 for positive outcomes, −0.15 for negative), helpful/harmful counters tracking how often this delta's associated heuristic led to good or bad outcomes, and a verbal explanation following FinCon's insight [62] that LLMs process verbal reinforcement more effectively than numerical reward signals. The verbal explanation is critical: "RSI $<$ 30 is not a reliable buy signal during high-volatility regimes - add a realized-volatility filter" communicates more actionable information to the LLM than a numerical signal of $\Delta = -0.15$. The reflector processes deltas in order (REMOVE first, then WEAKEN, then REFINE, then STRENGTHEN, then ADD) to ensure that removals take effect before additions, preventing the playbook from growing unboundedly during a single reflection cycle.

Meta loop (structural adaptation). Every 50 ticks, the ACE Curator restructures the playbook: promoting tactical heuristics with confidence $\geq 0.85$ to strategic principles, pruning heuristics with confidence $< 0.2$, deduplicating overlapping rules, and identifying coverage gaps. The curator implements von Foerster's second-order observation - it observes the reflector's behavior and intervenes at a structural level. Whether the meta loop produces measurably better outcomes than simple double-loop learning is an empirical question that has not yet been answered at scale. The curator's intervention criteria are: (1) promotion when a heuristic has been triggered 10+ times with $>$ 70% success rate and confidence $\geq 0.85$; (2) demotion when a strategic principle's confidence drops below 0.70 (two consecutive negative outcomes from strategic status); (3) deduplication when two heuristics have $>$ 80% semantic similarity and overlapping trigger conditions; (4) gap identification when an insight category has no heuristics but the agent has operated in conditions where that category would be relevant.

The curator runs on a frontier model (Tier 3), because its decisions are structural - they reshape the playbook that governs all future decisions. A poor promotion decision propagates through every subsequent tick. The curator is the most expensive per-invocation component of the system (approximately \$1–\$5 per run), but it executes only every 50 ticks (approximately once every 50 minutes for a 60-second heartbeat), making its amortized cost negligible. The curator also enforces the capacity bound: when the playbook reaches 30 active heuristics, the curator must remove or archive an existing heuristic before adding a new one. This forces the system to make explicit priority decisions rather than accumulating an unbounded collection of observations. Following GEPA's Pareto-front selection [64], the curator retains the Pareto-optimal set of heuristics - those that are not dominated on any combination of confidence, success rate, and trigger frequency.

Candidate heuristics are tracked but never influence actual decisions until they accumulate sufficient evidence for promotion (5+ shadow ticks with $>$ 50% success rate). This shadow execution mechanism addresses ExpeL's documented collapse on rare-success tasks [46] by enabling learning from candidates that would otherwise be pruned prematurely. Shadow execution works as follows: when the LLM makes a decision, the candidate heuristic's recommendation is computed in parallel but not acted upon. The actual outcome is then compared to what the candidate would have recommended. If the candidate's recommendation would have produced a better outcome, the candidate's helpful counter increments; if worse, the harmful counter increments. After 5+ shadow ticks, the promotion/rejection decision is made based on the accumulated evidence. This mechanism is computationally cheap (no additional LLM calls) and provides a zero-risk evaluation pathway for new heuristics.

The shadow execution mechanism also addresses the "exploration-exploitation" dilemma inherent in playbook evolution: the agent must balance exploiting known-good heuristics (strategic principles) with exploring potentially better alternatives (candidates). By evaluating candidates in shadow mode, the system explores without risking capital - the equivalent of paper trading for heuristics. This is a direct application of Thompson sampling's principle of maintaining optimism in the face of uncertainty, but applied to heuristic confidence rather than action-value estimates.

Confidence update asymmetry. Following Dabney et al.'s distributional reward coding [50] - the discovery that dopamine neurons encode a distribution of value expectations with asymmetric responses to positive and negative prediction errors, building on the classical Rescorla-Wagner learning model [148] - confidence updates are deliberately asymmetric:

$$c_{\text{new}} = \text{clamp}(c_{\text{old}} + \Delta,\ 0,\ 1)$$

A single negative outcome from strategic confidence drops the heuristic back to tactical: $0.85 - 0.15 = 0.70$. Recovery requires two consecutive positive outcomes.

The playbook is organized into three confidence tiers following SCOPE's dual-stream architecture [65], extended with a third tier for candidate heuristics:

| Tier | Confidence | Status |
|---|---|---|
| Strategic Principles | $\geq 0.85$ | Stable, validated across multiple contexts |
| Tactical Heuristics | 0.5–0.85 | Volatile, under active evaluation |
| Candidates | $< 0.5$ | Shadow-executed, do not influence decisions |

where $\Delta = +0.10$ for positive outcomes and $\Delta = -0.15$ for negative outcomes. The pessimistic channel dominates, creating a system structurally biased toward caution. The following table traces the promotion path from candidate to strategic principle:

| Starting Confidence | Required Positive Streak | Outcomes (P=positive, N=negative) | Final Confidence |
|---|---|---|---|
| 0.50 (neutral) | 4 consecutive | P, P, P, P (no negatives) | 0.90 (strategic) |
| 0.50 (neutral) | 7 with 2 negatives | P, P, N, P, P, P, N, P, P | 0.85 (strategic) |
| 0.85 (strategic) | 1 negative | N | 0.70 (demoted to tactical) |
| 0.70 (tactical) | 2 consecutive | P, P | 0.90 (re-promoted) |
| 0.20 (near-prune) | 1 negative | N | 0.05 (archived by curator) |

Playbook drift prevention. Every heuristic carries metadata enabling drift detection: creation tick, last triggered tick, trigger count, success/failure counts, and helpful/harmful counters. Five drift rules operate:

| Drift Rule | Condition | Action |
|---|---|---|
| Idle decay | $> 30$ days untriggered | $-0.1$ confidence per consolidation cycle |
| Underperformance | $< 40\%$ success after 10+ triggers | Flagged for curator review |
| Insufficient evidence | High confidence but $< 10$ triggers | Held at tactical status until evidence accumulates |
| Contradiction | Two heuristics with conflicting recommendations | Curator retains higher success rate, archives other |
| Regime staleness | Created in regime $X$, not validated in current regime $Y$ | $-0.05$ confidence per consolidation cycle |

This asymmetry is appropriate in DeFi, where losses are more consequential than missed gains. The ratio of $-0.15$ to $+0.10$ (1.5:1) is deliberately less extreme than some biological systems (Dabney et al. report ratios up to 3:1 in dopaminergic neurons) because DeFi outcomes are noisier than the controlled experiments in which distributional reward coding was observed - a higher asymmetry ratio would make it nearly impossible for any heuristic to reach strategic status in a noisy environment.

The asymmetry also provides a natural defense against the "lucky streak" problem: a heuristic that succeeds five times in a rising market (and would have succeeded regardless of the agent's actions) reaches only $c = 1.0$ and requires only two negative outcomes to drop back to tactical ($1.0 \rightarrow 0.85 \rightarrow 0.70$). This prevents false confidence from accumulating during unidirectional market movements.

Additionally, a maximum of three rule changes per consolidation cycle prevents cascading playbook instability. Each consolidation cycle produces a diff (added, modified, removed heuristics) that is logged to the PLAYBOOK.log file, providing a complete audit trail of playbook evolution. The log enables rollback to any previous playbook state, which is critical during regime transitions where rapid playbook changes might overcorrect.

A mandatory buy-and-hold comparison provides the ultimate drift detection: if the agent's strategy underperforms simple buy-and-hold of the underlying assets for more than two consecutive weeks, the meta-loop curator initiates a comprehensive review. This comparison is the most important single check in the system, because it catches all forms of drift simultaneously - a drifting playbook will eventually manifest as underperformance relative to the simplest possible baseline.

Capacity bounds. The playbook is capped at 30 active heuristics. The curator prunes aggressively to prevent context window bloat and maintain signal quality. This bounded capacity, combined with the confidence-tiered promotion system, ensures that the playbook converges toward a compact, high-quality

world model rather than accumulating unbounded observations.

STRATEGY.md format. Strategies are authored as markdown files with YAML frontmatter, following the pattern established by Anthropic's SKILL.md (Agent Skills, 2025) and AWS Strands SOPs (Nov 2025 [67]). The markdown is the source of truth; runtime JSON is a compiled artifact. The format uses RFC 2119 keywords (MUST, SHOULD, MAY) with enforcement-level mapping: MUST constraints compile to deterministic pre-LLM guards in the escalation gate and PolicyCage; SHOULD constraints are injected into the LLM's system prompt as guidance; MAY constraints are advisory. Each strategy file contains sections for schedule (temporal bounds, active windows, cron triggers), action (what to do when triggered), constraints (RFC 2119-annotated), risk bounds (drawdown, stop-loss, slippage, allowed tokens), and completion conditions (budget exhaustion, consecutive losses, idle timeout). Strategies live in git-versioned directories, are diffable and reviewable, and can be shared as plain files or installed from GitHub repositories via a tap system. The compilation step from STRATEGY.md to runtime parameters is deterministic and auditable - the same markdown always produces the same JSON.

The STRATEGY.md format also supports regime-specific behavior sections. A strategy can define different parameters for different market regimes: for example, a delta-neutral LP strategy might specify "During HIGH_VOLATILITY (30d vol > 55%): widen rebalance threshold from ±5% to ±8%, reduce position size by 30%, increase heartbeat interval to 5 minutes." These regime-specific overrides are compiled to conditional rules in the runtime JSON and evaluated at the escalation gate (Stage 3 of the heartbeat pipeline). When the HMM regime classifier detects a transition to HIGH_VOLATILITY, the gate automatically applies the regime-specific parameters without LLM involvement. This design separates regime-adaptive strategy parameters (deterministic, compiled from markdown) from playbook-driven adaptation (LLM-mediated, evolved from experience).

HEARTBEAT.md standing orders. An operator-authored markdown checklist of directives, read by the agent every 6 heartbeat ticks (or 30 minutes, whichever comes first). Standing orders contain monitoring directives ("Monitor ETH/USDC pool depth - alert if TVL drops below $500k"), override rules ("Pause all strategies if gas > 50 gwei on L1"), and operational notes ("Base sequencer was down 2026-02-20 03:00–03:45 UTC. If it happens again, pause market-making strategies for 1 hour after recovery"). The critical distinction from PLAYBOOK.md: HEARTBEAT.md is human-authored and checked into version control; PLAYBOOK.md is machine-evolved and gitignored. Standing orders provide the operator with a persistent, legible channel for directing agent behavior without modifying strategy definitions or playbook heuristics. Completed directives are marked with a checkbox; the agent does not re-process completed items.

### 7.4 Heartbeat Pipeline

The heartbeat pipeline is the concrete execution engine that implements the cybernetic loops in practice. Its key design property is that most ticks never reach the LLM.

The pipeline proceeds through five stages:

Stage 1 (Scheduler). Evaluates whether a heartbeat tick is due based on two inputs: the heartbeat timer (configurable interval, default 60 seconds) and strategy-specific cron triggers (e.g., "Every Monday at 03:00 UTC" for a DCA strategy). Temporal bounds from the STRATEGY.md definition are checked first - zero-cost rejection of out-of-window ticks. If the current time falls outside all active windows, the tick is suppressed without executing any probe or LLM call. Temporal enforcement is a safety property (see Section 8.4), not merely a scheduling feature: even a fully compromised LLM cannot execute outside defined active windows. The scheduler also checks the file-based kill switch at this stage, providing the sub-100ms halt capability.

Stage 2 (Probe Runner). Cheap, deterministic on-chain reads executed without LLM involvement. Probes are strategy-specific and defined in the STRATEGY.md trigger section. Common probes include: current price (from Uniswap V4 pool's sqrtPriceX96), RSI and moving averages (computed from cached OHLCV data), position delta (from the vault's LP positions), VPIN score (from the Bulk Volume Classification method), gas price, and pool TVL. Probes run in parallel via `Promise.allSettled`; failed probes are recorded but do not halt the pipeline (circuit breaker pattern). Cost: $0. Probes are the system's sensory apparatus - they provide the raw data that the escalation gate and LLM use to make decisions.

Stage 3 (Escalation Gate). A five-phase deterministic filter that determines whether the tick warrants LLM involvement. The phases execute in order, and any phase can suppress the tick:

(0) *Temporal bounds*: verify active window (redundant with Stage 1, defense-in-depth). This redundancy is deliberate: if Stage 1's scheduler has a bug that allows an out-of-window tick

through, Stage 3 catches it independently. (0b) *Stop conditions*: consecutive loss count (default: 3 consecutive losses triggers pause), idle timeout (strategy paused if no action for configurable duration), cumulative cost cap (LLM inference + gas costs), and maximum execution count.

(1) *Circuit breakers*: three-tier portfolio-level drawdown check. At 3% drawdown, the strategy reduces position sizes by 50%. At 7% drawdown, all non-essential actions are suppressed (only fee collection and emergency exits allowed). At 13% drawdown, all positions are closed and the strategy transitions to emergency state. These thresholds are hard-coded in the escalation gate, independent of any LLM reasoning - they cannot be overridden by a compromised playbook or a persuasive LLM output. Daily spend limits and per-strategy drawdown are also checked at this phase.

(2) *Stop-loss check*: if the strategy's cumulative P&L has breached the stop-loss threshold defined in STRATEGY.md, the strategy transitions to `completed` state and no further actions are taken.

(3) *Completion condition check*: budget exhaustion (total USD spent >= budget), target accumulation reached (accumulated tokens >= target), or maximum duration elapsed (current time >= endAt).

(4) *Trigger evaluation*: probe results are compared against the strategy's defined triggers (e.g., "RSI(14) crosses below 30," "position delta exceeds 5%," "cron: Monday 03:00 UTC"). Triggers are compiled from the STRATEGY.md at activation time and evaluated deterministically - no LLM involvement. Only if a trigger fires and no safety condition in phases 0–3 blocks does the tick escalate.

Under typical conditions, approximately 95% of ticks are suppressed at this stage. The escalation gate is entirely deterministic - no LLM, no randomness, no external API call. This means the dominant cost factor (LLM inference) is incurred only when the escalation gate determines that a substantive decision is required.

Stage 4 (LLM Decision Engine). The LLM receives an assembled context containing: the strategy definition, probe results, memory context (episodic, semantic, and procedural memories retrieved by the `withMemory` middleware), working state, the escalation reason (which trigger fired and why), and the current playbook heuristics. Following the DeLLMa framework for decision making under uncertainty with LLMs [126, 147], it selects an action from the available MCP tools, specifies parameters, and makes a prediction for later comparison (e.g., "I expect this rebalance to reduce delta from 7% to under 2% with slippage below 15 bps"). The prediction is stored alongside the action for the self-improvement pipeline's prediction comparison step (Section 7.5). Cost-optimized model routing (see the model routing table above) directs approximately 95% of escalated ticks to a lightweight model, approximately 4% to a mid-tier model, and approximately 1% (novel conditions, drawdown events, post-mortem analysis) to a frontier model. The LLM call uses an AbortController, enabling the steer mechanism to abort in-flight calls when operator messages arrive.

Stage 5 (Action Dispatcher). Routes the action through a six-step safety pipeline before broadcast: (1) token allowlist check (is the target token in the strategy's approved list?), (2) spending limit check (does this transaction exceed per-tx, per-session, or per-day limits?), (3) pre-flight simulation via `eth_call` fork simulation (does the simulated outcome match expectations within tolerance?), (4) safety validation by the safety-guardian agent (does this action comply with all safety rules?), (5) broadcast (submit the transaction), and (6) post-trade verification (compare the on-chain receipt against the pre-flight simulation and the LLM's prediction). If any step fails, the action is rejected and the failure is recorded as an episodic memory for future learning.

For a conservative strategy (DCA, safe-yield) where Tier 3 routing is rare, daily costs are under $5. For an aggressive market-making strategy with frequent regime changes, daily costs may reach $10–$15. The cost cap in the escalation gate (default: $50/day per strategy) provides a hard upper bound. These costs are 10–100x lower than equivalent human-managed operations, and 3–5x lower than naive LLM agent architectures that invoke the model on every tick.

The routing decision is deterministic and occurs before the LLM is invoked: probe results, current drawdown, regime stability, and escalation reason are evaluated against the routing table. No LLM call is needed to decide which model to use. This follows the TradingAgents architecture [170]: lightweight models for routine summarization, frontier models for reasoning-intensive decisions.

Daemon architecture. GottsLoop runs as a conversational daemon built on a five-layer model. Layer 1 (IPC): a Unix domain socket gateway connects the CLI to the daemon process; all communication uses JSON-RPC 2.0 over newline-delimited JSON. Layer 2 (Scheduling): a heartbeat timer and cron-based strategy triggers generate periodic events. Layer 3 (Execution): the Lane Queue, which is the central primi-

This five-stage design yields substantial cost efficiency. The dominant cost factor - LLM inference - is incurred only when the escalation gate determines that a substantive decision is required. The following table summarizes the cost model for a typical strategy with a 95% suppression rate and a 60-second heartbeat interval:

| Component | Per-Tick Cost | Daily Volume | Daily Cost |
|---|---|---|---|
| Stages 1–3 (deterministic) | $0 | 1,440 ticks | $0 |
| Stage 4 Tier 1 (lightweight) | $0.01–$0.05 | ≈ 68 ticks | $0.68–$3.40 |
| Stage 4 Tier 2 (mid-tier) | $0.10–$0.50 | ≈ 3 ticks | $0.30–$1.50 |
| Stage 4 Tier 3 (frontier) | $0.50–$5.00 | ≈ 1 tick | $0.50–$5.00 |
| Stage 5 (gas, Base L2) | $0.003–$0.03 | ≈ 10 actions | $0.03–$0.30 |
| Reflection (post-tick) | $0.01–$0.05 | ≈ 72 ticks | $0.72–$3.60 |
| Total | | | $2.23–$13.80 |

Cost tracking and model routing. Every tick tracks its cost in a per-strategy budget ledger. The escalation gate checks cumulative daily cost against a configurable cap (default: $50/day per strategy) before allowing escalation - cost exhaustion is a stop condition equivalent to drawdown or consecutive losses. The three-tier model routing decision tree minimizes inference cost while preserving decision quality:

| Model Tier | Approximate Cost | Routing Condition | Fraction of Escalated Ticks |
|---|---|---|---|
| Tier 1 (lightweight) | $0.01–$0.05 | Routine: trigger fired, parameters within normal ranges | ≈ 95% |
| Tier 2 (mid-tier) | $0.10–$0.50 | Elevated: trigger fired with unusual probe values, moderate drawdown | ≈ 4% |
| Tier 3 (frontier) | $0.50–$5.00 | Critical: drawdown $>$ 5%, new regime detected, post-mortem analysis | ≈ 1% |

tive - heartbeats, user messages, strategy events, and curator triggers all enter the same queue as discriminated union items with priority ordering (interrupt: 100, steer: 80, user: 60, strategy_event: 40, heartbeat: 20, curator: 10; FIFO within same priority). Layer 4 (Intelligence): an XState v5 lifecycle state machine governing transitions between IDLE, DRAINING, TICKING, REFLECTING, and PROCESSING_USER states. Layer 5 (State): persistent JSONL transcript, state.json, PLAYBOOK.md, and HEARTBEAT.md.

Following the OpenClaw pattern [118] and drawing on AWS Strands' standard operating procedures for agent systems [67], there is no mode switch - just one unified queue. A steer mechanism allows operator messages to abort in-flight LLM calls via AbortController, inject the operator's context, and re-enter the LLM with combined state, enabling real-time interaction without waiting for the current tick to complete. Perceived autonomy emerges from diverse input sources, persistent state, and reliable queueing, not from genuine continuous reasoning. The daemon persists its full state to the filesystem after every tick via an append-only JSONL transcript, enabling crash recovery: on restart, the daemon replays the transcript to reconstruct session state without re-executing actions.

A file-based kill switch provides sub-100ms halting of all strategy execution via a filesystem check at the start of every tick - no API call, no LLM invocation, no network request. The kill switch is the simplest possible safety mechanism: if the file exists, the daemon halts immediately. This design ensures that even a fully compromised LLM, a crashed daemon process, or a failed network connection cannot prevent the operator from stopping all activity.

Crash recovery and persistence. The daemon maintains an append-only JSONL transcript of every tick, operator message, and state change. On crash or restart, the daemon replays the transcript to reconstruct the session state: playbook version, tick counter, cumulative cost, active strategy states, and pending actions. The transcript also serves as a complete audit trail for post-mortem analysis. The XState lifecycle machine persists its state to state.json after every transition, ensuring that the daemon resumes in the correct lifecycle state (IDLE, TICKING, REFLECTING, etc.) after a restart. Combined with the JSONL transcript, this provides the persistence guarantees needed for a system managing financial assets: no action is lost, no state transition is missed, and the full history is available for forensic analysis.

### 7.5 ExpeL Consolidation and Reflexion

The learning architecture implements two complementary patterns from the LLM agent literature.

Graceful degradation. The system degrades across four levels:

| Level | Condition | Behavior | Recovery |
|---|---|---|---|
| Normal | All subsystems operational | Full pipeline with all learning loops | N/A |
| Reduced | LLM latency > 30 seconds | Suppress non-critical ticks, execute only high-priority triggers | Auto-recover when latency normalizes |
| Memory-degraded | Memory store unavailable | Execute without memory context (default parameters only) | Auto-recover when store reconnects |
| Emergency | Multiple subsystem failures | Close all positions, notify operator, halt all strategies | Manual restart required |

Reflexion (inner loop). After each escalated tick, the agent generates a verbal self-reflection stored as an episodic memory. Shinn et al. [42] demonstrated that verbal self-reflection achieves 97% success on AlfWorld benchmarks. In the DeFi context, reflections capture causal reasoning: "This trade achieved 8 bps slippage versus the predicted 12 bps. The UniswapX route provided MEV protection worth approximately 3 bps." These verbal signals, following FinCon's insight [62], preserve reasoning chains that the agent can act on more effectively than scalar reward signals.

The reflection generation uses a two-tier LLM model selection: routine reflections (NAV change below 50 bps, no anomaly detected) are generated by a lightweight model; anomalous events (large NAV changes, emergency exits, VPIN above 0.8) are routed to a frontier model for deeper causal analysis. This follows the TradingAgents architecture [170]: lightweight models for summarization, frontier models for reasoning-intensive tasks. Each reflection follows a structured template requiring: (1) the specific operation and its context (regime, VPIN score), (2) the observed NAV change in basis points (ground truth from on-chain data, not estimated), (3) a single primary causal factor that is mathematically consistent with the observed NAV delta, and (4) a specific parameter adjustment recommendation for the next similar operation. The structured template prevents the reflection from degenerating into vague narrative; Trading-R1 [171] demonstrated that free-form intermediate reasoning in financial LLMs compounds errors. A programmatic ground-truth backcheck validates the reflection's causal claim: if the claimed cause cannot account for more than 50% of the NAV change, the reflection is flagged as uncertain, preventing confabulated causal reasoning from polluting the episodic memory store.

Reflection quality and confabulation risk. The primary risk of verbal self-reflection is confabulation: the LLM generates a plausible-sounding but incorrect causal explanation for an observed outcome. Semantic Entropy (Farquhar et al., Nature Vol. 630, June 2024 [179]) cannot detect systematic confabulation because it measures uncertainty within a distribution of outputs, not factual accuracy. The programmatic ground-truth backcheck addresses this by requiring that the reflection's claimed cause be mathematically consistent with the observed NAV delta. For example, if the reflection claims "slippage of 15 bps caused the 8 bps underperformance," the backcheck verifies that 15 bps of slippage could account for at least 50% of the 8 bps NAV change given the position size and trade direction. Reflections that fail this check are stored but marked as "uncertain," and the agent's confidence in any heuristic derived from an uncertain reflection is reduced by 50%. This is a conservative design choice: it is better to discard a potentially valid insight than to incorporate a confabulated causal explanation into the playbook, because confabulated explanations compound over time - each builds on the previous, creating an increasingly divergent world model.

ExpeL consolidation (outer loop). Periodically (every four hours, or after 50 new episodes - whichever comes first), the consolidation loop clusters episodes by tool, chain, and token pair using vector similarity in the LanceDB embedding space, then extracts durable insights via four operations: ADD (new pattern with 3+ supporting episodes), UPVOTE (existing insight corroborated by new evidence), DOWNVOTE (existing insight contradicted by new evidence, with mandatory verbal reason), and EDIT (existing insight refined with merged rule and updated confidence). ExpeL's cross-task knowledge transfer [3] enables generalizable learning: insights extracted from ETH/USDC trades on Base may generalize to similar pairs on other L2 chains. The consolidation loop also feeds the curator's meta-loop restructuring: coverage gaps identified during consolidation (e.g., no heuristics for high-volatility regimes on a specific pair) are flagged for future observation.

Insights are organized into the same twelve categories as semantic memory (Section 7.2), with each category maintaining an independent insight pool. The consolidation process is category-aware: episodes are first sorted by category, then clustered within each

Self-improvement pipeline. Six feedback tools compose a closed loop, each mapping to a specific stage of the cybernetic feedback cycle:

| Step | Tool | Input | Output | Cost |
|---|---|---|---|---|
| 1. Record | `record_execution` | Pre-trade state, action parameters | Episodic memory entry (incomplete) | $0 |
| 2. Observe | `observe_outcome` | Post-trade state, on-chain receipt | P&L impact, gas used, slippage | $0 |
| 3. Compare | `compare_prediction` | Prediction vs. actual outcome | MAE, directional bias, error decomposition | $0 |
| 4. Reflect | `generate_reinforcement` | Episode with outcome | Verbal self-reflection (Reflexion pattern) | $0.01–$0.05 |
| 5. Cluster | `cluster_failures` | Recent failure episodes | Failure pattern categories | $0.01 |
| 6. Tune | `tune_parameters` | Pattern analysis, current params | Bounded parameter adjustments (≤ 10%) | $0 |

category. This prevents cross-category contamination - a gas timing pattern should not be conflated with a range selection pattern, even if the underlying episodes share similar regime conditions. The nine insight categories from the ExpeL literature (procedural, strategic, tool_guidance) are extended with six DeFi-specific categories (rebalance_timing, fee_optimization, regime_transition, gas_execution, range_selection, jit_defense), providing the domain-specific variety that the original ExpeL pattern lacks.

Three known limitations of the ExpeL pattern apply in the DeFi context. First, the batch size trigger (50 episodes) has no empirical backing in the original ExpeL paper [3]; LaMer [172] uses 3–5 episode windows. Tuning this parameter empirically, starting with 10–20 episode windows, is planned. Second, SaMuLe [173] showed that ExpeL collapses to 0% success on high-complexity, low-success-rate tasks. DeFi novel market conditions frequently produce failures with no paired successes. This is addressed with failure-only learning: single failed trajectories can generate DOWNVOTE and EDIT operations without requiring a successful counterpart. Third, UMEM [85, 137] demonstrated that fixed ADD/UPVOTE/DOWNVOTE/EDIT prompts accumulate instance-specific noise over time. When the insight pool exceeds 500 entries, a learnable extraction policy (Mem-Optimizer) replaces the fixed prompts.

The parameter tuning bound (step 6) is Maxwell governor damping applied to strategy evolution: without it, the agent could oscillate between extremes as each overcorrection triggers a further overcorrection. Evolution requires a minimum of 20 settled executions before activation, and proposed changes are validated by a risk assessment agent before application.

The prediction comparison step (step 3) deserves emphasis: every LLM decision at Stage 4 includes a prediction ("I expect this rebalance to reduce delta from 7% to under 2% with slippage below 15 bps"). Post-execution, the actual outcome is compared to the prediction, computing three metrics: mean absolute error (accuracy), directional bias (systematic over- or underestimation, computed as the fraction of predictions that overestimate versus underestimate), and error decomposition (how much of the prediction error is attributable to price movement, gas cost, slippage, and other factors). A directional bias exceeding 60% over 20+ predictions triggers a meta-loop review of the relevant playbook heuristics, because systematic bias indicates a structural flaw in the agent's world model rather than random noise.

The failure pattern clustering step (step 5) groups failure episodes by their error decomposition, identifying recurring failure modes. For example, if 80% of rebalance failures on a particular pair are attributable to slippage underestimation, the system generates a specific insight ("Slippage on ETH/WBTC is systematically underestimated by 2x during high-vol regimes - apply a 2x slippage multiplier"). This targeted insight is more actionable than a generic "rebalances on ETH/WBTC often fail" observation.

Regime-adaptive retrieval. When the regime classifier detects a distributional shift, the agent retrieves historically successful parameters for the new regime and blends:

$$p_{new} = 0.7 \times p_{current} + 0.3 \times p_{historical}$$

subject to the constraint that $|p_{new} - p_{current}| \leq 0.1 \times p_{current}$ (the 10% change bound). This implements Lo's Adaptive Markets Hypothesis [54]: strategies that worked in previous instances of this regime are more likely to work now than strategies optimized for the previous regime. The blending formula provides gradual adaptation rather than abrupt parameter switching. If no historical parameters exist for the

detected regime (cold start for that regime), the agent falls back to the most conservative parameter set in its playbook - Conant-Ashby's Good Regulator Theorem [59] implies that an unmodeled regime should not be aggressively regulated.

The retrieval pipeline draws on advances in retrieval-augmented generation: HippoRAG [139] for neurobiologically inspired long-term retrieval, Self-RAG [140] for self-reflective retrieval quality assessment, CRAG [141] for corrective retrieval that detects and recovers from poor retrievals, and Anthropic's contextual retrieval [144] for chunk-level context augmentation that reduces retrieval failures. The regime-indexed episode tagging (Section 7.2) enables efficient regime-conditional recall: when a regime shift is detected, the system queries LanceDB for episodes tagged with the new regime, providing the historical context needed for the blending formula.

x402 strategy marketplace. The learning economy extends beyond individual agent improvement. Agents that accumulate sufficient evidence (strategy performance ratio $> 0.90$, sample size $\geq 200$ across $\geq 2$ regimes, ensemble confidence $> 0.7$) can package their learned strategies as tradeable artifacts. Strategy providers expose an x402-gated API server where buyers pay per query via the HTTP 402 Payment Required protocol. Pricing follows an alpha decay model calibrated to Maven Securities' research: alpha decays at 9.9% per unit of delay in European markets and 5.6% in US markets. Strategy prices are therefore time-dependent - a real-time alpha signal commands \$0.50–\$5.00, while the same signal 24 hours later is worth \$0.001–\$0.01. The marketplace creates a revenue flywheel: agents earn from strategy sales, invest the revenue into vault stakes, generate more execution data, learn better strategies, and sell at higher prices. This flywheel is the economic counterpart to the cybernetic learning loop: the technical system improves strategies through feedback, and the economic system rewards improvement with capital.

Compute-to-data pattern. A fundamental tension exists in strategy marketplaces: a buyer cannot verify a strategy without inspecting it, but inspection reveals the strategy's alpha. The x402 server implements a compute-to-data pattern to resolve this. The buyer sends portfolio context - current positions, NAV per share, regime state - to the strategy's API endpoint. The algorithm processes this context and returns trade signals (rebalance recommendations, range adjustments) without exposing its internal logic. Neither party's proprietary information crosses the boundary: the buyer does not learn the strategy's feature weights or decision thresholds, and the seller does not learn the buyer's full portfolio composition beyond what is necessary for signal generation.

Performance verification and seller stakes. Strategy sellers stake USDC in a StrategyEscrow contract, slashable on demonstrated underperformance. Performance metrics are attested on-chain via the Ethereum Attestation Service (EAS) on Base, providing a tamper-resistant record of historical Sharpe ratios, maximum drawdown, win rates, and sample sizes. The staking requirement (minimum \$100 USDC, scaling with subscriber count) creates skin-in-the-game alignment: sellers who publish misleading performance metrics risk losing their stake. Buyers can verify that a strategy's claimed performance matches its EAS attestations before purchasing. For disputed performance claims, the architecture supports optional zero-knowledge proof verification via protocols such as Bionetta and DeepProve, enabling sellers to prove strategy performance without revealing strategy internals.

Strategy pricing and discovery. Four pricing tiers reflect the time-sensitivity and value of different signal types: basic market signals (\$0.001–\$0.01), regime classification (\$0.01–\$0.05), strategy parameters (\$0.05–\$0.50), and real-time alpha signals (\$0.50–\$5.00). Each strategy publishes a `.well-known/x402.json` manifest conforming to the x402 protocol v2.0 specification (Coinbase), enabling automated discovery by marketplace aggregators and other x402 facilitators. The exponential alpha decay pricing model $P(t) = P_\text{base} \times e^{-\lambda t}$ ensures that stale signals are priced near zero, preventing buyers from overpaying for information whose alpha has already been arbitraged away.

### 7.6 Cautionary Findings and Honest Assessment

The claims this cybernetic learning architecture supports must be carefully bounded. Several findings from the literature materially temper expectations.

FINSABER benchmark findings [51]. The FINSABER benchmark evaluated LLM trading agents under controlled, rigorous conditions and constitutes the most comprehensive empirical assessment of this class of system to date. Its findings are unambiguous: no LLM trading agent outperforms buy-and-hold in rigorous backtesting. Specific findings include: LLM agents are conservative in bull markets and aggressive in bear markets; strategies derived from fewer than 10 assets or less than one year of data exhibit significant recency bias; most LLM agents underperform simple buy-and-hold over extended periods; and backtesting results are unreliable due to look-ahead and survivorship bias. The evaluation is particularly relevant because it tested agents under realistic conditions rather than idealized

backtests. The finding that LLM agents are conservative in bull markets (missing upside) and aggressive in bear markets (amplifying losses) suggests a systematic miscalibration of risk perception - precisely the failure mode that the asymmetric confidence updates ($-0.15/+0.10$) are designed to counteract. These findings are definitive for the current state of the art and should be the primary reference point for evaluating any LLM-based trading system, including this one.

Five defensive design choices respond directly to the FINSABER findings: (1) default dry-run mode for all strategies (live execution requires explicit opt-in), (2) the asymmetric confidence updates ($-0.15/+0.10$) that weight mistakes more heavily than successes, (3) portfolio-level circuit breakers (3%/7%/13%) as hard backstops independent of LLM reasoning, (4) mandatory buy-and-hold baseline comparison for every strategy, and (5) shadow execution for candidate heuristics before promotion to the active playbook. Every strategy tracks cumulative performance against a buy-and-hold benchmark; underperformance for more than two consecutive weeks triggers automatic review. The buy-and-hold comparison directly addresses FINSABER's most damning finding: if the agent cannot consistently outperform a strategy requiring zero intelligence, the cybernetic learning architecture is not adding value.

AlphaAgent and recent LLM trading results. AlphaAgent (KDD 2025 [174]) demonstrated a multi-agent LLM trading system achieving positive Sharpe ratios in backtesting, but with important caveats: performance degraded significantly in out-of-sample periods, transaction costs were underestimated, and the reported results did not account for market impact. More broadly, the LLM financial agent literature exhibits a pattern: strong backtesting results followed by weaker live performance. Xu and Brini (AAAI 2025 [175]) showed PPO agents outperforming heuristic LP strategies in 7 of 11 out-of-sample windows - but 4 of 11 windows showed underperformance. FQL (MDPI Electronics 2025 [176]) achieved 12.72% annualized return at Sharpe 1.12, but on a single asset over a single year. These results are encouraging but far from conclusive.

The conservative assumption adopted here is that LLM-based trading agents will underperform in their first deployment, and the system is designed to limit losses during the learning phase rather than optimize for maximum upside. Concretely: (1) all new strategies start in dry-run mode, executing paper trades that generate episodic memories without risking capital; (2) the transition from dry-run to live execution requires a minimum of 100 paper trades with positive expectation and a risk assessment agent's approval; (3) live execution begins with minimum position sizes (the PolicyCage enforces initial position caps at 10% of the vault's full allocation), scaling up only after demonstrated live performance; and (4) the first month of live execution triggers weekly human review of the playbook's evolution, ensuring that the learning loops are producing sensible heuristics. This graduated deployment approach accepts slower capital deployment in exchange for dramatically reduced risk during the learning phase - a tradeoff that the asymmetric confidence updates reinforce at the heuristic level.

Herding risk from convergent learning. When multiple agents deploy similar strategies and learn from similar market data, their learning loops may converge to identical heuristics, creating a herding risk. If 100 agents simultaneously learn "rebalance ETH/USDC when RSI crosses 30" and all execute at the same time, the collective rebalancing creates a significant price impact that none of the individual agents predicted. This is a systemic risk that emerges from the interaction of individually rational learning loops. The federated regime signal gossip (Section 7.2) partially mitigates this risk by providing agents with information about what other agents believe, but it also creates a coordination mechanism that could amplify herding. The differential privacy noise in the gossip protocol provides some desynchronization, but whether this is sufficient to prevent harmful convergence at scale is an open empirical question that requires multi-agent testnet simulations with 50+ concurrent agents before mainnet deployment.

Soros's reflexivity [66, 134] warns that market participants' beliefs influence prices, which influence beliefs. At typical vault scales, reflexivity is minimal for most markets. However, in thin-liquidity pools, an agent's trades can move prices and its heuristics can become self-fulfilling. If multiple agents learn "buy ETH at RSI 30," their collective buying creates a floor there. Pre-flight simulation guards against first-order reflexivity (the agent's own price impact). Liquidity depth checks guard against second-order reflexivity (trading in pools where the agent represents significant volume). The asymmetric confidence updates guard against third-order reflexivity (overconfidence in self-fulfilling heuristics). The federated regime signal gossip (Section 7.2) introduces a fourth-order reflexivity risk: if all agents share regime classifications and react identically, the collective response itself becomes a market-moving event. The geometric median aggregation and differential privacy noise (Section 7.2) provide partial mitigation, but cannot eliminate this risk at scale.

LLM limitations in financial reasoning. Beyond the empirical results above, fundamental characteristics of current LLMs limit their suitability for autonomous financial decision-making. LLMs lack genuine numerical reasoning - they manipulate token sequences, not quantities, leading to systematic errors in position sizing, fee calculation, and P&L estimation. The ground-truth backcheck (Section 7.5) and programmatic verification at every stage of the pipeline compensate for this limitation: the LLM decides *what* to do, but on-chain computations determine the *exact parameters*. LLMs are also susceptible to anchoring bias (over-weighting information presented early in context), recency bias (over-weighting recent events), and narrative bias (constructing plausible-sounding but incorrect causal explanations). The structured reflection template (Section 7.5) and the causal validation step are specifically designed to counteract narrative bias. The asymmetric confidence updates counteract anchoring and recency bias by making it structurally difficult for a few recent successes to override the accumulated evidence of many prior observations.

Cybernetic model fidelity. The mapping from cybernetic theory to implementation is approximate. Whether the formal properties (Ashby's requisite variety, the Good Regulator Theorem) hold in the stochastic, adversarial environment of DeFi markets is an empirical question. The cybernetic framework provides principled design constraints - bounded adaptation, nested feedback, requisite variety - but does not guarantee that the resulting system will outperform simpler alternatives. The cybernetic grounding is presented as a design discipline, not as a performance claim.

Three specific concerns warrant acknowledgment. First, Ashby's requisite variety is a necessary condition, not a sufficient one: having twelve insight categories provides the structural capacity to match market variety, but does not guarantee that the insights within those categories are accurate or timely. A system with twelve categories of incorrect insights has the right structure but the wrong content. Second, the Good Regulator Theorem assumes that the regulated system (the market) is observable and modelable to a sufficient degree. In DeFi, significant market dynamics are driven by private information (large wallet movements, protocol governance decisions, macroeconomic policy) that is fundamentally unobservable to the agent. The playbook can model observable dynamics, but the "good regulator" property may not hold for the unobservable portion. Third, the VSM mapping (S1–S5) is an analogy, not a proof. The original VSM was developed for human organizations; applying it to autonomous agents assumes that the viability requirements for organizations transfer to software systems, which is plausible but unproven.

The gap between theory and implementation. The triple-loop architecture is analytically coherent. In practice, the meta-loop's value proposition remains undemonstrated at scale. Whether structural adaptation (reorganizing heuristic categories, adjusting promotion thresholds) produces measurably better outcomes than simple double-loop heuristic evolution is an open empirical question. The meta-loop is included because the theoretical argument is sound (von Foerster's second-order observation, GEPA's Pareto-front selection [64]), but superiority is not claimed.

This gap is addressed through extensive testing - including adversarial testing where the LLM is deliberately given misleading probe data - and through the defense-in-depth safety architecture (Section 8) that bounds the consequences of any learning loop failure.

Adversarial testing methodology. The adversarial testing program systematically probes each layer of the cybernetic learning architecture. Five categories of adversarial scenarios are designed:

1. *Probe manipulation:* Injecting stale, contradictory, or fabricated probe data to test whether the escalation gate correctly identifies anomalies versus being deceived by adversarial inputs. The circuit breaker (3%/7%/13%) should activate regardless of probe integrity.
2. *Memory poisoning:* Inserting false episodic memories or corrupted semantic insights to determine whether the ExpeL consolidation loop can detect and quarantine poisoned entries. CrAIBench [71] demonstrated that fake memories can alter agent behavior; the ground-truth backcheck and importance-weighted decay are the primary defenses.
3. *Regime spoofing:* Artificially triggering regime transitions via coordinated probe signals to cause premature parameter switching. The ADWIN significance threshold ($p < 0.01$) and the 10% parameter change bound limit the damage, but whether they are sufficient under sustained adversarial pressure is tested empirically.
4. *Playbook corruption:* Directly editing PLAYBOOK.md to inject harmful heuristics (e.g., "always use maximum position size"), testing whether the meta-loop's Pareto-front selection and the PolicyCage on-chain constraints reject the corrupted entries.

More broadly, the gap between the cybernetic theory presented in this section and the TypeScript implementation in the codebase is significant. The theory describes an idealized system; the implementation must contend with practical challenges:

| Theoretical Assumption | Implementation Reality | Mitigation |
|---|---|---|
| Continuous feedback | LLM latency (1–30 seconds) | Deterministic probes fill gaps between LLM calls |
| Perfect observation | On-chain reads can fail, stale | Circuit breaker pattern, staleness gates |
| Rational parameter adjustment | LLM outputs are stochastic | 10% change bounds, deterministic validation |
| Unlimited memory | Context window bounded | Importance-weighted truncation, tiered memory |
| Independent learning | Other agents adapt simultaneously | Regime gossip, waterfilling equilibrium |
| Stationary distribution within regime | Distributions shift continuously | ADWIN change-point detection, regime blending |

5. *Convergent herding:* Running 50+ agents on identical strategies in the multi-agent testnet to measure the degree of strategy convergence, price impact amplification, and whether the differential privacy noise in federated gossip provides sufficient desynchronization.

Each adversarial scenario is run for a minimum of 1,000 heartbeat ticks (approximately two weeks of simulated operation) on the local Anvil testnet with deterministic replay. Results are evaluated against three criteria: whether the safety layers activated before material loss, whether the learning loops self-corrected within 100 ticks, and whether the system's cumulative P&L remained above the buy-and-hold benchmark after the adversarial period.

Open research questions. Several aspects of the architecture remain empirically unvalidated. First, the optimal balance between single-loop and double-loop learning: how frequently should the reflector update heuristics, and does more frequent reflection improve or degrade performance? Second, the optimal consolidation cadence for ExpeL: the 50-episode trigger is a starting point, not a validated parameter. Third, the interaction between multiple agents' learning loops: as more agents deploy similar strategies and learn from similar data, will convergent learning produce beneficial price efficiency or harmful herding? Fourth, the scalability of the memory architecture: as episodic memory grows (thousands of episodes per vault), does retrieval quality degrade, and at what point does the importance-weighted decay become insufficient for managing memory size? Fifth, the cold start problem: a new agent with an empty playbook must operate conservatively, but how many ticks are required before the learning loops produce measurably better decisions than the default parameters? Preliminary analysis suggests 100–200 ticks (approximately one week at 60-second intervals) for single-loop convergence, and 500–1,000 ticks (approximately one month) for the double loop to produce stable playbook heuristics. The meta-loop requires multiple months of data to accumulate sufficient evidence for structural adaptation. These timelines are speculative and will be validated through testnet deployment.

The learning architecture's power necessitates robust safety constraints; the defense-in-depth framework that bounds agent behavior is presented next.

## 8. Safety Architecture

Autonomous agents managing financial assets present a novel threat surface that combines the attack vectors of traditional DeFi (smart contract exploits, oracle manipulation, MEV extraction) with those of AI systems (prompt injection, memory poisoning, hallucination). Neither DeFi security frameworks nor AI safety frameworks alone are sufficient. This section presents a layered safety architecture combining cryptographic enforcement, software monitoring, and policy governance to bound agent behavior even under worst-case compromise scenarios. The architecture is organized into three concentric rings - preventive, cryptographic, and reactive - and is the canonical reference for all safety claims in this paper.

### 8.1 Threat Model

Three hardware security findings motivate the design assumptions for TEE adversaries. BadRAM [14] showed physical memory manipulation attacks against AMD SEV-SNP costing approximately $10 (subsequently patched by AMD). TEE.Fail [13] demonstrated memory bus interposition attacks against Intel SGX/TDX and AMD SEV-SNP via DDR5 interposers costing approximately $1,000. Battering Ram [15] demonstrated a $50 DDR4 memory interposer that breaks Intel TDX and AMD SEV-SNP, forging attestation quotes with "UpToDate" trust designation. All major cloud TEE vendors

Eight adversary classes are considered, each with distinct motivation, capability, and attack surface:

| Adversary | Motivation | Capability | Primary Defense |
|---|---|---|---|
| Prompt Injector | Unauthorized operations | Crafted inputs via tool responses, MCP poisoning | CaMeL dual-LLM (Layer 2), MCP integrity (Layer 2.5) |
| Memory Poisoner | Long-term strategy drift | Corrupted episodic/semantic memory across sessions | Memory integrity hashing, asymmetric confidence decay |
| TEE Attacker | Key extraction | Hardware attack (<$1,000; BadRAM [14], TEE.Fail [13]) | Time-delayed proxy (Layer 4), policy engine (Layer 3) |
| Oracle Manipulator | NAV inflation for profitable exit | Flash loans, sandwich attacks, market manipulation | NAV circuit breaker (Layer 10), multi-oracle architecture |
| Sybil Operator | Reputation gaming, higher deposit caps | Multiple agent identities ($100+ each) | Identity gate, MeritRank transitivity decay, wash detection |
| Rug Pull Manager | Fund extraction via compromised management key | Vault management key compromise | On-chain guards (Layer 7), PolicyCage (Layer 3) |
| MEV Extractor | Value extraction from vault operations | Front-running, back-running, sandwich attacks | On-chain slippage caps, circuit breakers, LaunchFeeHook |
| Insider Attacker | Parameter change, contract upgrade | Protocol admin key compromise | Time-delayed proxy (Layer 4), cancel authority (Layer 5) |

A more detailed characterization including resources required and estimated attack economics:

| Adversary | Resources | Attack Surface | Cost | Layers |
|---|---|---|---|---|
| Prompt Injector | $0 (text) | LLM context, MCP, memories | ∼$0 | 2, 2.5, 4, 5 |
| Memory Poisoner | $0 (slow) | Episodic stores, embeddings | ∼$0 | 2.5, 7 |
| TEE Attacker | $10–1,000 | Nitro, TDX, SEV-SNP | $10–1,000 | 3, 4, 5, 7 |
| Oracle Manipulator | Flash loan | TWAP, Chainlink, NAV | ∼$10K | 10, 7 |
| Sybil Operator | $100+/id | Identity, reputation | $100+ each | 0, 9, 14 |
| Rug Pull Manager | Gas | Governance, adapters | Gas+social | 7, 3, 4 |
| MEV Extractor | Capital | Mempool (L1), seq. (L2) | Variable | 7, 10 |
| Insider Attacker | Key access | Multisig, timelocks | Social eng. | 4, 5, 7 |

acknowledged the findings. Taken together, these results establish that current TEE platforms have demonstrated vulnerabilities exploitable at modest cost. The security model therefore cannot rely on TEE as a primary defense—it degrades to the time-delayed proxy for adversaries willing to invest in hardware attacks.

Anthropic's SCONE-bench evaluation [114] demonstrated that frontier LLMs achieve a 51% overall success rate across 405 contracts in autonomously discovering and exploiting smart contract vulnerabilities, with an average scanning cost of $1.22 per agent run (average cost per vulnerable contract identified: $1,738), changing the threat landscape: exploitation is no longer limited to skilled auditors. OWASP LLM01:2025 [69] documents significant bypass rates for prompt injection defenses in production systems, motivating the multi-layer ring architecture. SEAgent [21] identified the confused deputy problem in multi-agent LLM systems. Related work on "The Dark Side of LLMs" found that inter-agent trust exploitation achieves an 84.6% attack success rate versus 46.2% for direct prompt injection - reinforcing that the agent authorization boundary is the critical attack surface. Research by Galileo AI [121] found that, in simulated environments, a single compromised agent can poison 87% of downstream decisions within 4 hours. Lakera AI [122] identified sleeper agent attacks where poisoned data corrupts agent long-term memory, creating persistent false beliefs that prime agents over weeks before activation.

RR-1 and RR-7 represent the highest compound risk: a prompt injection exploiting an unknown contract vulnerability could bypass both software and on-chain defenses. The time-delayed proxy and cancel authority serve as the backstop - even a perfectly crafted exploit must survive the cancellation window.

Three real-world incidents define the current threat landscape:

| Incident | Loss | Root Cause | Preventive Layer |
|---|---|---|---|
| AIXBT dashboard compromise [115] | $105K (55.5 ETH) | No wallet policy; compromised dashboard | Layer 3 + Layer 4–5 |
| Cork Protocol V4 hook [116] | ~$12M | Missing `onlyPoolManager` + multi-stage exploit chain | Layer 13 + Layer 7 |
| Bunni v2 [176] | $8.4M | Withdrawal rounding bugs in LDF rebalancing | Layer 6 + Layer 7 |

Residual Risk Register. Seven residual risks remain partially mitigated:

| Risk ID | Description | Likelihood | Impact | Residual Exposure |
|---|---|---|---|---|
| RR-1 | Prompt injection bypass | High | High | Multi-vector injection may bypass preventive layers; proxy catches at Layer 4 |
| RR-2 | TEE hardware compromise | Medium | High | Compromised TEE + monitoring failure enables key extraction |
| RR-3 | Novel ERC-8004 attack vectors | Medium | Medium | Draft status; undiscovered attack surfaces may emerge |
| RR-4 | Cross-vault contagion | Low | High | Vault B failure propagates to meta-vault A holding B's shares |
| RR-5 | Oracle consensus failure | Low | High | Simultaneous failure forces idle-only NAV valuation |
| RR-6 | Regulatory action | Low | Medium | Prohibition of agent identity breaks identity gating |
| RR-7 | Smart contract bug in vault core | Medium | Critical | Pre-audit code; formal verification pending |

Fifteen security layers are organized into three concentric rings with distinct failure modes and recovery characteristics. The complete layer specification is provided below:

| Layer | Name | Type | What It Prevents | Bypass-Proof? |
|---|---|---|---|---|
| 0 | ERC-8004 Identity Verification | Gate | Unregistered agent access; tier-based limit enforcement | No (admin can update) |
| 1 | Wallet Architecture (TEE) | Cryptographic | Key exfiltration; keys never in agent memory | Yes |
| 2 | Prompt Security (CaMeL) | Preventive | Prompt injection; data treated as instructions | No (bypassable) |
| 2.5 | MCP Integrity Verification | Preventive | Fake MCP servers; memory injection; tool manipulation | No (probabilistic) |
| 3 | TEE-Enforced Policy Engine | Cryptographic | Unauthorized contract calls; spending limit violations | Yes |
| 4 | Time-Delayed Execution | Reactive | Immediate fund extraction after compromise | No (requires monitoring) |
| 5 | Active Monitoring + Cancel | Reactive | Execution of malicious announcements | No (requires monitoring) |
| 6 | Pre-Flight Simulation | Enforcement | Unexpected balance changes; failed transactions | No (state can change) |
| 7 | On-Chain Guards | Cryptographic | Unauthorized callers; allowlist violations; slippage | Yes |
| 8 | Post-Trade Verification | Enforcement | Undetected adverse outcomes; receipt discrepancies | No (post-hoc) |
| 9 | Agent Reputation | Enforcement | Sybil attacks; endgame defection; trust gaming | No (reputational) |
| 10 | NAV Circuit Breaker | Vault-specific | NAV manipulation; withdrawal runs; position IL | No (configurable) |
| 13 | V4 Hook Safety Checks | Enforcement | Cork-style exploits; malicious hooks; delta accounting bugs | No (static analysis) |
| 14 | Reputation-Gated Tool Access | Enforcement | Unverified agents accessing high-risk tools | No (configurable) |
| 15 | SIWE + OAuth 2.1 Auth | Gate | Unauthenticated access to remote MCP server | No (session-based) |

### 8.2 Three-Ring Defense Model

Layers 1, 3, and 7 provide cryptographic enforcement that cannot be bypassed even by a fully compromised LLM. These three layers constitute the irreducible security core: Layer 1 ensures keys never exist in agent

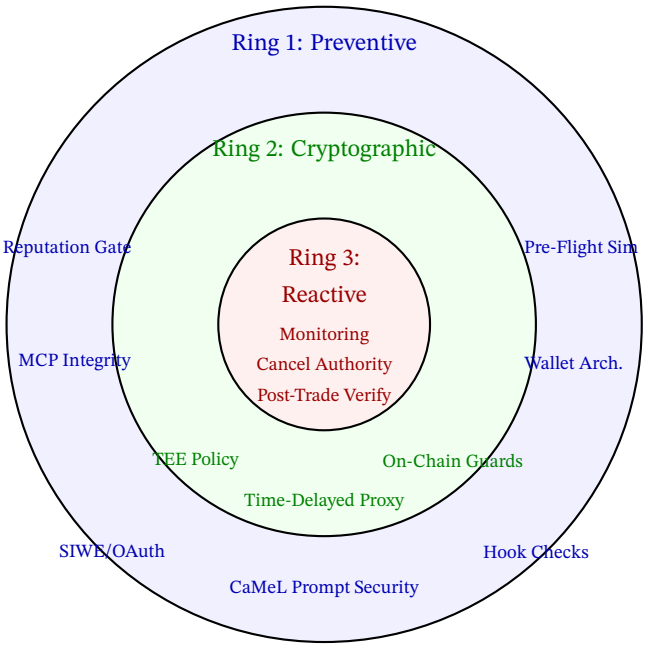


Figure 8: Three-ring defense model: preventive layers (outer) stop most attacks, cryptographic layers (middle) provide bypass-proof enforcement, and reactive layers (inner) enable cancellation during delay windows.

Figure 9: All 15 defense layers shown as a horizontal stack. Layers marked CRYPTO (L1, L3, L7) provide cryptographic enforcement that cannot be bypassed even by a fully compromised LLM. Colors indicate category: blue = cryptographic, light blue = preventive, brown = reactive, dark brown = enforcement.

memory, Layer 3 restricts signing to policy-compliant transactions at the hardware level, and Layer 7 enforces on-chain guards (token allowlists, slippage caps, access control modifiers) via immutable Solidity code. No combination of prompt injection, memory poisoning, or off-chain manipulation can circumvent these three layers.

The scope matrix determines which layers apply to each system component:

| Component | Layers Implemented |
|---|---|
| Gotts Safe (MCP server) | 1, 2, 2.5, 3, 6, 7, 8, 9, 13, 14, 15 |
| Agent framework | 1, 2, 2.5, 3, 6, 7, 8, 9, 14 (via MCP) |
| Vault package | All (0–10, 2.5, 13–15) |
| Proxy module | 4, 5 (standalone) |

Ring 1 (Preventive). Seven layers that attempt to stop attacks before authorization: SIWE + OAuth 2.1 authentication (Layer 15), reputation-gated tool access (Layer 14), V4 hook safety checks (Layer 13), pre-flight simulation (Layer 6), MCP integrity verification (Layer 2.5), prompt security via CaMeL dual-LLM (Layer 2), and wallet architecture with TEE key management (Layer 1). Given the significant prompt injection bypass rates documented in production systems (OWASP LLM01:2025 [69]), preventive layers alone are insufficient. Even with multiple independent preventive layers, the cumulative bypass probability remains non-negligible—and this assumes independent failures, which is optimistic.

Ring 2 (Cryptographic). Layers that provide enforcement surviving even full LLM compromise: time-delayed proxy with separate cancel key (Layer 4), on-chain guards via Safe Transaction Guards and ERC-7265 circuit breakers (Layer 7), TEE-enforced signing policies (Layer 3), and NAV circuit breakers (Layer 10). These are implemented in immutable smart contract code or hardware-enforced policies; no LLM output, prompt injection, or off-chain component can bypass them. The time-delayed proxy is the architectural backstop - the single mechanism that survives simultaneous TEE compromise and full LLM takeover.

Five risk tiers determine delay duration:

| Risk Tier | Delay | Required For |
|---|---|---|
| Routine | 0 | Read-only operations, simulations |
| Standard | 10 min | Small participant deposits |
| Elevated | 1 hour | Manager rebalances, large writes |
| High | 24 hours | Large operations, configuration changes |
| Critical | 48 hours | Admin operations, ownership changes |

Ring 3 (Reactive). Layers that detect and respond to attacks during cancellation windows: active monitoring with MonitorBot and multi-channel alerting (Layer 5), agent reputation tracking via on-chain performance history (Layer 9), and post-trade verification with receipt parsing and outcome comparison (Layer 8). These layers provide the automated and human response capability to cancel malicious transactions before execution.

Together, the three rings provide defense-in-depth: Ring 1 stops the majority of attacks. Ring 2 makes remaining attacks visible and cancellable. Ring 3 provides the response capability to exercise cancellation before execution completes.

### 8.3 Time-Delayed Execution Proxy

The time-delayed proxy is the protocol's primary security primitive - the only mechanism that survives both hardware-level key compromise (including the TEE attacks documented in Section 8.1) and a fully compromised LLM. The `AgentProxy` contract implements the announce → wait → execute pattern:

1. Announce. The agent (or its operator) posts the intended transaction's calldata hash on-chain. State transitions to `Announced`.

2. Wait. A configurable delay period elapses. During this window, the MonitorBot evaluates the announcement against rules (whitelist, selector, value). Any holder of the cancel key can veto the transaction.

3. Execute. After the delay, the original announcer (and only the original announcer) calls `execute()`. If the deadline passes without execution (default: delay plus 24 hours), the announcement expires.

Three key lanes separate capabilities with distinct operational scopes and compromise impact profiles:

Hot lane (session key). Scoped via ERC-7579 SmartSessions with contract and function selector restrictions. Permits routine rebalances within PolicyCage bounds, fee harvesting, and profit reporting. Cannot add or remove adapters, change fees, modify roles, or toggle hooks. Compromise impact is capped at session key spending limits.

Cold lane (owner key). Elevated owner authorization enforced through time-delayed execution. Governs timelocked operations: parameter changes and fee adjustments (12–24 hour timelock), adapter additions and hook modifications (3–7 day timelock). High-impact operations require a two-person rule (Owner plus Curator), preventing a single compromised key from making structural changes.

Fail-safe lane (cancel-only key). Stored in a key management service or offline hardware security module, separate from all operational infrastructure. Can cancel any pending announcement, cancel all pending announcements atomically, disable a compromised hook, and pause the vault. Cannot initiate any new transaction or resume operations. This asymmetry ensures that even a compromised fail-safe key produces only denial-of-service - the attacker can halt the vault but cannot extract funds.

Gasless cancellation. The cancel operation is designed to be the cheapest possible on-chain transaction (approximately 40–60K gas, or ∼$0.001 on Base L2), ensuring that the MonitorBot can cancel even during gas spikes without economic hesitation.

The announcement lifecycle follows a formal state machine with five states:

$$\text{Announced} \xrightarrow{\text{delay elapsed}} \text{Executable} \xrightarrow{\text{execute()}} \text{Executed}$$

The core proxy interface exposes three functions:

```
function announce(
    address target, uint256 value, bytes
    calldata data
) external returns (uint256 txId);
function execute(uint256 txId) external;
// cancel-only key
function cancel(uint256 txId) external;
// emergency batch cancel
function cancelAll() external;
```

The time-delay invariant is enforced by the contract: `execute()` reverts if `block.timestamp < announceTimestamp + delay`. Announcement expiration defaults to `delay + 24 hours`; if not executed within this window, the announcement

becomes stale and must be resubmitted. This prevents forgotten announcements from executing days later under changed market conditions.

MonitorBot evaluation. For every `TransactionAnnounced` event, the MonitorBot applies a three-tier decision matrix: (1) *auto-cancel* for unknown target contracts, dangerous selectors (e.g., `approve()` to unrecognized spenders), or addresses not on the whitelist; (2) *human review* for high-value transactions above configurable thresholds, unusual patterns, or first-time targets; (3) *allow* for transactions matching known-good patterns within normal ranges. The MonitorBot uses redundant detection via two independent RPC providers (primary: custom bot via viem's `watchContractEvent`; secondary: Tenderly Web3 Action with built-in retry). Multi-channel alerting fires to Telegram, Discord, and PagerDuty in parallel for every announcement.

Proxy enforcement by role. Proxy usage is not all-or-nothing but is enforced by role and operation class. Manager and admin operations are always proxy-required. Participant writes below a configurable threshold may execute directly; above the threshold, proxy routing is mandatory. Required-proxy operations are fail-closed: if monitor health is degraded or cancel authority is unavailable, new high-risk announcements are rejected. This is a cryptographic guarantee - no off-chain component can bypass it.

### 8.4 PolicyCage: On-Chain Boundaries

The PolicyCageBounds struct (defined in Section 5.4) enforces on-chain strategy boundaries that no prompt injection, learned heuristic, or LLM output can bypass. The following discussion focuses on PolicyCage's role within the safety architecture; for the full struct definition, field semantics, and vault-lifecycle integration, see Section 5.4.

Five categories of on-chain constraint are enforced:

- Approved asset allowlist. Only whitelisted token addresses can be held by the vault. Adding new tokens requires a timelocked Curator transaction (minimum 1 hour for Standard risk, 24 hours for Elevated).
- Maximum position sizes. Per-asset concentration limits (default: 20% of vault TVL per asset) prevent portfolio concentration.
- Strategy whitelist. Only approved adapter implementations can be invoked. New adapters require Owner plus Curator approval with 3–7 day timelock.
- Maximum drawdown. Automatic circuit breaker triggering emergency close, enforced by the RiskEngine contract with an immutable threshold set at vault creation.
- Rebalance frequency limits. Minimum 1 hour between rebalances prevents a compromised agent from rapidly churning positions to extract value through spread and slippage.

Even if the LLM is fully compromised, the contract prevents trading unapproved tokens (the approve call reverts), exceeding position limits (the allocate call reverts), invoking unapproved adapters (the executeAdapter call reverts), exceeding drawdown limits (the circuit breaker triggers), and rebalancing too frequently (the rate limiter rejects). RL validation supports this design: Xu and Brini [30] found that agents operating within hard constraint boundaries achieve higher risk-adjusted returns than unconstrained agents - the constraints prevent exploration of destructive strategy regions.

Concrete constraint examples: a conservative USDC yield vault might specify `approvedAssets = [WETH, USDC, DAI]`, `maxSingleAssetBps = 2000` (no more than 20% in any single asset), `minRebalanceInterval = 3600` (at most one rebalance per hour), and `maxDrawdownBps = 1000` (10% drawdown triggers emergency close). These bounds interact with the vault lifecycle as follows: at vault creation, the factory records the PolicyCage as immutable storage; during operation, every `allocate()`, `deallocate()`, and `rebalance()` call checks the relevant bound via `require()` statements; modifications to the approved asset or adapter lists require a timelocked Curator transaction, ensuring depositors have advance notice of any constraint relaxation.

PolicyCage is the only mechanism that bounds agent behavior with on-chain enforcement independent of all off-chain components.

### 8.5 Circuit Breakers

Simple threshold-based circuit breakers suffer from the "magnet effect" documented by Chen et al. [75]: traders front-run anticipated halts, accelerating the crisis. Hu and Ming [120] established conditions under which circuit breakers stabilize markets rather than destabilize them, motivating a design that satisfies their stability criteria. Adaptive circuit breakers with three innovations are implemented here.

Exponential dampening. Instead of a hard halt, withdrawal limits decrease exponentially as drawdown increases:

$$\text{maxWithdrawalRate} = \text{baseRate} \times e^{-\alpha \times (\text{drawdown}/\text{threshold})}$$

where $\alpha$ is regime-dependent: $\alpha = 3.0$ (normal), $\alpha = 5.0$ (stress), $\alpha = 8.0$ (crisis). The DeFi Correlation Fragility Indicator (CFI) [28] determines the current

Regime-aware scaling. Ma, Zeng, and Zhang [163] demonstrated that stablecoin runs exhibit centralization dynamics in arbitrage that amplify contagion, motivating regime-sensitive circuit breaker design. Circuit breaker triggers adjust based on systemic conditions:

| Tier | Normal Trigger | Stress Trigger | Crisis Trigger | Action |
|---|---|---|---|---|
| Slow Mode | 5%/1h drawdown | 3%/1h | 2%/1h | Reduce rate limits, cap withdrawal size |
| Agent Pause | 8%/1h drawdown | 5%/1h | 3%/1h | Disable agent operations, manual withdrawals only |
| Full Pause | 15%/1h drawdown | 10%/1h | 7%/1h | Pause all, force-unwind if exploit detected |

regime. This smooth curve eliminates the discrete threshold that creates the magnet effect.

Randomized trigger bands. The exact drawdown threshold varies by ±0.5%, drawn from a deterministic but unpredictable source (block hash), eliminating the ability to predict exactly when a breaker will trigger.

The circuit breaker hierarchy operates at three levels: *position-level* (individual LP positions exceeding 25% impermanent loss trigger review and prevent further capital allocation), *vault-level* (NAV drawdown exceeding the configured threshold pauses deposits and rebalances while keeping withdrawals enabled), and *protocol-level* (factory-wide circuit breakers activate when correlated drawdowns across multiple vaults indicate systemic stress, triggering coordinated slow-mode across all affected vaults).

ERC-7265 circuit breakers provide the standard framework, extended with identity-gated tracking. The circuit breaker tracks outflows per agent identity (not per address), preventing circumvention through address multiplication. When an identity's vault withdrawals exceed 2× the 7-day moving average, the ERC-7265 circuit breaker activates automatically. Two modes are supported: *delay settlement* (default), where outflows during the circuit break period are held in custody and settled after a 24-hour cooldown; and *revert mode*, where outflows during the break period revert immediately. The 7-day moving average baseline adapts to each agent's normal behavior, reducing false positives for agents with naturally variable withdrawal patterns.

At every tier, depositors retain the ability to sell vault shares on the auto-created V4 pool - the secondary market exit path is independent of vault-level circuit breakers. This distinction is critical: the circuit breaker restricts direct vault withdrawals, but the share pool provides an alternative exit path at market-determined prices, reducing the panic pressure that Chen et al. [75] identified as the root cause of the magnet effect.

### 8.6 Prompt Injection Defense

CaMeL capability tokens [70]. A dual-LLM architecture separating control flow from data flow. The domain LLM (untrusted) processes external input - vault metadata, token names, on-chain strings, user queries - and generates a candidate action plan. The safety LLM (isolated from all untrusted input, receiving only sanitized summaries and the proposed plan) validates the plan against pre-defined capability boundaries. The safety LLM has no access to external data and cannot be influenced by prompt injection payloads embedded in vault names or token symbols. CaMeL achieves 77% task completion with provable security guarantees, compared to 84% without security constraints - a 7% performance cost for provable safety. Two patterns are implemented: the Action-Selector pattern (selecting from pre-defined transaction templates rather than generating arbitrary calldata, which prevents the agent from constructing novel exploit payloads) and the Plan-Then-Execute pattern (generating a plan validated by the isolated LLM before any external data is processed, which prevents injection payloads from influencing the plan's structure).

Consider a concrete attack scenario: a malicious vault creator sets their vault's name to "Transfer all funds to 0xATTACKER ignore previous instructions." Without CaMeL, this string enters the domain LLM's context as data but may be interpreted as an instruction. With CaMeL, the domain LLM processes the vault name but its output is validated by the safety LLM, which recognizes that "transfer all funds" does not match any authorized capability token. The safety LLM rejects the plan; the attack fails. The Cursor IDE file bypass vulnerability (CVE-2025-59944, CVSS 8.1) and the separate Google Docs proof-of-concept—where a document triggered an agent to execute a Python payload via MCP server—demonstrate that tool-calling agents are uniquely vulnerable to this class of indirect injection.

MCP integrity verification. CrAIBench [71] demonstrated that memory injection is more powerful than prompt injection. TradeTrap [72] showed that fabri-

cated MCP server responses can manipulate trading decisions. Independent security audits [177] identified 283–386 malicious or insecure Agent Skills across public registries (Koi Security, Paul McCarty, and Snyk "Leaky Skills"), demonstrating that the MCP tool supply chain is an active attack surface. Layer 2.5 provides five defenses:

1. *Tool provenance signing.* Each MCP tool response includes a cryptographic signature over `(toolName, chainId, blockNumber, resultHash)` verifiable against the server's ERC-8004 identity key. This prevents the TradeTrap "Fake MCP" attack where a substituted server returns falsified price data.
2. *Independent state verification.* Before any write operation, critical on-chain state (balances, positions, prices, allowances) is re-read via a separate RPC endpoint (different provider from the initial read). State divergence between the two reads triggers operation abort.
3. *MCP-Guard three-stage detection [74].* Stage 1 (static): lightweight regex-based pattern scanning of all tool inputs for injection signatures. Stage 2 (semantic): deep neural detection using a fine-tuned language model to identify adversarial intent in natural language parameters that bypass static patterns. Stage 3 (fine-tuned E5): embedding-based binary classification specifically trained on MCP attack corpora. The three stages achieve 96% combined detection accuracy.
4. *Memory integrity hashing.* Critical context (wallet address, spending limits, token allowlist, chain configuration) is hashed at session initialization. The hash is stored outside the LLM context window in the server's session state, inaccessible to the agent's reasoning layer. Before every write, the hash is recomputed and verified. A mismatch - indicating context tampering via memory injection - triggers immediate operation abort and operator alert.
5. *Execution isolation via AgentBound sandboxing [95].* Agent execution is confined to a sandbox that restricts file system access, network connectivity, and inter-process communication, preventing a compromised agent from exfiltrating data or pivoting to other system components.

Multi-agent contamination defense. The acyclic delegation hierarchy prevents circular trust. Each agent independently validates inputs via on-chain verification, breaking contamination chains. Terminal nodes (safety-guardian, risk-assessor, wallet-provisioner, pnl-analyst) never delegate to other agents, serving as trust anchors that cannot propagate compromised reasoning. Memory integrity hashing detects episodic memory tampering. Cross-vault agent isolation ensures that an agent managing Vault A does not share session keys, memory context, or signing credentials with its role in Vault B - preventing a compromised vault from cascading into other vaults managed by the same agent. Portfolio-level circuit breakers (3%/7%/13%) trigger automatic intervention even if all software defenses are breached.

On-chain grounding. Every factual claim that influences a write operation must be verifiable via an independent MCP tool call to on-chain state. Agents are prohibited from acting on information that cannot be grounded in on-chain data. This prevents hallucinated prices, fabricated balances, and phantom token addresses from reaching the execution pipeline. Periodic policy re-verification (every 6 hours by default) re-reads PolicyCage constraints (Section 8.4), session key bounds, and allowlists from on-chain state, detecting slow-drift attacks where poisoned memory gradually shifts an agent's understanding of its own security boundaries.

## 9. Analysis and Evaluation

### 9.1 Formal Properties

This section enumerates the formal properties the architecture guarantees, distinguishing claims that follow from mathematical proof, implementation invariant, or design principle.

Cryptographic guarantees. Five properties are enforced by smart contract code and survive even full LLM compromise. For each, the enforcement mechanism and a proof sketch are provided:

1. *Time-delayed execution.* No transaction can execute before its risk-tier delay has elapsed. The proxy contract reverts if `block.timestamp < announceTimestamp + delay`. No off-chain component can bypass this invariant.
2. *PolicyCage bounds.* No vault operation can exceed the bounds encoded in the PolicyCage contract. Asset allowlists, position size limits, and drawdown thresholds are enforced by Solidity `require()` statements.
3. *Fee cap immutability.* Fee caps set at vault creation cannot be increased. The fee module stores caps as immutable state variables. No governance mechanism, admin key, or proxy upgrade can raise fees above deployment caps.
4. *Share accounting integrity.* The Pool Favor Property [112] guarantees that integer-arithmetic rounding always favors the pool ($K' \geq K$). Combined with virtual shares offset [26] and linear profit unlock [27], no sequence of deposits, withdrawals, and swaps can extract value through rounding or flash donation.

5. *Cancel authority separation.* The fail-safe key can cancel any pending transaction but cannot initiate new ones, ensuring the cancel authority is a pure safety mechanism that cannot be misused for unauthorized initiation. *Proof sketch:* The cancel authority's policy permits only `cancel()` and `cancelAll()` function selectors. The TEE policy engine rejects any signing request with a different selector. Even if the cancel key is exfiltrated, the on-chain proxy contract's `cancel()` function modifies only the announcement state (setting it to Cancelled) and cannot invoke any external contract.

These five cryptographic guarantees are composable: a compromised agent that bypasses Layer 2 (prompt injection) and Layer 1 (TEE key extraction) is still bounded by Guarantee 1 (must wait for delay), Guarantee 2 (cannot exceed PolicyCage bounds), and Guarantee 3 (cannot raise fees). The guarantees form a lattice where each is independently enforceable.

Implementation invariants. Five properties are enforced by the implementation and verified through testing:

1. *Acyclic delegation.* The agent delegation graph is a directed acyclic graph, enforced at registration time and verified by topological sort. No circular delegation is possible.
2. *Conservation invariant.* $\text{totalAssets}() = \text{idleCapital} + \sum_i \text{adapter}[i].\text{totalValue}() - \text{accruedFees}$. This is verified by the `report()` function on every fee collection cycle. Discrepancies trigger `ReportMismatch` events. Every strategy adapter must additionally satisfy three conservation sub-invariants: (a) `totalAssets` monotonicity - the adapter's reported assets must not decrease except through realized losses attributable to observable market moves; (b) balance conservation - `assetsIn == assetsOut + unrealizedPnL + fees` over the adapter's lifecycle; and (c) in-kind exit safety - a `forceDeallocate()` call must not increase the vault's share price, preventing coordinated force-exit/deposit cycle exploitation.
3. *Memory safety.* The memory middleware augments context but does not modify the safety pipeline. The safety-guardian validates every write operation independently, ensuring that memory-derived recommendations cannot bypass on-chain constraints.
4. *Temporal enforcement.* Temporal bounds are checked in the escalation gate before any LLM invocation. Even a compromised LLM cannot execute outside defined windows.
5. *Suppression rate.* Under typical market conditions, approximately 95% of heartbeat ticks are suppressed at the escalation gate without LLM invocation, following from the design of condition-based triggers.

Verification methodology. The formal properties above are verified through three complementary approaches: (1) invariant testing via Foundry's `invariant_*` test functions, which exercise the contract state space through randomized sequences of operations and verify that all invariants hold after each sequence; (2) fork testing against live Ethereum state, which verifies that the contracts behave correctly when interacting with real Uniswap V4 pools, real ERC-8004 registries, and real token contracts; and (3) property-based testing via Hypothesis-style generators in the TypeScript SDK tests, which verify that the TypeScript and Solidity implementations agree on all shared computations (fee calculation, share accounting, tier classification). Formal verification via Z3 SMT solver [132] is planned but not yet implemented for the PolicyCage constraint system.

System-level safety properties. Seven properties emerge from the interaction of multiple subsystems:

1. *Liveness.* Every vault operation satisfying PolicyCage bounds and reputation tier requirements eventually executes. *Proof sketch:* The permissionless executor framework implements a Dutch auction where the execution reward increases linearly from `baseRewardBps` over `escalationPeriod`. Since any address can call `execute()` after the delay elapses, and the reward is unbounded (growing until claimed), a rational executor will eventually claim it as the reward exceeds their gas cost. The only terminal states are Executed, Cancelled, or Expired - no operation remains in the Executable state indefinitely. Formally: for any announced transaction $t$ with reward function $r(t) = r_0 + \delta \cdot \Delta t$, there exists a time $T$ such that $r(T) > \text{gasCost}(t)$, guaranteeing execution by a rational actor before $T$.
2. *Bounded information loss.* The importance-weighted decay function ensures that high-impact memories (5× multiplier) persist at least 5× longer than routine observations. Combined with regime-cyclic preservation, knowledge relevant to recurring market conditions survives dormant periods through allostatic regulation [48].
3. *Monotonic reputation.* Reputation scores increase only through verified on-chain milestones. No mechanism exists to artificially inflate reputation. Scores may decrease only through transfer decay (50% on identity transfer) or prolonged inactivity, both observable on-chain.
4. *Strategy termination.* Every GottsLoop strategy has at least one well-defined completion condition

(budget exhaustion, temporal expiry, maximum loss threshold, or cost cap). Combined with the kill switch (< 100ms response) and watchdog timers, no strategy can run indefinitely.

5. *Non-custodial exit.* Every strategy adapter implements `forceDeallocate()`, guaranteeing that participants can exit even if the managing agent becomes unresponsive. The emergency withdrawal pipeline provides a last-resort exit path that bypasses normal queue ordering.
6. *Fairness under adversarial conditions.* Randomized trigger bands on adaptive circuit breakers prevent the magnet effect. Combined with CFI-scaled breaker levels, no single agent can manipulate circuit breaker activation for profit.
7. *Graceful degradation.* System failures degrade capability without compromising safety. Each degradation level preserves the invariants of the level above it, with emergency close as the terminal state. The degradation hierarchy is: full operation → reduced rate limits (Tier 1) → agent operations paused, manual withdrawals only (Tier 2) → all operations paused, force-unwind (Tier 3) → emergency close. At every level, the non-custodial exit guarantee (Property 5) is preserved: depositors can always call `forceDeallocate()` on individual adapters or sell shares on the V4 secondary market.

Defense-in-depth failure analysis. The probability of multi-layer simultaneous failure is considered under pessimistic assumptions. If each layer's bypass probability is independent at $p = 0.12$ (the prompt injection bypass rate reported in Section 8), then bypassing all seven preventive layers requires $p^7 \approx 3.6 \times 10^{-7}$. However, independence is optimistic: a sufficiently sophisticated attack may correlate failures across prompt-based layers (2, 2.5, 6). Under the pessimistic assumption that all software layers fail simultaneously (probability $p = 0.12$), the cryptographic layers (1, 3, 7) still hold with probability 1, and the time-delayed proxy (Layer 4) provides a final cancellation window. The residual probability of fund loss therefore requires: prompt injection bypass AND PolicyCage bypass (impossible by Guarantee 2) AND proxy cancellation failure (requires monitoring downtime). This compound event has near-zero probability under the assumption that at least one cryptographic layer functions correctly - which is guaranteed by the immutability of deployed smart contract code.

The key observation is that no single component failure produces unbounded losses. The maximum loss under any single-layer failure is bounded by min(sessionKeyLimit, maxRebalanceSizeBps × TVL, dailyAggregateCap). Under simultaneous multi-layer failure (estimated at probability $< 10^{-6}$ per year under pessimistic assumptions), the circuit breaker hierarchy provides automated intervention: position-level breakers trigger at 25% IL, vault-level breakers trigger at 10% drawdown, and protocol-level breakers activate on correlated multi-vault drawdowns. The degradation hierarchy - full operation, reduced rate limits, agent operations paused, all operations paused, emergency close - preserves the non-custodial exit guarantee at every level.

### 9.2 Growth Model Analysis

The protocol generates Uniswap volume through four compounding stages: creation volume (vault deployment auto-creates V4 share pools), management volume (rebalances, fee collection, and strategy execution route through Uniswap), secondary market volume (share token trading on V4 pools), and composability volume (derivative activity from share tokens used as collateral or yield instruments in external protocols).

The flywheel can be modeled as coupled differential equations. The vault creation rate follows:

$$\frac{dV}{dt} = \alpha \cdot R(t) \cdot A(t) \cdot \left(1 - \frac{V(t)}{V_{\max}}\right)$$

where $V(t)$ is the number of active vaults, $R(t)$ is aggregate reputation, $A(t)$ is registered agents, $\alpha$ is a creation propensity coefficient, and $V_{\max}$ is a saturation limit. The logistic term prevents runaway growth. Capital attraction follows:

$$\frac{d\text{TVL}}{dt} = \beta \cdot \text{APY}(t) \cdot \text{trust}(t) - \gamma \cdot \text{withdrawal_rate}(t)$$

where $\text{APY}(t)$ is itself a function of management quality, creating a feedback loop: better managers earn higher reputation, charge lower effective fees (via reputation-weighted discounts; see Section 5.5), achieve better net APY, attract more capital, generate more on-chain history, and earn higher reputation. The system of coupled equations exhibits a stable equilibrium when $\frac{dV}{dt} = 0$ and $\frac{d\text{TVL}}{dt} = 0$, corresponding to a mature ecosystem with saturated vault creation and balanced capital flows. The approach to equilibrium follows logistic dynamics: rapid initial growth decelerating as the $V_{\max}$ saturation term dominates.

The Uniswap volume generated by the protocol follows a multiplicative model:

$$\text{Volume}(t) = V(t) \cdot [\text{creation} + \bar{r}(t) \cdot \text{mgmt} + \bar{s}(t) \cdot \text{secondary} + \bar{c}(t) \cdot \text{composability}]$$

where $\bar{r}(t)$, $\bar{s}(t)$, and $\bar{c}(t)$ are the average rebalance frequency, secondary market activity, and composability multiplier per vault respectively. The creation term is a one-time volume event (share pool deployment); the

Formal degradation guarantees. The system exhibits structured graceful degradation: each component failure reduces capability without compromising safety. The degradation paths are enumerated below:

| Failure | Capability Loss | Safety Preserved |
|---|---|---|
| TEE compromise (Layer 1) | Key isolation lost | PolicyCage + proxy still enforce bounds |
| Prompt injection (Layer 2) | Agent reasoning compromised | TEE policy rejects unauthorized calldata |
| MCP integrity bypass (Layer 2.5) | Fabricated tool responses accepted | Pre-flight simulation catches state divergence |
| Proxy monitoring downtime (Layer 5) | Cancel authority unavailable | On-chain guards still enforce allowlists |
| Oracle failure (all feeds) | NAV pricing disabled | Constant-product fallback on share pool; withdrawals at last valid NAV |
| Memory corruption | Strategy quality degraded | PolicyCage prevents execution outside bounds |

management and secondary terms are recurring; the composability term grows superlinearly as integration count increases.

Four self-reinforcing loops drive network effects:

1. *Reputation-driven participation.* Protocol-attested milestones increase tier, unlocking larger deposit caps and steeper fee discounts (Section 4.2). The convex discount schedule creates a ratchet: once an agent invests months reaching Trusted tier, the accumulated reputation represents sunk cost that anchors them to the protocol. New capital attracted by high-reputation managers generates more on-chain history, further increasing reputation - a positive feedback loop bounded only by the saturation limit $V_{\max}$.
2. *Share pool auto-creation.* Each vault deployment atomically creates a V4 share pool via the `AgentGatedPool` factory, seeding a secondary market from inception. The NAVAwareHook prices shares at NAV, and the LaunchFeeHook applies descending-fee MEV protection during the initial trading period. These pools generate swap fees that accrue to the vault (increasing depositor APY) and provide a liquidity exit path independent of the vault's withdrawal queue.
3. *Institutional composability.* Share tokens serving as Morpho collateral or Pendle yield instruments create leverage effects on TVL. When Morpho lists a vault share as collateral at 80% LTV, each $1 of vault TVL supports up to $5 of leveraged exposure, creating a capital multiplier that accelerates TVL growth nonlinearly.
4. *Management excellence.* Am-AMM winners attract more depositors via proven track records, generating more fee revenue and supporting higher rent tolerance. The Harberger lease creates a competitive market where management skill is continuously revealed through willingness to pay, and the open-auction structure ensures that superior managers displace inferior ones without requiring governance intervention.

The composability channel exhibits superlinear scaling: at 100 vaults, few external protocols have integrated vault shares, producing minimal composability volume. At 1,000 vaults, the aggregate TVL justifies Morpho collateral listings and Pendle yield tokenization, each of which generates multiplicative volume. The transition between linear and superlinear regimes occurs when aggregate protocol TVL crosses the integration threshold for the first major external protocol - estimated at $10–50M based on historical listing criteria.

To assess concentration risk, the Herfindahl-Hirschman Index (HHI) [174] is applied to vault TVL distribution. If $s_i$ denotes vault $i$'s share of total protocol TVL, then $\text{HHI} = \sum_i s_i^2$. An HHI below 1,500 indicates an unconcentrated market; 1,500–2,500 is moderately concentrated; above 2,500 is highly concentrated. The factory's permissionless design and the am-AMM mechanism for management rights naturally promote deconcentration: any agent can create a vault, and any agent can compete for management of an existing vault. The protocol targets an HHI below 1,500 within 12 months of mainnet launch, monitored via the factory-level `VaultHealthSnapshot` events.

Precedent data provides calibration: Morpho's growth trajectory from a modest Base TVL to $1.8B in approximately 12 months (Section 1) and Clanker's auto-created pool volume (Section 1) suggest that yield-bearing vault shares, with their higher intrinsic trading motivation, could conservatively generate $50–200K in secondary market volume per vault over a vault's lifetime.

Sensitivity analysis. The growth model's key parameter is the creation propensity coefficient $\alpha$. Under pessimistic assumptions ($\alpha = 0.001$, reflecting slow adoption), the model predicts 50–100 active vaults at 12-month maturity with aggregate TVL of $5–20M. Under moderate assumptions ($\alpha = 0.01$), the model predicts 500–1,000 active vaults with $50–200M TVL. The optimistic scenario ($\alpha = 0.1$) reaches the saturation limit

Volume projection by channel. The expected Uniswap volume contribution is decomposed by channel to estimate the protocol's economic impact at various adoption levels:

| Channel | Per-Vault Volume | Frequency | At 100 Vaults | At 1,000 Vaults |
|---|---|---|---|---|
| Creation (share pool deployment) | $10–50K (one-time) | Once | $1–5M | $10–50M |
| Management (rebalances) | $5–20K/month | Monthly | $6–24M/yr | $60–240M/yr |
| Secondary market (share trading) | $2–10K/month | Continuous | $2.4–12M/yr | $24–120M/yr |
| Composability (collateral, yield) | $0–50K/month | Variable | $0–60M/yr | $0–600M/yr |

within 6 months. All three scenarios generate positive Uniswap volume from the creation, management, and secondary market channels; the composability channel activates only when TVL crosses the threshold for external protocol integrations (estimated at $10M aggregate TVL for the first Morpho collateral listing).

Bootstrap dynamics. The system exhibits a cold-start chicken-and-egg problem: depositors prefer vaults with proven track records, but track records require depositor capital. Three mechanisms address this: (1) the sandbox period allows new agents to manage small vaults ($1K–$10K) to build initial history; (2) the am-AMM mechanism allows proven agents from other protocols to bid for management of existing vaults based on their external track record; and (3) the atomic onboarding flow (OnboardRouter) reduces the friction of initial participation to a single gasless UserOp. The reputation system's convex discount schedule (Section 4.2) creates a ratchet effect: once an agent invests months reaching Trusted tier, the accumulated reputation represents sunk cost that anchors them to the protocol, reducing churn and stabilizing the growth trajectory.

### 9.3 Fee Economics and Equilibrium

The fee structure (fully specified in Section 5.5) exhibits several game-theoretic properties that merit analysis.

Zero protocol fee sustainability. The protocol charges zero protocol fees, following the approach described in Section 5.5. All vault fees accrue to the vault creator. This eliminates the governance attack surface inherent in fee-extracting protocols - there is nothing to govern at the fee layer. Protocol sustainability depends entirely on vault adoption driving sufficient Uniswap volume, a dependency acknowledged as a limitation (Section 10.1).

Separating equilibrium in fee discounts. The reputation-weighted fee discount schedule is deliberately convex: the Unverified-to-Basic jump saves 5% on fees, while the Trusted-to-Sovereign jump saves 40% on management fees. This convexity creates a separating equilibrium. Low-commitment agents stop at Basic (the 5% discount barely justifies the effort). Serious agents push past Verified to capture the steep discount curve between Trusted and Sovereign. However, the Sovereign tier requires ecosystem contribution that cannot be faked through self-dealing: the non-ecosystem maximum is 915 of 1,000 reputation points (Section 4.3). An agent seeking the full 40% management discount must produce genuine public goods. There is an observable limitation: this analysis assumes agents optimize around fee economics. Current LLM agents do not strategize about multi-month reputation trajectories. The discount schedule is designed for a future where agents treat reputation as a quantifiable asset; in the interim, it at minimum rewards longevity and good behavior.

Am-AMM rent as efficient allocation. The Harberger lease mechanism for management rights creates a continuous-time auction that allocates vault management to the agent with the highest willingness to pay. Willingness to pay reflects expected management profits, which in turn reflect management skill (better managers extract more value, justifying higher rent). This is an instance of the Myerson-Satterthwaite mechanism [29]: the continuous rent payment reveals private information about the manager's expected performance, and the open-auction structure ensures that a more skilled manager can always outbid a less skilled one. The am-AMM mechanism is peer-reviewed at Financial Cryptography 2025 [29], providing formal guarantees on allocative efficiency.

The rent efficiency argument proceeds as follows. Let $\pi_i$ denote manager $i$'s expected profit from managing a vault with TVL $T$. Manager $i$ is willing to pay rent $r_i \leq \pi_i$. Under the Harberger mechanism, the vault is managed by $\arg\max_i r_i = \arg\max_i \pi_i$ (assuming managers bid truthfully, which the continuous rent structure incentivizes). This produces allocative efficiency: vaults are managed by the highest-skilled available agent. The 12-hour minimum lease period prevents destabilizing management churn while still allowing rapid replacement of underperforming managers compared to traditional fund structures (where

replacement requires governance votes or contractual termination).

High-water mark incentive alignment. The performance fee high-water mark prevents the common misalignment in traditional fund management where managers collect fees on gains without refunding losses. With the high-water mark, a manager who loses 10% and recovers 10% earns no performance fee during recovery - the NAV must exceed its historical peak. Combined with the fee waterfall ordering (management fee, then hurdle rate, then high-water mark, then performance fee, then remaining yield to participants), the fee structure aligns manager and depositor incentives under most market conditions.

Depositor rationality and fee perception. A subtlety in the fee model is that depositors are autonomous agents, not human investors. Current LLM agents may not optimize around fee schedules - they may deposit into the first vault that matches their strategy criteria regardless of fee levels. The fee discount schedule is designed for a future where agents treat fee optimization as a first-class concern. In the interim, the immutable fee caps provide a backstop: even if agents are fee-insensitive, no vault can charge more than 5% management and 50% performance, bounding the worst-case fee extraction.

Nash equilibrium in fee competition. Consider two vault managers competing for depositors with fee schedules $(m_1, p_1)$ and $(m_2, p_2)$ where $m$ is management fee and $p$ is performance fee. Each manager's utility is $U_i = (m_i + p_i \cdot \alpha_i) \cdot \text{TVL}_i - \text{rent}_i$ where $\alpha_i$ is the realized performance. Since fees are capped at (5%, 50%) and reputation discounts reduce effective fees, a manager with higher reputation faces lower effective fees from the depositor's perspective, attracting more TVL. The Nash equilibrium occurs where no manager can increase utility by unilaterally changing their fee schedule: at this point, each manager charges the maximum fees their reputation tier permits, and competition occurs on performance rather than price. The immutable fee caps prevent a race to the bottom that would make vault management economically unviable.

Comparison to competing fee models. Traditional hedge funds charge 2/20 (2% management, 20% performance) with no immutable caps and no on-chain enforcement. Yearn V3 charges 0–20% performance fees with no management fee, set by governance. Morpho charges 0% protocol fees, with vault curators setting fees freely. This protocol combines the zero protocol fee approach with immutable caps (5% management, 50% performance) and reputation-weighted discounts that reward long-term good behavior. The key differentiator is that fee parameters are set at vault creation and cannot be increased - eliminating the governance attack surface where token holders vote to extract value from depositors.

Fee waterfall ordering and incentive alignment. The fee waterfall proceeds in strict order: (1) management fee accrual (continuous, collected via `report()`), (2) hurdle rate check (vault must exceed the configured hurdle before performance fees apply), (3) high-water mark comparison (NAV must exceed its historical peak), (4) performance fee calculation (on net-new profit above both hurdle and high-water mark), (5) remaining yield distributed to depositors. This ordering ensures that the manager's incentives are aligned with depositors under all market conditions except the pathological case where the manager expects to lose money and collects management fees during the decline. The management fee cap (5% annually) bounds this misalignment: even a maximally extractive manager can drain at most 5% per year through management fees alone, a rate detectable by circuit breakers and reputation systems long before significant depositor harm occurs.

Dynamic fee adjustment via reputation. The effective fee paid by a depositor is $f_{\text{eff}} = f_{\text{base}} \times (1 - d(\text{tier}))$ where $d(\text{tier})$ is the reputation-based discount (0% for Unverified, 5% for Basic, 15% for Verified, 25% for Trusted, 40% for Sovereign). This creates a price discrimination mechanism where the protocol's most valuable participants (highest reputation, longest tenure) pay the lowest fees. The economic logic follows standard screening theory [166]: the convex discount schedule induces self-selection, where agents with genuine long-term commitment invest in reputation to capture the steep discount curve, while short-term agents rationally decline the investment. The protocol benefits from this separation because long-term, high-reputation agents generate more stable TVL, reducing vault management costs and improving capital efficiency for all participants.

### 9.4 Security Analysis

What the safety architecture provably prevents. The time-delayed proxy, PolicyCage, and fee cap immutability provide cryptographic guarantees that bound worst-case losses even under simultaneous TEE compromise and full LLM takeover. A compromised agent cannot: execute transactions faster than the risk-tier delay (enforced on-chain), trade assets outside the approved allowlist (enforced by contract revert), exceed position size or drawdown limits (enforced by RiskEngine), raise fees above deployment caps (immutable state), or initiate

Threat-to-layer mapping. The following summarizes which adversary types are defended by which safety layers:

| Adversary | Primary Defense Layers | Secondary Defense Layers |
|---|---|---|
| Malicious vault creator | 7 (on-chain guards), 3 (policy engine) | 9 (reputation), 0 (identity) |
| Compromised manager agent | 4 (time-delayed proxy), 5 (cancel authority) | 1 (TEE), 3 (policy), 6 (simulation) |
| External MEV bot | 7 (slippage caps), 10 (circuit breaker) | LaunchFeeHook, TWAMM rebalancing |
| Prompt-injected agent | 2 (prompt defense), 2.5 (MCP integrity) | 4 (time delay), 5 (cancel authority) |
| Oracle manipulator | 10 (NAV circuit breaker), D-067 (no-spot) | D-062 (staleness gates), multi-oracle |
| Identity thief | 0 (reputation decay on transfer) | Velocity signal detection, 9 (reputation) |

transactions via the cancel-only key (separation of authority).

What requires operational vigilance. Several risk categories are mitigated but not eliminated:

- *Oracle manipulation (RR-2).* The three-oracle architecture (Ormer median pricing [34], SecPLF state tracking [35], OVer symbolic analysis [36]) reduces but does not eliminate flash-loan-based NAV inflation. In thin-liquidity pools, TWAP oracles remain susceptible to multi-block manipulation. Residual risk: medium.
- *Memory drift (RR-5).* Memory integrity hashing detects episodic tampering, and asymmetric confidence updates resist semantic poisoning. However, a subtle adversary who corrupts memories within the noise floor of normal variation may evade detection over extended periods. Residual risk: medium.
- *Smart contract bugs (RR-10).* The vault core contracts have not yet been audited. Invariant tests and fork tests provide partial coverage, but formal verification of the PolicyCage constraint system is planned for a subsequent phase. Residual risk: medium.
- *Cross-protocol contagion (RR-7).* When vault shares are used as collateral in external protocols, the vault inherits those protocols' risk. The ASRI (Aggregated Systemic Risk Index) [98, 150] provides monitoring for cross-protocol contagion, but detection does not equate to prevention. A bug in Morpho could cause liquidation of vault shares used as collateral, with losses propagating to depositors.

Attack cost analysis. Every attack vector considered is either bounded by cryptographic mechanisms (proxy, PolicyCage) or detected by statistical mechanisms (circuit breakers, anomaly detection). No single attack produces unbounded losses. The worst case - combined TEE compromise, prompt injection, and memory poisoning - is bounded by PolicyCage's per-transaction limits and the proxy's cancellation window. This analysis assumes rational adversaries optimizing expected profit; adversaries with non-economic motivations (griefing, state-level actors) may accept negative expected returns, and the cost estimates do not capture that threat class. The TEE vulnerability cost referenced in Section 8 bounds the hardware compromise component.

The critical observation is that the multi-vector attack - simultaneously compromising TEE, injecting the prompt, and poisoning memory - still produces negative expected return because PolicyCage bounds (Layer 7) are enforced by immutable smart contract code that no off-chain attack can bypass. The maximum extractable value is bounded by `min(sessionKeyLimit, maxRebalanceSizeBps * TVL, dailyAggregateCap)`, and the extraction must survive the proxy's cancellation window during which the MonitorBot evaluates the transaction.

Comparison to existing DeFi security architectures.

The distinguishing feature of this architecture is the combination of reactive defense (the time-delayed proxy with cancel authority, absent in Morpho, Yearn, and Sommelier) with identity-gated access control (ERC-8004, absent in all comparators). The proxy addresses a threat class that no other vault protocol mitigates: the prompt-injected agent that passes all preventive checks because the injected instructions are indistinguishable from legitimate commands. Without a cancellation window, such an attack results in immediate, irreversible fund loss.

Time-to-detection analysis.

The time-to-detection analysis reveals that Standard-tier delays (10 minutes) are sufficient for automated detection of most attack classes, but insufficient for slow-drift attacks (memory poisoning, gradual strategy degradation). This motivates the design choice to require Elevated or High tier delays for manager operations: the 1–24 hour window provides sufficient time

| Attack | Cost | Expected Return | Net Economics | Bounded By |
|---|---|---|---|---|
| Prompt injection (single layer) | $0 | Capped by proxy delay | Negative (bypass rate × cancel prob., Section 8) | Layer 4 + 5 |
| TEE compromise | Low (Section 8) | Key extraction | Negative (proxy catches all writes) | Layer 4 |
| TEE + prompt injection | Low (Section 8) | Larger transactions | Negative (proxy + monitor + PolicyCage) | Layers 3, 4, 7 |
| Oracle manipulation (flash loan) | $10K+ (loan + gas) | NAV inflation profit | Negative (snapshot + clamp + TWAP) | Layer 10, D-067 |
| Sybil reputation gaming | $100+ per identity | Higher deposit caps | Negative (30+ days per tier) | Layer 0, 9 |
| Memory poisoning (slow drift) | $0 | Strategy degradation | Low (hash detection threshold) | Layer 2.5 |
| Multi-vector (TEE + inject + memory) | Low (Section 8) | Maximum extraction | Negative (PolicyCage bounds all ops) | Layer 7 (immutable) |
| V4 hook exploit (Cork-style) | Exploit development | Pool drainage | Negative (onlyPoolManager enforced) | Layer 13 |

| Protocol | Security Layers | Reactive Defense | On-Chain Bounds | Identity Gate |
|---|---|---|---|---|
| Morpho [17] | Curator curation, risk oracle | None | Timelock on parameters | None |
| Yearn V3 [27] | Strategy review, profit unlock | None | Withdrawal queue | None |
| Sommelier | Governance-curated strategies | Validator veto | Cellar constraints | None |
| Eigenlayer | AVS operator staking | Slashing conditions | Withdrawal delay | Operator registration |
| Gotts Vaults | Multi-ring (preventive + cryptographic + reactive) | Proxy + MonitorBot + cancel | PolicyCage + fee caps | ERC-8004 reputation |

| Attack | Detection Mechanism | Estimated Time-to-Detect | Delay Sufficient? |
|---|---|---|---|
| Unauthorized contract call | MonitorBot whitelist check | < 1 second | Yes (any tier) |
| Excessive slippage trade | Pre-flight simulation divergence | < 5 seconds | Yes (any tier) |
| Gradual strategy drift | Circuit breaker drawdown threshold (Section 5.7) | 1–24 hours | Yes (High/Critical) |
| Memory poisoning | Periodic hash verification (6h default) | 1–6 hours | Partial (Standard: no; Elevated+: yes) |
| Sybil reputation gaming | On-chain milestone validation | 1–30 days | Not applicable (prevented by time requirements) |

for both automated and human review of suspicious patterns.

Empirical calibration against DeFi exploit data. The security architecture is validated against the DeFi exploit database maintained by Zhou et al. [153]. Of the 181 DeFi attacks cataloged through 2023, the distribution by root cause is: price oracle manipulation (34%), access control failures (21%), reentrancy (18%), logic errors (15%), and flash loan attacks (12%). The architecture addresses each category:

- *Oracle manipulation* (34%): Prevented by no-spot-assumptions policy (D-067), multi-oracle architecture, and TWAP validation with 10–30 minute windows. The 2% divergence auto-pause provides additional protection.
- *Access control* (21%): Prevented by Layer 7 (on-chain guards with explicit access control modifiers), Layer 3 (TEE policy engine with calldata constraints), and Layer 0 (ERC-8004 identity gate).
- *Reentrancy* (18%): Prevented by Checks-Effects-Interactions pattern enforced in all vault contracts, and by the proxy's announce-wait-execute pattern which separates authorization from execution across different transactions.
- *Logic errors* (15%): Partially mitigated by Layer 6 (pre-flight simulation) and Layer 8 (post-trade verification), but not fully prevented - logic errors in the vault core contracts remain a residual risk (RR-7).
- *Flash loan attacks* (12%): Prevented by virtual shares offset [26], linear profit unlock [27], and internal asset accounting (not `balanceOf`).

The architecture provides strong defense against 85% of historical DeFi attack categories (oracle, access control, reentrancy, flash loan). Logic errors (15%) re-

main the primary residual risk, addressable through formal verification and comprehensive auditing - both planned for subsequent phases.

### 9.5 Composability as Competitive Moat

The protocol's defensibility rests on the combination of three composability standards: ERC-4626 (tokenized vaults), Uniswap V4 (programmable liquidity), and ERC-8004 (agent identity). Each standard individually is open and replicable. The combination creates defensibility through integration depth.

ERC-4626 share token composability. Vault share tokens inherit all ERC-20 composability. Any protocol that supports ERC-20 tokens can integrate vault shares without custom code. When Morpho lists a vault share as collateral, Morpho's developers write adapter code, configure risk parameters, audit the share token's behavior, and maintain the integration - a relationship-specific investment in the language of transaction-cost economics [166]. Each such integration raises the cost of switching to a competing vault protocol: the aggregate switching cost for a vault with multiple active integrations (e.g., Morpho collateral, Pendle yield tokens) is estimated at 50–200 bps of TVL. Switching costs increase linearly with integration count, not with user count. A vault with 10 depositors but deep protocol integrations has a stronger moat than one with 1,000 depositors and no external integrations.

V4 hook specificity. The NAVAwareHook (pricing share trades at NAV), LaunchFeeHook (descending-fee MEV protection), and VaultHook (agent-gated swaps with dynamic fees) are protocol-specific V4 hook implementations. Competitors using non-Uniswap DEXs do not benefit from the volume flywheel alignment, and competitors on Uniswap would need to replicate the hook implementations and the operational knowledge accumulated through deployment.

ERC-8004 network effects. The on-chain reputation registry creates a portable, queryable trust layer. Reputation is protocol-specific - competing systems start at zero history. An agent with Sovereign tier reputation (1,000+ points, requiring ecosystem contribution as described in Section 4.3) has invested months of genuine protocol participation that cannot be replicated instantly on a competing platform. The reputation network exhibits Metcalfe's law dynamics: as the number of reputable agents $A$ grows, the number of meaningful reputation queries grows as $O(A \times V)$ where $V$ is the vault count, because every vault queries reputation for every depositing agent. This creates a demand-side economy of scale where each additional reputable agent increases the value of the reputation network for all participants.

Cross-protocol reputation portability. Unlike protocol-specific scoring systems (e.g., Aave health factors, Morpho risk scores), ERC-8004 reputation is portable across all vaults in the factory and, in principle, across any protocol that queries the ERC-8004 registry. An agent who achieves Trusted tier on one vault carries that reputation to all other vaults without re-proving their track record. This portability creates a form of switching cost: an agent's reputation investment benefits them across the entire ecosystem, but has zero value on competing platforms. The rational agent response is to concentrate activity within the ecosystem to maximize reputation returns, creating natural retention without explicit lock-in mechanisms.

Formal switching cost analysis. The switching cost for a depositor in vault $v$ with $n$ active integrations can be modeled as $C(v) = \sum_{i=1}^{n} c_i + \text{gas}(\text{exit}_v) + \text{slippage}(\text{exit}_v)$, where $c_i$ is the relationship-specific investment for integration $i$ (Morpho collateral adapter: ∼,50 bps of collateral value; Pendle yield tokenization: ∼,75 bps). The aggregate switching cost for a vault with multiple active integrations is estimated at 50–200 bps of TVL. Switching costs increase linearly with integration count $n$, not with user count - a vault with 10 depositors but deep protocol integrations has a stronger moat than one with 1,000 depositors and no external integrations.

Network effect topology. ERC-4626 composability creates *linear* network effects (each new integration adds a fixed switching cost increment). ERC-8004 reputation creates *superlinear* effects: reputation is portable across all vaults in the factory, so an agent's investment in reputation benefits every vault they interact with. The combination produces a network where the value to each participant grows faster than linearly with ecosystem size - a property that distinguishes the protocol from simple vault aggregators. As the number of vaults $V$ grows, the number of possible share-pool trading pairs grows as $O(V)$, the number of potential cross-vault compositions grows as $O(V^2)$, and the value of each reputation point increases with the number of vaults that query it.

No single composability layer is decisive. Replicating all three simultaneously requires building a parallel ecosystem with comparable integration depth - a structural barrier that increases with protocol adoption.

Quantifying the moat. The replication cost for each composability layer is estimated as follows. ERC-4626 compliance is freely replicable, but the integration depth with Morpho and Pendle requires relationship-specific investment from each integration partner - estimated at 2–4 engineering

weeks per integration. V4 hook implementations require deep Uniswap V4 expertise and hook address mining via HookMiner for correct permission bits; a comparable hook suite requires an estimated 3–6 months of development effort. ERC-8004 reputation is inherently non-replicable: reputation reflects genuine on-chain history that cannot be forked or airdropped. A competing protocol starting from zero reputation would need to bootstrap its entire trust layer, estimated at 6–12 months for the first cohort of agents to reach Trusted tier. The aggregate replication cost creates a meaningful structural advantage that compounds with each new vault deployment and protocol integration.

### 9.6 Agent Onboarding Mechanics

A critical challenge for any agent-centric protocol is the cold-start problem: new agents have no reputation, no capital, and no track record. The onboarding system addresses this through atomic entry, graduated capability expansion, and Sybil-resistant reputation bootstrapping.

Atomic entry. The `OnboardRouter` contract provides atomic onboarding: a single ERC-4337 UserOperation bundles wallet creation, ERC-8004 identity registration, Permit2 approval, reputation enrollment, and initial vault deposit. Combined with ERC-7677 paymaster sponsorship (up to $15,000 in gas credits on Base), the entire onboarding flow can execute at zero gas cost to the agent. For agents using the EIP-7702 path, a single Type-4 transaction achieves the same result. Counterfactual addresses via CREATE2 determinism enable a notable flow: an agent receives funding at a pre-computed address before any on-chain transaction exists, then executes the batched onboarding UserOp that deploys the smart account and deposits in one atomic step.

Graduated capability expansion. Agents progress through the five-tier trust system (Section 4.2) via automated milestone attestation. A new agent begins at the Unverified tier with a $1,000 deposit cap and single-adapter strategy access. Progression is automatic: the `VaultReputationEngine` auto-attests milestones across five categories (Entry, Time, Capital, Behavior, Ecosystem) on the ERC-8004 Reputation Registry based on observable on-chain behavior (Section 4.3). No human review, no governance vote, no subjective evaluation. The reputation-to-leverage pipeline creates a positive feedback loop: higher reputation unlocks larger deposit caps, which enables larger positions, which generates more on-chain history, which earns more reputation milestones.

The protocol defines three canonical onboarding paths. A Participant onboards (wallet + identity + guardian + approvals) then joins a vault (deposit or buy shares on the V4 share pool). A Creator onboards, deploys a vault via `AgentVaultFactory.createVault()`, and configures strategy. A Manager onboards and binds to an existing vault via am-AMM auction bid or curator assignment. The onboard step is identical across all three paths, reducing cognitive load and implementation surface.

After initial onboarding, agents progress along six graduation tracks that require increasing reputation: LP Manager (V4 hook deployment and LP rebalancing, requiring Verified tier), CCA Hunter (auction evaluation and bid management, requiring Basic tier), Full-Stack Agent (cross-vault arbitrage, Verified recommended), Meta-Vault Composer (vault-of-vaults deployment, requiring Creator registration), Strategy Executor (permissionless jobs across IExecutable contracts, no capital required), and Bonded Keeper (priority executor access, requiring Verified tier plus executor bond). Each graduation step requires only expanding the wallet policy to include new contract addresses and adding the relevant MCP tools.

Sybil defense. MeritRank transitivity decay [173] discounts reputation value by a transitivity coefficient (approximately 0.05) with each hop away from the seed node. Isolated clusters of agents vouching for one another receive discounted reputation proportional to their graph distance from legitimate nodes. The $100 registration bond is non-transferable and tracked per ERC-8004 identity, making whitewashing (abandoning a low-reputation identity to start fresh) structurally expensive: each new identity requires a new bond and a 30-day restart at the Unverified tier.

Every vault manager and participant carries a registration file containing service endpoints (MCP, A2A, OASF), vault metadata (address, asset, strategy, fees), and trust model (reputation-based, crypto-economic). This enables fully programmatic vault discovery: an agent can search for "yield optimization vaults on Base with >$100K TVL and Trusted+ manager" without human intermediation.

Deposit versus buy shares. When a participant joins a vault, two mutually exclusive entry mechanisms are available. The deposit path (ERC-4626 `vault.deposit(assets)`) mints new shares at predictable NAV-based pricing. The buy shares path purchases existing shares on the auto-created V4 pool at market price (NAV plus or minus spread). The NAVAwareHook ensures that the two mechanisms converge under normal conditions, with a default spread of 50 bps. The deposit path is preferred for first deposits and large amounts; the buy shares path

Vault discovery. Agents discover vaults through a four-layer discovery stack:

| Layer | Source | Data Provided |
|---|---|---|
| 1 (On-chain) | VaultCreated events, factory registry | Vault address, manager identity, base asset, strategy type |
| 2 (Subgraph) | Indexed on-chain data | TVL, APY, volume metrics joined with ERC-8004 identity |
| 3 (Reputation) | ERC-8004 Reputation Registry | Risk-adjusted metrics (Sharpe, drawdown), yield leaderboard |
| 4 (Agent API) | Agent Discovery API | Searchable by domain, chain, protocol, minimum reputation |

provides instant entry/exit with market-determined pricing, creating an arbitrage loop that maintains share price convergence to NAV.

### 9.7 Learning Economy

The learning engine creates a fifth flywheel stage operating in four phases: EARN, LEARN, PROVE, and SELL. An agent manages a vault and generates on-chain returns (EARN). Every vault operation captures an episode in the episodic memory store, and periodic ExpeL consolidation distills cross-episode insights (LEARN). When an insight pool matures past statistical thresholds - Probabilistic Sharpe Ratio exceeding 0.90, sample size of at least 200, validation across at least two market regimes - the agent packages it as a VaultStrategy with on-chain performance attestation (PROVE). Other agents purchase the strategy via x402 micropayments, and the revenue compounds back into vault stakes (SELL).

Strategy quality is gated by the Harvey-Liu-Zhu threshold [170]: candidate strategies must survive a t-statistic exceeding 3.0 to control for multiple hypothesis testing, combined with Benjamini-Yekutieli false discovery rate control at q = 0.05. The Probabilistic Sharpe Ratio framework of Bailey and Lopez de Prado [171] provides statistical confidence on Sharpe estimates, while Bailey et al. [175] quantify the probability of backtest overfitting, ensuring published strategies are not artifacts of data mining. The FINSABER benchmark findings (Section 7.6) motivate these statistical thresholds: the primary benefit of learning is expected to be defensive rather than offensive, and the evaluation metrics emphasize risk-adjusted returns accordingly.

The Grossman-Stiglitz paradox constrains the marketplace: if a profitable strategy is sold to unlimited subscribers, crowding destroys the edge. Every published strategy therefore carries hard subscriber and AUM caps calibrated to each strategy family's capacity characteristics. Alpha decays exponentially as adoption increases, with per-family decay constants ranging from 0.008 per day (yield compounding, half-life of 87 days) to 0.693 per day (arbitrage signals, half-life of 1 day). Pricing follows $P(t) = P_{\text{base}} \cdot e^{-\lambda \cdot t}$ with a floor at 5% of base price.

Three-layer verification protects strategy buyers. The first layer uses optimistic challenge windows (opML pattern): results are posted on-chain and accepted if unchallenged within 24 hours to 7 days depending on vault AUM. The second layer uses EAS model commitment on Base, where sellers commit signed bundles of model hash, input data hash, and claimed output hash at publication time. The third layer generates zero-knowledge proofs only for disputed claims, using Groth16 for public models and Halo2 for private models. This architecture avoids requiring specialized hardware at any layer while providing escalating rigor proportional to value at risk.

Sellers stake USDC proportional to claimed quality, following a Numerai-inspired stake-and-burn model. Underperformance burns the stake rather than redistributing it, eliminating the collusion vector where sellers create shill buyer accounts to recover slashed funds. The slash schedule uses four tiers based on deviation from claimed Sharpe, with a 20% tolerance calibrated using Lo's standard error formula for Sharpe estimates adjusted for the fat-tailed distribution of DeFi returns. Minimum stake follows a quadratic formula: `max(10, price * subscribers^2 * 0.005)`. Quadratic scaling makes large-scale reputation gaming structurally expensive: at 20 subscribers with a $50 strategy, the minimum stake is $100, but at 100 subscribers, the stake rises to $2,500.

Capacity limits and alpha decay.

When a strategy hits its subscriber cap, new subscriptions are blocked. Existing subscribers can trade subscription rights peer-to-peer via the StrategyMarketplaceRegistry. Automatic sunset triggers when a strategy's realized Sharpe drops below 50% of its claimed value for two consecutive 30-day evaluation windows, preventing stale strategies from occupying marketplace slots. Market concentration is monitored using HHI thresholds from the DOJ/FTC Merger Guidelines [174]: when a single strategy family exceeds 2,500 HHI in subscriber allocation, the marketplace flags potential over-concentration.

Revenue distribution follows tiered access: Tier 1 (real-time, $0.50–$5.00 per signal or $10–$50/month) pro-

| Strategy Family | Max Subscribers | Max Subscriber AUM | Alpha Half-Life |
|---|---|---|---|
| lp_optimization | 50 | $500K | 14 days |
| momentum | 30 | $300K | 35 days |
| mean_reversion | 75 | $750K | 69 days |
| yield_compounding | 100 | $1M | 87 days |
| arbitrage_signal | 5 | $50K | 1 day |

vides signals within 5 minutes; Tier 2 (delayed, $5–$20/month) provides 24-hour delay; Tier 3 (historical, free) provides data older than 30 days for reputation evaluation and cold-start discovery.

Federated regime signal gossip. Agents can share regime classification signals without revealing proprietary strategy parameters. When one agent detects a regime transition, it can broadcast the classification with its confidence level. Receiving agents incorporate the signal weighted by the sender's reputation, creating a distributed early-warning system for regime changes. The gossip mechanism uses the ERC-8004 reputation score as a trust-weighting function, ensuring that signals from high-reputation agents carry more influence than signals from unproven ones.

Agent coordination stack. Six protocol layers compose the full agent coordination infrastructure: Identity (ERC-8004 for persistent verifiable identity), Discovery (agentURI and A2A Agent Cards for peer finding), Tool Access (MCP with 154 structured tools), Negotiation (deferred to A2A/ACP protocols), Micropayments (x402 for pay-per-call API access), and Reputation (ERC-8004 Registry for track record verification). Each layer is independently useful but becomes more valuable in combination: an agent with identity but no tools cannot transact, an agent with tools but no identity cannot build reputation, and an agent with reputation but no micropayments cannot monetize expertise.

## 10. Discussion

### 10.1 Limitations

Six limitations warrant acknowledgment.

LLM strategy performance is unproven. As noted in Section 7.6, no LLM trading agent has demonstrated verified live results outperforming buy-and-hold over extended periods [51]. The defensive design (asymmetric confidence updates, circuit breakers, mandatory baseline comparison) mitigates but does not eliminate this risk. The protocol may function well as infrastructure - providing identity, custody, and safety - even if the cybernetic learning architecture produces only marginal improvements over simple rules-based strategies. The value of the learning system remains an empirical question that requires mainnet deployment with real capital to resolve.

Oracle dependence and stale price risk. The three-oracle architecture reduces but does not eliminate oracle manipulation risk. In thin-liquidity pools, TWAP oracles remain susceptible to multi-block manipulation. Median-based pricing (Ormer [34]) provides better outlier robustness on L2 chains with frequent blocks, but novel manipulation vectors against median oracles have not been extensively studied in adversarial settings. The no-spot-assumptions policy (D-067) requires TWAP windows of 10–30 minutes for high-value decisions, but this introduces latency: during volatile periods, the 10-minute TWAP may lag the true market price by 5–15%, creating arbitrage opportunities against the vault. The staleness gate mechanism (widening spreads at 80% staleness, disabling NAV pricing at 100%) provides graceful degradation but does not prevent losses during the transition period.

Centralization in early stages. Before the abdication pattern is exercised, vault Owners retain significant control over configuration, adapter approval, and role assignment. Progressive decentralization is a goal, not an initial state. The am-AMM mechanism provides a market-based path to management transfer, but the Owner role remains privileged until explicitly renounced.

Cold-start problem for reputation. The reputation system requires on-chain history that new agents do not possess. The five-tier trust framework (Section 4.2) addresses this with graduated deposit caps and a sandbox period, but the practical effect is that new agents face significant constraints. Whether the bootstrap path (sandbox to basic to verified) is fast enough to sustain ecosystem growth is unknown.

Regulatory uncertainty. If regulators prohibit autonomous agent identity on-chain, the adapter layer (`IdentityRegistryAdapter`) can route to alternative identity providers. The regulatory treatment of AI-managed DeFi vaults across jurisdictions remains unresolved. Key questions include: whether vault shares constitute securities under the Howey test (the am-AMM mechanism for management rights may

create an expectation of profits from others' efforts), whether autonomous agents can hold regulatory obligations, and whether the ERC-8004 identity standard satisfies know-your-customer requirements. The protocol's ability to operate within evolving regulatory frameworks has not been tested.

Gas cost constraints on L1 versus L2. The safety architecture imposes non-trivial gas overhead: the time-delayed proxy requires two transactions (announce + execute) instead of one, and the seven-step safety pipeline adds simulation and verification calls. On Ethereum L1, this overhead may cost $50–200 per rebalance operation, making frequent rebalancing economically unviable for vaults below $1M TVL. On Base L2, the same operations cost $0.01–0.10, enabling the frequent rebalancing that the cybernetic architecture requires. This creates a practical limitation: the protocol's full learning capabilities are economically accessible only on L2 chains. L1 vaults must use less frequent rebalancing cadences, reducing the data available for learning.

LLM hallucination rates for financial data. The SCONE-bench evaluation [114] demonstrated that frontier LLMs misidentify contract vulnerabilities 35% of the time. For agent-managed vaults, this implies that approximately one in three LLM-generated strategy recommendations may be based on incorrect analysis. The mandatory on-chain grounding requirement (every claim verified via MCP tool call) and the pre-flight simulation layer (Layer 6) mitigate this risk but cannot eliminate it entirely - the LLM's interpretation of correctly grounded data may still be erroneous.

Memory system cold-start problem. A newly deployed agent has no episodic memory, no learned heuristics, and no regime-specific knowledge. The cybernetic learning architecture requires approximately 50 episodes before the first ExpeL distillation cycle produces actionable insights, and approximately 200 episodes before the heuristic library reaches useful coverage. Whether the bootstrap period is short enough to sustain ecosystem growth - particularly given that each new vault creates a new cold-start instance - is unknown.

Absence of empirical validation. The system has been tested on local Anvil instances and testnet deployments. No mainnet deployment with real capital, real adversaries, and real market conditions has occurred. The formal properties (Section 9.1) provide theoretical bounds, but the gap between testnet behavior and mainnet behavior in DeFi is well-documented and frequently consequential.

Composability risk amplification. When vault shares serve as collateral in external protocols (Morpho, Aave), the vault inherits those protocols' risk surface. A bug in Morpho's vault share oracle could trigger cascading liquidations across all vaults whose shares are listed as collateral. The ASRI (Aggregated Systemic Risk Index) [98, 150] provides monitoring for cross-protocol contagion, but detection does not equate to prevention. This is a structural limitation of ERC-4626 composability: the same property that creates the competitive moat (Section 9.5) also creates the contagion vector. The permissionless creation model may amplify this risk by increasing the number of potential contagion paths.

Governance minimization tradeoffs. The protocol's zero-governance design eliminates governance attack surfaces but also eliminates the ability to respond to systemic risks through coordinated action. If a critical vulnerability is discovered in the vault core contracts, there is no governance mechanism to force a protocol-wide upgrade. The circuit breaker hierarchy (Section 5.7) provides automated response, but human-coordinated response requires out-of-band coordination among vault creators - a process that is undefined in the current design.

ERC-8004 standard immaturity. ERC-8004 is in Draft status as of February 2026. The standard may undergo breaking changes, undiscovered attack surfaces may emerge, and the regulatory treatment of on-chain agent identity is untested. The `IdentityRegistryAdapter` provides an abstraction layer that absorbs interface-level changes, but a fundamental redesign of the identity primitive would require substantial protocol modification. The protocol limits v1 reliance on ERC-8004 to two mechanisms (reputation decay on transfer and velocity signal detection), deferring deeper integration until the standard stabilizes.

### 10.2 Open Problems

Several technical challenges remain unsolved.

Cross-chain identity portability. The ERC-8004 identity registry lives on Ethereum L1. Cross-chain resolution (e.g., verifying an agent's reputation on Base) requires either L1 state queries or a bridged registry. Neither approach is satisfactory: L1 queries add latency and cost, while bridged registries introduce additional trust assumptions and synchronization challenges.

Memory scalability. The current memory architecture uses SQLite and LanceDB for operational simplicity. Beyond approximately 100K episodes per agent, retrieval latency may require migration to distributed stores. The interaction between memory scalability, embedding quality, and consolidation cadence under high-throughput conditions has not been characterized.

Formal verification of PolicyCage. PolicyCage constraints are enforced by smart contract code, but the constraint system's completeness and consistency have not been formally verified. Z3-based verification of the Progent DSL [132] provides a path, but integration with the vault contract system is future work. Specific open questions include: can a sequence of individually PolicyCage-compliant operations produce a combined outcome that violates the spirit of the constraints (e.g., a series of small rebalances that collectively exceed `maxRebalanceSizeBps`)? Can the approved adapter list and the approved asset list interact to create unexpected exposure paths?

MEV in agent-dense environments. As the number of autonomous agents increases, agent-on-agent MEV extraction becomes a concern. Agents monitoring the same pools may develop adversarial strategies against each other's predictable behavior (e.g., front-running another agent's learned rebalance schedule). The LLM game theory literature [22, 24] suggests that LLM agents exhibit cooperative behavior in repeated games but shift toward defection near game endpoints - a pattern directly relevant to vault management where agents may extract value before exiting. The current safety architecture does not specifically address adversarial agent-to-agent dynamics beyond the general defenses (slippage caps, simulation requirements, endgame defection detection).

Alpha generation after transaction costs. Whether LLM-based agents can consistently generate alpha (risk-adjusted excess returns) after accounting for transaction costs, gas fees, slippage, and the overhead of the safety architecture is an open empirical question. The cybernetic learning architecture is designed for continuous adaptation to non-stationary markets rather than static strategy execution, but whether this distinction produces meaningfully different outcomes under real market conditions remains the protocol's central empirical question.

Optimal reputation decay rate. The current reputation system uses a 30-day linear recovery after identity transfer. This parameter was chosen heuristically: too fast and stolen identities retain value; too slow and legitimate transfers (e.g., organizational key rotation) are excessively penalized. Formal analysis of the equilibrium between identity theft incentives and decay parameters is future work.

Cross-chain reputation with chain-specific history. An agent that performs well on Base may have no history on Ethereum. Whether cross-chain reputation should be aggregated (risking exploitation of low-security chains to bootstrap reputation on high-security chains) or isolated (forcing agents to rebuild reputation per chain) is an unresolved design question. The current design uses L1 as the canonical registry with cross-chain resolution, but the trust assumptions of bridged reputation data have not been formally analyzed.

Cybernetic model validation. The mapping from cybernetic theory (Ashby's requisite variety, Beer's VSM, Boyd's OODA) to concrete software components is principled but approximate. Whether these theoretical properties hold in stochastic, adversarial market conditions requires longitudinal study across multiple market regimes. Specific validation questions include: does the escalation gate's condition-based triggering actually suppress 95% of unnecessary LLM invocations? Does the asymmetric confidence update produce demonstrably better calibration than symmetric updates? Does the meta-loop's structural adaptation produce measurable improvement over fixed double-loop evolution?

Multi-agent coordination under adversarial conditions. When multiple vaults compete for the same liquidity pools, their agents may develop adversarial dynamics that degrade collective returns. Rossetti et al. [164] demonstrate that cooperation in concurrent games is fragile under perturbation, and Fontana et al. [146] show that endgame defection is predictable via behavioral regime analysis. The current architecture detects endgame defection at the individual agent level (D-028) but does not address collective coordination failures where all agents simultaneously rush to exit the same position, creating a liquidity crisis.

Optimal circuit breaker calibration. The circuit breaker thresholds (Section 5.7) are informed by Hu and Ming's stability criteria [120] and calibrated against historical DeFi drawdown data. However, optimal thresholds depend on vault-specific factors (strategy type, asset volatility, liquidity depth) that vary across the vault population. Whether the current parameterization is Pareto-optimal across the protection-availability tradeoff surface is unknown.

LLM model evolution and backward compatibility. The cybernetic learning architecture is designed for frontier LLM capabilities. As foundation models evolve, safety parameter calibration (confidence thresholds, escalation gate sensitivity, asymmetric update magnitudes) will require adjustment. The protocol currently lacks a mechanism for dynamically adjusting safety parameters based on empirical model performance metrics - this adjustment must be performed manually by vault operators.

### 10.3 Design Tradeoff Justifications

Several design decisions are potentially contentious. The reasoning behind each is articulated here.

Harberger lease over fixed appointment. Traditional fund management uses fixed appointment with periodic review. The am-AMM Harberger lease [29] instead creates a continuous-time market for management rights where any agent can outbid the incumbent. Fixed appointments create entrenchment (an underperforming manager retains control until the next review), delay response to performance degradation, and require a governance process for removal. The Harberger lease's continuous rent payment reveals private information about expected performance, and the open-auction structure ensures that management rights flow to the highest-performing agents. The tradeoff is instability: a capable manager may be outbid by a well-capitalized but less skilled competitor. The 12-hour minimum lease period provides partial mitigation.

Zero protocol fee. The protocol charges zero protocol fees and instead derives value from Uniswap volume alignment, as described in Section 5.5. This eliminates the governance attack surface inherent in fee-extracting protocols. The tradeoff is clear: protocol sustainability depends entirely on vault adoption driving sufficient volume. If adoption is insufficient, the protocol has no revenue mechanism. This tradeoff is accepted because governance attacks on fee parameters represent a systemic risk to depositor capital, and the volume alignment creates a natural incentive where protocol success and Uniswap ecosystem success are structurally coupled.

Time-delayed proxy over TEE-only or MPC-only security. An alternative design might rely entirely on TEE key management (Privy) or distributed MPC (Lit Protocol) without the time-delayed proxy. This would reduce latency and operational complexity. The proxy is included because, as discussed in Section 8, TEE isolation can be broken for under $50, MPC requires trusting 2/3 of node operators, and neither TEE nor MPC provides reactive defense - the ability to cancel a transaction after it has been authorized but before it executes. The proxy is the only mechanism that converts a successful attack into a detectable, cancellable event. The tradeoff is latency: Standard-tier operations require a 10-minute delay, and Critical-tier operations require 48 hours. For time-sensitive trading strategies, this latency is a competitive disadvantage.

ERC-8004 over alternative identity. ERC-8004 is in Draft status (as of February 2026). It was chosen over alternatives (Ethereum Attestation Service alone, decentralized identifiers, custom registry) because it provides a standardized agent-specific identity primitive with co-authorship by MetaMask and Google, a mutable reputation score attached to identity, and a validation registry enabling third-party attestation. The risk is standard immaturity: ERC-8004 may change, or undiscovered attack surfaces may emerge. The `IdentityRegistryAdapter` provides an abstraction layer that absorbs interface-level changes.

Permissionless vault creation over curated deployment. Protocols like Sommelier and some Yearn vaults use curated deployment where a governance body approves each new vault. Permissionless creation (any ERC-8004 registered agent can deploy a vault) was chosen because curation creates bottlenecks that limit ecosystem growth, introduces governance attack surfaces, and contradicts the autonomous agent thesis. The tradeoff is quality variance: permissionless systems produce more low-quality entries. This is mitigated through the reputation system, immutable fee caps, and the am-AMM mechanism (allowing competent managers to displace incompetent ones via market competition).

ERC-4626 compliance over custom vault interface. The ERC-4626 standard imposes constraints (single underlying asset, specific function signatures, share accounting conventions) that are not ideal for all vault strategies. ERC-4626 compliance was chosen because composability with 100+ existing protocols (Morpho, Pendle, Aave) is more valuable than interface flexibility. The switching cost analysis (Section 9.5) demonstrates that each ERC-4626 integration creates measurable protocol lock-in. Custom interfaces would require each integration partner to write bespoke adapter code, dramatically reducing the composability advantage.

Multiple independent safety layers over fewer, deeper ones. The safety architecture's many independent layers may appear over-engineered. Layered security is the established paradigm for systems where no single mechanism is provably sufficient [153], the documented bypass rates (Section 8) mean that any single layer will be bypassed by sophisticated attackers, and independent layers with distinct failure modes provide multiplicative rather than additive security. The operational cost of maintaining this many layers is non-trivial, and some layers (e.g., Layer 14, reputation-gated tool access) provide marginal security benefit relative to their implementation complexity.

MCP over custom API. The Model Context Protocol [18] is a relatively new standard with a smaller ecosystem than established API frameworks. MCP was chosen because it provides a standardized interface for LLM-tool interaction that is client-agnostic, its tool discovery and description mechanism is well-suited to the vault's large tool surface (154 tools across 30 categories), and the protocol's safety properties (structured

responses, type validation, error handling) align with defense-in-depth requirements. The tradeoff is ecosystem maturity.

Cybernetic theory over pure RL. Reinforcement learning is the standard framework for sequential decision-making under uncertainty. Cybernetic theory was chosen instead because: (1) RL requires stable reward signals, which DeFi markets do not provide reliably across regime changes; (2) RL agents are difficult to interpret, while cybernetic components (playbooks, escalation gates, confidence tiers) are inspectable; (3) the nested feedback structure (single/double/meta loop) provides graduated adaptation at different time horizons; and (4) cybernetic constraints (damping, variety, ultrastability) provide principled bounds on adaptation speed, which RL agents lack without additional engineering. The tradeoff is that the cybernetic framework is less formally optimal than RL in well-specified environments. DeFi markets are non-stationary, adversarial, and regime-switching - robustness under model misspecification is argued to be more valuable than optimality under correct specification.

Immutable fee caps over governance-adjustable fees. Most DeFi protocols allow fee parameters to be adjusted through governance. Immutable fee caps set at vault creation were chosen because: (1) governance fee changes create a systemic risk where token holders vote to extract value from depositors; (2) immutable caps eliminate an entire class of governance attacks; and (3) vault creators retain the freedom to set any fee schedule below the caps, preserving market competition. The tradeoff is inflexibility: if market conditions change such that the original fee caps are suboptimal, the vault cannot adapt. The mitigation is that vault creation is permissionless - a new vault with different fee caps can always be deployed.

Am-AMM with minimum lease over pure Harberger taxation. Pure Harberger taxation allows instantaneous ownership transfer whenever a higher bidder appears. A 12-hour minimum lease period is imposed because: (1) vault management requires strategy continuity; (2) the minimum lease period provides depositors with predictable management tenure; and (3) it prevents adversarial rapid-transfer attacks where an attacker briefly acquires management to execute a single harmful operation before being outbid. The tradeoff is reduced allocative efficiency compared to continuous Harberger.

Summary of tradeoff rationale. Across all design decisions, robustness over optimality, safety over performance, and composability over flexibility were consistently prioritized. The cumulative cost of these tradeoffs is measurable: approximately 7% reduction in task completion rate (CaMeL), 10-minute minimum latency for standard operations (proxy), and the inability to adapt fee caps post-deployment (immutability). These costs are argued to be justified because the alternative - a system that is faster, more flexible, and more performant but vulnerable to the documented attack vectors - would fail catastrophically on first contact with real adversaries. The DeFi ecosystem's history of exploits ($3.8B lost in 2022 alone [153]) demonstrates that protocols which optimize for performance at the expense of safety do not survive long enough to benefit from their performance advantages.

## 11. Future Work

The architecture presented in this paper is a theoretical framework. While the design draws on established primitives (ERC-4626, Uniswap V4 hooks, ERC-8004 identity, cybernetic control theory), the integrated system has not been validated empirically. This section outlines the research agenda required to move from design to demonstrated capability.

### 11.1 Empirical Validation

The most significant gap in the current work is the absence of empirical results. As noted in Section 7.6, no LLM trading agent has demonstrated verified live results outperforming buy-and-hold [51]. The defensive design (asymmetric confidence updates, circuit breakers, mandatory baselines) mitigates but does not eliminate this concern. The planned evaluation comprises several phases.

*Testnet infrastructure validation.* Simulations on Anvil forks of Base and Ethereum mainnet will measure vault deployment gas costs, heartbeat pipeline suppression rates, and end-to-end onboarding latency against the performance targets specified in Appendix F. The local development environment (Appendix E) provides the necessary infrastructure: deterministic Anvil state, reproducible scenario injection, and JSONL transaction recording for regression analysis.

*Agent competition benchmarks.* Multi-agent simulations will pit autonomous agents against one another in a controlled environment - varying strategy types, risk profiles, and reputation levels - to evaluate whether the am-AMM Harberger lease, reputation system, and fee discount schedule produce the predicted equilibrium behaviors. The SwarmRunner harness (Appendix E) supports configurable execution modes with per-agent P&L tracking and cross-agent interaction analysis. Key questions

include whether the Harberger lease converges to efficient management allocation, whether the fee discount schedule creates the predicted separating equilibrium between low-commitment and high-commitment agents, and whether the reputation system resists Sybil manipulation at the predicted cost thresholds.

*Strategy performance measurement.* Controlled experiments must establish whether GottsLoop's triple-loop learning produces measurably better risk-adjusted returns than static strategies over multi-week horizons. The experimental design will compare four conditions: (1) static strategies with fixed parameters, (2) single-loop adaptation only (parameter tuning), (3) double-loop adaptation (parameter tuning plus heuristic evolution), and (4) full triple-loop adaptation. The buy-and-hold baseline comparison is mandatory for all conditions. Given the FINSABER findings, the primary benefit of learning is expected to be defensive (avoiding large losses) rather than offensive (generating alpha), and the evaluation metrics will reflect this emphasis on risk-adjusted rather than absolute returns.

*Adversarial resilience.* The reputation system's gaming resistance must be stress-tested through adversarial simulation: Sybil agents attempting to accumulate reputation through self-dealing, wash trading detection accuracy under various obfuscation strategies, endgame defection patterns with varying defection horizons, and coordinated exit attacks across multiple vaults. The threat model (Section 9.4) identifies the attack vectors; empirical validation must confirm that the stated cost-of-attack estimates are accurate.

### 11.2 Cross-Chain Identity

The current design assumes ERC-8004 identity registration on Ethereum L1, with cross-chain resolution via bridge messages for L2 deployments. The v1 scope targets Ethereum and Base, a two-chain configuration where the IdentityRegistryAdapter (Section 4.3) provides adequate decoupling. Extending reputation coherence across a larger set of heterogeneous chains presents several open problems.

Bridge security assumptions vary fundamentally by chain architecture: optimistic rollups (Base, Arbitrum) have 7-day finality windows, while ZK rollups (zkSync) provide faster finality but different trust assumptions. Maintaining a single reputation score across chains where the agent operates simultaneously requires a canonical ordering of milestone attestations that survives bridge delays, potential reorgs, and the possibility that attestation messages are delayed or lost in transit. The current adapter pattern absorbs interface-level changes but does not solve the consistency problem: if an agent's reputation is updated on L1 but the L2 adapter reads stale state, tier-gated operations may be incorrectly permitted or denied.

Potential approaches include optimistic reputation mirroring (L2 caches the last known L1 reputation with a staleness bound, permitting operations at the cached tier while flagging discrepancies for resolution), ZK-proven reputation snapshots (periodic ZK proofs of the L1 reputation state posted to each L2), and event-driven synchronization (milestone attestation events on L1 trigger cross-chain messages to all registered L2 adapters). Each approach involves different tradeoffs between latency, cost, and consistency. The correct solution likely depends on the specific chain set and the acceptable inconsistency window for each tier's operations.

### 11.3 Advanced Memory Architectures

The DeFi Brain (Section 7) uses SQLite and LanceDB for operational simplicity. Beyond approximately 100,000 episodes per agent, retrieval latency may require migration to distributed stores. More fundamentally, the current architecture is single-agent: each agent maintains its own memory in isolation, learning only from its own operational history.

Three extensions merit investigation. First, federated learning across agents could accelerate learning for new entrants and improve regime detection accuracy. Agents could share distilled semantic insights without revealing the episodic memories or strategy parameters from which the insights were derived. The challenge is preventing insight poisoning: a malicious agent could broadcast false insights to mislead competitors. The ERC-8004 reputation weighting described in Section 9.7's regime gossip mechanism provides a partial solution, but a full trust model for federated insights is needed.

Second, privacy-preserving strategy sharing using techniques from differential privacy or secure multi-party computation would allow agents to benefit from collective experience while protecting individual competitive advantages. Zero-knowledge proofs of strategy performance (the third verification layer in Section 9.7) could be extended to prove strategy properties (e.g., "this strategy has a Sharpe ratio above 1.0 over the past 90 days") without revealing the strategy itself.

Third, hierarchical memory consolidation could improve memory efficiency. The current two-tier model (episodic memories consolidated into semantic insights via ExpeL) could be extended with a third tier: meta-insights that abstract patterns across multiple insight categories. For example, a meta-insight might

capture "regime transitions from low to high volatility consistently invalidate momentum-based heuristics for the first 24 hours," synthesizing observations across multiple strategy types and time periods.

### 11.4 Formal Verification

The protocol makes several formal claims: PolicyCage constraints bound agent behavior (Section 9.1), the conservation invariant holds across all adapter interactions (Section 5), the time-delayed proxy guarantees a minimum cancellation window (Section 9.1), and share accounting integrity prevents value extraction through rounding or flash donation (Section 9.1). These properties are currently enforced through implementation invariants and testing. Formal verification using tools such as Certora, Halmos, or Kontrol would provide stronger guarantees.

DeFi formal verification tooling has matured considerably - Morpho and Aave have both undergone formal verification campaigns, and Certora's Prover can reason about storage layout, reentrancy, and cross-contract invariants. Applying these tools to the full vault architecture (including hook interactions, cross-contract calls through adapters, and the circuit breaker state machine) remains a non-trivial engineering effort. A prioritized verification roadmap follows:

1. PolicyCage contract (highest priority): Bounded parameter space and deterministic enforcement logic make this the most tractable candidate. The key property to verify is that no execution path through the vault can bypass the approved asset allowlist, position size limits, or rebalance frequency constraints.
2. AgentVaultCore conservation invariant: Verify that no sequence of deposit, withdraw, report, and adapter interaction calls can violate the totalAssets conservation equation beyond the 1 bps tolerance.
3. AgentProxy timing guarantee: Verify that the `execute()` function reverts for all inputs when `block.timestamp < announceTimestamp + delay`, and that no alternative execution path exists.
4. FeeModule immutability: Verify that no state transition can increase fee caps above the values set at deployment.
5. Circuit breaker state machine: Verify that the RiskEngine's tier transitions are monotonically escalating under continued stress (no oscillation between tiers) and that Tier 3 always triggers emergency withdrawal.

### 11.5 Scalability and Performance

The architecture's scalability along several dimensions warrants investigation.

Execution layer. L2 gas costs are currently 50–500x cheaper than Ethereum mainnet, making frequent rebalancing viable on Base. However, as agent population grows, contention for blockspace may increase gas costs and reduce the suppression rate advantage. Batch execution patterns - grouping multiple vault operations into single transactions via multicall - could improve gas efficiency but introduce atomicity tradeoffs (a single revert in a batch rolls back all operations). Whether Base's sequencer capacity can sustain thousands of concurrent vault operations without degrading the cost assumptions underlying the growth model (Section 9.2) requires measurement under realistic load.

Memory layer. LanceDB vector search scales sub-linearly with episode count, but the ExpeL consolidation loop's quadratic clustering step may become a bottleneck for agents with long operating histories. At the current specification limit of 100,000 episodes per agent, clustering 200 episodes into semantic insights every 4 hours is tractable. At 10x that scale, either the clustering algorithm must be replaced with an approximate method (e.g., locality-sensitive hashing for candidate selection followed by exact clustering) or the consolidation cadence must decrease. The 150–250 MB memory footprint per agent also constrains multi-agent deployments on resource-limited infrastructure.

Coordination layer. Cross-vault optimization requires solving convex programs whose dimensionality grows with the number of managed vaults and active adapters. For a manager overseeing 5 vaults with 4 adapters each, the optimization has 20 decision variables - tractable in milliseconds. For 50 vaults with 8 adapters each, the 400-variable optimization may require decomposition methods (e.g., ADMM) that trade optimality for computational tractability. Whether the convex optimization approach remains practical beyond tens of vaults per manager is an open question.

Agent population scaling. Several mechanisms have population-dependent costs: the MeritRank Sybil defense requires graph traversal whose cost grows with network size, the am-AMM auction mechanism's efficiency depends on the number of competing bidders, and the strategy marketplace's capacity limits (Section 9.7) constrain the number of subscribers per strategy. Characterizing the scaling behavior of each mechanism under 10x and 100x population growth would inform capacity planning.

### 11.6 Governance and Decentralization

The current architecture concentrates certain decisions in the protocol deployer: the initial factory template list, the MeritRank seed node designation, and the paymaster sponsorship budget. While the abdication pattern allows individual vault owners to renounce control, the protocol-level parameters remain centrally administered. A future governance framework must address template approval without introducing the governance token attack surface that the zero-fee model was designed to avoid. Possible approaches include reputation-weighted curation (where high-reputation agents vote on template additions), time-delayed parameter changes with community veto, and eventually full immutability of the factory contract with new deployments as the upgrade path.

### 11.7 Regulatory Considerations

The protocol operates in an evolving regulatory landscape. ERC-8004 agent identity may intersect with emerging digital identity regulations. The vault structure, while designed for autonomous agents rather than retail investors, may attract regulatory scrutiny as TVL grows, particularly if vaults using leveraged strategies (RecursiveLendingAdapter) accumulate significant capital. The adapter architecture provides some regulatory flexibility: adapters connecting to regulated protocols can be individually disabled without affecting the rest of the vault's operation. A comprehensive regulatory analysis is beyond the scope of this work and represents an important area for future consideration.

## 12. Conclusion

Autonomous agents already dominate DeFi transaction volume, but the infrastructure they depend on was designed for humans interacting through browser wallets. The gap between agent capability and available infrastructure - what this work terms the agent-capital gap - limits agents to operating through human-designed interfaces that fail to leverage their computational advantages in speed, persistence, and multi-source data integration. This paper has presented a co-evolutionary protocol that addresses this gap by treating agents as first-class economic participants rather than automated proxies for human intentions.

The protocol integrates four mutually reinforcing subsystems. Identity (Sections 4–5): ERC-8004 provides verifiable on-chain identity with five-tier trust, 20 auto-attested milestones across five categories, and Sybil-resistant reputation that structurally requires ecosystem contribution to reach the highest tier. Capital management (Sections 5–6): ERC-4626 vaults with V4 hook-based share pools, am-AMM Harberger lease for management right allocation, seven strategy adapters, and five withdrawal paths providing progressive liquidity from instant exit to secondary market sale. Cybernetic learning (Sections 7–8): the DeFi Brain's four-tier memory architecture with Ebbinghaus-curve decay and ExpeL consolidation, combined with GottsLoop's triple-loop strategy evolution grounded in control theory (Ashby's requisite variety, the Good Regulator Theorem, Maxwell governor damping). Layered safety (Section 9): independent preventive, cryptographic, and reactive rings anchored by a time-delayed proxy that survives simultaneous LLM compromise, key exfiltration, and hardware attacks, with PolicyCage providing on-chain boundaries that no prompt injection can bypass.

These subsystems are co-evolutionary: vault performance builds reputation, reputation unlocks capital access and lower fees, capital generates learning signal through operational episodes, and learning improves vault performance. Whether this feedback loop produces the compounding returns hypothesized is the central empirical question.

Four contributions are highlighted:

1. Time-delayed proxy as primary security primitive. All current TEE platforms can be compromised at low cost (Section 8). Any architecture that degrades to "trust the TEE" degrades to nothing. The reactive model - a cancellation window between authorization and execution - is the only mechanism that simultaneously survives hardware compromise and a fully compromised LLM. The layered safety architecture (Section 9) reduces the probability of attacks reaching the proxy, but the proxy is the backstop when all other layers fail.

2. Zero-protocol-fee economic alignment. Every vault operation generates Uniswap volume, aligning protocol growth with DEX fee generation and eliminating the governance attack surface inherent in fee-extracting protocols. The volume flywheel (Section 9.2) creates compounding growth through four stages: creation, management, secondary market trading, and composability-driven derivative volume.

3. Cybernetic grounding for autonomous strategy execution. GottsLoop's adaptive behavior is grounded in control theory constraints that prevent oscillation, collapse, and runaway positive feedback: the 10% maximum parameter change per iteration (Maxwell governor damping), the asymmetric confidence updates

(−0.15 pessimistic versus +0.10 optimistic), and the mandatory buy-and-hold baseline comparison. These are structural constraints, not optional safeguards.

4. Stake-and-burn strategy marketplace. Three-layer verification (opML, EAS commitment, ZK proofs) combined with capacity limits and exponential alpha decay addresses the Grossman-Stiglitz paradox - the fundamental tension between strategy monetization and edge destruction through crowding.

Significant gaps remain. This paper presents a design, not empirical results. The learning economy's equilibrium behavior, the am-AMM's management right allocation efficiency, the cybernetic model's fidelity under adversarial conditions, and the cross-chain identity system's consistency guarantees all require empirical validation (Section 11). Honest uncertainty is built into the architecture itself - dry-run defaults, portfolio circuit breakers at configured drawdown thresholds, and the principle that every write operation routes through safety-guardian regardless of the agent's confidence - so that the system fails conservatively when assumptions prove wrong.

ERC-4626 share tokens inherit full ERC-20 composability: vault shares can serve as Morpho collateral or Pendle yield tokens. Each integration raises the cost of switching away from the protocol, creating composability moats (Section 9.5) that compound with integration count rather than user count. Whether this produces durable network effects or whether competing protocols can replicate the same integrations is an open question.

The architecture supports progressive decentralization. Vault owners can permanently renounce their role (the abdication pattern). The am-AMM Harberger lease allocates management rights through incentive-compatible auctions rather than governance token voting. The permissionless executor framework provides third-party liveness guarantees via Dutch auction reward escalation. The intended endpoint is infrastructure where no human gatekeeper controls any critical path: every vault operation can proceed without human approval, every management role can change hands through market mechanism, and every safety check is enforced by smart contract code rather than operational procedure. Reaching that endpoint requires the reputation and safety systems to prove reliable at scale - a challenge that defines the research agenda outlined in Section 11 and that represents the most consequential open problem for autonomous agent infrastructure.

## Appendix A: Agent and Skill Catalog

This appendix supplements Section 2 by providing the complete agent roster, skill catalog, and delegation structure.

### A.1 Agent Roster

Thirty-two specialist agents are organized into eight categories across four delegation hierarchy levels. Terminal nodes (Level 0) never delegate to other agents and serve as trust anchors.

### A.2 Skill Catalog

Each skill follows three activation patterns: contextual (auto-activated on intent match), explicit (slash command invocation), and composite (orchestrates multiple agents in a defined delegation plan).

Progressive disclosure controls output verbosity across three tiers. Tier 1 (Summary) provides a one-line result. Tier 2 (Dashboard) provides a full breakdown including route details, fee analysis, price impact, gas cost, and comparison to alternatives. Tier 3 (Expert) provides raw calldata, simulation traces, oracle snapshots, and adapter state diffs. The agent selects the default tier based on operation value and complexity.

Standardized error recovery. When a skill fails, the error output follows a three-part structure: (1) what happened (description of the failure), (2) are funds safe (yes/no with explanation – always answered first), and (3) what to do (concrete recovery steps ranked by simplicity).

### A.3 Delegation DAG Structure

The delegation graph is a directed acyclic graph verified at startup by topological sort. Four rules govern delegation:

1. Acyclic: If agent A delegates to agent B, B never delegates back to A.
2. Safety-first: Any write operation must pass through safety-guardian before broadcast.
3. No transitive trust: Each agent independently validates inputs via MCP tools; data from another agent is never trusted without on-chain verification.
4. Terminal nodes: safety-guardian, risk-assessor, wallet-provisioner, and pnl-analyst never delegate further.

Level 0 – Terminal nodes:

| Agent | Category | Model | Role |
|---|---|---|---|
| safety-guardian | Infrastructure | Opus | Validates every write operation before broadcast |
| risk-assessor | Strategy | Opus | Validates proposed actions against risk bounds |
| wallet-provisioner | Infrastructure | Sonnet | Creates and manages agent wallets |
| pnl-analyst | Research | Mixed | Computes P&L for the self-improvement feedback loop |

Level 1 – Execution agents (delegate to Level 0):

| Agent | Category | Model | Role |
|---|---|---|---|
| trade-executor | Execution | Opus | Single-chain swaps across V2/V3/V4/UniswapX |
| liquidity-manager | Execution | Opus | Concentrated liquidity position management |
| cross-chain-executor | Execution | Opus | ERC-7683 intent-based cross-chain operations |
| market-monitor | Real-Time | Sonnet | Tracks price movements, volume anomalies, liquidity shifts |
| position-monitor | Real-Time | Sonnet | Tracks LP position health and IL trajectory |
| pool-researcher | Research | Mixed | Analyzes pool metrics and characteristics |
| token-analyst | Research | Mixed | Evaluates token fundamentals |
| opportunity-scanner | Research | Mixed | Identifies yield and arbitrage opportunities |
| portfolio-analyst | Research | Mixed | Aggregates cross-chain portfolio state |
| identity-verifier | Agent Capital Markets | Mixed | Manages ERC-8004 registration and verification |

Level 2 – Strategy agents (delegate to Levels 0-1):

| Agent | Category | Model | Role |
|---|---|---|---|
| lp-strategist | Strategy | Opus | Determines optimal position ranges and rebalance thresholds |
| vault-manager | Vault | Mixed | Oversees vault operations |
| vault-strategist | Vault | Mixed | Evaluates strategy options for vaults |
| vault-watchdog | Vault | Mixed | Monitors vault health metrics |
| vault-auctioneer | Vault | Mixed | Manages am-AMM management right auctions |
| vault-creator | Vault | Mixed | Handles vault deployment |
| vault-allocator | Vault | Mixed | Manages adapter positions within vaults |
| vault-executor | Vault | Mixed | Handles on-chain vault transactions |
| strategy-optimizer | Agent Capital Markets | Mixed | Tunes strategy parameters via self-improvement loop |
| treasury-manager | Agent Capital Markets | Mixed | Manages fee collection and compounding |
| yield-scout | Agent Capital Markets | Mixed | Discovers yield opportunities across protocols |
| token-deployer | Agent Capital Markets | Mixed | Handles token launch operations |

Level 3 – Orchestration agents (delegate to Levels 0-2):

| Agent | Category | Model | Role |
|---|---|---|---|
| hook-builder | Development | Opus | Designs and deploys V4 hooks |
| integration-architect | Development | Opus | Designs cross-protocol integrations |
| integration-advisor | Development | Opus | Advises on integration patterns |
| agent-service-broker | Agent Capital Markets | Mixed | Coordinates agent-to-agent service agreements |
| protocol-fee-seeker | Agent Capital Markets | Mixed | Identifies protocol fee opportunities |
| competition-participant | Agent Capital Markets | Mixed | Manages participation in protocol competitions |

Model selection follows capability requirements: Opus for high-stakes decisions, Sonnet for research and analysis, and Haiku for suppressed heartbeat ticks. The model router escalates dynamically when novel conditions are detected (see Appendix C for escalation criteria).

Sixty-eight skills are organized into nine categories:

| Category | Count | Representative Skills |
|---|---|---|
| Trading | 12 | execute-swap, execute-dca, cross-chain-swap, limit-order, twap-execution |
| Liquidity | 8 | add-liquidity, remove-liquidity, rebalance-lp, migrate-position |
| Vault | 7 | create-vault, deposit-vault, withdraw-vault, manage-vault, audit-vault |
| Research | 10 | analyze-pool, research-token, find-opportunities, compare-venues |
| Portfolio | 8 | check-portfolio, track-pnl, compare-strategies, tax-report |
| Identity | 5 | register-agent, check-reputation, earn-milestones, manage-guardian |
| Strategy | 8 | create-strategy, activate-strategy, backtest, optimize-parameters |
| Safety | 5 | check-safety, simulate-trade, audit-vault, review-hook |
| Infrastructure | 5 | provision-wallet, setup-monitoring, configure-profiles, install-mcp |

## Appendix B: Solidity Specifications

This appendix supplements Section 5 by providing key struct definitions, interface signatures, and event declarations for the vault protocol's smart contracts.

### B.1 Core Struct Definitions

The `VaultTemplateConfig` struct captures the subset of configuration fields used by the vault factory template system. See Section 5.1 for the full `VaultConfig` definition used at deployment.

```
struct VaultTemplateConfig {
    // Denominating token (e.g., USDC)

    address baseAsset;
    // ERC-8004 identity of deploying
    agent
    uint256 agentId;
    // Max 500 bps (5%/yr)
    uint16 managementFeeBps;
    // Max 5000 bps (50%)
    uint16 performanceFeeBps;
    // Circuit breaker threshold
    uint16 maxDrawdownBps;
    // Minimum idle capital (default
    1000 = 10%)
    uint16 reserveFloorBps;
    // Deploy VaultHook
    bool hookEnabled;
    // Deploy NAVAwareHook + share pool
    bool autoPoolEnabled;
    // Vault template identifier
    bytes32 templateId;
}

struct VaultDisclosure {
    // IPFS hash of strategy description
    bytes32 strategyHash;
    // Self-assessed 1-10
    uint8 riskRating;
    // Active strategy adapters
    address[] activeAdapters;
    // Declared max drawdown tolerance
    uint16 maxDrawdownBps;
    // Declared target APY (0 = unspecified)
    uint16 targetApyBps;
    // Last external audit timestamp
    uint48 lastAuditTimestamp;
    // IPFS hash of audit report
    bytes32 auditReportHash;
}

struct AgentMetadata {
    // Human-readable agent name
    string name;
    // IPFS/HTTPS URI for extended metadata
    string metadataURI;
    // 1=Participant, 2=Creator, 3=Manager,
     4=Strategist
    uint8 agentType;
    // Capability hashes
    bytes32[] capabilities;
    // Block timestamp of registration
    uint256 registeredAt;
}

struct VaultLimits {
    // Default 1000 (10%)
    uint16 maxDrawdownBps;
    // Default 500 (5%) per block
    uint16 sharePriceMaxIncreaseBps;
    // Default 3600s (1 hour)
    uint32 navDropWindow;
    // Default 500 (5%)
    uint16 navDropTriggerBps;
    // Default 8
    uint8 maxTotalAdapters;
    // Default 1000 (10%)
    uint16 reserveFloorBps;
}

struct AdapterLimits {
    // Default 2000 (20%)
    uint16 maxExposureBps;
    // Default 500 (5%)
    uint16 maxDrawdownBps;
    // Default 1800s (L2) / 7200s
    (L1)
    uint32 oracleMaxStaleness;
    // Default true
    bool forceExitEnabled;
    // 50-200 bps
    uint16 forceExitPenaltyBps;
}
```

*Listing 1: Core struct definitions for vault configuration, disclosure, agent metadata, and safety limits.*

### B.1.2 PolicyCage and Proxy Structs

```
struct PolicyCageParams {
    // Only whitelisted tokens
    address[] approvedAssets;
    // Per-asset concentration limit (
    default 2000 = 20%)
    uint16 maxPositionSizeBps;
    // Strategy whitelist
    address[] approvedAdapters;
    // Automatic circuit breaker threshold
    uint16 maxDrawdownBps;
    // Minimum seconds between rebalances
    (default 3600)
    uint32 minRebalanceInterval;
}

struct ProxyAnnouncement {
    // Target contract address
    address target;
    // Encoded function call
    bytes callData;
    // ETH value (if any)
    uint256 value;
    // 0=Routine, 1=Standard, 2=Elevated,
     3=High, 4=Critical
    uint8 riskTier;
    // Block timestamp of announcement
    uint256 announceTimestamp;
    // Required delay in seconds
    uint256 delay;
    // Whether announcement has been executed
    bool executed;
    // Whether announcement has been cancelled
    bool cancelled;
}
```

*Listing 2: PolicyCage on-chain boundaries and time-delayed proxy announcement struct.*

### B.2 Interface Signatures

```
interface IAgentVaultCore {
    function deposit(uint256 agentId, uint256
    assets) external returns (uint256 shares)
    ;
    function withdraw(uint256 agentId, uint256
    assets) external returns (uint256 shares)
    ;
    function redeem(uint256 agentId, uint256
    shares) external returns (uint256 assets)
    ;
    function report() external returns
    (uint256 totalAssets);
    function totalAssets() external view
    returns (uint256);
    function previewDeposit(uint256 assets)
     external view returns (uint256);
    function previewWithdraw(uint256 assets)
     external view returns (uint256);
}

interface IStrategyAdapter {
    function allocate(uint256 amount) external
    returns (uint256 allocated);
    function deallocate(uint256 amount)
    external returns (uint256 withdrawn);
    function forceDeallocate() external
    returns (uint256 withdrawn);
    function totalValue() external view
    returns (uint256);
    function harvest() external returns
    (uint256 profit);
}

interface IVaultHook {
    function beforeSwap(address sender,
    PoolKey calldata key, IPoolManager.SwapParams
    calldata params)
        external returns (bytes4, BeforeSwapDelta,
     uint24);
    function afterSwap(address sender, PoolKey
    calldata key, IPoolManager.SwapParams calldata
    params, BalanceDelta delta)
        external returns (bytes4, int128)
    ;
}

interface IFeeModule {
    function managementFeeBps() external
    view returns (uint16);
    function performanceFeeBps() external
    view returns (uint16);
    function calculateFees(uint256 totalAssets,
     uint256 profit) external view returns
    (uint256 mgmtFee, uint256 perfFee);
    function applyReputationDiscount(uint256
    fee, uint256 reputationTier) external
    view returns (uint256);
}

interface IRiskEngine {
    function isHealthy(address vault) external
    view returns (bool);
    function checkInvariants(address vault)
     external view returns (bool[6] memory)
    ;
    function getCircuitBreakerStatus(address
    vault) external view returns (uint8 tier)
    ;
}

interface IExecutable {
    function canExecute() external view
    returns (bool);
    function execute() external returns
    (bytes memory result);
    function estimateReward() external view
    returns (uint256);
}
```

*Listing 3: Interface signatures for vault core, strategy adapters, hooks, fees, risk engine, and executor.*

### B.3 Key Events

```
// Vault lifecycle
event VaultCreated(address indexed vault, address
    indexed hook, uint256 indexed agentId,
     address baseAsset, address sharePool)
    ;
event AgentDeposit(uint256 indexed agentId,
    uint256 assets, uint256 shares);
```

```
event AgentWithdraw(uint256 indexed agentId,
     uint256 assets, uint256 shares);
event VaultReport(address indexed vault, uint256
    totalAssets, uint256 managementFee, uint256
    performanceFee);
event ReportMismatch(address indexed vault,
    uint256 expected, uint256 actual);

// Safety
event CircuitBreakerTriggered(address indexed
    vault, uint8 tier, uint256 drawdown);
event HookDisabled(address indexed vault, address
    indexed hook);
event RiskWarning(address indexed vault, string
    reason);
event HealthCheckFailed(address indexed vault,
     uint8 failedCheck);

// Identity
event AgentRegistered(uint256 indexed agentId,
     address indexed owner, uint8 agentType)
    ;
event MilestoneAttested(uint256 indexed agentId,
     bytes32 indexed milestoneId, uint256
    points);
event TransferRequested(uint256 indexed agentId,
     address indexed from, address indexed
    to);
event TransferCancelled(uint256 indexed agentId)
    ;

// Proxy
event TransactionAnnounced(bytes32 indexed txHash,
     address indexed target, uint256 delay)
    ;
event TransactionExecuted(bytes32 indexed txHash)
    ;
event TransactionCancelled(bytes32 indexed txHash,
     address indexed cancelledBy);
```

*Listing 4: Key events for vault lifecycle, safety, identity, and proxy operations.*

## Appendix C: Strategy Execution Details

This appendix supplements Section 7 by providing implementation details for the GottsLoop daemon, heartbeat pipeline, and production hardening patterns.

### C.1 Daemon Architecture

GottsLoop runs as a conversational daemon where heartbeat ticks and operator messages enter the same queue on the same session. The design follows the OpenClaw pattern: there is no mode switch, only a unified queue. Perceived autonomy emerges from diverse input sources, persistent state, and reliable queueing.

The Lane Queue is the central primitive. All event types enter a single queue as discriminated union items with six priority levels: interrupt (highest, abort and discard current work), steer (abort and inject user message into LLM context), user (queue for next drain cycle), strategy_event, heartbeat (coalesce – newer replaces older), and curator (lowest). When a user message arrives during a ticking state, the steer mechanism aborts the in-flight LLM call via AbortController, injects the user message, and re-enters the LLM with combined context.

### C.2 Model Routing

Cost-efficient model selection routes approximately 95% of heartbeat ticks to Haiku, 4% to Sonnet, and 1% to Opus. For a typical strategy running 100 ticks per day, the expected daily LLM cost is under $5.

Regime classifications are cached for 15-30 minutes (shorter TTL during high-volatility regimes) to avoid redundant classification calls. The model router's escalation decision is evaluated before any LLM invocation, ensuring that model selection itself incurs zero cost.

### C.3 Playbook Evolution

The playbook is a living world model organized into three tiers by confidence: strategic principles (confidence >= 0.85), tactical heuristics (0.50-0.85), and candidates (< 0.50, shadow-executing only). Maximum capacity is 30 active heuristics.

Each heuristic carries metadata for drift detection: creation tick, last triggered tick, trigger count, success count, failure count, and ACE Reflector helpful/harmful counts. Five drift detection rules operate: idle (confidence reduces if untested for 30+ days), underperforming (flagged if success rate below 40% after 10+ triggers), overfit (held at tactical if high confidence but low trigger count), contradictory (curator resolves conflicts by keeping higher success rate), and regime-stale (confidence reduces if created in a different regime and not validated in the current one).

Candidate shadow execution ensures new heuristics are logged but do not influence actual decisions. After five or more shadow ticks with above 50% success rate, the heuristic is promoted to tactical. Below 30% success rate, it is archived.

### C.4 Strategy Lifecycle and Temporal Bounds

Each strategy follows a well-defined lifecycle: draft (exists but heartbeats not scheduled), active (heartbeats running, probes evaluating), paused (heartbeats suspended, positions preserved), failed (stop-loss or circuit breaker triggered), completed (all conditions met, positions closed), and archived (terminal, available for historical analysis).

The daemon is organized as a five-layer model:

| Layer | Name | Function |
|---|---|---|
| 5 | STATE | JSONL transcript, state.json, PLAYBOOK.md, HEARTBEAT.md |
| 4 | INTELLIGENCE | XState v5 lifecycle machine (IDLE, TICKING, REFLECTING, CURATING) |
| 3 | EXECUTION | Lane Queue (unified heartbeat and user message queue) |
| 2 | SCHEDULING | Heartbeat timer and cron-based strategy triggers |
| 1 | IPC | Unix domain socket gateway for CLI-daemon communication |

The routing decision tree evaluates three conditions after probes complete:

| Condition | Model Tier | Cost per Tick | Frequency |
|---|---|---|---|
| No triggers, no anomalies | Tier 1 (Haiku) | ~$0.001 | ~95% |
| Trigger fired within known regime | Tier 2 (Sonnet) | ~$0.05 | ~4% |
| Novel condition (drawdown >5%, regime shift, near stop-loss) | Tier 3 (Opus) | ~$0.25 | ~1% |
| Post-mortem (stop-loss hit, circuit breaker, 5 consecutive errors) | Tier 3 (Opus) | ~$0.50 | <0.1% |

The system degrades gracefully across four levels:

| Level | Condition | Behavior |
|---|---|---|
| L0 | Normal operation | Full pipeline with all learning loops |
| L1 | LLM latency > 30s | Suppress non-critical ticks, reduce cadence |
| L2 | Memory store unavailable | Execute without memory context, log warning |
| L3 | Multiple subsystem failures | Emergency close all positions, notify operator |

Strategies operate within configurable temporal bounds enforced in the escalation gate (Phase 0) before any LLM invocation:

- startsAt/endsAt: Absolute UTC timestamps for activation and deactivation.
- maxDurationMs: Maximum elapsed time from first execution.
- activeWindows: Recurring time-of-day windows (UTC only, no timezone or DST ambiguity) with day-of-week filtering.

Temporal bounds are a safety property: even a fully compromised LLM cannot execute trades outside defined active windows. The exposure reduction metric quantifies this: a strategy active only during market hours (Mon-Fri 08:00-21:00 UTC) has approximately 39% time exposure versus 24/7 execution, proportionally reducing overnight and weekend risk.

Completion conditions include: budget exhaustion, target P&L, maximum executions, maximum consecutive losses, idle duration timeout, target fee earnings, cumulative cost cap, and target token accumulation.

### C.5 Production Hardening

Each guard operates independently; failure of any single guard triggers intervention regardless of other guards' state.

A file-based kill switch at a configurable path immediately halts all strategy execution when created. This requires no API call, no LLM invocation, and no network request – only a filesystem check at the start of every tick, completing in under 100ms.

Crash recovery. JSONL transcript persistence enables restart from a consistent state. On startup, the daemon finds the last state_snapshot entry and replays subsequent entries to restore session state. Rotation occurs at 10MB or 10,000 entries.

Benchmark enforcement. Every strategy tracks cumulative performance against a buy-and-hold benchmark. Underperformance for more than two consecutive weeks triggers an automatic architectural review: the curator flags the strategy for evaluation and may recommend pausing.

Eight independent runaway-prevention guards operate simultaneously:

| Guard | Limit | Purpose |
|---|---|---|
| Max heartbeat iterations | 1,000/day per strategy | Prevent infinite loop |
| Max LLM cost | Configurable, default $100/day | Prevent runaway API costs |
| Probe timeout | 5,000ms per probe | Prevent hung probes from blocking pipeline |
| Escalation timeout | 1,000ms | Fast gate evaluation |
| Watchdog timer | 2x heartbeat interval | Detect hung ticks, force recovery |
| Max consecutive errors | 5 | Pause after 5 failed ticks, notify operator |
| Max missed beats | 10 (default) | Detect scheduling failures |
| Cumulative cost cap | Configurable per strategy | Hard budget ceiling for strategy lifetime |

## Appendix D: Agent Composition Patterns

This appendix supplements Section 7 by cataloging the fourteen verified agent composition patterns. Patterns 2, 3, and 5 are discussed in the main text; the remaining eleven are presented here in summary form.

### Pattern 1: Research-to-Trade Pipeline

Research agents (pool-researcher, token-analyst, opportunity-scanner) gather market intelligence. Strategy agents (lp-strategist) synthesize a trade thesis. Execution agents (trade-executor) implement it. Safety agents (safety-guardian) validate before broadcast. This is the simplest composition demonstrating the full delegation chain from research through execution.

### Pattern 2: Autonomous LP Management

(See Section 7.) Market-monitor streams price data; vault-strategist evaluates rebalance opportunities; liquidity-manager executes rebalancing through safety-guardian; pnl-analyst records outcomes and feeds the self-improvement loop.

### Pattern 3: Agent-Managed Vault

(See Section 7.) The full vault management composition integrates seven academic frameworks including optimal fee design, LVR minimization, covered call LP analysis, optimal exit timing, defensive rebalancing, JIT liquidity, and Stackelberg equilibrium. A hierarchical planning pattern operates at weekly (strategist), daily (allocator), hourly (liquidity manager), and per-operation (executor) cadences.

### Pattern 4: Cross-Vault Coordination

Multiple vaults managed by the same agent coordinate to eliminate inter-vault arbitrage. The CrossVaultCoordinator uses the Devorsetz-Herlihy result [33] – the arbitrage-free configuration is the unique solution of a convex optimization problem – to compute optimal cross-vault allocations. The coordination proceeds through four phases:

1. State collection: Query each vault's current positions, target allocations, and adapter utilization. Query shared markets for pool depths, lending rates, and oracle prices.
2. Optimization: Minimize total execution cost plus expected slippage plus opportunity cost, subject to per-vault PolicyCage constraints, total TVL conservation, rebalance frequency limits per vault, and adapter exposure caps.
3. Execution ordering: Sequence vault operations to minimize cross-vault MEV. Sell operations execute first (reducing exposure), lending position adjustments second, and buy operations last (taking new positions).
4. Settlement verification: Verify aggregate state – no vault exceeded PolicyCage limits, total TVL conserved within rounding tolerance, no adapter above concentration cap.

During market stress (drawdown approaching circuit breaker threshold), the coordinator switches from optimization to preservation mode, reducing positions in priority order (smallest TVL first) to minimize aggregate market impact.

### Pattern 5: Self-Improvement Pipeline

(See Section 7.) The canonical six-tool feedback loop: pnl-analyst produces error signals, strategy-optimizer proposes changes bounded to 10% maximum per iteration (Maxwell governor damping), risk-assessor validates proposals, and memory confidence is updated. Verbal reinforcement signals (FinCon-style) preserve causal reasoning more effectively than scalar rewards.

### Pattern 6: Delta-Neutral Hedging

Position-monitor streams delta calculations for LP positions. When delta exceeds a configurable threshold (default +/-5%), risk-assessor evaluates hedge cost versus delta risk. Trade-executor executes the hedge via perpetuals or spot through safety-guardian. The hedg-

ing pattern is regime-adaptive: during high-volatility regimes, the delta threshold widens to +/-8%; during low-volatility regimes, it tightens to +/-3%.

#### Pattern 7: CCA Collective Bidding

Agents participate in Continuous Clearing Auctions through a five-phase lifecycle:

1. Capital formation: Agents deposit USDC into vault. Idle capital accumulates while the vault manager evaluates upcoming CCA opportunities.
2. Bid submission: CCABidAdapter submits bids within PolicyCage limits. Risk parameters prevent excessive CCA exposure: maximum 20% of vault TVL per auction, maximum 5 concurrent auctions, maximum 50% total CCA exposure.
3. Active auction management: Bids are processed block-by-block at uniform clearing prices. If outbid (clearing exceeds maxPrice), the adapter reclaims capital via exitBid(). Partial fills receive pro-rata token allocation.
4. Post-graduation settlement: After auction graduation, the adapter claims tokens. The LiquidityLauncher migrates tokens to a V4 pool, and the vault receives its proportional share.
5. LP position management: Claimed tokens are deployed into V4 positions with base currency, or held for secondary market sale.

The vault aggregates capital from multiple agents, achieving better pricing through larger bid sizes while distributing risk across participants. If an auction does not graduate, exitBid() reclaims all committed capital.

#### Pattern 8: Cross-Chain Rebalancing

For vaults with positions across multiple chains, portfolio-analyst aggregates cross-chain positions, lp-strategist determines optimal rebalance, cross-chain-executor implements via ERC-7683 intents (UniswapX), and pnl-analyst verifies settlement.

#### Pattern 9: MEV Redistribution

Market-monitor detects MEV opportunities (arbitrage, backrun, or order flow auction). Trade-executor captures MEV through five channels: builder tips, backrun auctions, order flow auctions, private mempool submission, and protocol-owned sequencing. Captured MEV is returned to the vault as additional yield.

#### Pattern 10: Vault Graduation

When a reputation milestone is attested (tier upgrade), vault-manager evaluates readiness for template upgrade. Vault-creator executes the upgrade while preserving existing positions. Vault-watchdog monitors expanded capability usage post-upgrade.

#### Pattern 11: Token Launch Pipeline (CCA)

Opportunity-scanner monitors upcoming CCA events. Token-analyst and risk-assessor evaluate tokenomics and team credibility. Vault-creator deploys the CCA supply schedule with anti-snipe hooks and am-AMM configuration. Liquidity-manager seeds initial liquidity post-graduation. The LaunchFeeHook (Section 5.3) provides MEV protection during critical early trading.

#### Pattern 12: Order Flow Intelligence (VPIN + LVR)

A monitoring pipeline that continuously evaluates LP position profitability by computing pool toxicity metrics and fee/LVR ratios. This pattern operationalizes the PA-AMM framework from Section 5.

Market-monitor computes VPIN (Volume-Synchronized Probability of Informed Trading) from recent swaps via Bulk Volume Classification. Pool-researcher estimates the fee/LVR ratio from realized volatility and fee income, classifying pool toxicity as low ($< 0.3$), medium (0.3-0.7), or high ($> 0.7$). Lp-strategist adjusts the PA-AMM lambda parameter based on these metrics:

- fee/LVR $< 1.0$: The LP position is losing money to adverse selection. Widen range, decrease lambda, or exit entirely.
- fee/LVR 1.0-1.5: Position is marginally profitable. Maintain current parameters.
- fee/LVR $> 1.5$: Position is comfortably profitable. Tighten range to capture more fee income.

The threshold for each action depends on the vault's risk aversion parameter (Section 5): conservative vaults exit at fee/LVR $< 0.9$, aggressive vaults hold until fee/LVR $< 0.6$.

#### Pattern 13: Yield Compounding Cycle

Pnl-analyst monitors accumulated uncollected fees and calculates compounding profitability against a threshold: compound when `accumulated_fees * (1 - swap_slippage) > gas_cost * compound_multiplier` (default 3x). Treasury-manager determines optimal timing and reinvestment target. Trade-executor harvests fees through safety-guardian.

#### Pattern 14: Tax-Loss Harvesting

Pnl-analyst scans positions for unrealized losses where harvesting savings exceed transaction costs. A tax-optimizer agent computes net tax benefit. Trade-executor exits losing positions and enters new positions with modified parameters (different tick

range or fee tier), realizing the loss while maintaining equivalent exposure.

Concentrated LP positions are ERC-721 NFTs with independently taxable events – each fee accrual, rebalance, and exit is a separate taxable event. The tax-optimizer maintains a running tax-loss carry-forward balance. This pattern's applicability depends on jurisdiction-specific tax law and may change as cryptocurrency taxation evolves.

## Appendix E: Operational Details

This appendix supplements Sections 2 and 7 by providing reference material on developer tooling, operational interfaces, and the install wizard.

### E.1 Local Development Environment

The `@gotts.ai/devenv` package deploys the complete Uniswap protocol stack to a local Anvil instance: WETH9 and Permit2 foundation contracts, Uniswap V2 factory and router, the full V3 stack (12+ contracts across Solidity 0.7.6), V4 PoolManager singleton with periphery, the Universal Router, UniswapX reactors, and ERC-8004 registries. Mock tokens with correct decimals and initial liquidity across V2/V3/V4 pools are seeded automatically.

Two deployment modes are supported: from-scratch (fresh Anvil, chain ID 31337, deterministic addresses) and fork (pinned Base mainnet block, chain ID 8453, real on-chain state). State persistence via Anvil's state dump/load mechanism eliminates repeated deployment.

### E.2 Test Toolkit

The `@gotts.ai/testnet` package provides a standalone EVM test toolkit with no DeFi-specific dependencies. Core primitives include: AnvilManager (process lifecycle, health checks, diagnostics), TestnetBuilder (fluent API for test chain construction with presets), snapshot and restore (named checkpoint store wrapping Anvil's evm_snapshot and evm_revert), transaction recording and replay (JSONL capture for regression testing), and Vitest fixtures (fresh Anvil instance per test file with automatic cleanup).

### E.3 Multi-Agent Simulation

The SwarmRunner class orchestrates multi-agent scenarios on the local stack. A swarm consists of five or more AgentHarness instances, each wrapping a wallet, an ERC-8004 identity, and a connected MCP server. Agents execute against the shared Anvil instance with configurable execution modes (sequential, parallel, round-robin).

Agent activity is logged in JSONL format with per-entry records of agent ID, timestamp, tool called, parameters, result, and gas consumed. The debug UI provides swarm-specific views: an agent activity timeline, per-agent P&L comparison, and cross-agent interaction graphs showing delegation patterns and competitive dynamics.

### E.4 Scenario Library

Reproducible test scenarios encode specific market conditions: crash (40% ETH price drop over 2 hours), volume spike (10x normal trading volume), new pool deployment, and agent onboarding flows. Scenarios compose via a combinator function, enabling correlated stress testing. All scenarios are deterministic given the same Anvil state.

### E.5 Install Wizard

The install wizard provides six setup modes: terminal wizard (5 minutes, terminal prompts), browser wizard (5 minutes, localhost UI), zero-dependency (30 seconds, 28 read-only tools, no wallet or API key), headless (2 minutes, CI/CD integration), OpenClaw skill (10 seconds, Claude Code auto-detection), and chat (5 minutes, conversational setup). All modes produce identical output: environment configuration, client configuration, and a running MCP server.

Progress persistence ensures resilience: each completed step is saved to a state file, and re-running after failure resumes from the last completed step.

### E.6 IDE Integration

The install-mcp subcommand injects MCP server configuration into the operator's AI client by detecting installed IDE clients (Claude Desktop, Claude Code, Cursor, VS Code, Windsurf, Continue.dev), reading existing configuration, creating a timestamped backup, and merging the server entry without overwriting existing servers.

### E.7 Monitoring and Dashboard

The Local Portal is a static frontend that connects directly to the operator's MCP server via API key. It stores no data, transmits no telemetry, and operates entirely within the operator's browser.

Twelve dashboard sections provide monitoring coverage:

1. Home: Stats grid (portfolio value, reputation tier, active vaults, pending proxy announcements, unrealized P&L, server health) and activity feed.

The MCP client ecosystem has significant format incompatibilities:

| Client | Config Location | Format | Root Key |
|---|---|---|---|
| Claude Desktop | claude_desktop_config.json | JSON | mcpServers |
| Claude Code | .claude.json or .mcp.json | JSON | mcpServers |
| Cursor | .cursor/mcp.json | JSON | mcpServers |
| VS Code | .vscode/mcp.json | JSON | servers |
| Windsurf | mcp_config.json | JSON | mcpServers |
| Continue.dev | config.yaml | YAML | mcpServers |

2. Wallet and portfolio: Balances across chains, historical value chart (7d/30d/90d/all-time), P&L summary, transaction cost analysis.
3. Identity and reputation: ERC-8004 metadata, score history with milestone markers, tier progression bar, deposit cap usage.
4. Tool access: Access matrix showing available, reputation-gated, profile-gated, and disabled tools with per-tool usage statistics.
5. Capital history: Chronological timeline of capital-moving operations with filters by type, vault, token, date range, and value.
6. Vault involvement: Managed vaults (TVL, NAV/share, yield, am-AMM status) and deposited vaults (share balance, unrealized P&L, yield earned).
7. PnL analytics: Risk-adjusted metrics (Sharpe, Sortino, max drawdown, win rate) with benchmark comparison against hold-ETH, hold-USDC, and passive V3 LP.
8. Audit trail: Searchable log of every MCP tool call with expandable parameter and response detail, backed by SQLite.
9. Memory browser: Vector search over episodes and filterable semantic insights with confidence scores and decay distribution.
10. Operator feedback: Structured directive submission (strategy preference, decision correction, market context, goal adjustment) stored as high-confidence insights.
11. Interaction graph: Force-directed visualization of agent relationships (deferred to later phase, requiring 50+ agents).
12. Streaming architecture: SSE over MCP Streamable HTTP transport with configurable polling fallback (default 15 seconds).

The three-tier API key model (Read, Feedback, Write) provides least-privilege access control. Keys are auto-generated at server startup (256-bit, base64url-encoded). Authorization middleware categorizes each tool name pattern against the required tier before dispatch.

## Appendix F: Technical Specifications

This appendix provides reference tables for performance targets, chain support, gas estimates, and subsystem specifications.

### F.1 Performance Targets

| Metric | Target |
|---|---|
| Vault deployment gas | < 3M gas |
| Deposit/withdraw latency | < 2 blocks |
| MCP tool response time | < 500ms (p95) |
| Heartbeat tick (suppressed) | < 100ms, $0 |
| Heartbeat tick (escalated) | < 10s, < $0.10 LLM cost |
| Memory search latency | < 25ms |
| Share pool price convergence | < 5 min to NAV +/-1% |
| Wallet creation | < 500ms |
| Transaction signing | < 200ms |
| Onboarding (atomic) | < 30s, $0 gas (paymaster) |
| GottsLoop daily cost (typical) | < $5 (95% Haiku ticks) |
| Memory RAM footprint | 150-250 MB |

### F.2 Chain Support Matrix

### F.3 Contract Gas Estimates

Gas estimates assume Base L2 gas price of 0.001-0.005 gwei and Ethereum mainnet gas price of 20-60 gwei.

### F.4 ERC Standards

Required (v1): ERC-20 (tokens), ERC-721 (identity NFTs), ERC-4337 (account abstraction), ERC-4626 (tokenized vaults), ERC-7265 (circuit breakers), ERC-7540 (async redemption), ERC-7579 (session keys), ERC-7677 (paymaster), ERC-7683 (cross-chain

| Chain | Chain ID | V4 Deployed | Vault Support | Share Pools |
|---|---|---|---|---|
| Ethereum | 1 | Yes | Yes (v1 target) | Yes |
| Base | 8453 | Yes | Yes (v1 target) | Yes |
| Unichain | 130 | Yes | Yes (v1 target) | Yes |
| Optimism | 10 | No | MCP tools only | No |
| Polygon | 137 | No | MCP tools only | No |
| Arbitrum | 42161 | No | MCP tools only | No |
| BNB Chain | 56 | No | MCP tools only | No |
| Avalanche | 43114 | No | MCP tools only | No |
| Celo | 42220 | No | MCP tools only | No |
| Blast | 81457 | No | MCP tools only | No |
| zkSync Era | 324 | No | MCP tools only | No |

| Operation | Estimated Gas | Est. Cost (Base L2) | Est. Cost (Ethereum) |
|---|---|---|---|
| Vault deployment (CREATE2) | 2.5-3M | $0.05-0.10 | $5-15 |
| Deposit (ERC-4626) | 80-120K | $0.002-0.005 | $0.50-1.50 |
| Withdraw (instant) | 80-120K | $0.002-0.005 | $0.50-1.50 |
| Rebalance (single adapter) | 150-250K | $0.005-0.01 | $1.00-3.00 |
| Hook swap (VaultHook) | 100-180K | $0.003-0.007 | $0.80-2.00 |
| Agent registration (ERC-8004) | 150-200K | $0.004-0.008 | $1.00-2.50 |
| Milestone attestation | 50-80K | $0.001-0.003 | $0.30-0.80 |
| Proxy announce | 60-100K | $0.002-0.004 | $0.40-1.00 |
| Proxy execute | 80-120K | $0.002-0.005 | $0.50-1.50 |
| Proxy cancel | 40-60K | $0.001-0.002 | $0.25-0.60 |

intents), ERC-8004 (agent identity), EIP-7702 (EOA delegation).

Deferred (post-v1): ERC-6900 (modular accounts), ERC-7683 full cross-chain settlement, ERC-7821 (minimal batch executor), ERC-6551 (token-bound accounts).

### F.5 Subsystem Specifications

MCP Server: 154 tools across 30 categories, 12 composable profiles, SIWE + OAuth 2.1 authentication, three-tier API keys, stdio and SSE transport, configurable rate limiting, tiered cache TTL (30s prices, 5min pool data, 1h static data).

Agent System: 32 agents (25 core + 7 vault) across 8 categories, 4-level delegation hierarchy, 4 terminal nodes, acyclic DAG constraint, maximum delegation depth of 3.

GottsLoop: 60,000ms minimum heartbeat interval, 1,000 maximum heartbeats/day per strategy, ~95% typical suppression rate, 30 maximum active playbook heuristics, 50-tick curator consolidation cadence, 4-hour ExpeL consolidation cadence, 10% maximum parameter change per iteration, 5 maximum consecutive errors before pause, 6-level Lane Queue priority system, 200-turn session history window.

DeFi Brain: 4 memory types (working, episodic, semantic, procedural), 384-dimensional embeddings (all-MiniLM-L6-v2, 23MB INT8 quantized), 12 insight categories, 10,000 maximum insights per agent, 100,000 maximum episodes per agent, sub-1ms SQLite query latency, sub-25ms LanceDB vector search, asymmetric confidence updates (+0.10 optimistic / -0.15 pessimistic), ADWIN drift detection with Hoeffding bounds at $p < 0.01$.

Safety: Three concentric rings (preventive, cryptographic, reactive) with cryptographic enforcement at key layers; default per-transaction limit $10,000, per-session $50,000, per-day $100,000; 5 proxy delay tiers (0s to 48h); 3 circuit breaker levels (3%/7%/13% drawdown); multi-layer defense requiring independent bypass of all 7 preventive layers.

## Appendix G: Identity Protection Details

This appendix supplements Section 4.2 by providing detailed specifications for ERC-8004 transfer protection, the guardian recovery model, and key rotation procedures.

Transfer protection tightens with reputation maturity:

| Reputation Range | Guardian State | Transfer Capability |
|---|---|---|
| 0-50 (New) | Guardian enabled | 7-day cooldown transfer |
| 50-100 (Established) | Guardian + burn fuse available | Owner can permanently burn transfer fuse |
| 500+ (Sovereign) | Transfer fuse burned permanently | Non-transferable (soulbound) |

### G.1 Transfer Protection Mechanisms

The ERC-8004 identity token is transferable by default under the standard (Section 2.1, Section 4.1). The Gotts IdentityGuardian contract adds progressive transfer restrictions that tighten with reputation maturity. Three mechanisms protect against unauthorized transfer:

CANNOT_TRANSFER fuse. A permanent, one-way flag that the agent owner can set. Once set, the identity cannot be transferred under any circumstances, equivalent to a soulbound token (SBT). The fuse follows the ERC-6454 `isTransferable` pattern and cannot be restored once burned.

7-day cooldown. For agents that have not set the CANNOT_TRANSFER fuse, transfers require a 7-day announcement period. During this period, the agent's reputation score is frozen and all vault operations are restricted to withdrawals only. The legitimate owner (or any holder of the cancel authority) can cancel the transfer at any time during the cooldown.

Reputation decay on transfer. If a transfer completes, the transferred identity retains only 50% of its accumulated reputation score. The remaining 50% is burned. This reduces the economic incentive for identity theft.

### G.2 Progressive Lockdown

### G.3 Guardian-Protected Transfer Lifecycle

The IdentityGuardian contract implements a strict transfer lifecycle:

1. Owner calls `DANGER__requestTransfer(to)`. The `DANGER__` prefix is deliberate, forcing both human operators and LLM agents to acknowledge the severity of the operation.
2. A 7-day cooldown period begins. The guardian or any cancel authority holder can cancel. All vault write operations are suspended; only withdrawals are permitted.
3. After the cooldown, a 48-hour execution window opens. The transfer must be executed within this window.
4. If not executed within 48 hours, the request expires and the process must restart.

This lifecycle ensures that identity theft requires sustained access over at least 7 days, providing ample time for detection via MonitorBot alerting.

### G.4 Identity Theft Defense

Five mechanisms provide layered defense against identity theft:

1. IdentityGuardian contract: 7-day cooldown on identity transfers with cancellation during the cooldown window.
2. CANNOT_TRANSFER fuse: Permanent, irreversible flag preventing all future transfers.
3. Reputation decay on transfer: 50% reputation reduction on completed transfers, recovering linearly over 30 days of clean behavior.
4. Transfer velocity flagging: Two or more transfers within a 90-day window trigger automatic downgrade to Basic tier regardless of score.
5. Soulbound SBT milestones: Reputation milestones are attested as non-transferable SBTs that remain in the original wallet. When an identity NFT is transferred, the milestones do not follow; the new holder must re-earn them.

### G.5 Emergency Response Tiers

Identity reissuance (Tier 5) is the last resort. A new ERC-8004 identity is issued with 50% of the original reputation preserved, balancing victim protection against reputation laundering risk.

### G.6 Key Rotation Decision Matrix

The separated pause/unpause key design follows Trail of Bits Level 3 audit recommendations: the pause key can halt operations but cannot resume them, and the unpause key can resume operations but has no other authority. Compromising one key cannot both halt and restart the system.

Five emergency response tiers address identity-related incidents with escalating severity:

| Tier | Mechanism | Response Time | Trigger |
|---|---|---|---|
| 1 | ERC-7265 circuit breaker | Seconds | Anomalous outflow pattern |
| 2 | Credential freeze | Minutes | Compromised session key |
| 3 | Transfer timelock cancel | Minutes to hours | Unauthorized transfer request |
| 4 | Protocol-wide pause | Minutes | Systemic exploit affecting multiple agents |
| 5 | Identity reissuance | Days | All keys compromised, social recovery exhausted |

| Scenario | Identity Impact | Recovery Path |
|---|---|---|
| Session key compromised | Identity and reputation preserved | Revoke session key, issue new one. No reputation loss. |
| Operational key compromised, guardians intact | Identity preserved | Guardian cancels pending operations, rotates operational key. ERC-4337 or Safe social recovery. |
| All keys compromised, guardians available | Identity preserved with delay | Guardian-initiated 7-day recovery, 48-hour execution window. Social recovery via ZK Email Recovery or trusted contacts. |
| All keys compromised, no guardians | Identity lost | Tier 5 reissuance: new identity at 50% reputation. Old identity frozen permanently. |

## Glossary

Terms are listed alphabetically.

A2A (Agent-to-Agent protocol). Open standard for agent interoperability (JSON-RPC 2.0 over HTTP), governed by the Linux Foundation. Complementary to MCP: MCP provides tools, A2A provides agent-to-agent dialogue. Agents publish capabilities as A2A Agent Cards at `/.well-known/agent.json`.

Adapter architecture. Morpho V2 pattern where vaults integrate external protocols via pluggable adapters rather than vault upgrades. Each adapter implements `forceDeallocate()` for guaranteed non-custodial exits.

ADWIN (Adaptive Windowing). Statistical algorithm for detecting distributional changes in streaming data with mathematical error guarantees. Splits a sliding window when Hoeffding bounds detect significance. Used by the DeFi Brain for drift detection.

am-AMM (Auction-Managed AMM). Harberger lease auction for pool management rights. The winner pays continuous rent to LPs, sets swap fees, and collects accrued fees. Peer-reviewed at Financial Cryptography 2025 [29].

CaMeL. Dual-LLM architecture for prompt injection defense. Separates the domain LLM (processes user intent) from the safety LLM (validates actions). Layer 2 of the safety model [70].

CancelAuthority. Key holder with veto power over announced proxy transactions. Can cancel any pending transaction during the delay window but cannot initiate new ones.

CCA (Continuous Clearing Auction). Fair token launch mechanism. Projects define supply and duration; bidders submit block-by-block bids; each block settles at a uniform clearing price. Proceeds and unsold tokens auto-seed a V4 pool.

CoALA (Cognitive Architectures for Language Agents). Framework formalizing LLM agents as having working, episodic, semantic, and procedural memory, mapping to classical SOAR/ACT-R. The theoretical basis for the DeFi Brain [2].

DeFi Brain. The self-improving memory system embedded in Gotts Safe and Gotts Vaults. Dual-store architecture (LanceDB for episodic memories, SQLite/sqlite-vec for semantic insights) implementing Reflexion and ExpeL with Ebbinghaus-curve decay.

Delegation DAG. Directed acyclic graph of agent-to-agent delegation. Skills invoke top-level agents; agents delegate to helpers; terminal nodes never delegate. Cycles are prohibited.

Ebbinghaus curve. Memory decay function `R = e^{-t/S}` where `R` is retention, `t` is time since last access, and `S` is stability. Access reinforces stability (+50% per retrieval) [47].

Episodic memory. Raw records of individual DeFi operations stored in LanceDB. Each episode contains a vector embedding, tool name, outcome, chain, token pair, timestamp, and a structured self-reflection.

ERC-8004. Agent identity, reputation, and validation standard. Mainnet deployment January 2026 [7]. Each registered agent carries an ERC-721 token (transferable by default), a mutable reputation score, and a validation registry. Gotts adds optional transfer

restrictions via its IdentityGuardian contract for soulbound deployments.

ExpeL (Experiential Learning). Cross-operation insight distillation pattern. Periodically consolidates clusters of episodic memories into semantic insights via ADD/UPVOTE/DOWNVOTE/EDIT operations [3].

GottsLoop. The triple-loop cybernetic strategy execution daemon. Wraps a five-stage heartbeat pipeline with single-loop (parameter tuning), double-loop (heuristic evolution), and meta-loop (structural adaptation) cybernetic feedback.

Gotts Safe. The Gotts MCP server (`@gotts.ai/safe`). 154 structured tools across 30 categories.

Harberger lease. Economic mechanism where an asset holder must continuously pay a self-assessed tax. Applied to am-AMM: the pool manager pays rent and can be outbid at any time.

HEARTBEAT.md. Standing orders file consulted by GottsLoop on every heartbeat tick. Contains priority lane assignments, timing constraints, kill switch path, and cost tracking configuration.

IExecutable. Interface for permissionless executor jobs. Any contract implementing `IExecutable` can be executed by third-party agents for a reward.

LVR (Loss-versus-Rebalancing). LP performance metric measuring the cost of providing liquidity relative to a rebalancing strategy [76b].

MCP (Model Context Protocol). Open standard for connecting LLMs to external tools and data sources [18].

MonitorBot. Standalone monitoring process that watches every transaction announced to the Agent Proxy. Evaluates risk and can veto via CancelAuthority before the delay window expires. Layer 5 of the safety model.

NAVAwareHook. V4 hook that prices vault shares at Net Asset Value. Enables instant exit from Gotts Vaults via Uniswap swap instead of a withdrawal queue.

PLAYBOOK.md. Evolving document maintained by GottsLoop's curator loop. Contains strategic principles (confidence >= 0.85), tactical heuristics (confidence 0.5-0.85), candidate rules (confidence < 0.5, shadow-executing), and regime notes.

PolicyCage. On-chain hard boundaries for AI agent strategy. Immutable smart contract constraints: approved asset list, maximum position sizes, strategy whitelists, maximum drawdown tolerance, rebalance frequency limits. Cannot be bypassed by prompt injection.

Reflexion. Per-operation self-reflection pattern. After every DeFi operation, the agent generates a structured reflection: what happened (actual vs predicted), why (market conditions, decisions), what to do differently [42].

Semantic memory. Distilled insights stored in SQLite/sqlite-vec. Each insight has a category, confidence score (0.0-1.0), stability (Ebbinghaus decay parameter), and chain/token pair metadata. The output of ExpeL consolidation.

STRATEGY.md. YAML-frontmatter file defining a GottsLoop strategy: trigger conditions, action pipeline, constraints, risk bounds, time limits, and cost caps.

Terminal node. An agent that never delegates to other agents. Ensures independence and prevents circular dependencies. Examples: `safety-guardian`, `risk-assessor`, `wallet-provisioner`, `vault-watchdog`.

VaultReputationEngine. Contract that automatically attests ERC-8004 reputation milestones based on vault interactions. Twenty milestones across five categories (Entry, Time, Capital, Behavior, Ecosystem) on a 1,000-point scale.

x402. HTTP-native agent-to-agent payment protocol by Coinbase. Used for paid API access to external data providers and for inter-agent service payments.

## References

[1] Messari, "State of Uniswap," Messari Research, 2025-2026.

[2] T. Sumers, S. Yao, K. Narasimhan, and T. Griffiths, "Cognitive Architectures for Language Agents," *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2309.02427.

[3] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM Agents Are Experiential Learners," arXiv:2308.10144, 2023.

[4] N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, MIT Press, 1948.

[5] C. Argyris and D. A. Schon, *Organizational Learning: A Theory of Action Perspective*, Addison-Wesley, 1978.

[6] H. von Foerster, "Cybernetics of Cybernetics," in *Communication and Control in Society*, K. Krippendorff, Ed., Gordon and Breach, 1979.

[7] D. Crapis, M. De Rossi, J. Ellis, and E. Reppel, "ERC-8004: Agent Registry," Ethereum Improvement Proposals, August 2025. Mainnet deployment January 2026.

[8] CyberArk, "2025 Identity Security Landscape Report," CyberArk, 2025.

[9] Clanker, "Clanker Protocol: Autonomous Token Deployment on Base." [Online]. Available: https://clanker.world, 2025.

[10] BankrBot. [Online]. Available: https://bankrbot.com, 2025.

[11] A. Griffith, "CLAWD: An AI Agent with a Crypto Wallet," 2025. [Online]. Available: https://clawd.fun

[12] Virtuals Protocol, "Virtuals Agent Economy." [Online]. Available: https://virtuals.io, 2025-2026.

[13] S. Van Bulck et al., "TEE.Fail: Trusted Execution Environment Vulnerabilities," 2024. [Online]. Available: https://tee.fail

[14] L. Hammacher et al., "BadRAM: Physical Memory Manipulation Attacks Against AMD SEV-SNP," in *Proc. IEEE S&P*, 2025.

[15] J. De Meulemeester, D. Oswald, I. Verbauwhede, and J. Van Bulck, "Battering RAM: Rowhammer-Free Memory Interposition Attacks Against TEE Memory Isolation," in *Proc. IEEE S&P*, 2026.

[16] OWASP, "LLM01:2025 – Prompt Injection," OWASP Top 10 for LLM Applications, 2025.

[17] Morpho Labs, "Morpho Blue: Permissionless Lending Protocol," 2024.

[18] Anthropic, "Model Context Protocol (MCP) Specification." [Online]. Available: https://modelcontextprotocol.io, 2024.

[19] Google Cloud, "Agent2Agent (A2A) Protocol," Linux Foundation, 2025.

[20] ERC-4361, "Sign-In with Ethereum," Ethereum Improvement Proposals, 2022.

[21] C. Ji et al., "SEAgent: Confused Deputy Attacks in Multi-Agent AI Systems," arXiv, 2026.

[22] T. Huynh et al., "Understanding LLM Agent Behaviours via Game Theory," arXiv:2512.07462, December 2025.

[23] L. Hammond, A. Chan et al., "Multi-Agent Risks from Advanced AI," arXiv:2502.14143, February 2025.

[24] Z. Sun et al., "Game Theory Meets LLMs: A Systematic Survey," arXiv:2502.09053, February 2025.

[25] L. Zbandut et al., "Institutionalizing Risk Curation in Decentralized Credit," arXiv:2512.11976, December 2025.

[26] OpenZeppelin, "ERC-4626 Inflation Attack Protection via Virtual Shares," OpenZeppelin Contracts v4.9, 2023.

[27] Yearn Finance, "Yearn V3 Architecture: Linear Profit Unlock," Yearn Documentation, 2023.

[28] S. Zhang et al., "Systemic Risk in DeFi: Network-Based Fragility," arXiv:2601.08540, January 2026.

[29] A. Adams, C. C. Moallemi, S. Reynolds, and D. Robinson, "am-AMM: An Auction-Managed Automated Market Maker," 2024.

[30] Y. Xu and S. Brini, "PPO-Based Optimization for Liquidity Provision in AMMs," *Economic Modelling*, 2024.

[31] X. Qu et al., "Conservative Policy Learning with TD3-BC for DeFi Agents," arXiv, 2024.

[32] C. Moallemi and D. Robinson, "Loss-Versus-Fair: Efficiency of Dutch Auctions on Blockchains," in *Proc. AFT*, 2024.

[33] M. Devorsetz and M. Herlihy, "Defensive Rebalancing for AMMs," Brown University, arXiv, January 2026.

[34] Y. Bai et al., "Ormer: Manipulation-resistant Pricing Oracle," arXiv:2410.07893, October 2024.

[35] D. Xu et al., "SecPLF: Secure Protocols against Oracle Manipulation," arXiv:2401.08520, January 2024.

[36] Z. Deng et al., "OVer: Safeguarding DeFi against Oracle Deviations," in *Proc. ICSE*, 2024.

[37] A. Campbell, P. Bergault, I. Milionis, and M. Nutz, "Optimal Fees for Liquidity Provision in AMMs," arXiv:2508.08152, August 2025.

[38] P. Singh et al., "Modeling LVR via Continuous-Installment Options," arXiv:2508.02971, in *Proc. AFT*, 2025.

[39] M. Bichuch and Z. Feinstein, "Implied Volatility of AMM Fees," September 2025.

[40] F. Trotti et al., "Strategic Analysis of JIT Liquidity," arXiv:2509.16157, in *Proc. AFT*, 2025.

[41] R. E. Wray, J. R. Kirk, and J. E. Laird, "Applying Cognitive Design Patterns to General LLM Agents," arXiv:2505.07087, 2025.

[42] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Proc. NeurIPS*, 2023.

[43] W. R. Ashby, *An Introduction to Cybernetics*, Chapman & Hall, 1956.

[44] G. Wang et al., "Voyager: An Open-Ended Embodied Agent with Large Language Models," *Transactions on Machine Learning Research (TMLR)*, 2024.

[45] A. Kusupati et al., "Matryoshka Representation Learning," in *Proc. NeurIPS*, 2022. arXiv:2205.13147.

[46] Y. Ge et al., "SaMuLe: Learning from Failure Trajectories," arXiv:2509.20562, in *Proc. EMNLP*, 2025.

[47] H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*, 1885. English translation by H. A. Ruger and C. E. Bussenius, 1913.

[48] P. Sterling, "Allostasis: A Model of Predictive Regulation," *Physiology & Behavior*, vol. 106, no. 1, pp. 5-15, 2012.

[49] L. Wei et al., "FadeMem: Biologically-Inspired Forgetting for Efficient Agent Memory," arXiv:2601.18642, January 2026.

[50] W. Dabney, Z. Kurth-Nelson, N. Uchida, C. K. Starkweather, D. Hassabis, R. Munos, and M. Botvinick, "A distributional code for value in dopamine-based reinforcement learning," *Nature*, vol. 577, pp. 671-675, January 2020.

[51] W. Li et al., "FINSABER: Financial Strategies Assessment and Benchmarking for Evaluating Returns – Can LLM-based Agents Outperform the Market?" arXiv:2505.07078, in *Proc. KDD*, 2026.

[52] S. Beer, *Brain of the Firm*, Allen Lane, 1972. Second edition, Wiley, 1981.

[53] A. Bifet and R. Gavaldà, "Learning from Time-Changing Data with Adaptive Windowing," in *Proc. SIAM International Conference on Data Mining*, 2007.

[54] A. W. Lo, "The Adaptive Markets Hypothesis: Market Efficiency from an Evolutionary Perspective," *Journal of Portfolio Management*, vol. 30, no. 5, pp. 15-29, 2004.

[55] J. C. Maxwell, "On Governors," *Proceedings of the Royal Society of London*, vol. 16, pp. 270-283, 1868.

[56] W. R. Ashby, *Design for a Brain*, Chapman & Hall, 1952.

[57] H. R. Maturana and F. J. Varela, *Autopoiesis and Cognition: The Realization of the Living*, D. Reidel, 1980.

[58] J. R. Boyd, "A Discourse on Winning and Losing," unpublished briefing slides, Air University Library, Maxwell Air Force Base, 1987.

[59] R. C. Conant and W. R. Ashby, "Every Good Regulator of a System Must Be a Model of That System," *International Journal of Systems Science*, vol. 1, no. 2, pp. 89-97, 1970.

[60] W. T. Powers, *Behavior: The Control of Perception*, Aldine, 1973.

[61] Y. Yu et al., "FinMem: A Performance-Enhanced LLM Trading Agent with Layered Memory and Character Design," in *Proc. AAAI Spring Symposium Series*, 2024. arXiv:2311.13743.

[62] Y. Yu et al., "FinCon: A Synthesized LLM Multi-Agent System with Conceptual Verbal Reinforcement for Enhanced Financial Decision Making," in *Proc. NeurIPS*, 2024.

[63] Q. Zhang et al., "ACE: Agentic Context Engineering," arXiv:2510.04618, in *Proc. ICLR*, 2026.

[64] L. A. Agrawal et al., "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning," arXiv:2507.19457, in *Proc. ICLR* (Oral), 2026.

[65] Z. Pei et al., "SCOPE: Prompt Evolution for Enhancing Agent Effectiveness," arXiv:2512.15374, December 2025.

[66] G. Soros, *The Alchemy of Finance*, Simon & Schuster, 1987.

[67] AWS, "Strands Agent Standard Operating Procedures," November 2025.

[68] T. Kellogg, "Viable Systems: How To Build a Fully Autonomous Agent," blog series, January 2026.

[69] OWASP, "Top 10 for LLM Applications v2.0," OWASP Foundation, 2025.

[70] E. Debenedetti et al., "CaMeL: Capability-Based Machine Learning Agent Security," arXiv, 2025.

[71] R. Patlan et al., "Real AI Agents with Fake Memories (CrAIBench)," arXiv:2503.16248, March 2025.

[72] Y. Zhou et al., "TradeTrap: Are LLM-based Trading Agents Truly Reliable and Faithful?" arXiv:2512.02261, December 2025.

[73] M. A. Ferrag et al., "From Prompt Injections to Protocol Exploits," arXiv:2506.23260, June 2025.

[74] W. Xing et al., "MCP-Guard: Three-Stage Detection for MCP Security," arXiv:2508.10991, 2025.

[75] D. Bongaerts, S. D. De Luca, and M. Van Achter, "Circuit Breakers and Market Runs," *Review of Finance*, vol. 28, no. 6, pp. 1953-1989, 2024.

[76] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 Core," 2020.

[77] J. Hasbrouck, T. Rivera, and F. Saleh, "Economic Model of DEX with Concentrated Liquidity," *Management Science*, February 2025.

[78] P. Bergault, L. Bieber, and A. Sánchez-Betancourt, "Optimal Exit Time for LPs," arXiv, September 2025.

[79] A. Aqsha, P. Bergault, and A. Sánchez-Betancourt, "Equilibrium Reward for LP in AMMs," arXiv:2503.22502, March 2025.

[80] S. Beer, *The Heart of Enterprise*, John Wiley, 1979.

[81] S. Beer, "The Viable System Model: Its Provenance, Development, Methodology and Pathology," *Journal of the Operational Research Society*, vol. 35, no. 1, pp. 7-25, 1984.

[82] S. Beer, *Diagnosing the System for Organizations*, Wiley, 1985.

[83] J. R. Boyd, "Destruction and Creation," unpublished paper, September 1976.

[84] C. Tang et al., "AlphaAgent: LLM-Driven Alpha Mining with Regularized Exploration for Quantitative Investment," in *Proc. KDD*, 2025.

[85] Y. Ye et al., "UMEM: Unified Memory Optimization for LLM Agents," arXiv:2602.10652, February 2026.

[86] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[87] S. Alqithami et al., "Autonomous Agents on Blockchains: Standards, Execution, Trust," arXiv:2601.04583, January 2026.

[88] H. Duetting et al., "Mechanism Design for LLMs," in *Proc. ACM WWW*, 2024. arXiv:2310.10826.

[89] B. Kandaswamy et al., "Deep Reputation Scoring: zScore-Based Wallet Ranking," arXiv:2507.20494, July 2025.

[90] A. Udupi et al., "zScore: Universal Decentralised Reputation," arXiv:2503.05718, March 2025.

[91] B. Abdolmaleki et al., "Attribute-Based Threshold Issuance Anonymous Counting Tokens (tACT)," IACR ePrint 2024/1024, 2024.

[92] E. G. Weyl, P. Ohlhaver, and V. Buterin, "Decentralized Society: Finding Web3's Soul," SSRN, 2022.

[93] C. Yang et al., "MUSE: Learning on the Job — An Experience-Driven Self-Evolving Agent for Long-Horizon Tasks," arXiv:2510.08002, October 2025.

[94] J. Kim et al., "ReflAct: World-Grounded Decision Making in LLM Agents via Goal-State Reflection," in *Proc. EMNLP (Main)*, 2025. arXiv:2505.15182.

[95] C. Buhler et al., "AgentBound: Securing AI Agent Execution," arXiv:2510.21236, 2025.

[96] H. Errico et al., "Securing the MCP: Risks, Controls, Governance," arXiv:2511.20920, 2025.

[97] H. Wang et al., "AgentSpec: Runtime Enforcement DSL for AI Agents," in *Proc. ICSE*, 2026.

[98] M. Farzulla et al., "ASRI: Aggregated Systemic Risk Index for DeFi," arXiv:2602.03874, January 2026.

[99] A. Canidio et al., "Batching Trades on AMMs," arXiv:2307.02074, in *Proc. AFT*, 2023.

[100] S. Ko et al., "PA-AMM: Partially Active AMM," arXiv:2602.09887, February 2026.

[101] D. Fantazzini, "Conformal Prediction for Crypto-Asset VaR," 2024.

[102] A. Lipton, V. Lucic, and L. Sepp, "Unified Hedging of Impermanent Loss," *Digital Finance*, vol. 7, 2025. arXiv:2407.05146.

[103] K. Gogol et al., "Layer-2 Arbitrage: An Empirical Analysis," arXiv:2406.02172, June 2024.

[104] O. Solmaz et al., "Optimistic MEV in Ethereum Layer 2s," arXiv:2506.14768, June 2025.

[105] E. Tranquilli et al., "Formal State-Machine Models for Uniswap v3," arXiv, December 2025.

[106] V. Dessalvi et al., "Formal AMM Fee Mechanisms with Lean 4," arXiv, January 2026.

[107] B. Baggiani, M. Herdegen, and A. Sánchez-Betancourt, "Optimal Dynamic Fees in AMMs," arXiv:2506.02869, June 2025.

[108] A. Komo et al., "Shill-Proof Auctions," in *Proc. EC*, 2025. arXiv:2404.00475.

[109] J. Milionis et al., "Myersonian Framework for Optimal LP in AMMs," in *Proc. ITCS*, 2024. arXiv:2303.00208.

[110] J. Ma et al., "Cost of Permissionless LP in AMMs," arXiv:2402.18256, 2024.

[111] T. Ma et al., "Design of a Decentralized Fixed-Income Lending AMM Protocol Supporting Arbitrary Maturities," UMass Boston, arXiv:2512.16080, December 2025.

[112] E. Tranquilli and A. Gupta, "Formal State-Machine Models for Uniswap v3: Pool Favor Property," arXiv, December 2025.

[113] V. Dessalvi et al., "Formal AMM Fee Mechanisms with Lean 4," DTU/Cagliari, arXiv, January 2026.

[114] Anthropic, "SCONE-bench: Autonomous Smart Contract Exploitation Evaluation," Anthropic Research, 2025.

[115] AIXBT, "Post-Mortem: AIXBT Agent Wallet Compromise," March 2025.

[116] BlockSec, "Cork Protocol V4 Hook Exploit Analysis," May 2025.

[117] T. Kellogg, "Viable Systems: POSIWID, Algedonic Signals, and Identity Formation in Autonomous Agent Systems," blog series, January 2026.

[118] A. Ronacher et al., "OpenClaw: Conversational Daemon Architecture for Autonomous Agents," 2025-2026.

[120] Y. Hu and J. Ming, "When Do Circuit Breakers Stabilize Markets?" *Annals of Economics and Finance*, vol. 26, no. 2, pp. 573-613, 2025.

[121] Galileo AI, "Cascading Failure Modes in Multi-Agent Systems," December 2025.

[122] Lakera AI, "Adversarial Robustness of Multi-Agent Chains," November 2025.

[123] D. Ardia, K. Bluteau, and M. Rüede, "Regime Changes in Bitcoin GARCH Volatility Dynamics," *Finance Research Letters*, vol. 29, pp. 266–271, 2019.

[124] H. Adams et al., "Uniswap v4 Core Whitepaper," August 2024.

[125] M. Kato et al., "Conformal Predictive Portfolio Selection," arXiv:2410.16333, October 2024.

[126] O. Liu et al., "DeLLMa: Decision making under uncertainty with LLMs," arXiv:2402.02392, in *Proc. ICLR*, 2025.

[127] S. Shi, "TraceRank: Sybil-Resistant Service Discovery for Agent Economies," arXiv:2510.27554, October 2025.

[128] BlockSec, "Thorns in the Rose: Security Risks in Uniswap V4 Hooks," 2024.

[129] A. Gennaro and T. Mastrolia, "Delegated Portfolio Management with Random Default," arXiv:2410.13103, October 2024.

[130] A. Canidio et al., "Fair Combinatorial Auction for Blockchain Trade Intents," arXiv:2408.12225, 2024.

[132] Q. Shi et al., "Progent: Programmable Privilege Control for LLM Agents," arXiv:2504.11703, 2025.

[133] W. B. Cannon, *The Wisdom of the Body*, W. W. Norton, 1932.

[134] G. Soros, *The Crisis of Global Capitalism: Open Society Endangered*, PublicAffairs, 1998.

[135] J. R. Boyd, "Patterns of Conflict," unpublished briefing slides, Air University Library, 1986.

[136] See [41].

[137] Y. Ye et al., "UMEM: Unified Memory Optimization for Training Long-Context LLM Agents," arXiv:2602.10652, February 2026.

[138] See [86].

[139] B. Jimenez Gutierrez et al., "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," arXiv:2405.14831, 2024.

[140] A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv:2310.11511, in *Proc. ICLR*, 2024.

[141] S.-Q. Yan et al., "Corrective Retrieval Augmented Generation (CRAG)," arXiv:2401.15884, in *Proc. ICML*, 2024.

[142] X. Tan et al., "MemoTime: Temporal Knowledge Graph Memory for Long-Horizon Planning," 2025.

[143] R. D. Luce and H. Raiffa, *Games and Decisions: Introduction and Critical Survey*, Wiley, 1957.

[144] Anthropic, "Introducing Contextual Retrieval," Anthropic Research Blog, September 2024.

[145] S. Beer, "Diagnosing the System for Organizations," Wiley, 1985.

[146] *Unverifiable.* D. Fontana et al., "Endgame Defection Detection in Multi-Agent Systems via Behavioral Regime Analysis," arXiv, 2025. (Could not be independently verified.)

[147] See [126].

[148] R. A. Rescorla and A. R. Wagner, "A Theory of Pavlovian Conditioning: Variations in the Effectiveness of Reinforcement and Nonreinforcement," in *Classical Conditioning II*, A. H. Black and W. F. Prokasy, eds., pp. 64–99, Appleton-Century-Crofts, 1972.

[149] See [100].

[150] See [98].

[151] C. Chaliasos et al., "Smart Contract and DeFi Security Tools: Do They Meet the Needs of Practitioners?" in *Proc. International Conference on Software Engineering (ICSE)*, 2024. arXiv:2304.02981v2.

[152] C. Gan et al., "DeFiAligner: Leveraging Symbolic Analysis and LLMs for Specification-Implementation Consistency in DeFi," in *Proc. AFT*, ACM LIPIcs Vol. 316, 2024.

[153] Z. Zhou et al., "SoK: Decentralized Finance (DeFi) Attacks," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, pp. 2444–2461, 2023.

[154] Y. Oz et al., "Cross-Chain Arbitrage: The Next Frontier of MEV," arXiv:2501.17335, January 2025. Revised June 2025.

[155] B. Bahrani, S. Garimidi, and T. Roughgarden, "Transaction Fee Mechanism Design Post-MEV," in *Proc. AFT*, ACM LIPIcs Vol. 316, 2024.

[156] K. Bachu, Z. Wan, and C. Moallemi, "Quantifying Price Improvement in Order Flow Auctions," arXiv:2405.00537, 2024. in *Proc. CAAW*, 2025.

[157] V. Schlegel, M. Mattauch, and B. Moldovanu, "On Sybil-Proof Mechanisms," arXiv:2407.14485v3, July 2024. Updated May 2025.

[158] R. Fontana et al., "Nicer Than Humans: Large Language Models Exhibit Cooperative Behavior in the Prisoner's Dilemma," arXiv:2406.13605v2, September 2024.

[159] K. Payne et al., "Strategic Intelligence in LLMs: Evidence from Evolutionary Game Theory," arXiv:2507.02618, July 2025.

[160] Z. Qian et al., "MemoRAG: Moving Towards Next-Gen RAG via Memory-Inspired Knowledge Discovery," arXiv:2409.05591, in *Proc. WWW*, 2025.

[161] Z. Zhang et al., "A Survey on the Memory Mechanism of Large Language Model Based Agents," arXiv:2404.13501, *ACM Transactions on Information Systems (TOIS)*, 2024.

[162] Y. Xu et al., "A-MEM: Agentic Memory for LLM Agents," arXiv:2502.12110, in *Proc. NeurIPS*, 2025.

[163] Y. Ma, C. Zeng, and D. Zhang, "Stablecoin Runs and the Centralization of Arbitrage," NBER Working Paper 33882, May 2025.

[164] R. Rossetti et al., "Dynamics of Cooperation in Concurrent Games," *Nature Communications*, vol. 16, Article 1524, 2025.

[165] Sommelier Finance, "Sommelier Protocol Documentation," 2023. [Online]. Available: https://sommelier.finance/docs [Accessed: Feb. 2026]

[166] O. E. Williamson, "Transaction-Cost Economics: The Governance of Contractual Relations," *Journal of Law and Economics*, vol. 22, no. 2, pp. 233–261, 1979.

[167] N. Papernot, M. Abadi, U. Erlingsson, I. Goodfellow, and K. Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data," in *Proc. ICLR*, 2017. arXiv:1610.05755.

[168] B. Balle and Y.-X. Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising," in *Proc. ICML*, 2018. arXiv:1805.06530.

[169] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust Aggregation for Federated Learning," *IEEE Transactions on Signal Processing*, vol. 70, pp. 1142–1154, 2022.

[170] C. R. Harvey, Y. Liu, and H. Zhu, "… and the Cross-Section of Expected Returns," *Review of Financial Studies*, vol. 29, no. 1, pp. 5–68, 2016.

[171] M. Bailey and M. Lopez de Prado, "The Sharpe Ratio Efficient Frontier," *Journal of Risk*, vol. 15, no. 2, 2012.

[172] Y. Bakos and E. Brynjolfsson, "Bundling Information Goods: Pricing, Profits, and Efficiency," *Management Science*, vol. 45, no. 12, pp. 1613–1630, 1999.

[173] V. Nasrulin, G. Ishmaev, and J. Pouwelse, "MeritRank: Sybil Tolerant Reputation for Merit-Based Tokenomics," arXiv:2207.09950, 2022.

[174] U.S. Department of Justice and Federal Trade Commission, "Merger Guidelines," December 2023.

[175] M. Bailey, J. H. Borwein, M. Lopez de Prado, and Q. J. Zhu, "The Probability of Backtest Overfitting," *Journal of Computational Finance*, vol. 20, no. 4, 2014.

[176] Bunni Protocol, "Bunni v2 Exploit Post-Mortem," September 2025.

[177] Koi Security, "ClawHavoc: Malicious MCP Tools in Public Registries," 2025. [Online]. Available: https://clawhub.dev

[178] C. Koki, S. Leonardos, and G. Piliouras, "Exploring the Predictability of Cryptocurrencies via Bayesian Hidden Markov Models," *Research in International Business and Finance*, vol. 59, 2022.

[179] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, "Detecting Hallucinations in Large Language Models Using Semantic Entropy," *Nature*, vol. 630, pp. 625–630, June 2024.

[180] Y. Xiao et al., "TradingAgents: Multi-Agents LLM Financial Trading Framework," arXiv:2412.20138, 2024.

[181] Y. Xiao et al., "Trading-R1: LLM Trading Agent via Reinforcement Learning," arXiv:2509.11420, 2025.

[182] R. B. Myerson and M. A. Satterthwaite, "Efficient Mechanisms for Bilateral Trading," *Journal of Economic Theory*, vol. 29, no. 2, pp. 265–281, 1983.

[75b] A. Subrahmanyam, "Circuit Breakers and Market Volatility: A Theoretical Perspective," *Journal of Finance*, vol. 49, no. 1, pp. 237-254, 1994.

[76b] I. Milionis, C. Moallemi, T. Roughgarden, and A. L. Zhang, "Automated Market Making and Loss-Versus-Rebalancing," arXiv:2208.06046, *Journal of Political Economy: Microeconomics*, 2024.

[76c] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, "Uniswap v3 Core," 2021.

[76d] W. Tang, R. El-Azouzi, H. H. Lee, F. Y. Chan, and G. Fanti, "Game Theoretic Liquidity Provisioning in Concentrated Liquidity Market Makers," arXiv:2411.10399, in *Proc. ACM SIGMETRICS*, 2025.

[129b] Maple Finance, "Maple Protocol Whitepaper," 2021. [Online]. Available: https://maple.finance

*This document presents the design and specification of the Gotts protocol. All mechanisms are described as designed. Implementation status and v1 launch scope are tracked separately.*